

Finishing I/O, and advanced I/O — PB158 Lecture 3

Contents

1 The claim

2 How this lecture is built

3 1. Recap by prediction

4 2 to 5. Finishing Lecture 2

4.1 Five slides inside the carried block are this lecture's own

5 6. So when is it safe?

5.1 The prediction

5.2 The observation

5.3 The explanation

5.4 Which rule fired

5.5 Visibility is not durability

5.6 What the last handoff costs

5.7 What `fsync()` promises

5.8 Windows, at equal depth

5.9 Even the kernel's *done* can be wrong

5.10 What the engines did instead

5.11 Windows here is neither "the same" nor "safe"

5.12 The half that software can fix

5.13 The respectable opposite view

5.14 So when is it safe?

5.15 Log first – and what if the log write fails?

6 7. Waiting is the new I/O, and `epoll` (announced tail)

6.1 The prediction

6.2 Where the question comes from

6.3 A thread each, and why that stopped working

6.4 The observation

6.5 What the counts do and do not say

6.6 `epoll` is three calls

6.7 Level-triggered and edge-triggered

6.8 Three ways to wait

6.9 Readiness is permission to try

6.10 The real difference is buffer ownership

6.11 It is not a Linux/Windows split

6.12 The trap this demonstration sets

6.13 Four Windows words

6.14 Windows, completion, traced

6.15 What may not be done with these two numbers

6.16 What a ring is

6.17 Where both platforms ended up

7 8. Three destinations: sockets and pipes (announced tail)

7.1 First leg: a pipe

7.2 What that run does not test

7.3 Third destination: the terminal, captured

7.4 Second leg: a TCP socket

7.5 Then what are `send` and `recv` for?

7.6 The same interface is not the same contract

7.7 Windows: where the destination *does* change the trace

7.8 A `SOCKET` is not a `HANDLE`

8 9. Scatter/gather and zero-copy: bytes you never touch (announced tail)

8.1 First prediction, and the observation

8.2 `writew` buys crossings, not copies

8.3 Two atomicity promises, and only one is unconditional

8.4 Second prediction, and the observation

8.5 What "zero-copy" does not cover

8.6 Windows, and a fallback that says nothing

9 10. Homework, and the next question

9.1 Assignment 1 is not set today

9.2 Voluntary homework

9.3 What Lecture 4 answers

10 What this lecture does not establish

11 References

12 Appendix — sources from the author's working notes

12.1 V4 — The Gap Between a Successful `write()` and Data Reaching Disk

12.2 V5 — The Gap Between a Successful `write()` and Data on Disk

12.3 V6 — Destination-dependent buffering of standard output

12.4 V8 — The placement of the stable syscall boundary in Windows and Linux

12.5 V9 — The layer at which Windows and Linux stabilize the syscall boundary

12.6 V10 — The System Call Boundary and Its Observability

12.7 V11 — Batching and the Disappearing Syscall Under `io_uring`/`SQPOLL`

12.8 V14 — Leaky operating system abstractions

12.9 V17 — Tracing mechanisms for observing a running system

[12.10 V21 – NtTrace, a user-mode Windows native-API tracer](#)

[12.11 V22 – NtTrace, a debug-interface tracer for the Windows Native API](#)

[12.12 V23 – NtTrace, a user-mode Native API tracer for Windows](#)

[12.13 V24 – PostgreSQL's fsync\(\) error-reporting failure \("fsyncgate"\)](#)

[12.14 V25 – The Trade Between Tracing Cost and Structural Blind Spot](#)

[12.15 V28 – The socket as a kernel object and which layer binds its contract](#)

[12.16 V29 – Sockets as Kernel Objects, Published Differently by Linux and Windows](#)

[12.17 V30 – File I/O Buffering and Durability Guarantees](#)

[12.18 V33 – File I/O durability across the write-to-device path](#)

[12.19 V36 – The differing layer at which Windows and Linux stabilize the syscall boundary](#)

[12.20 V37 – The gap between a successful write and a durable one](#)

[12.21 V38 – The placement of the stable syscall boundary on Windows versus Linux](#)

[12.22 V39 – The Cost/Blind-Spot Trade-off in System Tracing Mechanisms](#)

[12.23 V40 – The fsync\(\) error-reporting failure \(fsyncgate\)](#)

[12.24 V41 – PostgreSQL's fsync\(\) failure-reporting bug \("fsyncgate"\)](#)

[12.25 V43 – The layer at which Windows and Linux place the stable syscall boundary](#)

[12.26 V44 – File I/O durability across the buffering stack](#)

[12.27 V45 – The Boundary Between an Accepted Write and a Durable One](#)

[12.28 V46 – File I/O and durability](#)

[12.29 V47 – The four-boundary model of file I/O durability](#)

[12.30 V48 – File I/O and Durability](#)

[12.31 V49 – The four-boundary model of file I/O durability](#)

[12.32 V50 – The gap between an OS abstraction's promise and its actual behaviour](#)

[12.33 V51 – Leaky Operating-System Abstractions](#)

[12.34 V52 – Leaky Abstractions at the OS Syscall Boundary](#)

[12.35 V53 – The Non-Retryability of a Failed fsync\(\)](#)

[12.36 V54 – The unretryable fsync\(\) failure](#)

[12.37 V55 – The non-retryability of a failed fsync\(\)](#)

[12.38 V56 – PostgreSQL's fsync\(\) error-reporting failure](#)

[12.39 V57 – Readiness and Completion I/O Models](#)

[12.40 V58 – Readiness and Completion Models for Asynchronous I/O](#)

[12.41 V59 – Readiness and Completion I/O Models](#)

[12.42 V60 – Readiness and completion as the two models of asynchronous network I/O](#)

[12.43 V61 – The Cost of Waiting Behind the C10K Problem](#)

[12.44 V62 – The Cost Model Behind the C10K Problem](#)

[12.45 V63 – The Interest-Set Cost Behind the C10K Problem](#)

[12.46 V64 – The socket as a kernel object, published differently on Linux and Windows](#)

[12.47 V65 – Sockets as kernel-owned file objects, on Linux and on Windows](#)

[12.48 V66 – Sockets as kernel objects: the file-descriptor/Winsock divergence](#)

[12.49 V67 – Interprocess Communication](#)

[12.50 V68 – io_uring and Windows IoRing as Ring-Based Completion Interfaces](#)

[12.51 V69 – Blocking as a Thread-Scheduling Decision, Not a Device Property](#)

[12.52 V70 – Network Buffering and Delivery Semantics](#)

[12.53 V71 – Destination-dependent buffering of standard output](#)

[12.54 V72 – Virtual memory and the page cache across Linux and Windows](#)

[12.55 V73 – Ring-Based Asynchronous I/O: io_uring and Windows IoRing](#)

[12.56 V74 – Readiness and completion I/O notification models](#)

[12.57 V75 – Readiness and Completion Models](#)

[12.58 V76 – Readiness and Completion I/O Models](#)

[12.59 V77 – Readiness and Completion I/O Models](#)

[12.60 V78 – The gap between a successful write and durable storage](#)

[12.61 V79 – Ring-Based Completion I/O on Linux and Windows](#)

[12.62 V80 – Ring-Based I/O on Linux \(`io\_uring` \) and Windows \(`IoRing` \)](#)

[12.63 V81 – The Readiness/Completion Split in Network I/O](#)

[12.64 V82 – Interprocess Communication](#)

[12.65 V83 – The unretryability of a failed `fsync\(\)`](#)

[12.66 V84 – The `fsync\(\)` silent-data-loss defect \("`fsyncgate`"\)](#)

[12.67 V85 – The unretryability of a failed `fsync\(\)` call](#)

[12.68 V86 – File I/O durability across the buffering stack](#)

[12.69 V87 – The socket as a kernel object, and which layer each platform makes its contract](#)

[12.70 V88 – Ring-based I/O interfaces on Linux and Windows](#)

[12.71 V89 – The unretryable `fsync\(\)` failure](#)

[12.72 V90 – The non-retryability of a failed `fsync\(\)`](#)

[12.73 V91 – File I/O durability across the write-to-medium path](#)

[12.74 V92 – Readiness versus Completion I/O Models on Linux and Windows](#)

[12.75 V93 – PostgreSQL's `fsync\(\)` error-reporting failure \("`fsyncgate`"\)](#)

[12.76 V94 – The Waiting Cost Behind the C10K Problem](#)

[12.77 V95 – The C10K Bottleneck: Waiting, Not I/O](#)

[12.78 V96 – The C10K problem as a cost of waiting, not a cost of I/O](#)

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

Reading this document: *[S<n>] cites the numbered source list at the end. [inference] marks a claim that follows from those sources but is not stated by them. [speculative] marks the author's judgement, offered as judgement.*

This document stands on its own. You do not need the slides to read it, and it goes further in several places than I went in the lecture. [V<n>] entries are notes from the author's research vault. The appendix at the end summarises each one and lists every external source it draws on.

1 The claim

Your program never talks to the disk, the network, or another program. It talks to the layer next to it. Every layer says *done* about something different.

The rule underneath is one sentence, and it is not about operating systems in particular: "Success means "accepted," not "done." Almost every fast OS interface returns when the kernel has *taken responsibility* for the work, not when the work has happened" [V50]. Speed is why the report is early. A layer that waited for the layer below it would be at least as slow as the layer below it [inference].

So there is one question to ask of any return value. **Whose *done* did you measure?** I ask it three times today — of a file, of a wait, and of a pipe and a socket. It works outside this course too. An HTTP 202 and a `git push` both return long before the work is done [inference].

2 How this lecture is built

The deck runs as one sequence of ten parts. Parts 1 to 6 and part 10 are planned for the ninety-minute slot. Parts 7, 8 and 9 are an **announced tail**. I build them in full. The slides and the speaker notes are written. The demonstrations are captured, and their containers travel in the repository [S53] [S54] [S56]. I present the tail only if the clock allows. If the room does not reach it, Lecture 4 opens with it unchanged.

Parts 2 to 5 are Lecture 2's slides 33 to 62. I carry them across verbatim and renumber them into this deck's own sequence. The room then tracks one set of part numbers rather than two. Nothing in that range is reworded or dropped. Only the kicker labels change. Seven slides in that stretch are this lecture's own. Two are written predictions that Lecture 2's own build did not yet collect. The other five are described below.

Every demonstration is a **Predict / Observe / Explain** triple. The room writes a number or a claim down before anything runs. The observation is produced live, never as a screenshot. The explanation comes from the model the lecture built [S14].

That method is not a classroom preference. Crouch, Fagen, Callan and Mazur compared demonstration conditions against a no-demonstration control and found that "students who passively observe demonstrations understand the underlying concepts no better than students who do not see the demonstration at all", while "students who predict the demonstration outcome before seeing it, however, display significantly greater understanding" [S51]. That was a physics study at one institution, so I do not claim the effect size transfers exactly to operating systems [inference]. I do claim the direction is established well enough to run the course this way.

3 1. Recap by prediction

The room draws Lecture 2's four-level buffer stack from memory. It marks the level the bytes are in after `fflush()` returns. Then it answers one question: what does a sudden power outage lose?

The four levels are the application buffer, the C library buffer, the kernel page cache and the device write cache. Levels 1 and 2 are alternative routes into level 3 rather than a pile, because CPython "reimplements the same policy above libc rather than inheriting it" [V6]. Each arrow between levels is one call. The calls, in order down the stack, are `f.flush()` [S17], `fflush()` [S3], and `fsync()` or `FlushFileBuffers` [S13] [V37]. The last arrow is a cache-flush command the kernel issues to the device [V33].

The answer to the power-outage question is the bridge into the rest of the hour, and it separates two failures that students routinely conflate.

- **The process dies.** Levels 1 and 2 die with it [V14]. Anything already handed to the kernel survives, because the kernel writes it back on its own schedule [V14].
- **The machine loses power.** Levels 1, 2 and 3 are gone [V14]. Level 4 goes too, unless the device has power-loss protection [inference]. Most drives acknowledge a write "once it lands in an onboard DRAM cache, before it reaches NAND or platter", and "a power cut in that window loses it" [V20].

The distinction matters because part 2 opens with a `kill -9` demonstration. My own note is blunt: "the honest classroom demo is a power-cut, not a `kill -9`. Killing the process loses only the *runtime's* buffer; the kernel still writes back" [V14]. Students who test with a `kill`, watch the file survive, and conclude that `flush()` was enough have drawn a correct conclusion about the wrong level.

Nobody pulls a plug in a lecture theatre. Part 6 measures the level-3 window instead, which is the window a power cut would have caught.

The room writes the prediction for part 6 here, before part 2. That is a deliberate change from the obvious ordering. A ninety-minute lecture is recalled best at its start and its end, and part 6 sits in the middle. Collecting its prediction early means the room carries an open question through the carried hour [inference].

4 2 to 5. Finishing Lecture 2

Lecture 2 reached its slide 32 in a ninety-minute session. Slides 33 to 62 are carried here without a word changed. They occupy four parts of this deck:

- **2 • what dies with the process** — the `kill -9` triple, `ltrace`, and which levels a tracer can watch [V10].
- **3 • one write, three levels** — the `iodemo.c` walk [S37].
- **4 • two kernels** — the matched Linux/Windows trace pair [S36] [S37].
- **5 • buffering moves your errors** — the unresolved durability question this lecture then answers [S2].

The carried slides wear this deck's part labels. Lecture 2's own roman numerals are gone from the kickers. A deck that alternates between "Part 1" and "Part III" makes the room track two numbering schemes at once. The bodies and the speaker notes are byte-for-byte Lecture 2's. Nine of those notes still say "section two", "section three", "section five" or "section six" in Lecture 2's own numbering, which no longer matches any label on screen. Carrying them unaltered was the instruction, so I read the part number aloud instead.

Lecture 2's slide 63, "What to do on Monday", is **not** carried. A "Monday" close is a conference-talk device and makes no sense in a university course. Its content was the Assignment 1 hand-off, and part 10 does that on a slide of its own, keeping the same three working rules.

The two inserted slides are predictions. The first is collected before the `ltrace` run. How many `fwrite` lines will a library tracer print for three `fwrite` calls? Will `ltrace -s` show the single `write` the kernel saw? Both answers follow from the stack the room

drew an hour earlier [inference]. `ltrace` "intercepts and records the dynamic library calls which are called by the executed process", and with `-s` it displays "system calls as well as library calls" [S31] [S32].

The second is collected before the Windows capture. How many write calls does the Windows trace show for the same three `fwrite` calls? How many error codes does Windows report for the same failed open? The first question tests whether the room learned a Linux fact or a C fact. The second has a surprise in it. The Windows return value carries an `NTSTATUS` and the Win32 error it translates into. Linux carries one `errno` [S36] [S37].

4.1 Five slides inside the carried block are this lecture's own

Five slides in Parts 2 and 5 are written for Lecture 3 rather than carried from Lecture 2. Two are bridges — *Where Lecture 2 stopped* opens Part 2, and *Lecture 2 ends here*. The question does not. closes Part 5 into Part 6. The other three extend the carried slide *Never retry a failed close*, which stated the rule and showed no measurement.

The prediction. Every `write()` in a program returned the byte count it asked for. Can the `close()` still fail? And if it fails, is the descriptor still the program's? Both answers go on paper before anything runs [S14].

Why a close can fail. The kernel deferred the write's error. `close()` is where that error surfaces. The manual is direct: "it is quite possible that errors on a previous `write(2)` operation are reported only on the final `close()`", and failing to check it "may lead to silent loss of data. This can especially be observed with NFS and with disk quota" [S4]. Read that against the four-level stack. `write()` said *done* at level 3, and `close()` is the last call in the program that can still refuse the bytes [S4] [V30]. The mirror sentence bounds the repair: "A successful close does not guarantee that the data has been successfully saved to disk, as the kernel uses the buffer cache to defer writes" [S4]. A clean close is therefore necessary and not sufficient, which is part 6's question.

Closing twice, measured. `demo/close/` opens a file, writes, closes, and then closes the same descriptor number again. That second close is what a retry would do. Under `strace` on Linux, `close(3)` returns 0 and the second `close(3)` returns `-1 EBADF` [S39]. The reason is in the manual: Linux "always releases the file descriptor early in the close operation, freeing it for reuse", and "the steps that may return an error ... occur only later" [S4]. So a retry closes nothing, or worse, "a reused file descriptor from another thread" [S4]. Untraced on Windows, the second `CloseHandle` fails with error 6, `ERROR_INVALID_HANDLE` [S39]. Microsoft documents a twist for anyone tracing: under a de-

bugger the same call "will throw an exception if it receives ... a handle value that is not valid ... This can happen if you close a handle twice" [S38]. NtTrace is a debugger, so the traced run raises rather than failing quietly [S39].

These five are mine rather than Lecture 2's, and the slides carry the shorter form of each quotation while this document carries the full one. Where a slide shows an elision, the sentence above it here is the whole sentence from the manual, hedge included.

5 6. So when is it safe?

Here I answer the question I left open at the end of Lecture 2. `write()` returned success and the bytes are not on a disk. So when is it actually safe? The manual page states the problem in one sentence — "A successful return from `write()` does not make any guarantee that data has been committed to disk" [S2] — and Saltzer, Reed and Clark gave the general form in 1984, where a lower level may offer "an incomplete version of the function... useful as a performance enhancement" [S11].

5.1 The prediction

`save.py` writes a hundred records to a file with the default buffering, prints *done*, and exits. No `flush()`. No `fsync()`. Exactly the idiom every student already writes, and an idiom that "contains no durability step anywhere" [V4].

How long until the kernel actually writes those bytes to the device? Under a second, about five seconds, about thirty seconds, or not until shutdown?

5.2 The observation

`watch_dirty.py` runs `save.py`, then samples the kernel's own count of file data it has not yet written. Those are `Dirty` and `Writeback` in `/proc/meminfo`. It samples every half second, until the count falls back to its baseline [S52]. A page the kernel holds and has not written is a **dirty page**, and the kernel copying it out later is **writeback** [V78].

No tracer is involved, which is deliberate. Lecture 2 established that a tracer changes the program it watches [S1]. Here the program is not watched at all. The kernel publishes the number itself.

The captured run, 2026-09-20, in `python:3.12-slim` on Docker's Linux virtual machine, kernel 6.8.0-139-generic, root filesystem ext4 with `commit=30,data=ordered` [S52]:

```

t= 0.0s Dirty= 1176 kB baseline
t= 0.0s Dirty= 5132 kB <- save.py has EXITED: "done: 100 records x 40960 B"
t= 0.5s Dirty= 5284 kB
t= 2.5s Dirty= 5364 kB
t= 4.0s Dirty= 5388 kB
t= 5.0s Dirty= 676 kB <- the kernel has finished

```

Five runs across three file sizes [S52]:

WRITTEN	RUNS	THE KERNEL'S <i>DONE</i> , AFTER THE PROGRAM'S
100 × 40 KiB = 4 MiB	3 (two on overlayfs, one on plain ext4)	5.0 s, 5.5 s, 5.5 s
100 × 64 B = 6 KiB	1	5.5 s
1000 × 40 KiB = 40 MiB	1	0.5 s

Two limits travel with these numbers, and both are in the README. The `Dirty` counter belongs to the whole machine rather than to our file. The shape of the rise and fall is therefore the evidence, not the absolute value. And every figure is one machine, one kernel and one filesystem [S52].

The machine-wide counter has a visible consequence in the capture above, and it is worth naming before a student notices it unaided. The watcher samples until the count returns to its baseline, and this run finished at 676 kB against a baseline of 1176 kB [S52]. It went *below* where it started, because something else on the machine also finished writing back inside that five-second window. Read the fall, not the final figure.

5.3 The explanation

The program's *done* and the kernel's *done* are different events, about five seconds apart on this machine. For those five seconds the bytes sat in the page cache, level 3 of the stack, in memory owned by no process [V78]. A kernel timer set the delay [V78]. Neither the program nor the Python runtime set one.

The return value was correct. It reported that the kernel had accepted the bytes [V50]. It reported nothing about the device.

5.4 Which rule fired

The setting a student finds first is `dirty_expire_centisecs`. The kernel page defines it as "when dirty data is "old enough to be eligible for writeout by the kernel flusher threads" [S68]. The demo machine read it as 3000 — thirty seconds [S52]. Nothing here waited thirty seconds. The second setting, `dirty_writeback_centisecs`, controls how often "the kernel flusher threads will periodically wake up and write old data out to disk" [S68]. The same machine read it as 500, so five seconds [S52]. Five seconds matches that. Both numbers are this machine's readout, not values the kernel documentation states.

The 40 MiB run went out in half a second, before any timer, because it crossed a memory threshold: "Under memory pressure or above `dirty_background_ratio`, writeback starts sooner" [V48]. That is the answer to the obvious objection — more data did not become safer, it became large enough to trip a different rule [inference].

Writeback has several competing rules. Which one fired in this capture is an inference from the timings, not something this demonstration can show [inference]. Establishing it would need a kernel tracer, which this course does not teach. The lesson survives either way: the kernel's *done* is on the kernel's clock, and the program did not set it.

5.5 Visibility is not durability

The objection the room raises here is *but I can read the file back*. That is true, and it is guaranteed. After a successful `write()`, a later read returns the bytes, including a read from another process. My note names the distinction exactly: "That is **visibility**, which is a coherence property of the page cache. It is orthogonal to **durability**, which is a property of the medium. Conflating the two is the single most common error in this area" [V45].

So `cat results.csv` returning the data proves visibility and proves nothing about the disk [inference]. During the five-second window it would have shown every byte.

5.6 What the last handoff costs

The second prediction of this section: `flush()` per record costs 1.1 ms of wall time for 101 `write` calls. What does adding one `fsync()` per record cost?

The captured counts, four modes, 100 records of 64 bytes, under `strace -c -f -e trace=write,fsync,fdatasync` [S52]:

MODE	WRITE CALLS	FSYNC CALLS	WALL TIME
default	6	0	0.2 ms
flush per record	101	0	1.1 ms
fsync per record	101	100	67.3 ms
unbuffered	101	0	1.0 ms

Sixty times the wall time, for the same hundred `write` calls plus a hundred `fsync` calls. Only the write count is identical: the `fsync` mode crosses the boundary 201 times against the flush mode's 101. The extra hundred crossings are not what cost the 66 ms. Lecture 2 measured a crossing in microseconds. The price is the waiting, not the crossing [inference]. That is the price of the last software handoff, and it is why a program that is not a database never pays it [inference]. Microsoft concedes the same about its own equivalent: "the FlushFileBuffers function can be inefficient when used after every write to a disk drive device when many writes are being performed separately" [S40].

One number in that capture is worth its own sentence. `strace -c` reported 2.2 ms *inside* `fsync` against 67 ms of wall time [S52]. The other 65 ms were the process waiting for the device, which the CPU-time column does not see. That is Lecture 2's rule in one measurement: count under the tracer, time without it.

5.7 What `fsync()` promises

`fsync()` "transfers ("flushes") all modified in-core data... to the disk device", and this "includes writing through or flushing a disk cache if present" [S67]. It "blocks until the device reports that the transfer has completed" [S67].

Read the last four words again. *The device reports*. No system-call trace can distinguish "an honest flush from a fast lie" [V5]. Nor is that a theoretical worry. SQLite's documentation records that "often the IDE disk control lies and says that data has reached oxide while it is still held only in the volatile control cache" [V86].

The correct general statement is my note's: "Durability is not a property of a write. It is a property of a sequence of flushes, each of which crosses exactly one boundary, and each of which can fail independently" [V44]. `fsync()` is one crossing in that chain. It is the last one you can observe.

5.8 Windows, at equal depth

Windows has the same shape with different names. It "caches file writes in the system file cache — "a write-back cache" — under the direction of the Cache Manager, which "spawns a process every second called a lazy writer" that "queues one-eighth of the pages that have not been flushed recently to be written to disk" [V46]. `FlushFileBuffers` is the `fsync` analogue [S40] [V37].

Both observations now have Windows twins, captured on 2026-09-20 on a Windows 11 virtual machine (build 26200, ARM64) [S52].

The cost. `iocost.c` is the Win32 twin of `save.py`'s four modes. It writes a hundred 4 KiB records four ways:

- with `WriteFile` into the cache,
- with `FlushFileBuffers` after each record,
- with `FILE_FLAG_WRITE_THROUGH`,
- with `FILE_FLAG_NO_BUFFERING` [S52].

Timed with `QueryPerformanceCounter` and counted with `GetProcessIoCounters`, the wall times were 0.7 ms, **27.9 ms**, 25.2 ms and 9.9 ms [S52]. Flushing after every record costs forty times the cached write. Linux's figure was sixty. Do not divide one of those ratios by the other. These records are 4 KiB and the Linux records were 64 B [S52]. That is a 64-fold difference in payload per record, and this demonstration does not control for it [S52]. What survives is the shape. On both platforms, asking the device to confirm each record costs one to two orders of magnitude of wall time [inference]. The call count does not change [S52]. Microsoft's own page concedes it: the call "can be inefficient when used after every write to a disk drive device" [S40].

The interesting column is the one that does not move. The kernel counted 100 write operations in every mode, including the two that bypass the cache [S52]. For two lectures this course has taught the room to count crossings. Here the count is the number that says nothing and the wall time is the number that does. The count records how many times the program asked. The time records who waited [inference]. Under `NtTrace`, flush mode shows 101 `NtWriteFile` and 100 `NtFlushBuffersFile`, default mode the same writes and no flush at all [S52].

Two limits on those figures. The binary is x64 under emulation on an ARM64 virtual machine, so the shape is the claim and not the milliseconds. And under the tracer the 0.7 ms became 26.6 ms [S52] — count traced, time untraced, on Windows too.

The delay. `watch_dirty_win32.py` is the twin of `watch_dirty.py`. There is no `/proc/meminfo` on Windows. It samples the Cache Manager's own PDH counters instead, `\Cache\Dirty Pages` and `\Cache\Lazy Write Flushes/sec`, until the dirty count returns

to baseline. For a 4 MiB file the Cache Manager finished **0.5 s, 1.0 s and 0.5 s** after the program said *done*. Linux took 5.0, 5.5 and 5.5 for the same file. For a 40 MiB file the order reverses. Windows took **14.1 s**, trickling roughly 512 pages a pass, where Linux wrote everything in 0.5 s [S52].

Both platforms therefore answer the prediction slide's four options, and they answer differently for the same file. Neither clock belongs to the program. The 512-page pattern is my reading of the timeline rather than something the demo shows. Windows Internals is the source to read for the lazy writer's rules, and I am not paraphrasing it into a fact [inference].

5.9 Even the kernel's *done* can be wrong

In 2018 the PostgreSQL project discovered that a failed `fsync()` cannot be retried on Linux. Craig Ringer stated the contract disagreement precisely: "Retrying `fsync()` is not OK at least on Linux. When `fsync()` returns success it means "all writes since the last `fsync` have hit disk" but we assume it means "all writes since the last SUCCESSFUL `fsync` have hit disk"" [S41].

The behaviour reproduces. Against a `dmsetup` error target, Justin Pryzby observed under `strace`: `fsync(3) = -1 EIO`, then `close(3)`, `open(...)`, then `fsync(3) = 0` — annotated "fsync on file with still-dirty data+EIO returns success BAD", for a byte that "certainly never got written" [S41] [V53].

The mechanism is not careless code on anyone's part. **The retry does not re-attempt the failed I/O** [V54]. "It reads a status register that the previous read destroyed, over a page cache the data has already left" [V89]. The misreading stood long enough to be "how PostgreSQL lost data silently for twenty years" [V90].

5.10 What the engines did instead

Three shipped storage engines answered in the same year, and they answered the same way [V40]. "PostgreSQL PANICs and back-patched to every supported release" [V93]. "InnoDB shipped first, deleting an explicit sleep-and-retry loop and substituting `ib::fatal()`" [V84]. WiredTiger converted its sync paths and wrote the reason into the source comment: "we don't retry `fsync`-style calls and panic if a flush operation fails" [V24].

The argument for crashing deserves a sentence, because students read it as giving up. Completing a checkpoint is the moment a database discards its own last copy. The `fsync()` return code is the sole evidence that licenses the discard. Once that evidence

may be false, the in-memory model of what is on disk cannot be repaired locally. The sound move is to discard the model and rebuild it from the log. Crashing is what preserves the log [V40] [V41].

Two honesty notes. These were not three independent discoveries — WiredTiger's comment quotes the same LWN report that the other two were reading [V24]. And the answer was a stopgap: "The convergence on PANIC was a stopgap that has now lasted eight years" [V56].

The state of the art has not moved much. A fault-injection study at USENIX ATC 2020 covered PostgreSQL, LMDB, LevelDB, SQLite and Redis across ext4, XFS and Btrfs. It found that the applications "use many failure-handling strategies, none are sufficient: fsync failures can cause catastrophic outcomes such as data loss and corruption", and that "pages are always marked clean" after a failure [S42].

5.11 Windows here is neither "the same" nor "safe"

Eight years on, the PostgreSQL wiki still keeps a table of what each operating system does after a failed `fsync`. It covers Linux, FreeBSD, NetBSD, Darwin, Illumos and DragonflyBSD. "It contains no Windows row at all" [V47].

The reason is structural. On Linux the discard-and-report-once logic lives in shared kernel code that anybody can read. On Windows `FlushFileBuffers` "is a thin user-mode wrapper over `NtFlushBuffersFile`, which assembles an `IRP_MJ_FLUSH_BUFFERS` request and dispatches it via `IoCallDriver` to whichever filesystem driver owns the volume" [V83], so the answer may differ per driver: "The Linux contract failure was *discoverable* because the contract was inspectable; the Windows contract has not been discovered, and by construction cannot be settled by reading" [V55].

So the correct sentence is neither "Windows is safe" nor "Windows has the same bug". Linux documents a hostile contract, which is actionable. The Windows one has never been settled [V55], and "For a system that must not lose data, an unspecified contract is the worse of the two positions" [V91]. Reading what a specification does not say is a professional skill, and this is the clearest example this course has.

5.12 The half that software can fix

Five of the last six paragraphs concern things a programmer cannot control. This one does not.

- `fsync()` on a file does not necessarily save the file's directory entry [S67]. "For that an explicit `fsync()` on a file descriptor for the directory is also needed" [S67].

- Therefore "Create-write-fsync-close on a new file is the classic incomplete recipe" [V51]. The bytes are safe and the file may be unfindable [inference].
- Write-then-rename, which is how every editor saves, needs a sync first [S44]. The pattern exists because `rename()` is atomic for readers — "there is no point at which another process attempting to access newpath will find it missing" [S70] — and the trap is that the rename is metadata written eagerly while the temporary file's bytes are still dirty in the page cache; `fsync` the temporary file before the rename [S44]. On ext4 in 2009, without one, "that existing file gets replaced with a zero-length file" [S44].

Each of those is one line of code. The device you cannot check. The directory you can [inference].

5.13 The respectable opposite view

I argue one side of this in the lecture, so the handout owes you the other, and it is not a straw man.

SQLite documents its recommended write-ahead-logging setting as deliberately non-durable: "WAL mode does lose durability. A transaction committed in WAL mode with synchronous=NORMAL might roll back following a power loss or system crash", and the same paragraph calls it "a good choice for most applications running in WAL mode" [S43]. For an address book on a phone that is the right trade.

My own note concedes the general position: trusting the abstraction and covering the gap operationally "is a defensible engineering position for a great many systems, and it is the implicit default" [V52].

The strongest version of the objection is Valerie Aurora's, and it argues that the interface rather than the programmer is at fault: "It may not be POSIX, but the programmer's intent is clear - no one ever, ever wrote `creat()`; `write()`; `close()`; `rename()`; and hoped they would get an empty file if the system crashed during the next 5 minutes" [S45]. On that reading, a system that demands this knowledge of every programmer has already failed.

Where does the defensible position stop being defensible? My note names the line exactly. It "fails specifically where the abstraction's failure mode is *silent*" [V52]. Backups do not help against a loss nothing reported.

5.14 So when is it safe?

In software, never completely. Every *done* in the stack is a handoff, and the last one you can watch is `fsync()`, which trusts the device [S67] [V5].

In hardware, the honest answer is the one the room already heard in Lecture 2, said aloud rather than put on a slide. The only real answer to durability is a well-tested uninterruptible power supply, and it is still best effort. Keep the power on long enough for the last handoff to finish.

That answer covers exactly one failure. `fsync`-gate lost data with the power on and the machine running [V53]. A battery is no defence against a failure the kernel reports once and cannot report again [V54]. So the full answer has two halves: keep the power on, and write every change to a log first, because "the only way to avoid data loss is to recover from the WAL after any failure is reported" [V85].

SQLite states the position most plainly of anyone. It calls `fsync()` on Unix and `FlushFileBuffers()` on Windows and trusts neither, because "we have received reports that neither of these interfaces works as advertised on many systems", so "its correctness rests on journalling and checksums rather than on trusting either primitive" [V49]. That is the state of the art — not a better call, but a design that survives the call being wrong.

5.15 Log first — and what if the log write fails?

The write-ahead log protects a database because of an order, and the order covers its own failure. The database writes every change to the log and calls `fsync` on it first. Only then does it say *committed* to the client [V85]. It writes the data file later, at a checkpoint [S71]. If the log's `fsync` fails, nothing has been promised yet, and PostgreSQL panics there too — the 2018 commit notes "we already did the same for WAL data; this change extends that behavior to non-temporary data files" [S71]. If the data file's `fsync` fails later, "the only remaining copy of the data is in the WAL" [S71]: crash, replay the log. The database trusts the log more than the file for one reason. It makes the log durable first, at one `fsync` per commit — the 67 ms measured earlier in this part [S52].

6.7. Waiting is the new I/O, and `epoll` (announced tail)

6.1 The prediction

A hundred pipes that nobody ever writes to. Two programs. Both wait two seconds and receive nothing. They differ only in how they wait. How many system calls does each one make?

6.2 Where the question comes from

Dan Kegel named the C10K problem in 1999 and disposed of the hardware explanation in his opening: "So hardware is no longer the bottleneck". That sentence is in the earliest archived revision of the page, 8 May 1999 [S55]. The cost of waiting on the idle connections bounded these servers at roughly ten thousand. The cost of moving bytes for the busy ones did not.

The controlled experiment came five years later. Gammo, Brecht, Shukla and Pariag held the request workload fixed and varied only the number of idle connections. With none, "the select and poll event mechanisms perform comparably to the `epoll` event mechanism". With 10,000 idle connections added to the same workload, "the performance of select and poll degrade by up to 79%" [S49]. Same bytes, same requests, up to 79% of the throughput gone, and the only thing added was connections doing nothing [V61]. None of the answers made a `read` faster: "Every one of them changed **where the interest set lives and who re-states it**" [V62].

A precision note that matters, because students will find Kegel's page. The idleness clause everyone quotes is absent from the 8 May 1999 revision [S55] and present in the current one [S48]. The waiting-cost framing is "a *sharpening* of the 1999 original, not a quotation of it" [V63]. Cite the revision, or better, cite the measurement.

6.3 A thread each, and why that stopped working

The obvious answer to a hundred pipes is a hundred threads. Each thread blocks in `read()` on one pipe, and no program ever polls. This design is called *thread-per-connection*, and it is the blocking rung of the ladder above, run a hundred times over. It failed, and it failed on two separate bills.

The first bill is memory, because a thread needs a stack. Kegel did the arithmetic: "if each thread gets a 2MB stack (not an uncommon default value), you run out of *virtual memory* at $(2^{30} / 2^{21}) = 512$ threads on a 32 bit machine with 1GB user-accessible

VM" [V94]. The second bill is the scheduler. Banga, Mogul and Druschel stated it in 1999: the design "scales badly as the number of threads increases, because of the costs of thread scheduling, context-switching, and thread-state storage space. The use of a single process per connection is even more costly" [V95].

Put that beside the counts on the next page. A parked thread holds memory while nothing arrives, and a rescanned descriptor burns processor time while nothing arrives. Each model charges per connection rather than per byte. My own note joins the two rungs — "A parked thread and a scanned-but-idle descriptor are the same mistake at two altitudes: a *per-connection* charge levied whether or not the connection is doing anything" [V96] — and it says plainly that neither source wrote that sentence.

This rung is history and arithmetic. I measured none of it for this lecture, and the demonstration below counts system calls rather than threads [inference].

6.4 The observation

`strace -c -f` on both programs, 100 pipes, 2 seconds, in `python:3.12-slim` [S53]:

wait_blocking.py				wait_epoll.py			
% time	calls	errors	syscall	% time	calls	errors	syscall
97.75	28017	28000	read	16.97	100		epoll_ctl
				1.97	1		epoll_pwait

28,017 `read` calls, 28,000 of them failing with `EAGAIN`, against one `epoll_pwait` that slept for two seconds. The hundred `epoll_ctl` calls are the registration, paid once [S53].

`EAGAIN` is not a fault. It is the kernel answering "nothing yet" to a non-blocking read, and `strace -c` files it under a column headed `errors` [S53]. The polling program is working exactly as written.

6.5 What the counts do and do not say

- The polling count is a **rate** [S53]. "run it for twenty seconds and it makes ten times as many calls, all to learn nothing" [S53].
- The `epoll` count is a **constant** [S53]. One wait call, however long the wait [S53].
- 28,000 is a **floor**, not the true rate [S53]. The tracer slowed the loop, and "the laptop, untraced, asked 1.3 million times in one second" [S53].

That last point is the uncomfortable one, and it is the same instrument problem the whole course keeps meeting. The traced run made 28,017 reads in two seconds, so roughly 14,000 a second [S53]. The untraced laptop figure is 1.3 million in one second [S53]. Those two figures divide out to roughly 93× [inference]. They come from two different machines, so 93× gives the size of the gap and not a controlled ratio [inference]. A traced process "runs more slowly than a non-traced one" [S1]. Brendan Gregg timed one loop that does nothing but system calls: `dd if=/dev/zero of=/dev/null bs=1 count=500k` [S66]. It ran in 0.103851 s untraced and 45.9599 s under `strace`, which is the 442× [S66]. He states its limit himself — "It is worst-case, since `dd(1)` is performing syscalls as fast as it can" [S66] — so this is one person's measurement of one loop, not a property of the tool [V17]. The direction of the result is unaffected, and the ratio is if anything understated [inference].

6.6 `epoll` is three calls

The trace the room has just read is `epoll`, and the API deserves naming rather than implying. Three calls make the Linux readiness model [S46]:

- `epoll_create1` [S46] creates the *interest set* — a structure that lives in the kernel between calls, named by a descriptor the program holds.
- `epoll_ctl` [S46] adds, changes or removes one descriptor in that set. The hundred registrations in the trace are a hundred of these, paid once [S53].
- `epoll_wait` [S46] sleeps until a registered descriptor is ready or the timeout expires. The trace shows `epoll_pwait`, the variant that also takes a signal mask [S53].

That is the whole structural difference from `select` and `poll`, which take the full set as an argument on every call [S46]. `epoll` states the set once, which is why the polling count is a rate and the `epoll` count is a constant.

6.7 Level-triggered and edge-triggered

`epoll` has two notification disciplines and the choice between them is where Linux network code goes wrong quietly.

Level-triggered is the default, and it is what `wait_epoll.py` used. While a descriptor still has unread data, every wait reports it again. The man page's own summary of that mode is reassuring: without `EPOLLET`, "epoll is simply a faster `poll(2)`, and can be used wherever the latter is used since it shares the same semantics" [S46].

Edge-triggered, selected with `EPOLLET`, "delivers events only when *changes* occur on the monitored file descriptor" [S46]. The man page's own worked example is the bug in miniature: a writer puts 2 kB in a pipe, `epoll_wait` reports the pipe, the reader consumes 1 kB, and the next `epoll_wait` "will probably hang despite the available data still present in the file input buffer" [S46]. The unread kilobyte is not a change.

So the mode carries a two-part contract, both halves on the page: use it "(1) with nonblocking file descriptors; and (2) by waiting for an event only after `read(2)` or `write(2)` return `EAGAIN`" [S46]. Drop the first and the drain read that is *supposed* to return `EAGAIN` blocks the whole event loop. Drop the second and the descriptor goes quiet.

Nothing errors when the contract is broken. That silence is why this is worth a slide to students who will not write such a loop for another year [inference]. And the prescribed fix brings its own problem, which the same page names: "If there is a large amount of I/O space, it is possible that by trying to drain it the other files will not get processed causing starvation" [S46]. The remedy is a ready list of the program's own, on top of the kernel's. Level-triggered first, and edge-triggered only with a measured reason, is the defensible default [speculative].

6.8 Three ways to wait

Polling, readiness and completion are one ladder rather than a feature list. They "are three answers to one question — *who gets parked, and where*" [V69].

- **Polling.** The program asks every descriptor in turn [S53]. Our 28,000 failed reads [S53].
- **Readiness.** The program registers once, then makes one call that sleeps until the kernel says "you may read now" [S46]. The program still does the read itself [S46]. `epoll(7)` keeps "the interest list" and "the ready list" in the kernel across calls, which is what removes the rescan [S46].
- **Completion.** The program hands the kernel a buffer and is told "I have read for you" [V74]. Overlapped I/O and I/O completion ports on Windows [V60]. Not demonstrated here [S53].

The practical test is a trace [inference]:

- many failed reads means polling [S53]
- one sleeping call means readiness [S53]
- no reads at all, but completion notifications, means completion [inference]

The cost of each model is paid while nothing is happening [V61].

6.9 Readiness is permission to try

The word *ready* sounds like a guarantee and it is not one. `select(2)` says so about itself in its own BUGS section: it "may report a socket file descriptor as "ready for reading", while nevertheless a subsequent read blocks" [S47]. The same is true of `epoll` [V92].

The consequence is a rule: use non-blocking mode even in code that has just been told the descriptor is ready [S47]. The model to hold is "A readiness notification is permission to try, not a promise of success" [V58].

Notice what that is. It is this lecture's thesis one rung up the ladder. The kernel reported correctly about its own state at the instant of the call. The program read it as a report about its next call [inference].

6.10 The real difference is buffer ownership

	READINESS	COMPLETION
when you supply the buffer	after you learn the descriptor is ready	at submission, before any data exists
who owns it while waiting	you — the kernel holds no pointer	the kernel — you must not touch or free it
one buffer can	be shared across many connections	serve one outstanding operation

My own note's verdict on this comparison is "The load-bearing row is the fourth" [V57]. That note's table has seven rows. The row it means is the ownership row, which is the second of the three above [inference]. Everything else follows from it. In a readiness model "the kernel holds no pointer into your address space between notifications, so a buffer may be allocated *just in time*... and one scratch buffer may be shared across ten thousand connections" [V75]. In a completion model "the buffer must exist *before* the data does, one per outstanding operation, and it belongs to the kernel until the completion packet arrives", and clients "agree not to disturb the buffers in any way until the network has consumed the buffer's contents" [V74].

What completion buys is a genuinely zero-copy path and many operations in flight on one descriptor. What it costs is one buffer per outstanding operation, held for the whole wait. This is why porting an event loop between the models is a rewrite rather than a rename: code ported mechanically "is not slow — it is wrong" [V76].

6.11 It is not a Linux/Windows split

I use the platforms as labels in the lecture, and the label is a teaching shorthand that needs correcting. My note states it directly: "the readiness/completion split is not a Linux/Windows split" [V59]. It is a split in the design space that each platform entered from a different side.

The control case is `kqueue`: "BSD and macOS classify with `epoll`: readiness notification, application does the I/O" [V81]. Readiness is the Unix family's answer rather than a Linux quirk [V59].

6.12 The trap this demonstration sets

The demonstration uses pipes. A regular file behaves differently, and the difference is silent. `epoll` "has nothing useful to say about a regular file, which is always "ready,"" [V73]. Ask whether you may read from a disk file and the answer is yes, immediately, every time. A loop built on that spins and looks like a bug in the program.

Linux "grew a completion interface because readiness could not express what its users needed" [V88], and the regular file is the case it could not express [V73].

The rule to write down: readiness works on things that can be not ready [inference]. Sockets, pipes and terminals. Not files.

6.13 Four Windows words

The Windows pair uses vocabulary the Linux pair did not need. A *named pipe* is "a named, one-way or duplex pipe for communication between the pipe server and one or more pipe clients" [S72] — a pipe with a name, so unrelated processes can open it. `PeekNamedPipe` inspects a pipe without reading from it: it copies data "without removing it from the pipe" and "always returns immediately" [S73], the Windows twin of a non-blocking read. An *overlapped* `ReadFile` returns before the data exists and the kernel fills the caller's buffer later [V74]. A *completion port* is a queue of finished operations; `GetQueuedCompletionStatus` dequeues one, and "If there is no completion packet queued, the function waits" [S74] — the one sleeping call.

6.14 Windows, completion, traced

What runs on Windows today is I/O completion ports, and my own note's instruction to itself is to "teach IOCP as current Windows practice, mention IoRing as the direction of travel, and do not present RIO as something students will meet" [V60]. The completion rung now has a demonstration rather than a paragraph.

`demo/waiting/` carries a Windows pair matching the Linux one, in C and in PyWin32. `wait_peek.c` creates a hundred named pipes with connected clients that never write and calls `PeekNamedPipe` on each, round after round — the polling twin. `wait_iocp.c` is the completion twin. It posts one overlapped `ReadFile` per pipe *before any data exists*. It binds all hundred to one completion port with `CreateIoCompletionPort`. It then makes a single `GetQueuedCompletionStatus` that sleeps to its timeout.

Captured 2026-09-20 with NtTrace64 on a Windows 11 VM (build 26200) [S53]:

	NATIVE CALLS, 2 S	SHAPE
<code>wait_peek.exe</code>	8,153	2,500 peeks, each <i>three</i> native calls: <code>NtCreateEvent</code> , <code>NtFsControlFile</code> , <code>NtClose</code>
<code>wait_iocp.exe</code>	956	100 <code>NtReadFile</code> up front, 100 binds, one <code>NtRemoveIoCompletion</code> returning <code>STATUS_TIMEOUT</code>

Two things are worth stating from that table. First, the Win32 call the program wrote is not the unit the kernel charges for: one `PeekNamedPipe` is three crossings. So 2,500 asks cost 7,500 crossings to learn nothing [S53]. That is Lecture 2's lesson arriving one layer up. Second, count the reads the completion program makes *while it waits*: zero. The reads were submitted before the wait, and the kernel owns a hundred of the program's buffers for the whole two seconds. That is the third tell from the ladder, made visible rather than described.

The hundred "errors" NtTrace reports on `NtFsControlFile` in both traces are `ConnectNamedPipe` returning `STATUS_PIPE_CONNECTED` — an informational status, not a failure [S53]. The PyWin32 pair shows the same shape, with the interpreter's own file reads mixed in. That is the reason to compare two runs rather than trust one [S53].

`WSAPoll` is deliberately absent from the folder: it is readiness-shaped and would demonstrate the wrong thing about the platform [S53].

6.15 What may not be done with these two numbers

28,017 on Linux and 330,100 on Windows sit in the same section and invite a division. They must not be divided. 28,017 is a **traced** count. 330,100 is the **untraced** Windows run, whose traced count is 2,500 [S53]. They come from different machines, and the loops differ. One is a non-blocking `read` on a Unix pipe. The other is `PeekNamedPipe` on a named pipe, which costs three native calls where the read costs one [S53].

What survives the refusal is the shape *within* each platform, measured the same way on both sides of it. On Linux, tens of thousands of asks against one sleeping call. On Windows, thousands of asks against one sleeping dequeue. Two kernels that disagree about nearly everything answer this question the same way, and neither number ranks the other [inference]. Comparing across two measurements is the most common way a correct pair of numbers becomes a wrong claim.

6.16 What a ring is

A ring is two queues in memory that the program and the kernel both see: "ring buffers which are shared between user space and kernel space" [S75]. "You place I/O requests you want to make on the SQ, while the kernel places the results of those operations on the CQ" [S75]. Every wait I counted in this part was a system call. With a ring, the program queues many requests and crosses the boundary once [S75]. With a kernel-side polling thread, the program does not cross the boundary at all [V11]. The ring is the transport. Readiness or completion is what travels on it.

6.17 Where both platforms ended up

Linux, the readiness-native platform, built a completion interface: "Linux shipped this as `io_uring` in kernel 5.1" [V79]. Windows, which "has had overlapped I/O and I/O completion ports since NT", built a ring instead — "Windows shipped it as `IoRing` in Windows 11 build 22000" [V80].

The tempting summary is convergence, and it is wrong. "Linux adopted completion semantics; Windows adopted the shared-memory transport" [V68]. The ring and the completion model are separable ideas that `io_uring` happened to ship together, and each platform took the half it lacked.

One honest limit closes the section. A ring moves the system-call boundary, and the tracer this course teaches sits on that boundary. "with `IORING_SETUP_SQPOLL` a kernel-side polling thread means steady-state I/O submission needs **no syscalls at all**" [V11]. The tracer sees the setup and then nothing.

7 8. Three destinations: sockets and pipes (announced tail)

7.1 First leg: a pipe

The room knows the file run of `iodemo` by heart by this point. It is a failed `openat`, a successful `openat` returning descriptor 3, one `write` of fourteen bytes, and a `close` [S37]. The prediction: which of those four calls changes when the target is a pipe, and does the byte count change?

The captured pipe run, `./iodemo /dev/stdout | cat`, in the same container [S54]:

```
openat(AT_FDCWD, "no-such-file.txt", O_RDONLY) = -1 ENOENT
openat(AT_FDCWD, "/dev/stdout", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
write(3, "one\ntwo\nthree\n", 14)      = 14
close(3)                                = 0
```

The same four calls, the same fourteen bytes in one `write`, the same descriptor number [S54]. The `openat` *argument* changed, because a different path was passed. The call sequence, the call count and the byte total did not.

The reason is a kernel structure rather than a convention: "On Linux, anything that takes a file descriptor takes a socket, because the kernel routes both through the same dispatch table" [V64], and sockets, pipes and terminals "are all file descriptors" [V29]. The descriptor is the abstraction. The file is one of the things a descriptor can name.

And the thesis appears again. `write(3, ...)` said *done* when the bytes were in the pipe. Whether `cat` had read any of them at that moment is my reading rather than the evidence, because only the writer was traced [inference].

7.2 What that run does not test

A file and a pipe are both block-buffered, and that is not an accident of the demonstration. It is the C library's own rule: "Attached to a terminal, `stdout` is line-buffered: every `\n` pushes the line out with a `write()` syscall. Attached to a pipe or a regular file, `stdout` is fully (block) buffered" [V71].

So the two runs sit on the same side of the rule and agree by construction [inference]. Run `./iodemo /dev/stdout` without the pipe, straight to a terminal, and the same binary emits three `write` calls instead of one.

The accurate statement is therefore narrower than "the destination does not matter". The destination does change the trace, and it changes it at the C library rather than at the kernel [inference]. That is Lecture 2's takeaway reached from the other direction.

7.3 Third destination: the terminal, captured

I tested that bound on 2026-09-21 rather than leave it as voluntary homework. I ran the same binary with a terminal attached (`podman run -t`). It made three `write` calls — `"one\n"`, `"two\n"`, `"three\n"` [S54]. The file run and the pipe run each made one [S54]. The program did not change. The C library did: "Attached to a terminal, `stdout` is line-buffered... Attached to a pipe or a regular file, `stdout` is fully (block) buffered" [V71]. The kernel's calls kept their names and their descriptor. Only the number of calls changed. The destination changes the trace at the C library, not at the kernel.

7.4 Second leg: a TCP socket

`iodemo` cannot open a socket by path, because "a socket cannot be reopened through `/dev/stdout`" [S54]. That limitation is itself a fact about the boundary [V65]. So the pipe carries the fourteen bytes to a client `nc`, a listening `nc` receives them, and both netcats are traced [S54]:

```

== client nc (writer)                                == listener nc (reader)
socket(AF_INET, SOCK_STREAM...)      = 3      socket(AF_INET, SOCK_STREAM...) = 3
connect(3, ...9000...) = -1 EINPROGRESS      bind(3, ...9000...)              = 0
read(0, "one\ntwo\nthree\n", 16384) = 14      listen(3, 1)                    = 0
write(3, "one\ntwo\nthree\n", 14)   = 14      accept4(3, ...)                 = 4
close(3)                            = 0      read(4, "one\ntwo\nthree\n", ...) = 14

```

The client puts the bytes on the wire with `write(3, ...)`, which is the same call `iodemo` used for a file and for a pipe. The listener takes them off with `read(4, ...)`, the call it would use for a file. What is new is only how the descriptor came to exist. One side calls `socket` and `connect`. The other calls `socket`, `bind`, `listen` and `accept4` [S54].

One detail connects the two tail sections. The client's `connect` returned `EINPROGRESS` because this netcat opens its socket non-blocking and waits for readiness before writing [S54]. Part 7's mechanism, visible in passing.

7.5 Then what are `send` and `recv` for?

`send(2)` settles it in its opening description: "The only difference between `send()` and `write()` is the presence of flags. With a zero flags argument, `send()` is equivalent to `write(2)`" [S50].

So `send` is for when a flag is wanted — `MSG_DONTWAIT`, `MSG_NOSIGNAL` and the rest have no `write()` equivalent [inference]. Otherwise the choice is style, and a legitimate one: the spelling tells a human reader that the descriptor is a socket [inference]. It is not evidence of a different mechanism.

7.6 The same interface is not the same contract

Three boundaries on this result, and each is a bug students ship.

- **One send is not one receive** [V70]. A kernel buffer sits between the two programs, so TCP is a byte stream: "one `send()` does not correspond to one `recv()`", and framing is the caller's job. This is, empirically, the single most common practical bug in student socket code" [V70]. The clean one-write-one-read result above is partly a size effect [inference].
- **The pipe result is a size effect too** [V67]. "`PIPE_BUF` is the atomicity threshold: "POSIX.1 says that writes of less than `PIPE_BUF` bytes must be atomic"" [V67], and that threshold is 4 KiB on Linux. Above it, "records from two writers can be shredded into each other" [V82].
- **The default is not total** [V65]. "Everything is a file" is a strong default, not a total function" [V65]. `lseek` on a socket is meaningless — "the kernel marks sockets `FMODE_STREAM` precisely to skip file-position locking they cannot use" [V87].

7.7 Windows: where the destination *does* change the trace

On Linux the destination changed nothing. Windows is the contrast, and it has a capture of its own. `dest.c` sends the same fourteen bytes three ways in one process. The three legs are `WriteFile` to an anonymous pipe, `send()` on a loopback TCP socket, and `WriteFile` on **that same socket** cast to a `HANDLE`. It was captured 2026-09-20 with NtTrace64, in C and in PyWin32, and both give the same three lines [S57]:

LEG	NATIVE CALL	EVIDENCE
1 · pipe	NtWriteFile , Length=0xe	the pipe handle, status [0/0xe]
2 · send()	NtDeviceIoControlFile , IoControlCode=0x1201f	the AFD send code. afd.sys is the driver behind Winsock
3 · WriteFile on the socket	NtWriteFile , same handle as leg 2	the file API reached the socket's file object directly

So on Windows the API chosen changes the native call, and the kernel object is a file object either way [S57]. Linux is the mirror image: one call, three destinations, nothing visible. NtTrace does not name the AFD control codes, because its configuration has no AFD table. The decoding above therefore rests on a cited source rather than on the tool [S57].

The first run of that program failed, and the failure is the lesson. A plain `WriteFile((HANDLE)sock, ..., NULL)` returned `ERROR_INVALID_PARAMETER (87)`, because Winsock creates sockets as **overlapped handles** by default. Both programs now supply an `OVERLAPPED` structure and wait [S57]. The file API on a Windows socket is part 7's completion model whether the program asked for it or not.

7.8 A SOCKET is not a HANDLE

On Linux a socket is a file descriptor and the file-descriptor contract is binding. On Windows it is an opaque `SOCKET` and "Windows makes the *Winsock* API the contract, even though the NT kernel underneath is also holding a file object" [V28].

The accurate version is narrower than "Windows does not treat sockets as files". The kernel does hold a file object [V28] — which is exactly what leg 3 of the capture above wrote to [S57]. Microsoft's two pages look contradictory until the layers are separated: "These are not contradictory once you see which layer each is describing — the *type* is not a C-runtime file descriptor, while the *handle* is a kernel file handle" [V66]. The capability exists. The contract does not promise it.

So the platforms agree on the mechanism and disagree on which layer they promise [inference]. That is the third appearance of one idea in this lecture. System calls in part 4, waiting in part 7, sockets here. It holds at three different layers, which is why I keep returning to it.

8.9. Scatter/gather and zero-copy: bytes you never touch (announced tail)

Two calls, two savings, and they are not the same saving. `writew` removes **crossings**. `sendfile` removes **copies**. Collapsing them is the mistake this part exists to prevent, so the prediction asks for two numbers rather than one.

8.1 First prediction, and the observation

A message is three buffers in the program: a header, a body and a trailer. It must land in the file as one message. How many system calls does that take, three, one or zero? And how many times does anything copy the three buffers to make it happen?

`gather.py` under `strace -e trace=write,writew` answers the first question in one line [S56]:

```
== three write() calls
write(3, "HEADER  ", 8)                                = 8
write(3, "body body body body body body bo"... , 40) = 40
write(3, " TRAILER\n", 9)                               = 9

== one writew() call (wrapped to fit)
writew(3, [{iov_base="HEADER  ", iov_len=8},
           {iov_base="body body body body body body bo"... , iov_len=40},
           {iov_base=" TRAILER\n", iov_len=9}], 3) = 57
```

`writew()` "writes *iovcnt* buffers of data described by *iov*" [S58]. The vector entry is a `struct iovec` of a base address and a length, both visible in the trace. Three crossings became one, and the return value is the whole message: 57 bytes, not 8.

8.2 `writew` buys crossings, not copies

The second answer is the one students get marked wrong for getting right. The program copied nothing to make the `writew` happen — but it copied nothing in the three-`write` version either. What `writew` spares is the *other* design. That design concatenates header, body and trailer into a fresh staging buffer, so that a single `write` has something contiguous to point at. That buffer exists only to satisfy the shape of `write`, and removing it is the honest statement of the win [inference].

Below the crossing nothing changes. The four levels of Lecture 2's stack sit under a `writew` exactly as they sit under a `write`. `readv` is the mirror image on the read side: one crossing scattering into several buffers [S58].

The representative production use is unglamorous rather than headline, and it is in Valkey's own source. `writewToClient()` builds one `iovec` array per reply and submits it with `connWritew()` [S65]. The reason is in the comment at the call site: "use writew to save system calls and TCP packets" [S65]. One reply element — a *bulk string*, in the protocol's own term — costs exactly three entries, as `#define NUM_OF_IOV_PER_BULK_STR 3` says [S65]. The three are the length prefix of Valkey's RESP wire protocol, then `str_ref->str`, then the two-byte terminator. `str_ref->str` is the stored string's own buffer, reached by reference rather than copied [S65]. Assembling those three pieces by hand would need a staging copy. The call removes it.

8.3 Two atomicity promises, and only one is unconditional

It is tempting to write "`writew` is atomic" and stop. The guarantee is conditional on the destination.

On a **regular file** it is unconditional and it is the one that matters for a log: the data "is written as a single block that is not intermingled with output from writes in other processes; analogously, `readv()` is guaranteed to read a contiguous block of data from the file" [S58]. A log line written with one `writew` stays whole.

On a **pipe or socket** the rule is `PIPE_BUF`'s. POSIX "says that writes of less than `PIPE_BUF` bytes must be atomic", while "writes of more than `PIPE_BUF` bytes may be nonatomic: the kernel may interleave the data with data written by other processes" [S59]. On Linux `PIPE_BUF` is 4096 bytes [S59].

The trap is that the threshold applies to the **sum** of the `iov_len` values rather than to the largest entry [inference]. Eight buffers of 1 KiB total 8 KiB and may interleave, though no single buffer came near 4096. Splitting a record across `iovec` entries makes it exactly as atomic as concatenating it would have been.

One further limit, from the same page: "it is not an error for a successful call to transfer fewer bytes than requested" [S58], and the return value is a single total across the whole array. Recovering from a short `writew` means walking the entries to find where it stopped. On a regular file that almost never happens, so a naive implementation passes every local test and fails against a socket under load.

8.4 Second prediction, and the observation

4 MiB moved from a file into a pipe, two ways. Program A is the copy loop everyone has written: `read` 64 KiB into a buffer, `write` it out, repeat. Program B calls `sendfile`, which has no buffer in the program at all. How many data system calls does each make, and how many times does each byte cross into the process's address space and back?

`zerocopy.py` under `strace -c -e trace=read,write,sendfile` [S56]:

4 MIB INTO A PIPE	DATA SYSCALLS	COPIES THROUGH USER SPACE	WALL
copy loop, 64 KiB chunks	64 <code>read</code> + 64 <code>write</code> = 128	two per chunk	2.6 ms
<code>sendfile</code>	64 <code>sendfile</code>	none	2.0 ms

`sendfile()` "copies data between one file descriptor and another. Because this copying is done within the kernel, `sendfile()` is more efficient than the combination of `read(2)` and `write(2)`" [S60]. The tell in the trace is an absence: there is no `read` line at all. That is the same instruction as part 7's completion tell — a mechanism that removes work is observed by what is missing.

Why 64 calls, when the prediction offered three, one or zero? Because each call moved 64 KiB, which is what the pipe would accept at once: a pipe's capacity is 16 pages, 65,536 bytes [S59]. The chunk size is the pipe's, not `sendfile`'s [S56].

The wall times are deliberately not the claim. 2.6 ms against 2.0 ms on one machine once would not survive a second run's noise, and the README says so [S56]. The claim is the copies avoided and the calls halved, both of which are counted.

8.5 What "zero-copy" does not cover

The name counts the copies it removes at one boundary. That boundary is into and out of the calling process's address space. It claims nothing about the others.

- **The two DMA transfers remain** [S60]. The disk still moves bytes into memory and the device still takes them out. What went is the CPU carrying them through the process in between.
- **It requires the page cache rather than bypassing it** [inference]. `sendfile` works by handing the destination references to pages the kernel already has, so a file with nothing cached has nothing to reference. This is why the two best-known file-I/O speed-ups cancel: nginx's `directio` directive "automatically disables the use of `sendfile` for a given request" [S61].

- **It buys no durability** [inference]. The same four levels sit below. A `sendfile` that returned is a handoff like every other *done* in this lecture [S67].
- **It cannot help a program that looks at the bytes** [S60]. Compression, encryption and templating all require the plaintext in the process, and then there is nothing left to remove.

8.6 Windows, and a fallback that says nothing

For part 9 the Windows half is documentation rather than measurement. That is a real drop in evidence after three parts that had captures.

For sockets the honest counterpart of `writetv` is `WSASend`. It takes "a pointer to an array of `WSABUF` structures" [S69]. Each structure holds "a pointer to a buffer and the length, in bytes, of the buffer" [S69]. The page says the call "allows multiple send buffers to be specified making it applicable to the scatter/gather type of I/O" [S69]. The same page states today's thesis in Microsoft's words: "The successful completion of a `WSASend` does not indicate that the data was successfully delivered" [S69]. For files there is `WriteFileGather`, and it is not the same tool under another name: "Each buffer must be at least the size of a system memory page and must be aligned on a system memory page size boundary" [S62], on a handle opened with `FILE_FLAG_OVERLAPPED` and `FILE_FLAG_NO_BUFFERING`. It is a database primitive, not a general one. `TransmitFile` is the zero-copy call, sending a file over a connected socket [S63].

The part of this the room is most likely to trip over is in Python rather than in Win32. `socket.sendfile()` uses `os.sendfile()` where it exists, and where it does not — Windows — " `send()` will be used instead as a less efficient fallback" [S64]. No error and no warning. A student who measures `sock.sendfile()` on Windows and concludes that zero-copy does nothing has measured the fallback [inference]. Check which call was actually made before believing the number.

9 10. Homework, and the next question

9.1 Assignment 1 is not set today

Lecture 1 announced that Assignment 1 would be set on 22 September. That has moved. The submission infrastructure is not ready, so the assignments move back. There are no new dates yet. No date appears on any slide until one is known.

What it will measure has not changed: buffered against unbuffered, and sequential against random access, over a 1 MiB file. The room predicts the ratio, measures it, and explains the difference in operating-system crossings. That is Predict/Observe/Explain by construction [S14], which is what the room has been rehearsing for two weeks.

9.2 Voluntary homework

Everything demonstrated today ships with this deck, in a container, with a README recording what the lecturer measured. `demo/durability/` has the writeback watch and the four-mode counts. `demo/waiting/` has the two wait programs. `demo/iodemo-c/` has the file, pipe and socket runs.

Three rules, carried from Lecture 2:

1. **Predict, then look** [S51]. Write the number down first, because watching alone teaches no more than skipping the demonstration [S51].
2. **Report the version and the buffer size beside every number** [S7]. Python's default binary buffer moved from 8 KiB to 128 KiB, so a student laptop and the lecture machine can legitimately disagree [S7] [S6].
3. **Count traced, time untraced** [S1]. Two runs, because a traced process runs more slowly than an untraced one [S1].

A number that contradicts the README is a good week's work, not a mistake.

9.3 What Lecture 4 answers

Today the bytes sat in the page cache for five seconds [S52], in memory owned by the kernel rather than by any process [V78]. So what is that memory, and whose is it? If the machine is asked how much memory is free, does the page cache count as free or as used?

The question is harder than it sounds, and the reason is one sentence: "**Physical memory is a cache, not a container.**" [V72] So free RAM is mostly a bookkeeping category rather than a resource going to waste [inference].

Lecture 4 is that, and memory mapping with it. And if the clock beat this lecture, Lecture 4 opens with parts 7, 8 and 9, unchanged.

10 What this lecture does not establish

- **The device level is not observable from software** [V5]. No trace in this course can check whether a drive honoured a flush [V5]. That is where data is actually lost [V5].
- **Part 9 is the one part whose Windows half is documentation, not measurement** [inference]. Parts 4, 6, 7 and 8 each carry a traced Windows capture taken on the same day as their Linux twin [S37] [S52] [V5] [S57]. Part 9 does not, and the slides say so. The Windows figures that do exist come from one x64 binary under emulation on one ARM64 virtual machine [S52]. Their shape is the claim. The milliseconds are not.
- **Which writeback rule fired is inferred from timing** rather than observed [inference]. A kernel-side tracer would settle it [inference].
- **The C10K evidence is about servers, not about the hundred-pipe demo** [S49]. The demonstration shows the shape of the cost at demo scale [S53]. The 79% figure is somebody else's controlled experiment at a different scale [S49].
- **Asynchronous I/O breaks the observation method** [V11]. A ring with a kernel-side polling thread makes no system calls in steady state, so the tracer sees nothing [V11].

11 References

Every source behind every claim above, in registry order. [S<n>] entries are external and carry a URL. [V<n>] entries are notes from the author's research vault, each summarised in the appendix below together with the external sources it draws on.

- [S1] strace(1) — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html>
- [S2] write(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html>
- [S3] fflush(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>
- [S4] close(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/close.2.html>
- [S5] C99 draft, §7.19.3. <https://busybox.net/~landley/c99-draft.html>
- [S6] Built-in Functions — open(), Python documentation. <https://docs.python.org/3/library/functions.html#open>

- **[S7]** Increase `io.DEFAULT_BUFFER_SIZE` — CPython issue 117151. <https://github.com/python/cpython/issues/117151>
- **[S9]** `winternl.h` reference — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>
- **[S10]** NtTrace — Native API tracing for Windows. <https://rogorrr.github.io/NtTrace/>
- **[S11]** End-to-End Arguments in System Design. <https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>
- **[S13]** `fsync` — The Open Group Base Specifications. <https://pubs.opengroup.org/onlinepubs/9699919799/functions/fsync.html>
- **[S14]** Predict-Observe-Explain. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>
- **[S15]** The sad history of the Microsoft POSIX subsystem. <https://brianreiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/>
- **[S17]** `io.IOBase.flush()` — Python documentation. <https://docs.python.org/3/library/io.html#io.IOBase.flush>
- **[S18]** Process Monitor — Sysinternals. <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>
- **[S19]** Inside Native Applications. <https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>
- **[S20]** `os` — Miscellaneous operating system interfaces, `os.write()`, Python documentation. <https://docs.python.org/3/library/os.html#os.write>
- **[S21]** `fread(3)`, the page that documents `fwrite(3)` — Linux man-pages. <https://man7.org/linux/man-pages/man3/fwrite.3.html>
- **[S22]** Windows API index — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>
- **[S31]** `ltrace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/ltrace.1.html>
- **[S32]** `ltrace` — project page. <https://ltrace.org/>
- **[S33]** NtTrace — source repository. <https://github.com/rogorrr/NtTrace>
- **[S34]** NtTrace64 capture of `cmd.exe /c echo Hello World!`. <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/NtTrace64-CMD-ECHO-Output.txt>
- **[S36]** NtTrace64 capture of `iodemo.exe`. <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/iodemo-c/NtTrace64-IODEMO-Output.txt>

- **[S37]** iodemo on Linux and on Windows, the recorded runs (demo/iodemo-c/README.md). <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/iodemo-c/README.md>
- **[S38]** CloseHandle function (handleapi.h) — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/handleapi/nf-handleapi-close-handle>
- **[S39]** demo/close/README.md — the double-close capture on both platforms, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/close/README.md>
- **[S40]** FlushFileBuffers function (fileapi.h) — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-flushfilebuffers>
- **[S41]** fsyncgate — the pgsql-hackers thread, March 2018. <https://danluu.com/fsync-gate/>
- **[S42]** Can Applications Recover from fsync Failures? — USENIX ATC 2020. <https://www.usenix.org/system/files/atc20-rebello.pdf>
- **[S43]** PRAGMA synchronous — SQLite documentation. https://www.sqlite.org/pragma.html#pragma_synchronous
- **[S44]** Delayed allocation and the zero-length file problem — Theodore Ts'o (2009). <https://thunk.org/tytso/blog/2009/03/12/delayed-allocation-and-the-zero-length-file-problem/>
- **[S45]** Delayed allocation and the zero-length file problem — Valerie Aurora, LWN (2009). <https://lwn.net/Articles/351422/>
- **[S46]** epoll(7) — Linux man-pages. <https://man7.org/linux/man-pages/man7/epoll.7.html>
- **[S47]** select(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/select.2.html>
- **[S48]** The C10K problem — Dan Kegel (current revision). <http://www.kegel.com/c10k.html>
- **[S49]** Comparing and Evaluating epoll, select, and poll Event Mechanisms — Gammo et al. (2004). <https://www.kernel.org/doc/ols/2004/ols2004v1-pages-215-226.pdf>
- **[S50]** send(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/send.2.html>
- **[S51]** Classroom demonstrations: Learning tools or entertainment? — Crouch, Fagen, Callan and Mazur (2004). <https://works.swarthmore.edu/fac-physics/203/>

- **[S52]** demo/durability/README.md — the writeback timeline and the four-mode counts, captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>
- **[S53]** demo/waiting/README.md — the polling and epoll system-call counts, captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/waiting/README.md>
- **[S54]** demo/iodemo-c/README.md — the pipe run and the traced netcat pair, captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/iodemo-c/README.md>
- **[S55]** The C10K problem — Dan Kegel, Internet Archive snapshot of 8 May 1999. <https://web.archive.org/web/19990508164301/http://www.kegel.com/c10k.html>
- **[S56]** demo/scatter/README.md — the `writew` trace and the `sendfile` counts, captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md>
- **[S57]** demo/destinations-win32/README.md — the three-destination NtTrace64 capture, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/destinations-win32/README.md>
- **[S58]** `writew(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/writew.2.html>
- **[S59]** `pipe(7)` — Linux man-pages. <https://man7.org/linux/man-pages/man7/pipe.7.html>
- **[S60]** `sendfile(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/sendfile.2.html>
- **[S61]** Module `ngx_http_core_module` — nginx documentation. https://nginx.org/en/docs/http/ngx_http_core_module.html#sendfile
- **[S62]** `WriteFileGather` function (`fileapi.h`) — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-writefilegather>
- **[S63]** `TransmitFile` function (`mswsock.h`) — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/mswsock/nf-mswsock-transmitfile>
- **[S64]** `socket` — Low-level networking interface, Python documentation. <https://docs.python.org/3/library/socket.html#socket.socket.sendfile>
- **[S65]** `valkey/src/networking.c` — Valkey, pinned at commit 00a8a19 (read 2026-09-20). <https://github.com/valkey-io/valkey/blob/00a8a19554a39-facb591837ea41ba4350260919d/src/networking.c>
- **[S66]** `strace Wow Much Syscall` — Brendan Gregg (11 May 2014). <https://www.brendangregg.com/blog/2014-05-11/strace-wow-much-syscall.html>

- **[S67]** fsync(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/fsync.2.html>
- **[S68]** Linux kernel documentation — admin-guide/sysctl/vm.rst. <https://www.kernel.org/doc/html/latest/admin-guide/sysctl/vm.html>
- **[S69]** WSASend function (winsock2.h) — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winsock2/nf-winsock2-wsase>
- **[S70]** rename(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/rename.2.html>
- **[S71]** PostgreSQL commit 9ccdd7f — "PANIC on fsync() failure" (2018). <https://git.postgresql.org/gitweb/?p=postgresql.git;a=commit;h=9ccdd7f66e3324d2b6d3dec282cfa9ff084083f1>
- **[S72]** Named Pipes — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/ipc/named-pipes>
- **[S73]** PeekNamedPipe function (namedpipeapi.h) — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/namedpipeapi/nf-namedpipeapi-peeknamedpipe>
- **[S74]** GetQueuedCompletionStatus function (ioapiset.h) — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/ioapiset/nf-ioapiset-getqueuedcompletionstatus>
- **[S75]** io_uring(7) — Linux man-pages. https://man7.org/linux/man-pages/man7/io_uring.7.html
- **[V4]** A Successful write() Is a Handoff, Not a Save — a note from the author's research vault. The idiom the room already writes contains no durability step.
- **[V5]** A Successful write() Is a Handoff, Not a Save — a note from the author's research vault. The limit on the thesis. The bottom of the stack is not observable from software, and the deck says so.
- **[V6]** Standard output is buffered differently depending on where it points — a note from the author's research vault. Removes the misconception that a Python write passes through the C library buffer.
- **[V8]** Windows and Linux place the stable syscall boundary at different layers — a note from the author's research vault. Takeaway 4, stated as a placement question rather than a quality judgement.
- **[V9]** Windows and Linux place the stable syscall boundary at different layers — a note from the author's research vault. Stops section 5 reading as a complaint about Windows.
- **[V10]** The System Call Boundary — a note from the author's research vault. Makes "a library call is not a system call" falsifiable with two tools instead of asserting it.

- **[V11]** The System Call Boundary — a note from the author's research vault. The closing honest-limits slide.
- **[V14]** Where the Abstraction Leaks — a note from the author's research vault. Bounds section 3's vanishing-output demonstration so it does not teach the misconception it exists to remove.
- **[V17]** Observing a Running System — a note from the author's research vault. Why sections 1 and 2 need two runs.
- **[V20]** File IO and Durability — a note from the author's research vault. The device reports success before the bytes reach the medium, which is the sentence Lecture 2 stopped on.
- **[V21]** NtTrace — Native API tracing for Windows (Roger Orr) — a note from the author's research vault. Why this tool is usable in a teaching lab, and what its design bought.
- **[V22]** NtTrace — Native API tracing for Windows (Roger Orr) — a note from the author's research vault. The single string in the Windows trace that shows the layer difference.
- **[V23]** NtTrace — Native API tracing for Windows (Roger Orr) — a note from the author's research vault. Comparison slide names the defect it has just cleared.
- **[V24]** PostgreSQL's fsync() surprise — a note from the author's research vault. Shipped storage engines answer the deferred-error problem by refusing to retry.
- **[V25]** Every Tracing Mechanism Trades Observation Cost for a Blind Spot — a note from the author's research vault. The honest-limits slide. The tool taught here is a teaching and debugging instrument, not an operational one.
- **[V28]** Sockets and Network I-O — a note from the author's research vault. One-slide bridge to networking.
- **[V29]** Sockets and Network I-O — a note from the author's research vault. The pipe row on the same slide.
- **[V30]** File IO and Durability — a note from the author's research vault. The row of the note's per-layer instrumentation table covering the kernel-to-device boundary.
- **[V33]** File IO and Durability — a note from the author's research vault. The bottom arrow of the stack diagram, and the "emptied by" cell of level 4.
- **[V36]** Windows and Linux place the stable syscall boundary at different layers — a note from the author's research vault. The caveat on the outlier list.
- **[V37]** File IO and Durability — a note from the author's research vault. The Windows half of level 3's "emptied by" cell, and of the third arrow in the stack diagram.

- **[V38]** Windows and Linux place the stable syscall boundary at different layers — a note from the author's research vault. Why Linux nominates the layer it does, and why the other platforms need not.
- **[V39]** Every Tracing Mechanism Trades Observation Cost for a Blind Spot — a note from the author's research vault. What the lower-overhead mode named in [V25] actually buys, on a named workload.
- **[V40]** PostgreSQL's fsync() surprise — a note from the author's research vault. How many storage engines abandoned the retry, and when.
- **[V41]** PostgreSQL's fsync() surprise — a note from the author's research vault. The independence of the three engines, which is what makes the never-retry slide an argument rather than an anecdote.
- **[V43]** Windows and Linux place the stable syscall boundary at different layers — a note from the author's research vault. The named list behind the outlier correction.
- **[V44]** File IO and Durability — a note from the author's research vault. Core claim, stated as a conjunction.
- **[V45]** File IO and Durability — a note from the author's research vault. The objection the room raises during section 3 — *but I can read the file back*.
- **[V46]** File IO and Durability — a note from the author's research vault. Windows half at equal depth.
- **[V47]** File IO and Durability — a note from the author's research vault. The honest Windows answer is *undocumented*, which is a stronger and truer statement than *the same* or *safe*.
- **[V48]** File IO and Durability — a note from the author's research vault. Honesty slide. Why the 40 MiB file was written back sooner than the 4 MiB one, which is the first question the observation provokes.
- **[V49]** File IO and Durability — a note from the author's research vault. Close. A shipped database declines to trust the last software handoff and defends itself another way.
- **[V50]** Where the Abstraction Leaks — a note from the author's research vault. The thesis in one sentence, from the vault's own synthesis note.
- **[V51]** Where the Abstraction Leaks — a note from the author's research vault. Fixable half. The room needs the part software *can* get right, or the durability section reads as fatalism.
- **[V52]** Where the Abstraction Leaks — a note from the author's research vault. Counter-position. Trusting the abstraction is respectable, and the note says exactly where it stops being respectable.

- **[V53]** A failed `fsync()` is not retryable — a note from the author's research vault. The kernel's *done* observed lying, in a trace of exactly the kind this course teaches the room to read.
- **[V54]** A failed `fsync()` is not retryable — a note from the author's research vault. Why the obvious repair is not a repair. It has the same shape as Lecture 2's slide "Never retry a failed close", one level down.
- **[V55]** A failed `fsync()` is not retryable — a note from the author's research vault. Windows slide. The asymmetry is about documentation, not about which platform is safer.
- **[V56]** PostgreSQL's `fsync()` surprise — a note from the author's research vault. The industry answer to an untrustworthy *done* is crash and replay, and it was never meant to last this long.
- **[V57]** Readiness and Completion Models — a note from the author's research vault. Buffer ownership, not API taste, is what separates the two models, and it is why porting between them is a rewrite.
- **[V58]** Readiness and Completion Models — a note from the author's research vault. The sentence that stops the room reading *ready* as a guarantee, which is the misconception this model produces.
- **[V59]** Readiness and Completion Models — a note from the author's research vault. Correction to its own framing.
- **[V60]** Readiness and Completion Models — a note from the author's research vault. Windows slide. What the room should carry away as present tense, and what is only direction of travel.
- **[V61]** The C10K Problem Was About the Cost of Waiting, Not I-O — a note from the author's research vault. Predict slide. The controlled experiment behind the question, stated as what was held fixed and what was varied.
- **[V62]** The C10K Problem Was About the Cost of Waiting, Not I-O — a note from the author's research vault. Explanation. `epoll` did not make a read faster, and the room needs that said before it draws the wrong conclusion from the counts.
- **[V63]** The C10K Problem Was About the Cost of Waiting, Not I-O — a note from the author's research vault. The page most often cited for this argument did not make it in 1999, so the deck cites the revision and the measurement instead.
- **[V64]** Sockets and Network I-O — a note from the author's research vault. Mechanism. Why `read` and `write` work on a socket, stated as a kernel structure rather than as a convention.
- **[V65]** Sockets and Network I-O — a note from the author's research vault. Boundary. The same interface does not mean the same contract, and the note names where the default stops.

- **[V66]** Sockets and Network I-O — a note from the author's research vault. Windows slide. The platforms agree on the mechanism and disagree on which layer they promise, which is the accurate statement.
- **[V67]** Interprocess Communication — a note from the author's research vault. Boundary. The clean one-write-one-read result in the capture holds because fourteen bytes is small, and the room should know the threshold.
- **[V68]** io_uring and Windows IoRing — Completion on Both Platforms — a note from the author's research vault. Close. Where both platforms ended up, stated precisely enough not to read as "they converged".
- **[V69]** Blocking Is a Scheduler Decision, Not a Property of the I-O — a note from the author's research vault. The one question that orders polling, readiness and completion into a ladder rather than a feature list.
- **[V70]** Network Buffering and Delivery Semantics — a note from the author's research vault. Boundary. The capture shows one write and one read only because the payload is fourteen bytes, and the slide must say so.
- **[V71]** Standard output is buffered differently depending on where it points — a note from the author's research vault. Honest bound on its own demonstration.
- **[V72]** Virtual Memory and the Page Cache — a note from the author's research vault. Handover. The page cache held the bytes for five seconds, and what that memory actually is belongs to the next lecture.
- **[V73]** io_uring and Windows IoRing — Completion on Both Platforms — a note from the author's research vault. Boundary. The demonstration uses pipes, and a student who tries the same loop on a file gets a mechanism that always answers yes.
- **[V74]** Readiness and Completion Models — a note from the author's research vault. What a completion model demands of the program, in Microsoft's own words, and the memory bill that follows from it.
- **[V75]** Readiness and Completion Models — a note from the author's research vault. The other half of the ownership comparison.
- **[V76]** Readiness and Completion Models — a note from the author's research vault. Why the choice of model is structural rather than a matter of API taste.
- **[V77]** Readiness and Completion Models — a note from the author's research vault. Takeaway 2 and section 4's definition slide.
- **[V78]** File IO and Durability — a note from the author's research vault. What a dirty page is, what writeback is, and that the kernel rather than the program schedules it.
- **[V79]** io_uring and Windows IoRing — Completion on Both Platforms — a note from the author's research vault. Close. The version in which Linux acquired a completion interface.

- **[V80]** `io_uring` and Windows IoRing — Completion on Both Platforms — a note from the author's research vault. Close. The build in which Windows acquired a ring, paired with [V79].
- **[V81]** Readiness and Completion Models — a note from the author's research vault. Correction. The control case that shows readiness is the Unix family's answer rather than a Linux quirk.
- **[V82]** Interprocess Communication — a note from the author's research vault. Boundary. The threshold in bytes, and what happens above it, which the atomicity span [V67] does not state.
- **[V83]** A failed `fsync()` is not retryable — a note from the author's research vault. Windows slide. The mechanism behind the undocumented contract, which is why no single answer exists.
- **[V84]** PostgreSQL's `fsync()` surprise — a note from the author's research vault. What InnoDB actually changed, which the count-and-year span [V40] does not carry.
- **[V85]** A failed `fsync()` is not retryable — a note from the author's research vault. Close. The log-and-replay answer, in PostgreSQL's own shipped documentation rather than as the deck's summary of it.
- **[V86]** File IO and Durability — a note from the author's research vault. The device lying is a reported practice rather than a theoretical possibility, and a shipped database says so.
- **[V87]** Sockets and Network I-O — a note from the author's research vault. Boundary. A concrete operation that does not transfer, which the strong-default span [V65] asserts without instanting.
- **[V88]** `io_uring` and Windows IoRing — Completion on Both Platforms — a note from the author's research vault. Why the regular-file gap mattered enough to build a second model, which the `epoll`-and-regular-files span [V73] does not say.
- **[V89]** A failed `fsync()` is not retryable — a note from the author's research vault. The mechanism behind the failed retry, which [V54]'s one-sentence span states without explaining.
- **[V90]** A failed `fsync()` is not retryable — a note from the author's research vault. How long the misreading stood before anyone noticed, which is what makes it a contract failure rather than a careless bug.
- **[V91]** File IO and Durability — a note from the author's research vault. Windows slide. The judgement itself, from the note that makes it, rather than asserted by the deck on the strength of a missing wiki row.
- **[V92]** Readiness and Completion Models — a note from the author's research vault. The `select(2)` BUGS caveat applies to the interface the demonstration actually used.

- **[V93]** PostgreSQL's `fsync()` surprise — a note from the author's research vault. What PostgreSQL actually changed, and that the change reached every release rather than only the current one.
- **[V94]** The C10K Problem Was About the Cost of Waiting, Not I-O — a note from the author's research vault. Kegel's own arithmetic for why a thread per connection ran out of virtual memory.
- **[V95]** The C10K Problem Was About the Cost of Waiting, Not I-O — a note from the author's research vault. Banga, Mogul and Druschel on the scheduler half of the same bill, stated in 1999.
- **[V96]** The C10K Problem Was About the Cost of Waiting, Not I-O — a note from the author's research vault. What unifies the thread rung with the polling rung, marked in the note as the note's own reading.

12 Appendix — sources from the author's working notes

The talk cites material from a private research vault. Each entry below summarises one such note and lists every external source it draws on.

12.1 V4 — The Gap Between a Successful `write()` and Data Reaching Disk

This note argues that a successful `write()` on Linux or `WriteFile()` on Windows tells you only that the kernel accepted your bytes, not that any storage device holds them. It grounds this in each platform's own primary documentation: the Linux `write(2)` man page states outright that a successful return "does not make any guarantee that data has been committed to disk," and Microsoft's Win32 reference describes `WriteFile` as writing into an internal buffer that the operating system flushes on its own schedule, via a cache-manager "lazy writer" that runs once a second. The note applies this directly to the quoted claim: the durability idiom used in almost every mainstream language — open a file, write, let a `with` block, `using` block, or destructor close it — contains no durability step anywhere. It supports this by walking each boundary the idiom crosses and showing none of them reaches the device: Linux's `close(2)` man page explicitly denies that closing implies a disk write, `fflush()` only moves bytes from a language's user-space buffer into the kernel, and Python's, Java's, and .NET's stream-close contracts document resource release, not persistence. It reinforces the claim with kernel writeback defaults that can leave dirty pages unflushed for up to thirty seconds, and with PostgreSQL's 2018 admission that it had wrongly assumed a successful `fsync()` proved data reached storage — a mistaken model held for two decades and fixed by crashing the database on `fsync` failure rather than trusting a retry.

The note is explicit that this is a claim about guarantees, not about what usually happens: on a healthy machine the default writeback window is short enough that data typically survives, and it concedes that a programmer's years of not losing a file is accurate testimony rather than evidence against the claim. It also declines to present `fsync()` as a clean fix — POSIX explicitly permits an implementation that does nothing, and kernel-version-dependent error reporting can make a failed `fsync()` unsafe to retry. It names several configurations where the claim does not hold: explicit synchronous-write flags on either platform, a battery- or capacitor-backed device cache, deliberately disabled write barriers on protected hardware, and database engines such as PostgreSQL that have already built their own sync protocol on top of this exact gap. It leaves several questions open rather than resolved: no current measurement of the actual, as opposed to configured, loss window was found; whether Windows exposes a dirty-page counter comparable to Linux's is unconfirmed; whether a cited 2020 study of five database systems still holds against their current versions is unverified; what share of shipping consumer drives implement power-loss protection is unknown; and whether the note's power-cut classroom experiment reproduces reliably across hypervisors has not been validated.

Referenced sources

- [write\(2\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project HIGH
- [fsync\(2\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project HIGH
- [close\(2\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project HIGH
- [fsync — The Open Group Base Specifications Issue 7](#) — The Open Group / IEEE HIGH
- [ext4 General Information — The Linux Kernel documentation](#) — Linux kernel documentation project HIGH
- [File Caching — Win32 apps](#) — Microsoft Learn HIGH
- [FlushFileBuffers function \(fileapi.h\) — Win32 apps](#) — Microsoft Learn HIGH
- [fflush\(3\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project HIGH
- [CreateFileW function \(fileapi.h\) — Win32 apps](#) — Microsoft Learn HIGH
- [Querying for the Write Cache Property — Windows drivers](#) — Microsoft Learn HIGH
- [DTrace on Windows — Windows drivers](#) — Microsoft Learn HIGH
- [microsoft/DTrace-on-Windows](#) — Microsoft HIGH
- [_commit — C runtime library reference](#) — Microsoft Learn HIGH
- [os — Miscellaneous operating system interfaces \(Python 3 documentation\)](#) — Python Software Foundation HIGH

- [io — Core tools for working with streams \(Python 3 documentation\)](#) — Python Software Foundation **HIGH**
- [FileOutputStream \(Java SE 25 & JDK 25\)](#) — Oracle **HIGH**
- [FileStream.Flush Method](#) — .NET API reference — Microsoft Learn **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC (2020) **HIGH**
- [PANIC on fsync\(\) failure — PostgreSQL commit 6534d54](#) — Thomas Munro / PostgreSQL Global Development Group **HIGH**
- [PostgreSQL's fsync\(\) surprise](#) — Jonathan Corbet / LWN.net **MEDIUM**
- [Page Cache Writeback](#) — Linux Kernel Internals — kernel-internals.org **MEDIUM**
- [Fsync Errors](#) — PostgreSQL wiki — PostgreSQL Global Development Group **HIGH**
- [Linux Page Cache Basics](#) — Thomas-Krenn.AG wiki **MEDIUM**
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — winntfs.com **MEDIUM**
- [PostgreSQL Documentation: 28.1. Reliability](#) — PostgreSQL Global Development Group **HIGH**
- [Disk write cache buffer and separate power supply](#) — Super User (Stack Exchange) **LOW**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (archive of the pgsq-l hackers thread, 2018) **MEDIUM**

12.2 V5 — The Gap Between a Successful write() and Data on Disk

A successful `write()` on Linux or `WriteFile()` on Windows means only that the kernel accepted the bytes into its own buffers, not that any storage device holds them — a claim both platforms state in their own primary documentation: Linux's `write(2)` man page disclaims any guarantee that data reached disk, and Microsoft describes `WriteFile` as writing to an internal buffer that the cache manager's lazy writer flushes later, on its own schedule rather than the caller's. The usual escape hatch is `fsync()` or `FlushFileBuffers()`, but the note pushes past that to a harder limit on what any escape hatch can prove: once a flush call is issued, whether the drive actually honored the cache-flush command before acknowledging it is firmware behavior, entirely outside what the operating system — or any syscall trace — can observe. That is the quoted claim: "No syscall trace can distinguish an honest flush from a fast lie." The note grounds this in PostgreSQL's own reliability documentation, which warns that many

consumer drives and SSDs do not honor cache-flush commands by default and names independent hardware testing, not software tracing, as the only way to know for certain.

The note treats its central claim as a default rather than a universal law, and names the conditions under which it does not hold — synchronous I/O flags, a battery- or capacitor-backed device cache, or an application that has already delegated durability to an engine like PostgreSQL. On the specific question the quoted span raises, it does not go further than the claim itself: it argues the device layer is unverifiable from software, but does not offer a measured rate of drives lying about flushes, and it flags two related claims elsewhere as resting on weaker evidence — the Windows flush-to-SCSI-command mapping comes from a practitioner source rather than a Microsoft statement, and an SSD power-loss-protection comparison is explicitly labeled an inference rather than an observation. It also leaves one dispute between primary sources unresolved: POSIX explicitly permits a null `fsync()` implementation, while the Linux man page asserts stronger crash- and reboot-survivability, and the note does not adjudicate between the two.

Referenced sources

- [write\(2\) — Linux manual page](#), Michael Kerrisk / Linux man-pages project (2025) HIGH
- [fsync\(2\) — Linux manual page](#), Michael Kerrisk / Linux man-pages project (2025) HIGH
- [close\(2\) — Linux manual page](#), Michael Kerrisk / Linux man-pages project (2025) HIGH
- [fsync — The Open Group Base Specifications Issue 7](#), The Open Group / IEEE (2018) HIGH
- [ext4 General Information — The Linux Kernel documentation](#), Linux kernel documentation project HIGH
- [File Caching — Win32 apps](#), Microsoft Learn HIGH
- [FlushFileBuffers function \(fileapi.h\) — Win32 apps](#), Microsoft Learn HIGH
- [fflush\(3\) — Linux manual page](#), Michael Kerrisk / Linux man-pages project (2025) HIGH
- [CreateFileW function \(fileapi.h\) — Win32 apps](#), Microsoft Learn HIGH
- [Querying for the Write Cache Property — Windows drivers](#), Microsoft Learn HIGH
- [DTrace on Windows — Windows drivers](#), Microsoft Learn (2024) HIGH
- [microsoft/DTrace-on-Windows](#), Microsoft HIGH
- [_commit — C runtime library reference](#), Microsoft Learn HIGH

- [os — Miscellaneous operating system interfaces \(Python 3 documentation\)](#), Python Software Foundation **HIGH**
- [io — Core tools for working with streams \(Python 3 documentation\)](#), Python Software Foundation **HIGH**
- [FileOutputStream \(Java SE 25 & JDK 25\)](#), Oracle (2026) **HIGH**
- [FileStream.Flush Method — .NET API reference](#), Microsoft Learn **HIGH**
- [Can Applications Recover from fsync Failures?](#), Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau — USENIX ATC (2020) **HIGH**
- [PANIC on fsync\(\) failure — PostgreSQL commit 6534d54](#), Thomas Munro / PostgreSQL Global Development Group (2018) **HIGH**
- [PostgreSQL's fsync\(\) surprise](#), Jonathan Corbet / LWN.net (2018) **MEDIUM**
- [Page Cache Writeback — Linux Kernel Internals](#), kernel-internals.org **MEDIUM**
- [Fsync Errors — PostgreSQL wiki](#), PostgreSQL Global Development Group (2023) **HIGH**
- [Linux Page Cache Basics](#), Thomas-Krenn.AG wiki **MEDIUM**
- [Windows Write Caching — Part 2: An Overview for Application Developers](#), winntfs.com (2012) **MEDIUM**
- [PostgreSQL Documentation: 28.1. Reliability](#), PostgreSQL Global Development Group (2026) **HIGH**
- [Disk write cache buffer and separate power supply](#), Super User (Stack Exchange) **LOW**
- [Fsyncgate: errors on fsync are unrecoverable](#), Dan Luu (archive of the postgresql-hackers thread, 2018) **MEDIUM**

12.3 V6 — Destination-dependent buffering of standard output

Standard output's buffering mode is not fixed by a program's source code — the C runtime chooses it by looking at what the file descriptor is attached to: line-buffered for a terminal, fully (block) buffered for a pipe or a regular file. C99 §7.19.3 licenses this destination-dependent behaviour, and glibc implements it with an `fstat -plus- isatty` test performed lazily, on the stream's first use. The note's claim about Python is that CPython does not simply inherit this libc-level behaviour: "CPython reimplements the same policy above libc rather than inheriting it," applying the same interactive/non-interactive test inside its own `io` layer instead of deferring to the C runtime it sits on. Python's own documentation states the rule directly — `stdout` is line-buffered when interactive and block-buffered otherwise, while `stderr` is line-buffered in both cases. That last clause is itself a Python-specific choice rather than a straight port of the C

convention: since Python 3.9, non-interactive `stderr` changed from fully buffered to line-buffered, a change the note traces to a CPython tracker discussion. The evidence is Python's official documentation, read against the corresponding C99 and glibc behaviour described earlier in the note, plus the process-wide (`-u`, `PYTHONUNBUFFERED`) and per-call (`flush=True`) overrides that exist specifically at Python's own layer.

The note is explicit about what this does not settle. It marks as contested whether `stderr` is unbuffered or merely not-fully-buffered: C99 requires only the weaker property, the Linux man page asserts the stronger one, and Python's own choice is weaker than either, so any claim that "errors always get out" has to be pinned to a named runtime rather than assumed from the C standard. The note also scopes its conclusions to runtimes built on libc stdio and refuses to generalize beyond them: Java, .NET, Node and Rust "each need checking separately; nothing in this note transfers to them by analogy," since a runtime issuing its own `write()` calls owns its own flush policy regardless of what the C standard says. Two questions are left open rather than resolved: whether Windows' UCRT applies any destination-dependent test to `stdout` at all, and how `stdout` attached to a socket behaves, since the glibc test the note documents covers only character devices and regular files. Finally, the note's scope is the userspace stdio buffer alone — it makes no claim about whether a flushed `write()` has reached durable storage.

Referenced sources

- [setbuf\(3\) — Linux manual page](#) — Michael Kerrisk / man7.org — Linux man-pages project **HIGH**
- [The C99 Draft \(N869, 18 January 1999\), §7.19.3 Files](#) — ISO/IEC JTC1/SC22/WG14 (1999) **HIGH**
- [glibc source: libio/filedoalloc.c \(_IO_file_doallocate \)](#) — GNU C Library, revision glibc-2.39-288 (2024) **HIGH**
- [Output is buffered and lost if the program aborts · Issue #1235 · actions/runner](#) — GitHub / actions-runner issue tracker (2021) **HIGH**
- [sys — System-specific parameters and functions: sys.stdout / sys.stderr](#) — Python Software Foundation — CPython 3.14 documentation **HIGH**
- [1. Command line and environment — the -u option](#) — Python Software Foundation — CPython 3.14 documentation **HIGH**
- [Why pipes sometimes get "stuck": buffering](#) — Julia Evans (2024) **HIGH**
- [How stdbuf works](#) — Harry Marr (2022) **MEDIUM**
- [stdio buffering](#) — Pádraig Brady **HIGH**
- [setvbuf — C runtime library reference](#) — Microsoft Learn (2020, msvc-170) **HIGH**

- [Issue 13601: sys.stderr should be line-buffered when stderr is not a TTY](#) — Python issue tracker (2011–2019) **HIGH**
- [libio: Always use 8192 bytes as the default buffer size \[BZ #4099\]](#) — Florian Weimer / glibc patchwork (2016) **HIGH**
- [Implementation-Defined ISO/IEC C99 Behavior](#) — Oracle Developer Studio C User's Guide — Oracle Corporation **HIGH**
- [Stdout Buffering](#) — Evan Klitzke **MEDIUM**
- [Buffering in pipes](#) — Thomas Habets (2008) **MEDIUM**
- [Why won't journalctl show logs when the program output is piped?](#) — Super User (community Q&A) **MEDIUM**
- [Kubernetes "logs" does not show any output for a simple python program running in a container in a pod](#) — Stack Overflow (community Q&A, 2021) **MEDIUM**
- [Rationale for International Standard — Programming Languages — C, revision 5.10](#) — ISO/IEC JTC1/SC22/WG14 (2003) **HIGH**
- [coreutils source: src/stdbuf.c](#) — GNU coreutils **HIGH**
- [SSH output isn't line buffered?](#) — Unix & Linux Stack Exchange (community Q&A) **MEDIUM**

12.4 V8 — The placement of the stable syscall boundary in Windows and Linux

The note argues that Windows and Linux both promise application developers a stable interface to the kernel, but nominate that stable layer at different points, and the difference is structural rather than one platform being disciplined and the other careless. Linux fixes the contract at the syscall trap itself: numbers are assigned per architecture in an in-tree table, and the kernel's own ABI documentation states such interfaces are "expected to never change and always be available," so a program may bypass the C library, issue the trap directly, and expect it to keep working on future kernels. Windows fixes the contract one layer higher, at Win32; below it, `ntdll`'s Native API is named and resolvable but explicitly disclaimed by Microsoft's own SDK documentation as "subject to change from one release of Windows to another," and the System Service Number beneath it is neither documented nor stable — verified in the note by recomputing Windows's system-call tables directly from the canonical dataset, finding that 52% of functions' numbers shift at least once across versions and hundreds differ between concurrently shipping SKUs. From this the note draws its central, contestable thesis, quoted above: both systems are stable at the layer they nominate, and the disagreement is only

about which layer is nominated, not that one is stable and the other isn't. Everything downstream — what `open()` bottoms out in, where a tracer can attach, whether a syscall number is legitimate to hardcode — follows from that single placement decision.

The note explicitly does not claim Windows is the unusual case; it argues the opposite, that FreeBSD, OpenBSD, NetBSD, Solaris/illumos and macOS all decline a stable-number guarantee the way Windows does, making Linux the outlier among Unix-family systems, with a structural explanation that platforms controlling their own libc and forbidding static linking can move the kernel boundary freely while Linux historically could not. Several points are flagged as unsettled rather than resolved: whether Microsoft's `winternl.h` disclosure was compelled by the 2002 antitrust settlement or an independent decision, graded partially verified because the causal link is unestablished; whether the Win32/`ntdll` split reflects deliberate design intent or is a vestige of NT's abandoned multi-subsystem architecture, graded contested since both readings agree on the underlying mechanism; whether `ntdll` counts as a stable interface at all or only a traced convention, also contested; and whether FreeBSD belongs among the outliers, since a self-identified former core-team contributor disputes the vendor wiki's no-guarantee framing. The note also qualifies its own headline: Windows syscall tracing via DTrace now exists but ships off by default, and Linux's "never break userspace" guarantee, while genuine kernel policy, is bounded — compatibility is "guaranteed for at least 2 years," with exceptions for grave errors or security problems — rather than absolute.

Referenced sources

- [ABI stable symbols — The Linux Kernel documentation](#), The Linux Kernel Organization **HIGH**
- [j00ru/windows-syscalls — Windows System Call Tables](#), Mateusz "j00ru" Jurczyk **HIGH**
- [Windows X86-64 System Call Table \(XP/2003/Vista/7/8/10/11 and Server\)](#), Mateusz "j00ru" Jurczyk **HIGH**
- [NtQuerySystemInformation function \(winternl.h\) — Win32 apps](#), Microsoft **HIGH**
- [Calling Internal APIs — Win32 apps](#), Microsoft **HIGH**
- [arch/x86/entry/syscalls/syscall_64.tbl](#), Linux kernel source tree **HIGH**
- [include/uapi/asm-generic/unistd.h](#), Linux kernel source tree **HIGH**
- [Do other Unix-like kernels have stable syscall ABIs?](#), Unix & Linux Stack Exchange **MEDIUM**
- [FreeBSD Vendor Information](#), The FreeBSD Project **HIGH**
- [openbsd-tech: system call origin verification](#), OpenBSD project mailing list **HIGH**

- [all: stop using non-libc system calls on OpenBSD \(golang/go#36435\)](#), The Go Project **HIGH**
- [syscall\(2\) — NetBSD Manual Pages](#), The NetBSD Project **HIGH**
- [port-solaris mailing list, November 2010: syscall number reservation](#), port-solaris list archive (Carnegie Mellon mirror) **HIGH**
- [NtTrace — Native API tracing for Windows](#), Roger Orr **HIGH**
- [strace — linux syscall tracer](#), The strace project **HIGH**
- [DTrace on Windows — Windows drivers](#), Microsoft **HIGH**
- [DTrace — Windows Commands](#), Microsoft **HIGH**
- [Inside the Native API](#), Mark Russinovich, Sysinternals **HIGH**
- [NT OS/2 Subsystem Design Rationale \(NT Design Workbook\)](#), Mark H. Lucovsky, Microsoft **HIGH**
- [The Sad History of the Microsoft POSIX Subsystem](#), Brian Reiter (2010) **MEDIUM**
- [Syscall ABI compatibility: Linux vs Windows/macOS](#), Mark Mossberg (2023) **MEDIUM**
- [Never break userspace](#), LWN.net **HIGH**
- [Updating Syscall Tables — SysWhispers4 documentation](#), SysWhispers4 project documentation **LOW**
- [CRT-free in 2023: tips and tricks — discussion](#), Lobsters discussion thread **MEDIUM**
- [Architecture of Windows NT](#), Wikipedia **MEDIUM**
- [U.S. v. Microsoft Corporation — Information on the Settlement](#), U.S. Department of Justice, Antitrust Division **HIGH**
- [Syscalls on Windows 11](#), hammertux **MEDIUM**

12.5 V9 — The layer at which Windows and Linux stabilize the syscall boundary

This note argues that Linux and Windows both promise application developers a stable interface to the kernel, but place that promise at different layers, and it is the placement — not the presence or absence of a promise — that separates them. Linux nominates the syscall trap itself: syscall numbers are fixed per architecture in the kernel's in-tree table, and the kernel's own ABI documentation states such interfaces are "expected to never change and always be available." Windows nominates a layer above the kernel transition, at Win32; the Native API exported by `ntdll.dll` and the kernel-transition stub beneath it are both undocumented, and `ntdll` is explicitly disclaimed by Microsoft as "internal to the operating system and subject to change from one release of

Windows to another." The note backs this with a recomputation of Windows syscall-number drift from the canonical upstream dataset, showing that most Windows syscall numbers change between versions, sometimes within a single major release, and that concurrently shipping SKUs disagree with each other — evidence that the number, unlike the name, is never the contract on Windows. It then broadens the claim to the whole Unix family: FreeBSD, OpenBSD, NetBSD, Solaris/illumos, and macOS all decline a syscall-number stability guarantee the way Windows does, each in its own first-party statement. This is why the note's central, contestable claim inverts the intuitive framing: rather than Windows being the odd one out next to a stable Linux, Linux is the outlier — "the only mainstream system that treats the raw syscall interface as a public contract" — with a structural explanation offered for why: platforms that control their own libc and forbid static linking can move the kernel boundary freely, while Linux historically could not control its userspace and so had to freeze the boundary itself.

The note is explicit about what it does not claim. It does not treat Linux's guarantee as absolute in practice, citing pushback on "we don't break userspace" as an oversold slogan and noting that the kernel's own stable-ABI document hedges compatibility to "at least 2 years" and permits breaking changes for security fixes. It leaves two questions of intent explicitly unsettled rather than resolved: whether the Win32/`ntdll` split is a deliberate design choice or a vestige of NT's abandoned multi-subsystem architecture, and whether `ntdll` counts as a genuine interface or merely a traced convention — in both cases holding that the underlying mechanism is undisputed and only the framing is contested. It flags as open questions whether Microsoft's disclosure of Native API prototypes was compelled by the 2002 antitrust settlement or an independent decision, and whether Windows' syscall-number reshuffling is purely mechanical (a byproduct of alphabetically ordered service tables) or partly deliberate — asserting only the mechanical explanation, since no first-party statement of intent was found. It also qualifies its own "Windows has no strace" framing as now substantially outdated, since a working syscall tracer ships with a recent Windows Server release, though disabled by default, leaving open whether that default posture will change.

Referenced sources

- [ABI stable symbols — The Linux Kernel documentation](#) — *The Linux Kernel Organization* **HIGH**
- [j00ru/windows-syscalls — Windows System Call Tables](#) — *Mateusz "j00ru" Jurczyk* **HIGH**
- [Windows X86-64 System Call Table \(XP/2003/Vista/7/8/10/11 and Server\)](#) — *Mateusz "j00ru" Jurczyk* **HIGH**
- [NtQuerySystemInformation function \(winternl.h\) — Win32 apps](#) — *Microsoft* **HIGH**
- [Calling Internal APIs — Win32 apps](#) — *Microsoft* **HIGH**

- [arch/x86/entry/syscalls/syscall_64.tbl](#) — *Linux kernel source tree* **HIGH**
- [include/uapi/asm-generic/unistd.h](#) — *Linux kernel source tree* **HIGH**
- [Do other Unix-like kernels have stable syscall ABIs?](#) — *Unix & Linux Stack Exchange* **MEDIUM**
- [FreeBSD Vendor Information](#) — *The FreeBSD Project* **HIGH**
- [openbsd-tech: system call origin verification](#) — *OpenBSD project mailing list* **HIGH**
- [all: stop using non-libc system calls on OpenBSD \(golang/go#36435\)](#) — *The Go Project* **HIGH**
- [syscall\(2\)](#) — *NetBSD Manual Pages* — *The NetBSD Project* **HIGH**
- [port-solaris mailing list, November 2010: syscall number reservation](#) — *port-solaris list archive (Carnegie Mellon mirror)* **HIGH**
- [NtTrace](#) — *Native API tracing for Windows* — *Roger Orr* **HIGH**
- [strace](#) — *linux syscall tracer* — *The strace project* **HIGH**
- [DTrace on Windows](#) — *Windows drivers* — *Microsoft* **HIGH**
- [DTrace](#) — *Windows Commands* — *Microsoft* **HIGH**
- [Inside the Native API](#) — *Mark Russinovich, Sysinternals* **HIGH**
- [NT OS/2 Subsystem Design Rationale \(NT Design Workbook\)](#) — *Mark H. Lucovsky, Microsoft* **HIGH**
- [The Sad History of the Microsoft POSIX Subsystem](#) — *Brian Reiter (2010)* **MEDIUM**
- [Syscall ABI compatibility: Linux vs Windows/macOS](#) — *Mark Mossberg (2023)* **MEDIUM**
- [Never break userspace](#) — *LWN.net* **HIGH**
- [Updating Syscall Tables](#) — *SysWhispers4 documentation* — *SysWhispers4 project documentation* **LOW**
- [CRT-free in 2023: tips and tricks](#) — *discussion* — *Lobsters discussion thread* **MEDIUM**
- [Architecture of Windows NT](#) — *Wikipedia* **MEDIUM**
- [U.S. v. Microsoft Corporation — Information on the Settlement](#) — *U.S. Department of Justice, Antitrust Division* **HIGH**
- [Syscalls on Windows 11](#) — *hammertux* **MEDIUM**

12.6 V10 — The System Call Boundary and Its Observability

This note's organizing claim is that "the system call boundary" is actually two separate boundaries — a hardware mechanism boundary, where the CPU crosses privilege levels, and a contract boundary, where an operating system promises not to change an interface — and that debugging and tracing tools attach at one or the other, not both.

The quoted claim sits in its treatment of Linux userspace tooling: `strace` attaches at the kernel syscall entry and exit points via the `ptrace` mechanism, so it sees every actual crossing into the kernel with decoded arguments, while `ltrace` attaches one layer higher, at the dynamic linker, intercepting calls into shared libraries. Because a call like `printf` is a library call that only sometimes results in a `write` syscall underneath, `printf` is visible to `ltrace` and `write` to `strace` — the two tools are watching different layers of the same call stack. The note draws out the consequence directly: a deadlock that occurs entirely inside a library, without ever reaching the kernel, produces no evidence in `strace` output at all, because the kernel is never involved and therefore has nothing to report. This grounds the note's broader argument, from its tooling comparison table, that where a tracer can attach is dictated by where a stable, nameable boundary exists — the same logic it later applies to explain why Windows has no direct equivalent to `strace`.

The note does not claim that `ltrace` and `strace` between them cover everything a debugger might need to see. It documents, in a separate section, that some Linux calls — those served by the vDSO, such as `clock_gettime` — involve no kernel crossing whatsoever and are consequently invisible to `strace` too, not just to library-level tools; the note treats a call's absence from a trace as diagnostic information rather than a debugging dead end. The claim about the `ltrace`/`strace` layering itself rests on a single practitioner blog post graded Medium credibility, corroborated in shape by the note's separate, higher-graded citation of `strace`'s own documentation of the `ptrace` mechanism, but the `ltrace`-specific detail is not independently verified elsewhere in the note. The note is also explicit that Windows tooling has no clean analogue to this `strace`/`ltrace` split, since Windows exposes no stable numbered syscall list for any tracer to bind to — so the observation is scoped to Linux and is not offered as a general claim about tracing on any operating system.

Referenced sources

- [Anatomy of a system call, part 1](#) HIGH
- [Anatomy of a system call, part 2](#) HIGH
- [SYSCALL — Fast System Call \(x86 instruction reference\)](#) HIGH
- [x86-64 Syscall Entry — Linux Kernel Internals](#) MEDIUM
- [The Linux Kernel Driver Interface \(stable-api-nonsense\)](#) HIGH
- [Add kernel ↔ userspace ABI stability documentation](#) HIGH
- [x86 Assembly / Interfacing with Linux](#) MEDIUM
- [Do other Unix-like kernels have stable syscall ABIs?](#) MEDIUM
- [Libraries and Headers — Windows drivers](#) HIGH
- [winternl.h header — Win32 apps](#) HIGH

- [NTDLL Functions](#) **HIGH**
- [Windows X86-64 System Call Table \(XP through Windows 11 and Server\)](#) **HIGH**
- [Microsoft Windows library files — NTDLL.DLL](#) **MEDIUM**
- [Windows X86-64 System Call Table — discussion thread](#) **LOW**
- [strace — linux syscall tracer](#) **HIGH**
- [ptrace and Syscall Interception — Linux Kernel Internals](#) **MEDIUM**
- [Process Monitor — Sysinternals](#) **HIGH**
- [NtTrace — Native API tracing for Windows](#) **MEDIUM**
- [What Makes System Calls Expensive](#) **MEDIUM**
- [vDSO and Virtual System Calls — Linux Kernel Internals](#) **MEDIUM**
- [Two frequently used system calls are ~77% slower on AWS EC2](#) **MEDIUM**
- [Why you should use io_uring for network I/O](#) **HIGH**
- [Speed up open\(\).read\(\) pattern by reducing the number of system calls — cpython#120754](#) **HIGH**
- [win32file.CreateFile — PyWin32 documentation](#) **MEDIUM**
- [ltrace: When strace Is Not Enough](#) **MEDIUM**
- [System Calls and the User-to-Kernel Transition](#) **MEDIUM**
- [Windows vs. Linux Syscall Interfaces: Win32 API, NT Native API, and WSL](#) **LOW**
- [In Linux x86_64, are syscalls and int 0x80 related?](#) **MEDIUM**
- [A Deep Dive Into Malicious Direct Syscall Detection](#) **HIGH**
- [Linux kernel system calls table](#) **HIGH**
- [io_uring Interface Overview](#) **LOW**

12.7 V11 — Batching and the Disappearing Syscall Under io_uring/SQPOLL

The note's thesis is that a system call is a genuinely different hardware mechanism from a function call, with a real, largely payload-independent cost, and that syscall count — not bytes moved — is the right unit for reasoning about I/O-bound performance. Rather than hide the cases where this thesis strains, the note names three of them explicitly, and batching via `io_uring` is the second. `io_uring` moves submissions and completions into shared ring buffers, so a single `io_uring_enter` call can carry a large batch of operations; and with `IORING_SETUP_SQPOLL` a kernel-side polling thread means steady-state I/O submission needs no syscalls at all — the crossing itself disappears for the steady-state case, not merely its per-operation share. The note grounds the batching mechanism in a Red Hat Developers article on `io_uring`, graded high credibility as coming from the primary corporate steward of the subsystem's upstream development,

and attaches a magnitude to the claim: published microbenchmarks reportedly show per-submission cost falling from roughly 300 cycles unbatched to roughly 40 cycles at a batch size of 16, sourced to EmergentMind, a secondary aggregator summarizing an academic study it has not read at source.

The note is explicit about what this does not establish. It flags the 300-to-40-cycle figure [reported], describing it as a single benchmark study rather than a settled or general result, and its source for that number is graded low credibility precisely because it is a secondary summary rather than the underlying paper. Elsewhere the note treats SQ-POLL and kernel-bypass techniques as trading the crossing cost for a dedicated polling core, not eliminating the work — so "no syscalls at all" describes the submission path in steady state, not a claim that the cost has vanished outright. The note's stated conclusion is narrower than the quoted line alone suggests: where batching applies, per-call cost simply stops being the right unit of analysis, which is a claim about how to reason about the cost rather than a claim that no cost remains.

Referenced sources

- [Anatomy of a system call, part 1](#) — David Drysdale / LWN.net (2014) **HIGH**
- [Anatomy of a system call, part 2](#) — David Drysdale / LWN.net (2014) **HIGH**
- [SYSCALL — Fast System Call \(x86 instruction reference\)](#) — Felix Cloutier, transcribed from the Intel SDM (n.d.) **HIGH**
- [x86-64 Syscall Entry](#) — Linux Kernel Internals — kernel-internals.org (n.d.) **MEDIUM**
- [The Linux Kernel Driver Interface \(stable-api-nonsense\)](#) — Greg Kroah-Hartman / Linux kernel documentation (2004–present) **HIGH**
- [Add kernel ↔ userspace ABI stability documentation](#) — Greg Kroah-Hartman / LWN.net (2006) **HIGH**
- [x86 Assembly / Interfacing with Linux](#) — Wikibooks contributors (n.d.) **MEDIUM**
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange (2018–2023) **MEDIUM**
- [Libraries and Headers — Windows drivers](#) — Microsoft Learn (last updated 2022-07-12) **HIGH**
- [winternl.h header — Win32 apps](#) — Microsoft Learn (n.d.) **HIGH**
- [NTDLL Functions](#) — Geoff Chappell (ongoing) **HIGH**
- [Windows X86-64 System Call Table \(XP through Windows 11 and Server\)](#) — Mateusz "j00ru" Jurczyk (ongoing) **HIGH**
- [Microsoft Windows library files — NTDLL.DLL](#) — Wikipedia contributors (n.d.) **MEDIUM**

- [Windows X86-64 System Call Table — discussion thread](#) — Hacker News contributors (2019) **LOW**
- [strace — linux syscall tracer](#) — The strace project (ongoing) **HIGH**
- [ptrace and Syscall Interception — Linux Kernel Internals](#) — kernel-internals.org (n.d.) **MEDIUM**
- [Process Monitor — Sysinternals](#) — Microsoft Learn / Sysinternals (ongoing) **HIGH**
- [NtTrace — Native API tracing for Windows](#) — Roger Orr (ongoing) **MEDIUM**
- [What Makes System Calls Expensive](#) — Abhinav Upadhyay (2025) **MEDIUM**
- [vDSO and Virtual System Calls — Linux Kernel Internals](#) — kernel-internals.org (n.d.) **MEDIUM**
- [Two frequently used system calls are ~77% slower on AWS EC2](#) — packagecloud engineering (n.d.) **MEDIUM**
- [Why you should use io_uring for network I/O](#) — Red Hat Developers (2023) **HIGH**
- [Speed up open\(\).read\(\) pattern by reducing the number of system calls](#) — cpython#120754 — CPython issue tracker (2024) **HIGH**
- [win32file.CreateFile — PyWin32 documentation](#) — Mark Hammond / Tim Golden (PyWin32 docs, ongoing) **MEDIUM**
- [ltrace: When strace Is Not Enough](#) — Satesh / unixy.io (2026) **MEDIUM**
- [System Calls and the User-to-Kernel Transition](#) — Yunolay (2026) **MEDIUM**
- [Windows vs. Linux Syscall Interfaces: Win32 API, NT Native API, and WSL](#) — kinda-technical.com (2026) **LOW**
- [In Linux x86_64, are syscalls and int 0x80 related?](#) — Stack Overflow (2015–present) **MEDIUM**
- [A Deep Dive Into Malicious Direct Syscall Detection](#) — Palo Alto Networks / Unit 42 (2024) **HIGH**
- [Linux kernel system calls table](#) — Marcin Juskiewicz (regenerated from kernel source; table dated 2026-08-20 from 7.2.0 sources) **HIGH**
- [io_uring Interface Overview](#) — EmergentMind, summarizing Jasny et al. (updated 2026-01-17) **LOW**

12.8 V14 — Leaky operating system abstractions

"Where the Abstraction Leaks" argues that operating-system interfaces are contracts with a hidden scope clause: the interface states what a call does, not the conditions under which that promise holds, and a "leak" is simply the program running past those conditions. Its central case is durability. A successful `write()` or `f.write()` call means

the kernel has *accepted* the bytes into the page cache, not that they have reached the device; only `fsync()` (or Windows' `FlushFileBuffers`) forces the transfer, a claim the note backs with the Linux and Windows platform manuals directly. The note is explicit about how to teach this honestly: the honest classroom demo is a power-cut, not a `kill -9`, because killing the process destroys only the language runtime's own buffer while the kernel's page cache survives the process and is still written back normally — a power cut is the only test that actually reaches the layer being discussed. The evidentiary backing for this is observational rather than assertion: `strace` showing `write` calls with no accompanying `fsync`, `blktrace` on the block device showing nothing reaches disk until later writeback, and Evan Jones's own published `blktrace` measurements of ext4 and xfs issuing a cache flush and FUA write to the journal. The note treats this "accepted, not done" mismatch as the single most consequential of the five leaky-abstraction cases it documents.

The note does not present this as a complete or permanent guarantee. It notes that `fsync()` on a file does not by itself make a newly created file durably *findable* after a crash — the containing directory needs its own `fsync()` — and that `fsync()`'s honesty depends on the device: older kernels, some filesystems, and drives that ignore the FUA flag can silently defeat it, a separate hardware-level problem the note does not claim software can solve. It also flags one of its own numeric claims elsewhere as unsettled: a stated per-syscall cost in cycles is marked in-text as sourced only from aggregator content rather than a primary measurement, with the recommendation that students verify it themselves. More broadly, the note scopes every platform claim to a kernel or API version and lists mechanisms that have already changed — Windows' User-Mode Scheduling was withdrawn as of Windows 11, `io_uring` altered what a Linux syscall count means, and CPython's GIL is being unwound — presenting its cross-platform claims as things to re-verify rather than as timeless facts.

Referenced sources

- [close\(2\) — Linux manual page](#), Michael Kerrisk / man7.org (2024) **HIGH**
- [fsync\(2\) — Linux manual page](#), Michael Kerrisk / man7.org (2024) **HIGH**
- [Flushing System-Buffered I/O Data to Disk](#), Microsoft Learn (2021) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#), Microsoft Learn (2023) **HIGH**
- [Durability: Linux File APIs](#), Evan Jones (2020) **MEDIUM**
- [The Secret Life of fsync](#), Andrey Pechkurov / puzpuzpuz.dev (2022) **MEDIUM**
- [Revised notes on the reliability of FlushFileBuffers](#), Raymond Chen / Microsoft DevBlogs (2017) **HIGH**
- [fsync, fdatasync, and O_SYNC](#), kernel-internals.org (2025) **MEDIUM**

- [os — Miscellaneous operating system interfaces](#), Python Software Foundation (2026) **HIGH**
- [Bypassing the GIL for Parallel Processing in Python](#), Real Python (2024) **MEDIUM**
- [About Processes and Threads](#), Microsoft Learn (2023) **HIGH**
- [Fibers](#), Microsoft Learn (2023) **HIGH**
- [User-Mode Scheduling](#), Microsoft Learn (2022) **HIGH**
- [The Libuv Thread Pool Trap: Why Node.js Async APIs Stall Under Load](#), The Practical Developer (2026) **MEDIUM**
- [MapViewOfFile function \(memoryapi.h\)](#), Microsoft Learn (2022) **HIGH**
- [Which is fastest: read, fread, ifstream or mmap?](#), Daniel Lemire (2012) **HIGH**
- [Latency implications of virtual memory](#), Erik Rigtorp (2020) **MEDIUM**
- [Why is mmap\(\) Faster Than Sequential IO? Debunking Common Myths](#), linux-vox.com (2025) **LOW**
- [File Access: Memory-Mapped vs. I/O System Call Performance](#), Baeldung on Linux (2024) **MEDIUM**
- [strace\(1\) — Linux manual page](#), strace project / man7.org (2024) **HIGH**
- [DTrace on Windows](#), Microsoft Learn (2023) **HIGH**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#), Yarden Shafir / Winsider Seminars & Solutions (2021) **HIGH**
- [Linux I/O Models — Blocking, epoll, io_uring, AIO, Signal-Driven](#), systeminternals.dev (2026) **MEDIUM**
- [The Law of Leaky Abstractions](#), Joel Spolsky (2002) **HIGH**
- [There Is a Mountain Lion in the Datacenter](#), HFT University (2026) **LOW**
- [Sysinternals Suite 2026.19.08](#), Neowin (2026) **MEDIUM**
- [How Python's GIL Works](#), Codefinity (2025) **LOW**

12.9 V17 — Tracing mechanisms for observing a running system

This note argues that a tracing mechanism's capabilities and costs are fixed by where it attaches to a running program — kernel syscall entry, a user-mode library boundary, a rewritten instruction stream, an in-kernel probe, or an OS-published event pipeline — and that the attachment point mechanically determines visibility, blind spot, and cost. The quoted span anchors the cost claim for `ptrace / strace` on Linux, which stops a traced process twice per syscall, at kernel entry and at exit. Brendan Gregg's benchmark — `dd if=/dev/zero of=/dev/null bs=1 count=500k`, a program that issues nothing but syscalls — runs 442× slower under `strace`, and the note stresses that this

penalty holds even when the trace filter targets a syscall the program never issues, because the kernel pays the stop unconditionally, before it can know whether the event is one the tracer wanted. The note treats this as the sharpest illustration of a broader claim: `ptrace`-class overhead scales with syscall *rate*, not with program size, which it folds into a three-part model of attachment — visibility, blind spot, cost — applied across seven tracing mechanisms spanning Linux and Windows.

The note does not present 442× as typical. It cites independent measurements of the identical `dd` benchmark, on different hardware and speculative-execution mitigation states, ranging from roughly 37× to 136×, and separately notes that other sources quote `ptrace` overhead as "2–10×," "15×," or "100×." Its position is that this spread is not a contradiction to resolve but evidence that a bare overhead figure, quoted without its workload, carries no information — the number describes the benchmark, not the tool. It does not claim `strace` is unusable in general; elsewhere it recommends `strace -c -f` as a cheap first step to establish syscall rate before deciding whether a full trace is affordable. What the note leaves unsettled is exactly how overhead scales below this syscall-heavy extreme, and it makes no attempt to collapse the published range into one authoritative figure, treating the workload rather than the tool as the determining variable.

Referenced sources

- [ptrace and Syscall Interception, Linux Kernel Internals](#) MEDIUM
- [strace Wow Much Syscall, Brendan Gregg \(2014\)](#) HIGH
- [Windows vs. Linux Syscall Interfaces: Win32 API, NT Native API, and WSL, kindat-echnical.com \(2026\)](#) MEDIUM
- [Syscall ABI compatibility: Linux vs Windows/macOS, Mark Mossberg \(2023\)](#) MEDIUM
- [NtTrace — Native API tracing for Windows, Roger Orr](#) HIGH
- [rogerorr/NtTrace — "How it works", Roger Orr](#) HIGH
- [vdso\(7\) — Linux manual page, Linux man-pages project](#) HIGH
- [StraceNT — System Call Tracer for Windows NT, Pankaj Garg](#) MEDIUM
- [Debugging eBPF verifier: Best Practices for High-Performance Kernel Programming, Raven](#) MEDIUM
- [Teaching Operating Systems with FreeBSD through Tracing, Analysis, and Experimentation, George V. Neville-Neil et al., USENIX ;login: \(2017\)](#) HIGH
- [Tracing and its observer effect on concurrency, Indigo Orton, Alan Mycroft — ACM \(2021\)](#) HIGH
- [We Have It Easy, But Do We Have It Right? / observer effect and measurement bias, Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, Peter Sweeney \(2008\)](#) HIGH

- [System Calls and the User-to-Kernel Transition](#), Yunolay (2026) **MEDIUM**
- [Windows System Call Tables](#), Mateusz "j00ru" Jurczyk **HIGH**
- [strace: fight for performance \(FOSDEM 2020 slides\)](#), strace maintainers, FOSDEM (2020) **MEDIUM**
- [System Call Tracing With eBPF: Move Beyond Ptrace](#), SysTutorials **LOW**
- [Ptrace & You](#), Omar S. Navarro Leija **MEDIUM**
- [vDSO and Virtual System Calls](#), Linux Kernel Internals **MEDIUM**
- [System-wide syscalls tracing and monitoring in Linux using bpftrace](#), Narasimha Prasanna HN **MEDIUM**
- [Notes and Thoughts on Strace for Windows](#), Jarek Przygódzki **MEDIUM**
- [ipankajg/stracent — StraceNT](#), Pankaj Garg **MEDIUM**
- [Unveiling Dynamic Binary Instrumentation Techniques](#), arXiv preprint (2025) **MEDIUM**
- [How to do a million watchpoints: Efficient Debugging using Dynamic Instrumentation](#), Qin Zhao et al., CC (2008) **HIGH**
- [What is eBPF? An Introduction and Deep Dive into the eBPF Technology](#), eBPF Foundation **HIGH**
- [Inside the eBPF Verifier: Ensuring Program Safety and Complexity Bounds](#), How Tech — Systems Programming **MEDIUM**
- [Loops and Iterators in eBPF](#), Teodor J. Podobnik **MEDIUM**
- [CVE-2021-33624 — Red Hat Customer Portal](#), Red Hat **HIGH**
- [Eliminating eBPF Tracing Overhead on Untraced Processes](#), Milo Craun et al., eBPF '24 **HIGH**
- [DTrace Capabilities](#), Oracle — Solaris Dynamic Tracing Guide **HIGH**
- [Dynamic Instrumentation of Production Systems](#), Bryan M. Cantrill, Michael W. Shapiro, Adam H. Leventhal — USENIX ATC (2004) **HIGH**
- [DTrace on Windows](#), Microsoft Learn **HIGH**
- [Announcing DTrace for Windows Insider](#), Packt (2019) **MEDIUM**
- [ETW — Event Tracing for Windows](#), Mjolnir Security **MEDIUM**
- [NT Kernel Logger Trace Session](#), Microsoft Learn **HIGH**
- [Windows Performance Recorder](#), Microsoft Learn **HIGH**
- [Doubling Down: Detecting In-Memory Threats with Kernel ETW Call Stacks](#), Elastic Security Labs **MEDIUM**
- [prash-wghats/DTrace-win32](#), prash-wghats **MEDIUM**
- [CSE 422S Studio 5: Tracing the Linux Kernel](#), Christopher Gill, Washington University in St. Louis **MEDIUM**

- [microsoft/DTrace-on-Windows](#), Microsoft **HIGH**
- [bpf-helpers\(7\)](#) — Linux manual page, Linux man-pages project **HIGH**
- [bpf: Change value of MAX_TAIL_CALL_CNT from 32 to 33](#), Linux kernel mailing list / stable backport series (2022) **HIGH**
- [Windows 11, version 26H1](#), Microsoft Support (2026) **HIGH**
- [Updating Syscall Tables](#) — SysWhispers4, SysWhispers4 project documentation **MEDIUM**
- [strace 7.1 release notes](#), strace project (2026) **HIGH**

12.10 V21 — NtTrace, a user-mode Windows native-API tracer

This note documents NtTrace, a tool that reports the calls a Windows process makes into `ntdll.dll`, the boundary between application code and the NT kernel. Its central design choice is to trace via the debug interface so the tool needs no driver and no administrator rights: NtTrace attaches to its target the way a symbolic debugger does, using the Windows `WaitForDebugEvent / ContinueDebugEvent` machinery, and writes software breakpoints into `ntdll.dll` around the native call stubs. When a breakpoint fires, the debug interface delivers an exception event that lets the tool read arguments and return values before resuming the target. Because this stays entirely at user level — "no attempt is made to go into the kernel" — the tool can run without installing anything or elevating privileges, which the author states was a deliberate decision to avoid requiring administrative rights or permission to install device drivers. The note treats this as the tool's core contribution: not a feature, but a structural choice with direct consequences, corroborated by two independent third-party reviews of Windows tracing tools that confirm the same debug-interface mechanism.

The note is equally explicit about what that choice does not buy. Because the tool only ever sees code that executes through `ntdll`, it cannot observe kernel calls made by device drivers, nor any direct use of the `int 2e`, `syscall`, or `sysenter` instructions — and the note treats that second gap as the standard technique used to evade user-mode monitoring, not a hypothetical edge case. The debug-interface attachment also has a timing cost the documentation itself does not quantify: breakpoint-per-call tracing is event-based, so calls from multiple threads are serialised in the output, and two independent developers who tried the same design abandoned it as too slow for ordinary GUI programs. The note is careful to distinguish the privilege claim, which it treats as settled and structurally guaranteed by the debugger-based design, from the performance and coverage trade-offs, which it presents as real costs the author states rather than hides, and as unsettled in the sense that no published benchmark exists to size them precisely.

Referenced sources

- [NtTrace — Native API tracing for Windows](#) — Roger Orr **HIGH**
- [rogerorr/NtTrace — repository README](#) — Roger Orr **HIGH**
- [Home · rogerorr/NtTrace Wiki](#) — Roger Orr **HIGH**
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygodzki **MEDIUM**
- [AWStrace: Another Windows Strace](#) — downwithup (2026) **MEDIUM**
- [StraceNT - System Call Tracer for Windows NT](#) — Pankaj Garg **MEDIUM**
- [System Call Tracer for Windows \(drstrace\)](#) — Dr. Memory / DynamoRIO project **HIGH**
- [mxmssh/drltrace — README](#) — Maksim Shudrak **MEDIUM**
- [Introducing QTrace, a "zero knowledge" system call tracer](#) — Roberto Paleari (2014) **MEDIUM**
- [strace Wow Much Syscall](#) — Brendan Gregg (2014) **HIGH**
- [Introducing strace --seccomp-bpf](#) — Paul Chaignon (2019) **HIGH**
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange (2018-) **MEDIUM**
- [A 10-minute guide to the Linux ABI](#) — Alison Chaiken, opensource.com (2022) **HIGH**
- [A Deep Dive Into Malicious Direct Syscall Detection](#) — Or Chechik and Ofir Ozer, Palo Alto Networks (2024) **HIGH**
- [Overload Content by Article](#) — ACCU — ACCU (Association of C and C++ Users) **HIGH**
- [Windows System Calls & Native API: A Guide to Assembly Calling Conventions Explained](#) — linuxvox.com **LOW**
- [mandiant/STrace — A DTrace on Windows Reimplementation](#) — Mandiant **MEDIUM**

12.11 V22 — NtTrace, a debug-interface tracer for the Windows Native API

The note documents NtTrace, a Windows tool that traces calls crossing from user-mode applications into the OS by placing breakpoints in `ntdll.dll` via the Windows debug interface — a design chosen specifically so the tool needs no driver and no administrator rights. Its central piece of evidence for why Windows tracing differs structurally from Linux's is a single string from the project's own published sample capture: a traced `NtOpenFile` call whose object attributes are printed as `\\??\\C:\\WWW\\NtTrace\\`. The note explains: "The `\\??\\` prefix is the object-manager namespace, which a Win32 program never types. This is the layer difference, visible in one string." That prefix

marks the NT object-manager path from the kernel resolves, distinct from the Win32 path a caller supplies, and the note treats it as concrete, visible proof that NtTrace observes a lower, undocumented layer than any ordinary Windows program ever addresses directly. This grounds the note's larger claim: Windows offers no published, version-stable syscall boundary the way Linux does, so a Windows tracer must instead attach to `ntdll`, an interface whose entry points and argument layouts NtTrace's own configuration file reconstructs through reverse engineering rather than vendor documentation.

The note is careful about what this single capture does not show. It is one unpaired trace of `cmd.exe`, not a matched comparison against an equivalent Linux `strace` run of the same program, and it is an old, 32-bit capture whose specific addresses should not be read as representative of current Windows output. One line within it — `NtFsControlFile` listing `UserApcContext` twice — is flagged as an unresolved oddity, possibly a transcription slip, still to be checked against the live source. The note does not claim `ntdll` is documented or guaranteed stable across releases; it explicitly disputes other sources that describe it that way. Nor does it claim the capture shows everything a traced process does — driver-originated kernel calls and direct-syscall stubs are named as invisible to this technique, and the note treats any timing or ordering read from such a trace as unsafe evidence about the untraced program, since breakpoint-based tracing serialises concurrent calls and distorts performance by construction.

Referenced sources

- [NtTrace — Native API tracing for Windows](#) — Roger Orr — **HIGH**
- [rogerorr/NtTrace — repository README](#) — Roger Orr — **HIGH**
- [Home · rogerorr/NtTrace Wiki](#) — Roger Orr — **HIGH**
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygodzki — **MEDIUM**
- [AWStrace: Another Windows Strace](#) — downwithup — **MEDIUM**
- [StraceNT – System Call Tracer for Windows NT](#) — Pankaj Garg — **MEDIUM**
- [System Call Tracer for Windows \(drstrace\)](#) — Dr. Memory / DynamoRIO project — **HIGH**
- [mxmssh/drltrace — README](#) — Maksim Shudrak — **MEDIUM**
- [Introducing QTrace, a "zero knowledge" system call tracer](#) — Roberto Paleari — **MEDIUM**
- [strace Wow Much Syscall](#) — Brendan Gregg — **HIGH**
- [Introducing strace --seccomp-bpf](#) — Paul Chaignon — **HIGH**
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange — **MEDIUM**
- [A 10-minute guide to the Linux ABI](#) — Alison Chaiken, opensource.com — **HIGH**

- [A Deep Dive Into Malicious Direct Syscall Detection](#) — Or Chechik and Ofir Ozer, Palo Alto Networks — **HIGH**
- [Overload Content by Article](#) — ACCU — ACCU (Association of C and C++ Users) — **HIGH**
- [Windows System Calls & Native API: A Guide to Assembly Calling Conventions Explained](#) — linuxvox.com — **LOW**
- [mandiant/STrace — A DTrace on Windows Reimplementation](#) — Mandiant — **MEDIUM**

12.12 V23 — NtTrace, a user-mode Native API tracer for Windows

The note documents NtTrace, a Windows tool that traces a process's calls into `ntdll.dll` — the boundary between application code and the NT kernel — by attaching as a debugger and placing breakpoints around the native call stubs, a design chosen specifically so the tool needs no kernel driver and no administrator rights. To make that mechanism concrete rather than merely described, the note reproduces the tool's own published sample capture: a trace of `NtTrace -filter File cmd`, retrieved from the project's home page. From that one capture the note draws out several things a Windows trace shows that prose alone cannot — NT object paths distinct from the Win32 paths a program actually typed, a single return value carrying two error namespaces at once (an `NTSTATUS` translated into a Win32 message), and small console writes appearing unbuffered rather than coalesced. But the note draws a sharp line around what this evidence supports: "It is not a matched pair." The capture traces `cmd` alone; the note does not pair it with an equivalent `strace` run of the same program, so it demonstrates only what a Windows trace looks like, not how its cost or fidelity compares to a Linux one.

The note is correspondingly explicit that this capture cannot support a cross-platform comparison, stating that a genuine comparison would require both traces of the same program produced together, which this source does not supply. Several details about the capture itself are also left open rather than resolved: it is a 32-bit, Windows XP-era artifact, so its addresses should not be read as representative of a current 64-bit trace; one field, `UserApcContext`, appears twice in a single output line, and the note cannot say whether that is a quirk of the tool's configuration or an error on the source page; and because the capture reached the note through a fetch-and-convert step rather than a direct terminal copy, the note flags it for a spot-check against the live page before any further use. None of these caveats bear on the tracing mechanism itself, which the note treats as independently established elsewhere; they are scoped narrowly to how much trust this one illustrative capture deserves.

Referenced sources

- [NtTrace — Native API tracing for Windows](#) — Roger Orr **HIGH**
- [rogerorr/NtTrace — repository README](#) — Roger Orr **HIGH**
- [Home · rogerorr/NtTrace Wiki](#) — Roger Orr **HIGH**
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygodzki **MEDIUM**
- [AWStrace: Another Windows Strace](#) — downwithup (2026) **MEDIUM**
- [StraceNT - System Call Tracer for Windows NT](#) — Pankaj Garg **MEDIUM**
- [System Call Tracer for Windows \(drstrace\)](#) — Dr. Memory / DynamoRIO project **HIGH**
- [mxmssh/drltrace — README](#) — Maksim Shudrak **MEDIUM**
- [Introducing QTrace, a "zero knowledge" system call tracer](#) — Roberto Paleari (2014) **MEDIUM**
- [strace Wow Much Syscall](#) — Brendan Gregg (2014) **HIGH**
- [Introducing strace --seccomp-bpf](#) — Paul Chaignon (2019) **HIGH**
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange (2018-) **MEDIUM**
- [A 10-minute guide to the Linux ABI](#) — Alison Chaiken, opensource.com (2022) **HIGH**
- [A Deep Dive Into Malicious Direct Syscall Detection](#) — Or Chechik and Ofir Ozer, Palo Alto Networks (2024) **HIGH**
- [Overload Content by Article](#) — ACCU — ACCU (Association of C and C++ Users) **HIGH**
- [Windows System Calls & Native API: A Guide to Assembly Calling Conventions Explained](#) — linuxvox.com **LOW**
- [mandiant/STrace — A DTrace on Windows Reimplementation](#) — Mandiant **MEDIUM**

12.13 V24 — PostgreSQL's fsync() error-reporting failure ("fsyncgate")

This note establishes that Linux's `fsync()` reports a writeback error to the calling process only once: a failed writeback marks the dirty page clean and discards it, so the flag that signalled the error is consumed by the very act of reporting it. A retry that follows sees a clean flag and no dirty pages, and returns success — truthfully, by its own semantics, even though the data never reached disk. PostgreSQL's checkpoint, which typically opens a file only to flush it rather than holding descriptors open, hit exactly this window: it retried a failed checkpoint, saw success, and recycled the WAL that was its only remaining record of the lost writes. The response converged across three inde-

pendent storage engines during 2018. PostgreSQL, InnoDB, and WiredTiger each replaced `fsync` retry with an unconditional crash, discarding the untrustworthy in-memory model of persistent state and rebuilding it from the write-ahead log instead. WiredTiger's fix is the most direct evidence of how the industry treated the finding: its engineers quoted this very LWN article in the source comment as its justification for replacing retry with panic, a rare case of a code comment citing its own journalism.

The note is explicit about what remains unresolved. Whether POSIX actually settles the dispute in PostgreSQL's favour is contested — POSIX specifies the success case but says outstanding I/O is "not guaranteed to have been completed" after a failure, so neither side can be shown to have violated the standard. Whether the kernel behaviour is a defect is likewise contested: kernel developers defended it as coherent, if undocumented, given the need to reclaim pages from persistently failing devices. The note also flags that the fix is narrower than it looks — Linux's later per-descriptor error cursors made *reporting* reliable but did not make the *data* recoverable, and PostgreSQL's own commit message names a residual inode-eviction race as still open. A peer-reviewed fault-injection study cited in the note found that even after these 2018 fixes, no application-level failure-handling strategy is fully sufficient. The note treats all of this as an unsettled specification vacuum rather than a solved problem, and does not claim the crash-and-replay pattern eliminates the risk, only that it converts silent data loss into a loud, recoverable failure.

Referenced sources

- [PostgreSQL's `fsync\(\)` surprise](#) — Jonathan Corbet / LWN.net (2018) **HIGH**
- [Fsyncgate: errors on `fsync` are unrecoverable](#) — Dan Luu (archive of the `pgsql-hackers` thread, 2018) **HIGH**
- [PostgreSQL commit 9ccdd7f66e3324d2b6d3dec282cfa9ff084083f1](#) — "PANIC on `fsync()` failure" — Craig Ringer / Thomas Munro, PostgreSQL Global Development Group (2018) **HIGH**
- [PostgreSQL documentation — `data\_sync\_retry`](#) — PostgreSQL Global Development Group **HIGH**
- [MySQL Bug #90296](#) — "Hard error should be report when `fsync()` return EIO" — Oracle / MySQL bug tracker (2018) **HIGH**
- [mysql-server commit 8590c8e12a3374eecb547359750a9d2a128fa6a](#) — Bug#27805553 — Oracle / MySQL Server (2018) **HIGH**
- [WiredTiger commit ae8bccce3d8a8248afa0e4e0cf67674a43dede96](#) — WT-4045 "Don't retry `fsync` calls after EIO failure" — Keith Bostic / Alex Gorrod, MongoDB WiredTiger (2018) **HIGH**
- [Can Applications Recover from `fsync` Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20 (2020) **HIGH**

- fs: introduce new writeback error reporting and convert existing API as a wrapper around it — Jeff Layton, linux-fsdevel (2017) **HIGH**
- PATCH v2 3/5 vfs: track per-sb writeback errors and report them to syncfs — Jeff Layton, LKML (2018) **HIGH**
- Linux Filesystems API summary — `filemap_set_wb_err` / `filemap_check_wb_err` — The Linux Kernel documentation **HIGH**
- PostgreSQL 18 and beyond: From AIO to Direct IO? — Cybertec PostgreSQL International (2025) **MEDIUM**
- debug_io_direct — pgPedia — pgPedia, a PostgreSQL Encyclopedia **MEDIUM**
- All Your GUCs in a Row: data_sync_retry — The Build (PostgreSQL Experts) **MEDIUM**
- fsync — POSIX Programmer's Manual — IEEE Std 1003.1 / The Open Group Base Specifications **HIGH**

12.14 V25 — The Trade Between Tracing Cost and Structural Blind Spot

This note argues that every mechanism for observing a running program — from `strace`'s kernel-syscall interception to eBPF's in-kernel probes to Windows' ETW — makes one indivisible choice: where it attaches to the system. That attachment point simultaneously fixes what the tracer costs and what it structurally cannot see; the two are not independent, they are the same decision read from two sides. `strace`, which stops the traced process twice per system call and round-trips through a separate tracer process, is measured at up to 442x slowdown on syscall-saturated workloads and around 3x on a realistic compile. The note singles out `strace`'s own optimized path as the sharpest illustration of the thesis: even after `--seccomp-bpf` cuts overhead from roughly 37x down to about 1.5x, the feature's own author disqualifies it outright — "strace --seccomp-bpf still stops at each syscall of interest and should therefore not be used on production systems!" — because the mechanism still pauses execution at each observed event; the optimization reduces cost without removing either the perturbation or the blind spot. The note contrasts this with mechanisms that attach elsewhere — eBPF's nanosecond-scale in-kernel probes, Windows' provider-based ETW, DynamoRIO's code-cache instrumentation, and record-and-replay via `rr` — each trading a different cost for a different, mechanism-specific inability to see some class of behavior.

The note is explicit about what it does not claim. It refuses to rank the mechanisms on one scale, arguing that a whole-workload multiplier (`strace`'s 442x) and a per-event nanosecond cost (eBPF's 35ns) measure different things, so arithmetic across them would be a mistake even though an order-of-magnitude comparison is defensible. It

flags NtTrace's overhead as argued from mechanism rather than measured, and marks the Windows-side production-safety evidence as leaning more heavily on vendor sourcing than the independently benchmarked Linux side. It records as genuinely unsettled — not resolved — whether "production-safe" tracing means merely not destabilizing a live system (Brendan Gregg's reading) or cheap enough to run continuously at fleet scale (Dan Luu's dissenting reading), treating both as defensible. It also notes that a blind spot is not always permanent — `io_uring`'s invisibility to `strace` prompted a move to a different mechanism rather than a faster version of the old one — so the trade shifts location under engineering pressure without disappearing.

Referenced sources

- [strace Wow Much Syscall](#) — Brendan Gregg (2014) **HIGH**
- [Ptrace, Utrace, Uprobes: Lightweight, Dynamic Tracing of User Apps](#) — Keniston, Panchamukhi, Hiramatsu, Ottawa Linux Symposium (2007) **HIGH**
- [Introducing strace --seccomp-bpf](#) — Paul Chaignon (2019) **HIGH**
- [strace: fight for performance \(FOSDEM 2020 slides\)](#) — strace maintainers, FOSDEM (2020) **HIGH**
- [Sysdig vs DTrace vs Strace: A technical discussion](#) — Sysdig (n.d.) **MEDIUM**
- [NtTrace — Native API tracing for Windows](#) — Roger Orr (n.d.) **HIGH**
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygodzki (n.d.) **MEDIUM**
- [Transparent Dynamic Instrumentation](#) — Derek Bruening, Qin Zhao, Saman Amarasinghe, VEE (2012) **HIGH**
- [DynamoRIO Release Notes](#) — DynamoRIO project (n.d.) **HIGH**
- [Dynamic binary instrumentation \(DBI\) with DynamoRio](#) — Cisco Talos Intelligence (2025) **MEDIUM**
- [Tracing a Subset of Execution](#) — DynamoRIO project (n.d.) **HIGH**
- [DTrace for Linux 2016](#) — Brendan Gregg (2016) **HIGH**
- [Benchmarking Performance Overhead of DTrace on FreeBSD and eBPF on Linux](#) — Mateusz Piotrowski, AsiaBSDCon (2024) **HIGH**
- [ebpf_exporter eBPF overhead benchmark](#) — Cloudflare (n.d.) **HIGH**
- [Pitfalls of relying on eBPF for security monitoring \(and some solutions\)](#) — Artem Dinaburg, Trail of Bits (2023) **HIGH**
- [eBPF Hookpoints Gotchas \(FOSDEM 2026 slides\)](#) — FOSDEM (2026) **HIGH**
- [Instrumenting Your Code with ETW](#) — Microsoft (n.d.) **HIGH**
- [Event Tracing for Windows \(ETW\)](#) — Ryan Myers, Microsoft (archived blog, n.d.) **MEDIUM**

- [Performance Analysis with ETW: Event Tracing for Windows](#) — Atomic Object (n.d.) **MEDIUM**
- [Heisenbug](#) — Wikipedia (2026) **MEDIUM**
- [Engineering Record And Replay For Deployability](#) — O'Callahan, Jones, Froyd, Huey, Noll, Partush, USENIX ATC (2017) **HIGH**
- [rr: lightweight recording & deterministic debugging](#) — rr project (2024) **HIGH**
- [Sampling v. tracing](#) — Dan Luu (n.d.) **HIGH**
- [eBPF monitoring overhead: measure and reduce it](#) — groundcover (n.d.) **MEDIUM**
- [strace of io_uring events? \(strace-devel mailing list\)](#) — strace-devel (2020) **HIGH**
- [strace\(1\)](#) — Linux manual page — man7.org (n.d.) **HIGH**
- [DTrace pid Provider Overhead](#) — Brendan Gregg (2011) **HIGH**
- [Portable, Low-Overhead Observability for io_uring](#) — arXiv preprint (2026) **MEDIUM**
- [io_uring War Stories](#) — kernel-internals.org (n.d.) **MEDIUM**
- [Transparently Integrating Debugging and Dynamic Binary Instrumentation](#) — Rensselaer Polytechnic Institute (n.d.) **MEDIUM**
- [Event Tracing for Windows \(ETW\) — architecture and providers](#) — Salucci (n.d.) **MEDIUM**
- [Bypassing ETW For Fun and Profit](#) — White Knight Labs (2021) **MEDIUM**
- [Support io_uring](#) — rr issue #2613 — Robert O'Callahan, rr project (2020) **MEDIUM**
- [Sampling vs deterministic profilers: which should I use?](#) — Python Developer Tooling Handbook (n.d.) **MEDIUM**

12.15 V28 — The socket as a kernel object and which layer binds its contract

This note argues that a socket is a kernel object reached through a per-process handle, and that Linux and Windows diverge only in which layer makes that handle's contract binding, not in whether the kernel treats the object as a file. On Linux, `socket()` allocates a `struct socket` and binds it to a `struct file` on the `sockfs` pseudo-filesystem, with `f_op` set to `socket_file_ops`, so `read`, `write`, `close`, `fstat` and `select` dispatch through the ordinary VFS table into the network stack — a claim the note backs with the kernel source (`net/socket.c`), the `socket(2)` man page, and a reproduced experiment on Linux 6.8.0 showing `fstat` reporting `S_ISSOCK` and `/proc/self/fd/N` resolving to `socket:[<inode>]`. On Windows, `socket()` returns an opaque `SOCKET`, and Microsoft's own documentation states plainly that "a socket descriptor is not a file descriptor and must be used with the Winsock functions" — yet underneath Winsock,

the Ancillary Function Driver (`afd.sys`) does hold the socket as a file object on `\Device\Afd`, which is exactly what lets `ReadFile`, `WriteFile` and I/O completion ports work on it at all. The note treats two apparently conflicting Microsoft Learn pages — Socket Data Type versus Socket Handles — as reconcilable by layer rather than contradictory: the *type* is not a CRT file descriptor, while the *handle* is a kernel file handle Microsoft simply declines to promise support for outside Winsock calls.

The note is explicit about what it does not claim. It does not argue that Windows fails to model sockets as files at the kernel level — it corrects that exact misreading against its own starting brief. It flags the reconciliation of the two Microsoft Learn pages as [inference], not a fact either page states outright, and labels the AFD IOCTL call-shape claims [reported] rather than independently reproduced, since only the Linux side was verified experimentally in this environment. It treats Windows's later `AF_UNIX` support as a partial convergence: the address family matches Linux, but `SOCK_DGRAM`, `SOCK_SEQPACKET`, ancillary data (including passing file descriptors), and `socketpair` remain unsupported, so it declines to read this as closing the object-model gap. Finally, it flags the "readiness versus completion" shorthand between `epoll` and IOCP as aging given `io_uring`'s newer networking surface, without asserting the asymmetry has closed.

Referenced sources

- `socket(2)` — Linux manual page, Michael Kerrisk / Linux man-pages project — <https://man7.org/linux/man-pages/man2/socket.2.html> — HIGH
- `socket(7)` — Linux manual page, Michael Kerrisk / Linux man-pages project — <https://man7.org/linux/man-pages/man7/socket.7.html> — HIGH
- `linux/net/socket.c`, Linux kernel source tree — <https://github.com/torvalds/linux/blob/master/net/socket.c> — HIGH
- Socket Data Type — Win32 apps, Microsoft — <https://learn.microsoft.com/en-us/windows/win32/winsock/socket-data-type-2> — HIGH
- Socket Handles — Win32 apps, Microsoft — <https://learn.microsoft.com/en-us/windows/win32/winsock/socket-handles-2> — HIGH
- Error Codes - `errno`, `h_errno` and `WSAGetLastError` — Win32 apps, Microsoft — <https://learn.microsoft.com/en-us/windows/win32/winsock/error-codes-errno-h-errno-and-wsagetlasterror-2> — HIGH
- `WSAStartup` function (`winsock.h`), Microsoft — <https://learn.microsoft.com/en-us/windows/win32/api/winsock/nf-winsock-wsastartup> — HIGH
- `closesocket` function (`winsock.h`), Microsoft — <https://learn.microsoft.com/en-us/windows/win32/api/winsock/nf-winsock-closesocket> — HIGH

- CloseHandle function (handleapi.h), Microsoft — <https://learn.microsoft.com/en-us/windows/win32/api/handleapi/nf-handleapi-closehandle> — HIGH
- Select, fd_set, and FD_XXX Macros — Win32 apps, Microsoft — <https://learn.microsoft.com/en-us/windows/win32/winsock/select-and-fd---2> — HIGH
- socket — Low-level networking interface (Python documentation), Python Software Foundation — <https://docs.python.org/3/library/socket.html> — HIGH
- select — Waiting for I/O completion (Python documentation), Python Software Foundation — <https://docs.python.org/3/library/select.html> — HIGH
- AF_UNIX comes to Windows, Microsoft, Windows Command Line blog (2017) — https://devblogs.microsoft.com/commandline/af_unix-comes-to-windows/ — HIGH
- Improving AFD Socket Visibility for Windows Forensics & Troubleshooting, diversenok (2025) — <https://diversenok.github.io/2025/05/15/AFD-sockets.html> — MEDIUM
- 12 Days of HaXmas: Does it Blend Like a Duck? (Failing Portably), Rapid7 (2014) — <https://www.rapid7.com/blog/post/2014/12/29/12-days-of-haxmas-failing-portably/> — MEDIUM
- An Introductory 4.4BSD Interprocess Communication Tutorial, Computer Systems Research Group, UC Berkeley — <https://docs-archive.freebsd.org/44doc/psd/20.ipctut/paper.pdf> — HIGH
- An Advanced 4.3BSD Interprocess Communication Tutorial, Samuel J. Leffler et al., UC Berkeley — https://sites.units.it/mumolo/reti_netuno/bsdsocket.pdf — HIGH
- 4.4BSD Networking Implementation Notes, Computer Systems Research Group, UC Berkeley — <https://docs-archive.freebsd.org/44doc/smm/18.net/paper.pdf> — HIGH
- rogerorr/NtTrace, Roger Orr — <https://github.com/rogerorr/NtTrace> — HIGH
- Porting Socket Applications to Winsock — Win32 apps, Microsoft — <https://learn.microsoft.com/en-us/windows/win32/winsock/porting-socket-applications-to-winsock> — HIGH
- Under the Hood of AFD.sys Part 2: TCP handshake, Mateusz Lewczak (2025) — <https://leftarcode.com/posts/afd-reverse-engineering-part2/> — MEDIUM
- cpython/Modules/socketmodule.h, Python Software Foundation — <https://github.com/python/cpython/blob/main/Modules/socketmodule.h> — HIGH
- Closing non-blocking socket causes leak, Microsoft — <https://learn.microsoft.com/en-us/troubleshoot/windows/win32/close-non-blocked-socket-memory-leak> — HIGH
- vfs: mark pipes and sockets as stream-like file descriptors, Linus Torvalds, Linux kernel (2019) — <https://github.com/torvalds/linux/commit/d8e46ec-c17b4444e9a3e148a9748c4828c6328c> — HIGH

- `io_uring` zero copy Rx — The Linux Kernel documentation, The Linux Kernel Organization — <https://docs.kernel.org/networking/iou-zcrx.html> — **HIGH**
- Winsock Programmer's FAQ: General Winsock Information, Warren Young — <https://tangentsoft.com/wskfaq/general.html> — **MEDIUM**

12.16 V29 — Sockets as Kernel Objects, Published Differently by Linux and Windows

The note argues that a socket is a kernel object reached through a per-process handle, and that Linux and Windows differ only in which layer makes that handle binding — not in whether the kernel models the object as a file. On Linux, `socket()` allocates a `struct file` bound to `sockfs`, dispatched through the same `socket_file_ops` table that `read(2)`, `write(2)` and `close(2)` use for any other descriptor; the note reproduces this directly, tracing a live TCP connection through `/proc/self/fd` and matching the reported inode against `fstat`'s `S_ISSOCK` output on a Linux 6.8.0 system. On Windows, `socket()` returns an opaque `SOCKET`, and Microsoft's own documentation states that it "is not a file descriptor and must be used with the Winsock functions" — but underneath, the NT kernel holds a genuine file object on `\Device\Afd`, which is exactly what lets `ReadFile`, `WriteFile` and I/O completion ports work on it. The three practical consequences the note draws from this — what else can operate on the handle, how it is released (`close` versus `closesocket`), and how failure is reported (`errno` versus `WSAGetLastError`) — are each backed by first-party vendor or kernel-source citations, and the note frames a real LibreSSL porting failure as the concrete cost of missing the distinction.

The note is explicit about what it does not claim. It does not assert that Windows lacks a file-object model for sockets — a correction the author says departs deliberately from the brief that originally scoped the note around the Win32-level claim alone. It does not treat `select()`'s Windows behavior as a bug; it presents the array-of-sockets `fd_set` design as a documented, deliberate divergence. Two apparently contradictory Microsoft Learn pages — one saying a socket descriptor is not a file descriptor, the other that a socket handle can optionally be a file handle — are reconciled by distinguishing type from handle, but the note flags that reconciliation as its own reading rather than something either source states outright. Likewise, the claim that Windows `AF_UNIX`'s omission of `SCM_RIGHTS` is "the single most file-descriptor-dependent thing Unix does with sockets" is marked as inference, not sourced fact. The note treats the narrowing gap between `io_uring` and IOCP as real but unsettled over time, flagging its own shorthand as likely to age.

Referenced sources

- [socket\(2\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project **HIGH**
- [socket\(7\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project **HIGH**
- [linux/net/socket.c](#) — Linux kernel source tree **HIGH**
- [Socket Data Type — Win32 apps](#) — Microsoft **HIGH**
- [Socket Handles — Win32 apps](#) — Microsoft **HIGH**
- [Error Codes - errno, h_errno and WSAGetLastError](#) — Win32 apps — Microsoft **HIGH**
- [WSAStartup function \(winsock.h\)](#) — Microsoft **HIGH**
- [closesocket function \(winsock.h\)](#) — Microsoft **HIGH**
- [CloseHandle function \(handleapi.h\)](#) — Microsoft **HIGH**
- [Select, fd_set, and FD_XXX Macros](#) — Win32 apps — Microsoft **HIGH**
- [socket — Low-level networking interface \(Python documentation\)](#) — Python Software Foundation **HIGH**
- [select — Waiting for I/O completion \(Python documentation\)](#) — Python Software Foundation **HIGH**
- [AF_UNIX comes to Windows](#) — Microsoft, Windows Command Line blog (2017) **HIGH**
- [Improving AFD Socket Visibility for Windows Forensics & Troubleshooting](#) — diversenok (2025) **MEDIUM**
- [12 Days of HaXmas: Does it Blend Like a Duck? \(Failing Portably\)](#) — Rapid7 (2014) **MEDIUM**
- [An Introductory 4.4BSD Interprocess Communication Tutorial](#) — Computer Systems Research Group, UC Berkeley **HIGH**
- [An Advanced 4.3BSD Interprocess Communication Tutorial](#) — Samuel J. Leffler et al., UC Berkeley **HIGH**
- [4.4BSD Networking Implementation Notes](#) — Computer Systems Research Group, UC Berkeley **HIGH**
- [rogerorr/NtTrace](#) — Roger Orr **HIGH**
- [Porting Socket Applications to Winsock](#) — Win32 apps — Microsoft **HIGH**
- [Under the Hood of AFD.sys Part 2: TCP handshake](#) — Mateusz Lewczak (2025) **MEDIUM**
- [cpython/Modules/socketmodule.h](#) — Python Software Foundation **HIGH**
- [Closing non-blocking socket causes leak](#) — Microsoft **HIGH**
- [vfs: mark pipes and sockets as stream-like file descriptors](#) — Linus Torvalds, Linux kernel (2019) **HIGH**

- [io_uring zero copy Rx — The Linux Kernel documentation](#) — The Linux Kernel Organization **HIGH**
- [Winsock Programmer's FAQ: General Winsock Information](#) — Warren Young **MEDIUM**

12.17 V30 — File I/O Buffering and Durability Guarantees

File I/O and Durability establishes that durability is not a property of a single write call but of a sequence of four independent buffer boundaries — a language-runtime buffer, a libc/kernel copy, the kernel's page cache, and the storage device's own cache — each with its own flush trigger and its own cost to cross. A successful `write()` only confirms that the kernel has taken custody of the bytes in its page cache; it says nothing about whether they reached the physical medium. `fsync()` on Linux and `FlushFileBuffers()` on Windows are the calls meant to close that gap, and the note grounds this in the 2018 `fsync-gate` incident: PostgreSQL's checkpointer retried a failed `fsync()` believing the retry would surface any lingering failure, but Linux's page cache marks the failed pages clean and discards their contents after a single error report, so the retry silently reported success while the data stayed lost — a bug that went undetected for roughly two decades. The note frames this as a specification gap rather than a bug on either side: POSIX defines what a successful `fsync()` guarantees, but says nothing about state after a failed one, so the kernel developers and PostgreSQL developers read that silence in two different, internally coherent ways. PostgreSQL, InnoDB, and WiredTiger converged independently on treating a failed `fsync()` as unrecoverable and crashing to replay from a write-ahead log rather than retrying the flush.

It does not claim Windows shares or avoids Linux's non-retryable failure class — it states plainly that nobody has publicly characterized Windows' post-failure behavior and Microsoft has not documented it, noting that even the PostgreSQL project's own cross-platform failure matrix carries no Windows row. It does not claim the four-layer model is complete, flagging network filesystems, memory-mapped I/O, and copy-on-write filesystems as cases that reshape or add layers the model does not cover. It separately marks as speculative, rather than settled, the claim that no Windows tracing facility observes device-level flush commands as cleanly as Linux's `blktrace`, and marks as inferred, rather than documented, the mapping from Python's `os.fsync()` to the Windows CRT's `_commit()` call. On the PANIC-on-failure recommendation, the note states an explicit limit: that guidance was calibrated to 2018-era local disks, where an I/O error meant hardware that was not coming back, and it does not claim the same response is correct for cloud block storage, where an I/O error is often transient and self-clearing.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026) **MEDIUM**
- [fsync\(2\)](#) — Linux manual page — Linux man-pages project **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017) **HIGH**
- [io](#) — Core tools for working with streams — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn **HIGH**
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022) **MEDIUM**
- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla **HIGH**
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode **MEDIUM**
- [Page Cache Writeback](#) — Linux Kernel Internals **MEDIUM**
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn **HIGH**
- [File Caching](#) — Microsoft Learn **HIGH**
- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al. **HIGH**
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026) **LOW**
- [Life of a Write](#) — Linux Kernel Internals **MEDIUM**
- [The errseq_t datatype](#) — The Linux Kernel documentation — Linux kernel documentation **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018) **HIGH**
- [Fsync Errors](#) — PostgreSQL wiki **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020) **HIGH**
- [CreateFileW function — FILE_FLAG_WRITE_THROUGH and FILE_FLAG_NO_BUFFERING semantics](#) — Microsoft Learn **HIGH**
- [File Buffering](#) — Microsoft Learn **HIGH**
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — Dilip Naik (2012) **MEDIUM**

- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News (2019) **LOW**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow (2019) **MEDIUM**
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation **HIGH**
- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow **MEDIUM**
- [DTrace on Windows](#) — Microsoft Learn **HIGH**
- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn **HIGH**
- [Write-Ahead Logging \(SQLite\)](#) — SQLite **HIGH**

12.18 V33 — File I/O durability across the write-to-device path

This note argues that a program's write to a file is durable only after it has crossed four independent buffering boundaries, not one: the language-runtime buffer, the kernel's acceptance of the `write()` call, the kernel's page-cache writeback to the storage device, and the device's own onboard cache before data reaches the physical medium. Each boundary has its own flush primitive and its own way of failing, and a successful `write()` only establishes that the first of the four accepted the bytes — a statement about custody, not persistence. The evidence spans both platforms at equal depth: the Linux `fsync()` man page's documented guarantee; the 2018 PostgreSQL `fsync-error` incident, where the kernel's `errseq_t` mechanism reports a writeback failure to only the next caller and then discards the failed pages, a design later confirmed by the kernel documentation itself and by participants in the original mailing-list thread; a 2020 USENIX fault-injection study of five storage engines finding none had a sufficient recovery strategy; and the matching Windows documentation for `FlushFileBuffers`, the `CreateFileW` write-through and no-buffering flags, and the Cache Manager's lazy-writer cadence.

The note is explicit about what it does not claim. It does not assert that Windows shares Linux's non-retryable failure class — it states that nobody has publicly characterized Windows' post-failure behavior and that Microsoft has not documented it, so the defensible position is that Windows' contract is unspecified rather than equivalent to or safer than Linux's hostile-but-documented one. It marks as speculative the claim that no Windows tracing tool exposes device-level flush commands as cleanly as Linux's `blktrace`, and as inference the mapping from Python's `os.fsync` to the Windows CRT's

`_commit()`, since no source states that chain directly. It also calls the four-layer model itself a teaching simplification, noting that network filesystems, memory-mapped I/O, and copy-on-write filesystems each add or reshape layers the table does not capture.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026) **MEDIUM**
- [fsync\(2\)](#) — Linux manual page — Linux man-pages project **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017) **HIGH**
- [io](#) — Core tools for working with streams — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn **HIGH**
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022) **MEDIUM**
- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla **HIGH**
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode **MEDIUM**
- [Page Cache Writeback](#) — Linux Kernel Internals **MEDIUM**
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn **HIGH**
- [File Caching](#) — Microsoft Learn **HIGH**
- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al. **HIGH**
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026) **LOW**
- [Life of a Write](#) — Linux Kernel Internals **MEDIUM**
- [The errseq_t datatype](#) — The Linux Kernel documentation — Linux kernel documentation **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018) **HIGH**
- [Fsync Errors](#) — PostgreSQL wiki **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020) **HIGH**
- [CreateFileW function](#) — `FILE_FLAG_WRITE_THROUGH` and `FILE_FLAG_NO_BUFFERING` semantics — Microsoft Learn **HIGH**
- [File Buffering](#) — Microsoft Learn **HIGH**

- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — Dilip Naik (2012) **MEDIUM**
- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News (2019) **LOW**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow (2019) **MEDIUM**
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation **HIGH**
- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow **MEDIUM**
- [DTrace on Windows](#) — Microsoft Learn **HIGH**
- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn **HIGH**
- [Write-Ahead Logging \(SQLite\)](#) — SQLite **HIGH**

12.19 V36 — The differing layer at which Windows and Linux stabilize the syscall boundary

The note's central correction is that Linux, not Windows, is the outlier: Linux alone treats the numbered syscall trap as a public, version-stable contract, while Windows, FreeBSD, OpenBSD, NetBSD, Solaris/illumos and macOS all decline that guarantee at the kernel-transition layer. The quoted span settles a contested sub-claim inside that correction — whether FreeBSD genuinely belongs on the no-guarantee side. The note surfaces two disagreeing primary sources, found through its cross-engine search process. FreeBSD's own vendor wiki states flatly that "no compatibility for API and ABI is guaranteed from one to the next major release." A FreeBSD core-team contributor pushes back, arguing the syscall ABI is stable in practice, because the project's default kernel build has shipped compiled-in `COMPAT_FREEBSD*` compatibility layers reaching back to FreeBSD 4, spanning more than two decades. Both accounts agree on the underlying mechanism — versioned entries in `syscalls.master` selected by build-time COMPAT flags — and disagree only about whether a guarantee contingent on a kernel build option still counts as a guarantee. The quoted sentence is the note's own resolution: FreeBSD delivers strong ABI compatibility in practice through those compiled-in layers, while stopping short of a formal cross-major-release promise.

The note is explicit about what this nuance does not do. It does not overturn the headline correction: FreeBSD still publishes no Linux-style numbering guarantee, so it remains on the no-guarantee side of the comparison. Nor does it claim FreeBSD's practical stability is illusory, or treat FreeBSD as equivalent in instability to, say, OpenBSD's outright refusal of direct syscalls. The note grades the underlying claim "Contested,"

not "Verified," and explicitly declines to adjudicate between the vendor wiki and the practitioner account, since both describe the same mechanism and differ only in characterization. It does not extend this practice-versus-promise distinction to the other four systems on the outlier list, which it treats as more uniformly non-guaranteeing. Where the boundary between "compiled-in compatibility" and "no guarantee" properly lies is left unsettled by the note itself.

Referenced sources

- [ABI stable symbols — The Linux Kernel documentation](#) — *The Linux Kernel Organization* HIGH
- [j00ru/windows-syscalls — Windows System Call Tables](#) — *Mateusz "j00ru" Jurczyk* HIGH
- [Windows X86-64 System Call Table \(XP/2003/Vista/7/8/10/11 and Server\)](#) — *Mateusz "j00ru" Jurczyk* HIGH
- [NtQuerySystemInformation function \(winternl.h\)](#) — *Win32 apps* — *Microsoft* HIGH
- [Calling Internal APIs — Win32 apps](#) — *Microsoft* HIGH
- [arch/x86/entry/syscalls/syscall_64.tbl](#) — *Linux kernel source tree* HIGH
- [include/uapi/asm-generic/unistd.h](#) — *Linux kernel source tree* HIGH
- [Do other Unix-like kernels have stable syscall ABIs?](#) — *Unix & Linux Stack Exchange* MEDIUM
- [FreeBSD Vendor Information](#) — *The FreeBSD Project* HIGH
- [openbsd-tech: system call origin verification](#) — *OpenBSD project mailing list* HIGH
- [all: stop using non-libc system calls on OpenBSD \(golang/go#36435\)](#) — *The Go Project* HIGH
- [syscall\(2\)](#) — *NetBSD Manual Pages* — *The NetBSD Project* HIGH
- [port-solaris mailing list, November 2010: syscall number reservation](#) — *port-solaris list archive (Carnegie Mellon mirror)* HIGH
- [NtTrace](#) — *Native API tracing for Windows* — *Roger Orr* HIGH
- [strace](#) — *linux syscall tracer* — *The strace project* HIGH
- [DTrace on Windows](#) — *Windows drivers* — *Microsoft* HIGH
- [DTrace](#) — *Windows Commands* — *Microsoft* HIGH
- [Inside the Native API](#) — *Mark Russinovich, Sysinternals* HIGH
- [NT OS/2 Subsystem Design Rationale \(NT Design Workbook\)](#) — *Mark H. Lucovsky, Microsoft* HIGH
- [The Sad History of the Microsoft POSIX Subsystem](#) — *Brian Reiter (2010)* MEDIUM
- [Syscall ABI compatibility: Linux vs Windows/macOS](#) — *Mark Mossberg (2023)* MEDIUM

- [Never break userspace](#) — *LWN.net* **HIGH**
- [Updating Syscall Tables](#) — *SysWhispers4 documentation* — *SysWhispers4 project documentation* **LOW**
- [CRT-free in 2023: tips and tricks](#) — *discussion* — *Lobsters discussion thread* **MEDIUM**
- [Architecture of Windows NT](#) — *Wikipedia* **MEDIUM**
- [U.S. v. Microsoft Corporation](#) — [Information on the Settlement](#) — *U.S. Department of Justice, Antitrust Division* **HIGH**
- [Syscalls on Windows 11](#) — *hammertux* **MEDIUM**

12.20 V37 — The gap between a successful write and a durable one

This note argues that durability is not a property of a single write call but of a sequence of four independent flushes — runtime buffer to kernel, kernel buffer to device, and device cache to medium — each crossing one boundary and able to fail on its own. A successful `write()` (or Windows' `WriteFile`) only means the kernel accepted the bytes; that is custody, not persistence, and neither closing a file nor exiting a process forces the remaining crossings. The note grounds this in the 2018 PostgreSQL "fsyncgate" incident: when writeback failed, Linux reported the error once to the next `fsync()` call, then discarded both the failure record and the dirty pages, so a retrying checkpointer saw success on data never written. It treats this as a genuine contract disagreement — POSIX defines nothing about state after a failed `fsync` — resolved industry-wide by treating a failed `fsync` as unrecoverable and replaying from a write-ahead log instead. Extending to Windows, its central finding is that the same four layers exist under different names but carry no documented post-failure contract at all — worse than Linux's hostile-but-explicit one, because nothing undocumented can be designed against.

The note is explicit about limits. It does not claim Windows shares Linux's exact failure class — only that no one, including Microsoft and the PostgreSQL project's own per-platform matrix, has ever characterized it, which it treats as materially worse than either "safe" or "equally broken." It flags the four-layer model itself as a teaching simplification, noting that network filesystems, memory-mapped I/O, and copy-on-write filesystems reshape or add layers it does not cover, and it does not claim device flush commands are trustworthy — some drives acknowledge flushes they have not honored, unverifiable from software. It states the PANIC-on-failure response was calibrated for 2018-era local disks and flags as unsettled whether the same aggressiveness suits cloud block storage, where I/O errors are often transient. One further mapping — Python's `os.fsync` to Windows' `_commit()` — is marked as inference, not documented fact, and two related claims are named as still unwritten.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026) **MEDIUM**
- [fsync\(2\)](#) — Linux manual page — Linux man-pages project **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017) **HIGH**
- [io](#) — Core tools for working with streams — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn **HIGH**
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022) **MEDIUM**
- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla **HIGH**
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode **MEDIUM**
- [Page Cache Writeback](#) — Linux Kernel Internals **MEDIUM**
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn **HIGH**
- [File Caching](#) — Microsoft Learn **HIGH**
- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al. **HIGH**
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026) **LOW**
- [Life of a Write](#) — Linux Kernel Internals **MEDIUM**
- [The errseq_t datatype](#) — The Linux Kernel documentation — Linux kernel documentation **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018) **HIGH**
- [Fsync Errors](#) — PostgreSQL wiki **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020) **HIGH**
- [CreateFileW function — FILE_FLAG_WRITE_THROUGH and FILE_FLAG_NO_BUFFERING semantics](#) — Microsoft Learn **HIGH**
- [File Buffering](#) — Microsoft Learn **HIGH**
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — Dilip Naik (2012) **MEDIUM**

- Discussion: does FlushFileBuffers() on Windows also forget failed writes? / Files Are Hard thread — Hacker News (2019) **LOW**
- Is it safe to call the Win32 FlushFileBuffers() function again if it previously returned an error? — Stack Overflow (2019) **MEDIUM**
- os — Miscellaneous operating system interfaces — Python Software Foundation **HIGH**
- Monitoring certain system calls done by a process in Windows — Stack Overflow **MEDIUM**
- DTrace on Windows — Microsoft Learn **HIGH**
- FlushViewOfFile function (memoryapi.h) — Microsoft Learn **HIGH**
- Write-Ahead Logging (SQLite) — SQLite **HIGH**

12.21 V38 — The placement of the stable syscall boundary on Windows versus Linux

This note argues that Windows and Linux both promise application developers a stable interface to the kernel, but they draw that promise at different layers, and the difference is structural rather than a matter of one system being careless and the other careful. Linux fixes the contract at the syscall trap itself: the kernel's own ABI documentation states syscalls "are expected to never change and always be available," and the syscall number is part of that contract — `open` has been number 2 on x86-64 for the life of the architecture port, appended to but never renumbered. Windows fixes its contract one layer higher, at Win32. Below Win32 sits `ntdll`'s Native API, which Microsoft's own SDK documentation calls "internal to the operating system and subject to change from one release of Windows to another," and below that sits an undocumented, frequently renumbered System Service Number — measured directly from the canonical Windows syscall-table project across many builds, showing the same function's number both rising and falling, and even disagreeing between two concurrently shipping Windows SKUs. The practical corollary is a portability rule that is the same on both platforms but bites at different depths: on Windows, hardcoding a syscall number is a compatibility bug because the number is never contractual, while a Linux program calling the trap directly is using a genuinely supported interface, and the reason it normally goes through `libc` instead is ergonomic, not contractual.

The note is explicit that this does not make Windows the unusual case. It surveys FreeBSD, OpenBSD, NetBSD, Solaris/illumos and macOS and finds all of them decline a syscall-number stability guarantee the way Windows does, making Linux the outlier among Unix-family systems rather than the norm. It also bounds its own claim rather than overstating it: the "Linux never breaks userspace" framing is reported as retold

more absolutely than the kernel's own documentation states it, since that documentation hedges its guarantee to at least two years and allows changes for grave security problems. Two structural questions are left explicitly unsettled rather than resolved: whether the Win32/ntdll split reflects a deliberate design choice or a vestige of NT's abandoned multi-subsystem architecture, and whether ntdll counts as a stable interface at all given that tooling treats its named entry points as one while Microsoft's own documentation disclaims that status. A further claim — that Microsoft's disclosure of the Native API in `winternl.h` was compelled by the 2002 antitrust settlement — is graded only partially verified, since the temporal correlation is documented but no source draws the causal link directly.

Referenced sources

- [ABI stable symbols — The Linux Kernel documentation](#) — *The Linux Kernel Organization* **HIGH**
- [j00ru/windows-syscalls — Windows System Call Tables](#) — *Mateusz "j00ru" Jurczyk* **HIGH**
- [Windows X86-64 System Call Table \(XP/2003/Vista/7/8/10/11 and Server\)](#) — *Mateusz "j00ru" Jurczyk* **HIGH**
- [NtQuerySystemInformation function \(winternl.h\) — Win32 apps](#) — *Microsoft* **HIGH**
- [Calling Internal APIs — Win32 apps](#) — *Microsoft* **HIGH**
- [arch/x86/entry/syscalls/syscall_64.tbl](#) — *Linux kernel source tree* **HIGH**
- [include/uapi/asm-generic/unistd.h](#) — *Linux kernel source tree* **HIGH**
- [Do other Unix-like kernels have stable syscall ABIs?](#) — *Unix & Linux Stack Exchange* **MEDIUM**
- [FreeBSD Vendor Information](#) — *The FreeBSD Project* **HIGH**
- [openbsd-tech: system call origin verification](#) — *OpenBSD project mailing list* **HIGH**
- [all: stop using non-libc system calls on OpenBSD \(golang/go#36435\)](#) — *The Go Project* **HIGH**
- [syscall\(2\) — NetBSD Manual Pages](#) — *The NetBSD Project* **HIGH**
- [port-solaris mailing list, November 2010: syscall number reservation](#) — *port-solaris list archive (Carnegie Mellon mirror)* **HIGH**
- [NtTrace — Native API tracing for Windows](#) — *Roger Orr* **HIGH**
- [strace — linux syscall tracer](#) — *The strace project* **HIGH**
- [DTrace on Windows — Windows drivers](#) — *Microsoft* **HIGH**
- [DTrace — Windows Commands](#) — *Microsoft* **HIGH**
- [Inside the Native API](#) — *Mark Russinovich, Sysinternals* **HIGH**

- [NT OS/2 Subsystem Design Rationale \(NT Design Workbook\)](#) — *Mark H. Lucovsky, Microsoft* **HIGH**
- [The Sad History of the Microsoft POSIX Subsystem](#) — *Brian Reiter (2010)* **MEDIUM**
- [Syscall ABI compatibility: Linux vs Windows/macOS](#) — *Mark Mossberg (2023)* **MEDIUM**
- [Never break userspace](#) — *LWN.net* **HIGH**
- [Updating Syscall Tables](#) — *SysWhispers4 documentation* — *SysWhispers4 project documentation* **LOW**
- [CRT-free in 2023: tips and tricks — discussion](#) — *Lobsters discussion thread* **MEDIUM**
- [Architecture of Windows NT](#) — *Wikipedia* **MEDIUM**
- [U.S. v. Microsoft Corporation — Information on the Settlement](#) — *U.S. Department of Justice, Antitrust Division* **HIGH**
- [Syscalls on Windows 11](#) — *hammertux* **MEDIUM**

12.22 V39 — The Cost/Blind-Spot Trade-off in System Tracing Mechanisms

This note argues that every mechanism for observing a running program — from `strace`'s kernel-syscall interception to eBPF's in-kernel probes to Windows' ETW — makes the same underlying trade twice over: the point where a tracer attaches to the system simultaneously fixes what observation costs and what it structurally cannot see. It surveys six mechanisms (`ptrace` / `strace` , NtTrace-style breakpoint tracing, DynamoRIO's dynamic binary instrumentation, eBPF/DTrace, ETW, and record-and-replay via `rr`) and shows the cost side spans four orders of magnitude, from `strace`'s measured 442× slowdown on a syscall-saturated workload down to an eBPF tracepoint's 35-nanosecond per-event cost, while each mechanism's attachment point produces a distinct, non-shrinking blind spot: `strace` cannot see `io_uring` traffic that bypasses the syscall boundary, NtTrace cannot see driver-originated kernel calls, DynamoRIO cannot see code that escapes its code cache, and eBPF silently drops events under recursion or overload. The note also documents the observer effect directly — tracing overhead can alter a program's timing enough to manufacture bugs (DynamoRIO uncovering behavior that does not occur natively) or hide races (breakpoint serialization, single-stepping) — with record-and-replay tools like `rr` offered as the closest approach to escaping that trap, though only by serializing execution to one thread at a time.

The note is explicit that this is a claim about a ratio, not a universal law, and states two boundary conditions where the trade stops mattering: when event rates are low enough that cost drops out of the decision, and when a bug sits squarely inside a mech-

anism's field of view and is not timing-sensitive, in which case a trace can be trusted at face value. Several claims are flagged as contested or unmeasured rather than settled: whether in-kernel tracing (eBPF/DTrace) is genuinely cheap enough to run continuously at fleet scale is disputed between two named practitioners, and no published per-call overhead figure exists for NtTrace, so its place in the cost ordering is argued from mechanism rather than measurement. The note also cautions that its overhead figures are not commensurable across mechanisms — a whole-workload multiplier is not arithmetically comparable to a per-event absolute cost — and that ETW's own performance claims lean on Microsoft's self-reported, decades-old benchmark hardware rather than independent measurement. It leaves several questions explicitly open, including how often eBPF's undetectable event drops actually occur under a characterized production workload, and whether a mechanism can ever be both low-overhead and free of a timing blind spot at once.

Referenced sources

- [strace Wow Much Syscall](#) — Brendan Gregg (2014) **HIGH**
- [Ptrace, Utrace, Uprobes: Lightweight, Dynamic Tracing of User Apps](#) — Keniston, Panchamukhi, Hiramatsu, Ottawa Linux Symposium (2007) **HIGH**
- [Introducing strace --seccomp-bpf](#) — Paul Chaignon (2019) **HIGH**
- [strace: fight for performance \(FOSDEM 2020 slides\)](#) — strace maintainers, FOSDEM (2020) **HIGH**
- [Sysdig vs DTrace vs Strace: A technical discussion](#) — Sysdig (n.d.) **MEDIUM**
- [NtTrace — Native API tracing for Windows](#) — Roger Orr (n.d.) **HIGH**
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygodzki (n.d.) **MEDIUM**
- [Transparent Dynamic Instrumentation](#) — Derek Bruening, Qin Zhao, Saman Amarasinghe, VEE (2012) **HIGH**
- [DynamoRIO Release Notes](#) — DynamoRIO project (n.d.) **HIGH**
- [Dynamic binary instrumentation \(DBI\) with DynamoRio](#) — Cisco Talos Intelligence (2025) **MEDIUM**
- [Tracing a Subset of Execution](#) — DynamoRIO project (n.d.) **HIGH**
- [DTrace for Linux 2016](#) — Brendan Gregg (2016) **HIGH**
- [Benchmarking Performance Overhead of DTrace on FreeBSD and eBPF on Linux](#) — Mateusz Piotrowski, AsiaBSDCon (2024) **HIGH**
- [ebpf_exporter eBPF overhead benchmark](#) — Cloudflare (n.d.) **HIGH**
- [Pitfalls of relying on eBPF for security monitoring \(and some solutions\)](#) — Artem Dinaburg, Trail of Bits (2023) **HIGH**
- [eBPF Hookpoints Gotchas \(FOSDEM 2026 slides\)](#) — FOSDEM (2026) **HIGH**

- [Instrumenting Your Code with ETW](#) — Microsoft (n.d.) **HIGH**
- [Event Tracing for Windows \(ETW\)](#) — Ryan Myers, Microsoft (archived blog, n.d.) **MEDIUM**
- [Performance Analysis with ETW: Event Tracing for Windows](#) — Atomic Object (n.d.) **MEDIUM**
- [Heisenbug](#) — Wikipedia (2026) **MEDIUM**
- [Engineering Record And Replay For Deployability](#) — O'Callahan, Jones, Froyd, Huey, Noll, Partush, USENIX ATC (2017) **HIGH**
- [rr: lightweight recording & deterministic debugging](#) — rr project (2024) **HIGH**
- [Sampling v. tracing](#) — Dan Luu (n.d.) **HIGH**
- [eBPF monitoring overhead: measure and reduce it](#) — groundcover (n.d.) **MEDIUM**
- [strace of io_uring events? \(strace-devel mailing list\)](#) — strace-devel (2020) **HIGH**
- [strace\(1\)](#) — Linux manual page — man7.org (n.d.) **HIGH**
- [DTrace pid Provider Overhead](#) — Brendan Gregg (2011) **HIGH**
- [Portable, Low-Overhead Observability for io_uring](#) — arXiv preprint (2026) **MEDIUM**
- [io_uring War Stories](#) — kernel-internals.org (n.d.) **MEDIUM**
- [Transparently Integrating Debugging and Dynamic Binary Instrumentation](#) — Rensselaer Polytechnic Institute (n.d.) **MEDIUM**
- [Event Tracing for Windows \(ETW\) — architecture and providers](#) — Salucci (n.d.) **MEDIUM**
- [Bypassing ETW For Fun and Profit](#) — White Knight Labs (2021) **MEDIUM**
- [Support io_uring](#) — rr issue #2613 — Robert O'Callahan, rr project (2020) **MEDIUM**
- [Sampling vs deterministic profilers: which should I use?](#) — Python Developer Tooling Handbook (n.d.) **MEDIUM**

12.23 V40 — The fsync() error-reporting failure (fsyncgate)

This note summarizes LWN's 2018 reporting on fsyncgate: a Linux kernel behavior in which a failed writeback is reported to `fsync()` exactly once, after which the kernel marks the dirty page clean and discards it, so a retried `fsync()` call on the same file returns success even though the data was never durably written. It documents that PostgreSQL's checkpointer relied on retrying `fsync()` until it saw success, so a single missed error report let the database believe a checkpoint had completed, recycle its write-ahead log, and lose committed data with no error ever surfaced. The evidence is not limited to the original report: it is corroborated by a strace trace showing `fsync()`

returning `EIO` followed later by a successful `fsync()` on data that was never written, and by an independent fault-injection study across three filesystems finding that all three mark pages clean after a failed `fsync`, which defeats retry as a technique. Three separate storage engines — PostgreSQL, MySQL's InnoDB, and MongoDB's WiredTiger — responded within the same year by replacing retry-on-failure with an immediate crash, because a missed failure leaves an application's in-memory model of what is durable untrustworthy in a way no local repair can fix; in every case the fix was to discard that model and rebuild it from a durable log.

The note is explicit about what it does not claim. It does not say POSIX was violated: the standard specifies what a successful `fsync()` means but is silent on the state of the world after a failure, so the dispute between the PostgreSQL and kernel camps over whose behavior was wrong was never settled by either side conceding the point. It does not claim retry is now safe in general — the kernel-side fix that gave each file descriptor its own error cursor made the reporting reliable but did not make lost data recoverable, and a process that opens a file only to flush it, as PostgreSQL's checkpointer does, can still miss an error that occurred before the file was opened. And it does not present the story as finished: direct I/O is described as the long-term fix both sides endorsed, but the note states plainly that it still has not shipped as a production path years later, and it stops short of claiming the crash-based response makes buffered I/O durability correct — only that it turned a silent loss into a loud one.

Referenced sources

- [PostgreSQL's fsync\(\) surprise](#) — Jonathan Corbet / LWN.net (2018) **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (archive of the pgsql-hackers thread, 2018) **HIGH**
- [PostgreSQL commit 9ccdd7f66e3324d2b6d3dec282cfa9ff084083f1](#) — "PANIC on fsync() failure" — Craig Ringer / Thomas Munro, PostgreSQL Global Development Group (2018) **HIGH**
- [PostgreSQL documentation — data_sync_retry](#) — PostgreSQL Global Development Group **HIGH**
- [MySQL Bug #90296](#) — "Hard error should be report when fsync() return EIO" — Oracle / MySQL bug tracker (2018) **HIGH**
- [mysql-server commit 8590c8e12a3374eecb547359750a9d2a128fa6a](#) — [Bug#27805553](#) — Oracle / MySQL Server (2018) **HIGH**
- [WiredTiger commit ae8bccce3d8a8248afa0e4e0cf67674a43dede96](#) — WT-4045 "Don't retry fsync calls after EIO failure" — Keith Bostic / Alex Gorrod, MongoDB WiredTiger (2018) **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20 (2020) **HIGH**

- fs: introduce new writeback error reporting and convert existing API as a wrapper around it — Jeff Layton, linux-fsdevel (2017) **HIGH**
- PATCH v2 3/5 vfs: track per-sb writeback errors and report them to syncfs — Jeff Layton, LKML (2018) **HIGH**
- Linux Filesystems API summary — `filemap_set_wb_err` / `filemap_check_wb_err` — The Linux Kernel documentation **HIGH**
- PostgreSQL 18 and beyond: From AIO to Direct IO? — Cybertec PostgreSQL International (2025) **MEDIUM**
- debug_io_direct — pgPedia — pgPedia, a PostgreSQL Encyclopedia **MEDIUM**
- All Your GUCs in a Row: `data_sync_retry` — The Build (PostgreSQL Experts) **MEDIUM**
- fsync — POSIX Programmer's Manual — IEEE Std 1003.1 / The Open Group Base Specifications **HIGH**

12.24 V41 — PostgreSQL's fsync() failure-reporting bug ("fsyncgate")

This note establishes that Linux's fsync() reports a writeback error to the caller exactly once: when a dirty page fails to reach the device, the kernel marks that page clean and records the failure as a one-shot flag on the file, which the next fsync() call both reads and clears. A retry therefore finds a clean flag and no dirty pages, and returns success — not because the write succeeded, but because the record of its failure was already consumed and the data already evicted from the page cache. PostgreSQL's checkpointer relied on fsync() success as proof that WAL-covered data was durable, so a false-positive at checkpoint time let it recycle WAL segments that were the only remaining copy of the lost writes. The evidence is unusually strong: reported by the affected vendor, reproduced under strace against a fault-injecting block device in the same discussion thread, and independently confirmed under controlled fault injection across three filesystems. The note also documents the convergent industry response — PostgreSQL, MySQL/InnoDB, and MongoDB's WiredTiger each independently replaced retry with an immediate crash (PANIC/abort) during 2018, reasoning that once a write failure may have gone unreported, the in-memory model of what is durable can no longer be trusted, so the only sound recovery is to discard it and rebuild from the write-ahead log.

The note is explicit about what remains unresolved. It does not claim the kernel behaviour was a specification violation: POSIX constrains only the success case of fsync() and declines to guarantee anything about outstanding I/O after a failure, so the dispute between PostgreSQL and kernel developers was never settled by the standard and was instead routed around by every affected application changing its own behavior. Nor does it claim the 2018 fixes closed the problem — the PostgreSQL commit itself names a

residual race where an inode evicted under memory pressure between close and re-open can still hide an error, and a later peer-reviewed fault-injection study found that no application, PostgreSQL included, yet handles fsync failure correctly. The Linux 4.13 kernel-side fix is described as making the report reliable, not the underlying data recoverable. Finally, the note treats direct I/O as the long-term fix endorsed by kernel and database developers alike in 2018, but flags that it still had not landed as a production write path years later, leaving crash-and-replay a stopgap rather than a settled answer.

Referenced sources

- [PostgreSQL's fsync\(\) surprise](#) — Jonathan Corbet / LWN.net (2018) **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (archive of the postgresql-hackers thread, 2018) **HIGH**
- [PostgreSQL commit 9ccdd7f66e3324d2b6d3dec282cfa9ff084083f1](#) — "PANIC on fsync() failure" — Craig Ringer / Thomas Munro, PostgreSQL Global Development Group (2018) **HIGH**
- [PostgreSQL documentation](#) — `data_sync_retry` — PostgreSQL Global Development Group **HIGH**
- [MySQL Bug #90296](#) — "Hard error should be report when fsync() return EIO" — Oracle / MySQL bug tracker (2018) **HIGH**
- [mysql-server commit 8590c8e12a3374eecb547359750a9d2a128fa6a](#) — [Bug#27805553](#) — Oracle / MySQL Server (2018) **HIGH**
- [WiredTiger commit ae8bccce3d8a8248afa0e4e0cf67674a43dede96](#) — WT-4045 "Don't retry fsync calls after EIO failure" — Keith Bostic / Alex Gorrod, MongoDB WiredTiger (2018) **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20 (2020) **HIGH**
- [fs: introduce new writeback error reporting and convert existing API as a wrapper around it](#) — Jeff Layton, linux-fsdevel (2017) **HIGH**
- [PATCH v2 3/5 vfs: track per-sb writeback errors and report them to syncfs](#) — Jeff Layton, LKML (2018) **HIGH**
- [Linux Filesystems API summary](#) — `filemap_set_wb_err` / `filemap_check_wb_err` — The Linux Kernel documentation **HIGH**
- [PostgreSQL 18 and beyond: From AIO to Direct IO?](#) — Cybertec PostgreSQL International (2025) **MEDIUM**
- [debug_io_direct](#) — pgPedia — pgPedia, a PostgreSQL Encyclopedia **MEDIUM**
- [All Your GUCs in a Row: data_sync_retry](#) — The Build (PostgreSQL Experts) **MEDIUM**

- `fsync` — [POSIX Programmer's Manual](#) — IEEE Std 1003.1 / The Open Group Base Specifications HIGH

12.25 V43 — The layer at which Windows and Linux place the stable syscall boundary

This note argues that Linux and Windows both promise application developers a stable interface to the kernel, but nominate that promise at different layers, and that this placement — not a difference in rigor — explains the downstream differences in syscall numbering, tracing, and portability. Linux fixes the contract at the syscall trap itself: numbers are permanent per architecture, recorded in an in-tree table, and the kernel's own ABI documentation states they are expected never to change. Windows fixes the contract one layer higher, at Win32; the Native API in `ntdll` below it is named but explicitly disclaimed as subject to change, and the System Service Number beneath that is undocumented and demonstrably renumbered between builds — the note recomputed this directly from the canonical Windows syscall-table dataset, finding that most functions' numbers shift across versions and that concurrently shipping Windows SKUs disagree with each other. Anchored in the quoted span, the note's central corrective is that this makes Windows the normal case, not the anomaly: FreeBSD, OpenBSD, NetBSD, Solaris/illumos and macOS all decline Linux-style syscall-number stability guarantees, each in its own words, so Linux is the outlier among Unix-family systems rather than Windows. The structural explanation offered is that Windows and macOS control their own C library and can move the kernel boundary freely, while Linux's userland was historically developed separately, forcing the kernel boundary itself to become the stable interface.

The note is explicit about what it does not claim. It takes no position on intent: whether the Win32/`ntdll` split is deliberate design or a vestige of NT's abandoned multi-subsystem architecture, and whether `ntdll` counts as a genuine interface or merely a traced convention, are both left as disagreements over characterization rather than fact. It does not assert that Microsoft's `winternl.h` disclosure was compelled by the 2002 antitrust settlement — the temporal correlation is noted, but the causal link is called an unresolved inference. It qualifies "Windows has no syscall tracer" as now only partly true, since DTrace ships in Windows Server 2025 but remains opt-in. It also tempers its own Linux claim, noting that "Linux never breaks userspace" overstates what the kernel's documentation actually promises, which is scoped to at least two years and allows exceptions for security fixes. Finally, it flags FreeBSD's status as a genuine, unresolved disagreement between the project's own no-guarantee statement and a core contributor's account of a compiled-in compatibility layer.

Referenced sources

- [ABI stable symbols — The Linux Kernel documentation](#) — The Linux Kernel Organization **HIGH**
- [j00ru/windows-syscalls — Windows System Call Tables](#) — Mateusz "j00ru" Jurczyk **HIGH**
- [Windows X86-64 System Call Table \(XP/2003/Vista/7/8/10/11 and Server\)](#) — Mateusz "j00ru" Jurczyk **HIGH**
- [NtQuerySystemInformation function \(winternl.h\)](#) — Win32 apps — Microsoft **HIGH**
- [Calling Internal APIs — Win32 apps](#) — Microsoft **HIGH**
- [arch/x86/entry/syscalls/syscall_64.tbl](#) — Linux kernel source tree **HIGH**
- [include/uapi/asm-generic/unistd.h](#) — Linux kernel source tree **HIGH**
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange **MEDIUM**
- [FreeBSD Vendor Information](#) — The FreeBSD Project **HIGH**
- [openbsd-tech: system call origin verification](#) — OpenBSD project mailing list **HIGH**
- [all: stop using non-libc system calls on OpenBSD \(golang/go#36435\)](#) — The Go Project **HIGH**
- [syscall\(2\)](#) — NetBSD Manual Pages — The NetBSD Project **HIGH**
- [port-solaris mailing list, November 2010: syscall number reservation](#) — port-solaris list archive (Carnegie Mellon mirror) **HIGH**
- [NtTrace — Native API tracing for Windows](#) — Roger Orr **HIGH**
- [strace — linux syscall tracer](#) — The strace project **HIGH**
- [DTrace on Windows — Windows drivers](#) — Microsoft **HIGH**
- [DTrace — Windows Commands](#) — Microsoft **HIGH**
- [Inside the Native API](#) — Mark Russinovich, Sysinternals **HIGH**
- [NT OS/2 Subsystem Design Rationale \(NT Design Workbook\)](#) — Mark H. Lucovsky, Microsoft **HIGH**
- [The Sad History of the Microsoft POSIX Subsystem](#) — Brian Reiter (2010) **MEDIUM**
- [Syscall ABI compatibility: Linux vs Windows/macOS](#) — Mark Mossberg (2023) **MEDIUM**
- [Never break userspace](#) — LWN.net **HIGH**
- [Updating Syscall Tables — SysWhispers4 documentation](#) — SysWhispers4 project documentation **LOW**

- [CRT-free in 2023: tips and tricks — discussion](#) — Lobsters discussion thread

MEDIUM

- [Architecture of Windows NT](#) — Wikipedia **MEDIUM**
- [U.S. v. Microsoft Corporation — Information on the Settlement](#) — U.S. Department of Justice, Antitrust Division **HIGH**
- [Syscalls on Windows 11](#) — hammertux **MEDIUM**

12.26 V44 — File I/O durability across the buffering stack

This note establishes that durability is not a property of a single `write()` call but of a sequence of independent flushes across four buffering boundaries — the language runtime buffer, the libc/kernel boundary, the kernel page cache, and the device's own cache — each with its own trigger and its own failure mode. A successful `write()` on Linux or Windows confirms only that the kernel accepted the bytes into a cache: a custody guarantee, not a durability guarantee. Closing a file, or flushing a language-level buffer, does not cross the remaining boundaries either. The note grounds this in the Linux `fsync(2)` man page's flush-and-block wording, kernel documentation of the `errseq_t` mechanism, and the 2018 PostgreSQL "fsyncgate" incident: writeback failures were reported once to the next `fsync()` call and then silently cleared, so PostgreSQL's retry-on-`EIO` checkpointer lost data undetected for roughly two decades. The note frames this as a genuine contract disagreement between the kernel and PostgreSQL, not a bug on either side, since POSIX never specifies state after a failed `fsync`. From the absence of a Windows row in the PostgreSQL wiki's per-OS behaviour matrix, and unanswered developer questions on Hacker News and Stack Overflow, it further concludes that Windows has no documented equivalent contract at all — a position it judges worse than Linux's, because a hostile but documented contract is actionable and an undocumented one is not.

The note is explicit about its limits. It states the four-layer model as a teaching simplification that omits network filesystems, memory-mapped I/O, and copy-on-write filesystems, warning that a student who over-applies the table will misread NFS. It does not claim Windows shares Linux's non-retryable `fsync` failure behaviour: it states that nobody has publicly characterised this and Microsoft has not documented it, marking that conclusion as inference rather than verified fact. It does not claim any current application-level strategy is sufficient on Linux, citing a 2020 USENIX fault-injection study that found all five examined systems — including PostgreSQL, SQLite, and Redis — inadequate in some failure mode. It marks `fsync`'s meaningfulness on cloud storage as reported, not verified, and treats two related claims about `write()` and `fsync` failure semantics as unfinished, deferred to separate notes still in progress.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026) **MEDIUM**
- [fsync\(2\)](#) — Linux manual page — Linux man-pages project **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017) **HIGH**
- [io](#) — Core tools for working with streams — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn **HIGH**
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022) **MEDIUM**
- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla **HIGH**
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode **MEDIUM**
- [Page Cache Writeback](#) — Linux Kernel Internals **MEDIUM**
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn **HIGH**
- [File Caching](#) — Microsoft Learn **HIGH**
- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al. **HIGH**
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026) **LOW**
- [Life of a Write](#) — Linux Kernel Internals **MEDIUM**
- [The errseq_t datatype](#) — The Linux Kernel documentation — Linux kernel documentation **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018) **HIGH**
- [Fsync Errors](#) — PostgreSQL wiki **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020) **HIGH**
- [CreateFileW function — FILE_FLAG_WRITE_THROUGH and FILE_FLAG_NO_BUFFERING semantics](#) — Microsoft Learn **HIGH**
- [File Buffering](#) — Microsoft Learn **HIGH**
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — Dilip Naik (2012) **MEDIUM**

- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News (2019) **LOW**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow (2019) **MEDIUM**
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation **HIGH**
- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow **MEDIUM**
- [DTrace on Windows](#) — Microsoft Learn **HIGH**
- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn **HIGH**
- [Write-Ahead Logging \(SQLite\)](#) — SQLite **HIGH**

12.27 V45 — The Boundary Between an Accepted Write and a Durable One

The note argues that durability is a property of a sequence of independent flushes, not of a single write call, and names four boundaries between a program's write and the physical medium: a language-runtime buffer, a libc/kernel buffer, the kernel's page cache, and the storage device's own onboard cache — each with its own trigger and its own failure mode. A successful `write()`, it establishes, reports only that the kernel accepted the bytes — a statement about custody, not persistence — kept sharply distinct from visibility, the POSIX guarantee that a subsequent read will see the written bytes, which is page-cache coherence rather than durability. Its central evidence is the 2018 `fsync`-failure episode: after a writeback error, Linux reports the failure once to the next `fsync()` call and then discards the affected pages, marking them clean, so PostgreSQL's checkpoint — which retried on error, believing a later success meant all prior writes had landed — silently lost data for roughly two decades. The note frames this as a genuine disagreement between two internally coherent readings of a POSIX contract that never addressed the failure case, not a bug on either side, and traces the resulting convergence of PostgreSQL, InnoDB, and WiredTiger on PANIC-and-recover-from-write-ahead-log. Extending the same treatment to Windows, it finds — after checking the PostgreSQL wiki's cross-platform matrix, a Hacker News thread, and a Stack Overflow answer, none of which supply an answer — that Windows has no documented post-failure contract at all, which it argues is the worse position, since no correct response can be engineered against silence.

The note is explicit about where its model stops applying: network filesystems, memory-mapped I/O, and copy-on-write filesystems each reshape or add layers, and it warns that applying the four-layer table uncritically to those cases will mislead. It does not claim PANIC-on-failure is universally correct — the recommendation is calibrated

to 2018-era local disks, and it notes that on cloud block storage an I/O error is often transient, which changes the availability trade-off without changing the underlying correctness argument. Several claims are flagged as explicitly unsettled rather than asserted: whether any Windows tool exposes device-level flush commands as cleanly as Linux's `blktrace` is marked speculative; the chain from Python's `os.fsync` on Windows to `FlushFileBuffers` via the CRT's `_commit()` is marked inferred, not documented; and whether Windows exposes a knob comparable to `dirty_expire_centisecs` is listed as a sourcing gap rather than a settled negative.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium — **MEDIUM**
- [fsync\(2\)](#) — Linux manual page — Linux man-pages project — **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones — **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn — **HIGH**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs — **HIGH**
- [io](#) — Core tools for working with streams — Python Software Foundation — **HIGH**
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn — **HIGH**
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 — **MEDIUM**
- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla — **HIGH**
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode — **MEDIUM**
- [Page Cache Writeback](#) — Linux Kernel Internals — **MEDIUM**
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn — **HIGH**
- [File Caching](#) — Microsoft Learn — **HIGH**
- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al. — **HIGH**
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek — **LOW**
- [Life of a Write](#) — Linux Kernel Internals — **MEDIUM**
- [The errseq_t datatype](#) — The Linux Kernel documentation — Linux kernel documentation — **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants — **HIGH**
- [Fsync Errors](#) — PostgreSQL wiki — **HIGH**

- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC — **HIGH**
- [CreateFileW function — FILE_FLAG_WRITE_THROUGH and FILE_FLAG_NO_BUFFERING semantics](#) — Microsoft Learn — **HIGH**
- [File Buffering](#) — Microsoft Learn — **HIGH**
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — Dilip Naik — **MEDIUM**
- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News — **LOW**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow — **MEDIUM**
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation — **HIGH**
- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow — **MEDIUM**
- [DTrace on Windows](#) — Microsoft Learn — **HIGH**
- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn — **HIGH**
- [Write-Ahead Logging \(SQLite\)](#) — SQLite — **HIGH**

12.28 V46 — File I/O and durability

This note argues that durability is a property of a sequence of independent flushes, not of any single write call, and it sets out four boundaries — language-runtime buffer, kernel buffer, kernel cache, and device cache — each with its own trigger and its own way to fail silently. It establishes that a successful `write()` on Linux or `WriteFile()` on Windows only confirms that the kernel accepted the bytes into a cache; it says nothing about the medium, and page-cache visibility (reads see the write) is a distinct guarantee from durability (the write survives a power loss). The evidentiary core is the 2018 PostgreSQL fsync-failure incident: on Linux, a writeback error is reported to only the next `fsync()` call and then cleared, with the failed pages marked clean and discarded, so a retrying checkpointer can report success on data that never reached disk. The note treats this as a genuine contract disagreement between the kernel and PostgreSQL, not a bug on either side, documented in the kernel's own `errseq_t` design notes and the archived mailing-list thread. It extends the same analysis to Windows, concluding that Windows's `FlushFileBuffers` has no documented post-failure contract at all — a gap that public discussion (Hacker News, Stack Overflow) confirms went unanswered rather than merely unpublicized.

The note is explicit about what it does not claim. It does not conclude that Windows is safer or less safe than Linux after a flush failure — only that Linux's contract is documented and hostile while Windows's is undocumented, which it frames as the worse position for a system that must not lose data. It flags the current PANIC-on- EIO recommendation as calibrated for 2018-era local disks, noting that on cloud block storage the same error is often transient, which changes the cost of crashing without changing the correctness argument. Several claims are marked inline as inference or speculation rather than verified fact: the CRT `_commit()`-to-FlushFileBuffers mapping, and the claim that no Windows tool exposes device flush commands as cleanly as Linux's `blk-trace`. As a hub note, it also defers two atomic claims — on `write()`'s guarantee and on `fsync`'s non-retryability — to separate notes not yet written, and states outright that the four-layer model is itself a teaching simplification that breaks down for network filesystems and memory-mapped I/O.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026) **MEDIUM**
- [fsync\(2\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017) **HIGH**
- [io — Core tools for working with streams](#) — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn **HIGH**
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022) **MEDIUM**
- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla **HIGH**
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode **MEDIUM**
- [Page Cache Writeback](#) — Linux Kernel Internals **MEDIUM**
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn **HIGH**
- [File Caching](#) — Microsoft Learn **HIGH**
- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al. **HIGH**
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026) **LOW**
- [Life of a Write](#) — Linux Kernel Internals **MEDIUM**

- [The errseq_t datatype — The Linux Kernel documentation](#) — Linux kernel documentation **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018) **HIGH**
- [Fsync Errors](#) — PostgreSQL wiki **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020) **HIGH**
- [CreateFileW function — FILE_FLAG_WRITE_THROUGH and FILE_FLAG_NO_BUFFERING semantics](#) — Microsoft Learn **HIGH**
- [File Buffering](#) — Microsoft Learn **HIGH**
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — Dilip Naik (2012) **MEDIUM**
- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News (2019) **LOW**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow (2019) **MEDIUM**
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation **HIGH**
- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow **MEDIUM**
- [DTrace on Windows](#) — Microsoft Learn **HIGH**
- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn **HIGH**
- [Write-Ahead Logging \(SQLite\)](#) — SQLite **HIGH**

12.29 V47 — The four-boundary model of file I/O durability

This note argues that durability is a property of a sequence of four independent buffer-flush boundaries — the language-runtime buffer, the libc/kernel crossing, the kernel page cache, and the device cache — not of any single write call, and that almost every real data-loss bug is a program that flushed boundary n while believing it had flushed boundary $n+1$. A successful `write()` on Linux, or `WriteFile` on Windows, only confirms the kernel accepted the bytes into its own cache; only crossing all four boundaries, ending in `fsync()` or `FlushFileBuffers()`, gives the guarantee most programmers assume they already have. Its central evidence is the 2018 fsyncgate incident: PostgreSQL's checkpointer retried after an `EIO` from `fsync()`, correctly by its own reading of POSIX, but Linux reports a writeback error only once via `errseq_t` and then discards the failed pages — a genuine gap in the specification, not a bug on either

side, that silently lost data for roughly two decades until PostgreSQL, InnoDB and WiredTiger independently converged on PANIC-and-replay-from-WAL. Extending the same analysis to Windows, and after checking the PostgreSQL wiki's per-OS matrix, a Hacker News thread, and a Stack Overflow answer, the note concludes Windows documents no equivalent post-failure contract at all — a worse position than Linux's merely hostile one, since no correct response can be designed from undocumented behavior.

The note is explicit about its limits. It calls its four-layer table a teaching simplification that network filesystems, memory-mapped I/O, and copy-on-write filesystems reshape or add to. It does not claim any current Linux application strategy is sufficient, citing a fault-injection study that found all five systems it examined — including PostgreSQL, SQLite and Redis — inadequate in some failure mode. On Windows it treats the missing contract as an open question rather than a resolved one, stating outright that "nobody has publicly characterised it, and Microsoft has not documented it." Two supporting claims are flagged as speculative or inferred rather than verified: that no Windows tool cleanly exposes device-level `FLUSH_CACHE` commands, and that Python's `os.fsync` reaching `_commit()` on Windows implies a `FlushFileBuffers`-class operation. It also flags that PANIC-on-`EIO` may be miscalibrated for cloud block storage, where errors are often transient — a question it leaves open.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026) **MEDIUM**
- [fsync\(2\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017) **HIGH**
- [io — Core tools for working with streams](#) — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn **HIGH**
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022) **MEDIUM**
- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla **HIGH**
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode **MEDIUM**
- [Page Cache Writeback](#) — Linux Kernel Internals **MEDIUM**
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn **HIGH**
- [File Caching](#) — Microsoft Learn **HIGH**

- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al. **HIGH**
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026) **LOW**
- [Life of a Write](#) — Linux Kernel Internals **MEDIUM**
- [The errseq_t datatype](#) — The Linux Kernel documentation — Linux kernel documentation **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018) **HIGH**
- [Fsync Errors](#) — PostgreSQL wiki **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020) **HIGH**
- [CreateFileW function](#) — FILE_FLAG_WRITE_THROUGH and FILE_FLAG_NO_BUFFERING semantics — Microsoft Learn **HIGH**
- [File Buffering](#) — Microsoft Learn **HIGH**
- [Windows Write Caching](#) — Part 2: An Overview for Application Developers — Dilip Naik (2012) **MEDIUM**
- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News (2019) **LOW**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow (2019) **MEDIUM**
- [os](#) — Miscellaneous operating system interfaces — Python Software Foundation **HIGH**
- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow **MEDIUM**
- [DTrace on Windows](#) — Microsoft Learn **HIGH**
- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn **HIGH**
- [Write-Ahead Logging \(SQLite\)](#) — SQLite **HIGH**

12.30 V48 — File I/O and Durability

This note argues that durability in file I/O is not a property of a single write call but of a full sequence of independent flushes across four buffering boundaries: the language runtime, the user-space-to-kernel syscall boundary, the kernel's page cache, and the storage device's own cache. A program's write() call only reports that the first kernel-side buffer accepted the bytes — a statement about custody, not persistence — and each of the remaining boundaries can be crossed or skipped independently, with its own trigger and cost. The note traces this model in detail on Linux, where fsync() is docu-

mented to flush through the device cache and block until completion, and shows how the same four boundaries exist on Windows under different names (FlushFileBuffers, the Cache Manager's lazy writer) with a different flush history. It gives particular weight to the 2018 PostgreSQL fsync-failure incident: when writeback fails, Linux reports the error only once and then discards the failed pages, so a checkpoint that retried after a reported error silently believed writes had succeeded when they had not. The note frames this as a genuine contract disagreement between the kernel and PostgreSQL, not a bug on either side, and traces how PostgreSQL, InnoDB and WiredTiger all independently converged on treating a failed fsync as fatal and recovering from a write-ahead log instead of retrying.

The note is explicit about where its claims stop. On Windows, it states plainly that no one — including Microsoft's own documentation — has publicly characterised whether a failed flush behaves the same way as on Linux; its position is that Windows's contract is undocumented rather than safe or equivalently hostile. It qualifies its own guidance to always treat a failed fsync as fatal, PANIC rather than retry, as calibrated to 2018-era local disks, noting that on cloud block storage the same error is often transient, which changes the cost of crashing without changing the correctness argument. It also flags its four-layer model as a teaching simplification that does not fully cover network filesystems, memory-mapped I/O, or copy-on-write filesystems, and marks one of its own claims — that no Windows tool exposes device flush commands as cleanly as Linux's blktrace — as speculative rather than verified. It does not evaluate or improve on these open points; it states them as unresolved, including two related claims it deliberately defers to separate notes rather than settling here.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026) **MEDIUM**
- [fsync\(2\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017) **HIGH**
- [io — Core tools for working with streams](#) — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn **HIGH**
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022) **MEDIUM**
- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla **HIGH**

- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode **MEDIUM**
- [Page Cache Writeback](#) — Linux Kernel Internals **MEDIUM**
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn **HIGH**
- [File Caching](#) — Microsoft Learn **HIGH**
- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al. **HIGH**
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026) **LOW**
- [Life of a Write](#) — Linux Kernel Internals **MEDIUM**
- [The errseq_t datatype](#) — The Linux Kernel documentation — Linux kernel documentation **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018) **HIGH**
- [Fsync Errors](#) — PostgreSQL wiki **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020) **HIGH**
- [CreateFileW function](#) — FILE_FLAG_WRITE_THROUGH and FILE_FLAG_NO_BUFFERING semantics — Microsoft Learn **HIGH**
- [File Buffering](#) — Microsoft Learn **HIGH**
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — Dilip Naik (2012) **MEDIUM**
- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News (2019) **LOW**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow (2019) **MEDIUM**
- [os](#) — Miscellaneous operating system interfaces — Python Software Foundation **HIGH**
- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow **MEDIUM**
- [DTrace on Windows](#) — Microsoft Learn **HIGH**
- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn **HIGH**
- [Write-Ahead Logging \(SQLite\)](#) — SQLite **HIGH**

12.31 V49 — The four-boundary model of file I/O durability

The note's central claim is that durability is a property of a sequence of flushes, not of a single `write()` call: four independent boundaries sit between a program and persistent media — a language-runtime buffer, a user-space-to-kernel buffer, a kernel cache, and a device cache — each with its own trigger, cost, and failure mode, and a successful `write()` only confirms the first of these accepted the bytes. It grounds this in the Linux `fsync(2)` man page's wording (a flush "including writing through or flushing a disk cache if present") and in the 2018 "fsyncgate" incident: Linux reports a writeback error to only the next `fsync()` call and then discards the failed pages, so PostgreSQL's contract-compliant retry-on-`EIO` logic silently lost data for roughly two decades — a disagreement the note treats as two coherent readings of an underspecified POSIX contract, not a bug on either side. PostgreSQL, InnoDB, and WiredTiger each independently converged on PANIC-and-recover-from-WAL. Extending the model to Windows, the note argues Windows is structurally parallel (`FlushFileBuffers`, `FILE_FLAG_WRITE_THROUGH`, `FILE_FLAG_NO_BUFFERING`) but worse off, because Microsoft has never documented what happens to buffered data after a flush failure, so no correct application-level response can be designed the way PostgreSQL designed one for Linux.

The note is explicit about where its claims stop. It does not conclude that Windows shares Linux's exact non-retryable-failure bug, nor that Windows is safe — only that its contract is undocumented, and it treats an unspecified contract as worse than a documented but hostile one. Several points are flagged rather than asserted as fact: that no Windows tool exposes device-level flush commands as cleanly as Linux's `blktrace` is marked `[speculative]`; the mapping from Python's `os.fsync` to the Windows CRT's `_commit()` is marked `[inference]`, since that call chain is not stated in Python's documentation; and whether Windows exposes a knob comparable to `dirty_expire_centisecs` is left an open gap. It also cautions its four-layer model is a teaching simplification that omits network filesystems, memory-mapped I/O, and copy-on-write filesystems, and that its "PANIC on fsync failure" advice is calibrated for 2018-era local disks — on cloud storage an `EIO` is often transient, changing the availability trade-off without changing the correctness argument. It does not adjudicate which of the kernel's or PostgreSQL's readings of POSIX was "right," treating the disagreement as a specification gap rather than a resolved question.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026) **MEDIUM**
- [fsync\(2\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **HIGH**

- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017) **HIGH**
- [io](#) — Core tools for working with streams — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn **HIGH**
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022) **MEDIUM**
- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla **HIGH**
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode **MEDIUM**
- [Page Cache Writeback](#) — Linux Kernel Internals **MEDIUM**
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn **HIGH**
- [File Caching](#) — Microsoft Learn **HIGH**
- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al. **HIGH**
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026) **LOW**
- [Life of a Write](#) — Linux Kernel Internals **MEDIUM**
- [The errseq_t datatype](#) — The Linux Kernel documentation — Linux kernel documentation **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018) **HIGH**
- [Fsync Errors](#) — PostgreSQL wiki **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020) **HIGH**
- [CreateFileW function — FILE_FLAG_WRITE_THROUGH and FILE_FLAG_NO_BUFFERING semantics](#) — Microsoft Learn **HIGH**
- [File Buffering](#) — Microsoft Learn **HIGH**
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — Dilip Naik (2012) **MEDIUM**
- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News (2019) **LOW**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow (2019) **MEDIUM**
- [os](#) — Miscellaneous operating system interfaces — Python Software Foundation **HIGH**

- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow

MEDIUM

- [DTrace on Windows](#) — Microsoft Learn HIGH
- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn HIGH
- [Write-Ahead Logging \(SQLite\)](#) — SQLite HIGH

12.32 V50 — The gap between an OS abstraction's promise and its actual behaviour

This note argues that abstractions over the operating system leak in a specific, recurring shape: they are a contract with a silent scope clause, and the leak is the program running past that clause rather than a defect in the abstraction itself. It builds this claim through five concrete cases — a successful `write()` that is not durable, a `close()` that neither flushes nor reliably reports earlier errors, a "thread" that may or may not be a kernel thread depending on the runtime, an `mmap()` that can be faster or slower than `read()` depending on access pattern, and a profiler that attributes time to the wrong layer because kernel time and blocked time are invisible to it. Each case is backed by primary vendor and kernel documentation (Linux man pages, Microsoft Learn) for what the interface actually guarantees, cross-checked against independent measurement (blktrace traces, Daniel Lemire's read/mmap benchmarks) for what happens in practice, and illustrated with a named observation tool — `strace`, Process Monitor, ETW — on both Linux and Windows treated as structurally equal. The evidence is deliberately observational rather than deductive: the note's stated method is that you cannot reason your way to which layer is lying, you have to make the boundary crossing visible and count it.

The note is explicit about what it does not claim. It does not re-derive the underlying mechanisms (syscall boundary, page cache, scheduling), treating those as owned by sibling notes that, as of the note's writing, do not yet exist in the vault. It flags one claim — the cycle cost of a mode switch used to argue that syscall counts matter — as uncited arithmetic resting only on aggregator sources, and recommends students measure it themselves rather than trust the figure. It presents the mmap-versus-read question as genuinely contested between sources rather than resolved, and treats several of its own generalizations — the CPython GIL's future, Windows' now-removed User-Mode Scheduling, `io_uring`'s effect on syscall counting — as version-scoped facts likely to go stale rather than timeless properties of the platforms.

Referenced sources

- [close\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024) HIGH

- [fsync\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024) **HIGH**
- [Flushing System-Buffered I/O Data to Disk](#) — Microsoft Learn (2021) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn (2023) **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **MEDIUM**
- [The Secret Life of fsync](#) — Andrey Pechkurov / puzpuzpuz.dev (2022) **MEDIUM**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft DevBlogs (2017) **HIGH**
- [fsync, fdatasync, and O_SYNC](#) — kernel-internals.org (2025) **MEDIUM**
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation (2026) **HIGH**
- [Bypassing the GIL for Parallel Processing in Python](#) — Real Python (2024) **MEDIUM**
- [About Processes and Threads](#) — Microsoft Learn (2023) **HIGH**
- [Fibers](#) — Microsoft Learn (2023) **HIGH**
- [User-Mode Scheduling](#) — Microsoft Learn (2022) **HIGH**
- [The Libuv Thread Pool Trap: Why Node.js Async APIs Stall Under Load](#) — The Practical Developer (2026) **MEDIUM**
- [MapViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn (2022) **HIGH**
- [Which is fastest: read, fread, ifstream or mmap?](#) — Daniel Lemire (2012) **HIGH**
- [Latency implications of virtual memory](#) — Erik Rigtorp (2020) **MEDIUM**
- [Why is mmap\(\) Faster Than Sequential IO? Debunking Common Myths](#) — linux-vox.com (2025) **LOW**
- [File Access: Memory-Mapped vs. I/O System Call Performance](#) — Baeldung on Linux (2024) **MEDIUM**
- [strace\(1\) — Linux manual page](#) — strace project / man7.org (2024) **HIGH**
- [DTrace on Windows](#) — Microsoft Learn (2023) **HIGH**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#) — Yarden Shafir / Winsider Seminars & Solutions (2021) **HIGH**
- [Linux I/O Models — Blocking, epoll, io_uring, AIO, Signal-Driven](#) — systeminternals.dev (2026) **MEDIUM**
- [The Law of Leaky Abstractions](#) — Joel Spolsky (2002) **HIGH**
- [There Is a Mountain Lion in the Datacenter](#) — HFT University (2026) **LOW**
- [Sysinternals Suite 2026.19.08](#) — Neowin (2026) **MEDIUM**
- [How Python's GIL Works](#) — Codefinity (2025) **LOW**

12.33 V51 — Leaky Operating-System Abstractions

This note argues that every abstraction a programming language or runtime places over the operating system is a contract with an unwritten scope clause: it states what a call does, but not the conditions under which that guarantee holds. Three properties of the syscall boundary make these gaps different from ordinary application-level abstractions — the boundary is a privilege transition that can be interrupted or reordered rather than a simple jump, a fast return usually means the kernel has accepted the work rather than completed it, and language runtimes add a second layer of buffering on top of the kernel's own. From this the note builds five paired Linux/Windows cases, each with a stated belief, the actual kernel behaviour drawn from primary documentation (man pages, Win32 references, Python's own docs), and an observation procedure that makes the gap visible with a named tool — `strace`, `blktrace`, Process Monitor, ETW, `/proc/<pid>/task`. The cases cover: a successful write that is not yet durable; a `close()` that neither flushes nor reliably reports an earlier write's error; a Python or Node "thread" that is not what the word implies; an `mmap` whose cost model is page faults rather than syscalls; and a CPU profiler that misattributes kernel-boundary and blocking time to the wrong function.

The note is explicit about what it does not claim. It does not treat leaks as bugs to be designed away, and it does not argue that any given abstraction should be replaced — trusting the abstraction operationally, pushing durability into a dependency, bypassing the OS layer, or changing the boundary's economics with `io_uring` are all presented as legitimate, situation-dependent choices rather than a single recommended fix. Several claims are marked as unsettled or time-bound rather than settled fact: whether `mmap` outperforms `read()` is treated as a genuine disagreement between benchmark-first and practitioner sources, resolved only by the access pattern being measured; and the note flags that its mechanisms are moving targets — User-Mode Scheduling has already been withdrawn from Windows, `io_uring` has changed what a Linux syscall count means, and CPython's GIL is being incrementally unwound. One quantitative claim, the roughly 300-cycle cost of a mode switch used to argue that syscall counts matter in aggregate, is marked in the note itself as an inference without a primary-sourced measurement, and the note recommends students verify it themselves rather than treat it as established. More broadly, it concedes that some leaks — device write caches, NUMA topology, per-CCX cache slicing — sit below the operating system entirely, so "the next layer down" has no fixed floor.

Referenced sources

- [close\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024) HIGH
- [fsync\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024) HIGH

- [Flushing System-Buffered I/O Data to Disk](#) — Microsoft Learn (2021) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn (2023) **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **MEDIUM**
- [The Secret Life of fsync](#) — Andrey Pechkurov / puzpuzpuz.dev (2022) **MEDIUM**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft DevBlogs (2017) **HIGH**
- [fsync, fdatsync, and O_SYNC](#) — kernel-internals.org (2025) **MEDIUM**
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation (2026) **HIGH**
- [Bypassing the GIL for Parallel Processing in Python](#) — Real Python (2024) **MEDIUM**
- [About Processes and Threads](#) — Microsoft Learn (2023) **HIGH**
- [Fibers](#) — Microsoft Learn (2023) **HIGH**
- [User-Mode Scheduling](#) — Microsoft Learn (2022) **HIGH**
- [The Libuv Thread Pool Trap: Why Node.js Async APIs Stall Under Load](#) — The Practical Developer (2026) **MEDIUM**
- [MapViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn (2022) **HIGH**
- [Which is fastest: read, fread, ifstream or mmap?](#) — Daniel Lemire (2012) **HIGH**
- [Latency implications of virtual memory](#) — Erik Rigtorp (2020) **MEDIUM**
- [Why is mmap\(\) Faster Than Sequential IO? Debunking Common Myths](#) — linux-vox.com (2025) **LOW**
- [File Access: Memory-Mapped vs. I/O System Call Performance](#) — Baeldung on Linux (2024) **MEDIUM**
- [strace\(1\)](#) — Linux manual page — strace project / man7.org (2024) **HIGH**
- [DTrace on Windows](#) — Microsoft Learn (2023) **HIGH**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#) — Yarden Shafir / Winsider Seminars & Solutions (2021) **HIGH**
- [Linux I/O Models — Blocking, epoll, io_uring, AIO, Signal-Driven](#) — systeminternals.dev (2026) **MEDIUM**
- [The Law of Leaky Abstractions](#) — Joel Spolsky (2002) **HIGH**
- [There Is a Mountain Lion in the Datacenter](#) — HFT University (2026) **LOW**
- [Sysinternals Suite 2026.19.08](#) — Neowin (2026) **MEDIUM**
- [How Python's GIL Works](#) — Codefinity (2025) **LOW**

12.34 V52 — Leaky Abstractions at the OS Syscall Boundary

The note argues that every higher-level abstraction over the operating system — a file handle, a thread, a memory-mapped region — is a contract with a hidden "scope clause": it says what the call does but not the conditions under which that holds, and the leak is the abstraction confessing what was underneath it all along. It grounds this in three structural properties of OS boundaries: crossing into the kernel is a privilege transition that can be interrupted or reordered, not an ordinary function call; success from a fast OS interface almost always means the kernel accepted responsibility for work, not that the work happened; and language runtimes add a second, usually invisible layer of the same buffering the kernel already does. From there it walks five concrete, evidence-backed cases — a `write()` that returns success without being durable, a `close()` that neither flushes nor safely reports earlier write errors, "thread" meaning three different things depending on runtime (CPython's GIL-serialized OS threads, Node's four-thread libuv pool, Windows fibers and the now-removed User-Mode Scheduling), an `mmap` that trades syscalls for page faults and so is not simply faster than `read()`, and CPU profilers that misattribute kernel and blocked time so that syscall count, not bytes moved, explains latency. Each case names an observation procedure — `strace`, `blktrace`, Process Monitor, ETW/WPA, `/proc/<pid>`, Process Explorer — on both Linux and Windows, treating the platforms as structurally equal, and is anchored by production incidents: Facebook Scuba's `mmap`-based restart speedup, Firefox 3's ext3 `fsync` stall, and Node.js services with healthy event loops but unexplained tail latency from an exhausted thread pool.

The note is explicit about what it does not claim. It does not assert these mechanisms are permanent: User-Mode Scheduling was withdrawn, `io_uring` changed what a syscall count means, and CPython's GIL is being unwound, so every claim carries a version scope rather than being timeless. It flags observation itself as non-neutral — `strace` and ETW perturb the very thing they measure — so absolute timings deserve less trust than counts and orderings. One quantitative claim, that a mode switch costs on the order of hundreds of cycles, is marked in-text as ungrounded: it traces only to aggregator content rather than a primary measurement, and the note recommends either sourcing an `lmbench`-style benchmark or having students measure it themselves before the claim is exported into a student packet. It also treats the disagreement among its own sources over whether `mmap` beats `read()` on sequential access as unsettled by design — the resolution offered is that the question depends on access pattern, not that one side is wrong — and it does not attempt to re-derive the underlying syscall, page-cache, or scheduling mechanisms, deferring those to sibling notes that, as of this note's writing, do not yet exist in the vault.

Referenced sources

- [close\(2\)](#) — Linux manual page, Michael Kerrisk / man7.org (2024) — HIGH
- [fsync\(2\)](#) — Linux manual page, Michael Kerrisk / man7.org (2024) — HIGH
- [Flushing System-Buffered I/O Data to Disk](#), Microsoft Learn (2021) — HIGH
- [FlushFileBuffers function \(fileapi.h\)](#), Microsoft Learn (2023) — HIGH
- [Durability: Linux File APIs](#), Evan Jones (2020) — MEDIUM
- [The Secret Life of fsync](#), Andrey Pechkurov / puzpuzpuz.dev (2022) — MEDIUM
- [Revised notes on the reliability of FlushFileBuffers](#), Raymond Chen / Microsoft DevBlogs (2017) — HIGH
- [fsync, fdatasync, and O_SYNC](#), kernel-internals.org (2025) — MEDIUM
- [os](#) — Miscellaneous operating system interfaces, Python Software Foundation (2026) — HIGH
- [Bypassing the GIL for Parallel Processing in Python](#), Real Python (2024) — MEDIUM
- [About Processes and Threads](#), Microsoft Learn (2023) — HIGH
- [Fibers](#), Microsoft Learn (2023) — HIGH
- [User-Mode Scheduling](#), Microsoft Learn (2022) — HIGH
- [The Libuv Thread Pool Trap: Why Node.js Async APIs Stall Under Load](#), The Practical Developer (2026) — MEDIUM
- [MapViewOfFile function \(memoryapi.h\)](#), Microsoft Learn (2022) — HIGH
- [Which is fastest: read, fread, ifstream or mmap?](#), Daniel Lemire (2012) — HIGH
- [Latency implications of virtual memory](#), Erik Rigtorp (2020) — MEDIUM
- [Why is mmap\(\) Faster Than Sequential IO? Debunking Common Myths](#), linux-vox.com (2025) — LOW
- [File Access: Memory-Mapped vs. I/O System Call Performance](#), Baeldung on Linux (2024) — MEDIUM
- [strace\(1\)](#) — Linux manual page, strace project / man7.org (2024) — HIGH
- [DTrace on Windows](#), Microsoft Learn (2023) — HIGH
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#), Yarden Shafir / Winsider Seminars & Solutions (2021) — HIGH
- [Linux I/O Models — Blocking, epoll, io_uring, AIO, Signal-Driven](#), systeminternals.dev (2026) — MEDIUM
- [The Law of Leaky Abstractions](#), Joel Spolsky (2002) — HIGH
- [There Is a Mountain Lion in the Datacenter](#), HFT University (2026) — LOW
- [Sysinternals Suite 2026.19.08](#), Neowin (2026) — MEDIUM
- [How Python's GIL Works](#), Codefinity (2025) — LOW

12.35 V53 — The Non-Retryability of a Failed `fsync()`

On Linux, when a write-back to disk fails, the kernel marks the dirty page clean and discards it, then records the failure as a one-shot flag on the file. The next call to `fsync()` sees the flag, returns an error — and the act of returning that error clears the flag. A second `fsync()` call therefore finds no dirty pages and no error flag, and truthfully reports success, even though the original data was never written. This makes "just retry" unsound: the retry does not re-attempt the failed write, it reads a status register the previous read already destroyed. The evidence includes a 2018 PostgreSQL incident in which the database's checkpointer treated a retried `fsync()` success as proof a checkpoint had completed and recycled the log records that were the only surviving copy of that data; a reproduction under `strace` showing exactly this sequence; and fault-injection testing across ext4, XFS, and Btrfs confirming the clean-on-failure behavior is uniform across filesystems. PostgreSQL's fix, shipped in 2018 and back-ported to every supported release, replaced retry with an immediate crash so the database is forced to rebuild its state from its write-ahead log rather than trust an untrustworthy status code; InnoDB and WiredTiger converged on the same fix independently. A structurally similar discard-on-failure behavior exists in Windows's write-back cache, documented by Microsoft itself, with guidance to bypass buffering entirely for critical writes.

The note is explicit about where this stops. FreeBSD is a live counterexample: it remarks the page dirty rather than clearing the flag, so retrying is genuinely safe there. The claim does not extend to unbuffered or direct I/O, where there is no page cache to discard and a repeated write call is a real retry. And unlike the well-established Linux mechanism, whether a retried Windows flush call can falsely report success has never been publicly settled — extensive searching found the question asked but unanswered, and the note treats this as an open gap rather than assuming Windows behaves like Linux. It also flags as unresolved, not decided, whether crashing immediately is still the right operational trade-off on cloud storage where such errors are often transient.

Referenced sources

- PostgreSQL's `fsync()` surprise — Jonathan Corbet / LWN.net — <https://lwn.net/Articles/752063/> **HIGH**
- Fsyncgate: errors on `fsync` are unrecoverable — Dan Luu — <https://danluu.com/fsyncgate/> **HIGH**
- PostgreSQL commit 9ccdd7f6 — "PANIC on `fsync()` failure" — Craig Ringer / Thomas Munro, PostgreSQL Global Development Group — <https://git.postgresql.org/git-web/?p=postgresql.git;a=commit;h=9ccdd7f66e3324d2b6d3dec282cfa9ff084083f1> **HIGH**

- PostgreSQL documentation — `data_sync_retry` — PostgreSQL Global Development Group — <https://www.postgresql.org/docs/current/runtime-config-error-handling.html> **HIGH**
- fs: enhanced writeback error reporting with `errseq_t` (pile #1) — Jeff Layton, linux-fsdevel — <https://lwn.net/Articles/726774/> **HIGH**
- The `errseq_t` datatype — Linux Kernel documentation — <https://docs.kernel.org/7.0/core-api/errseq.html> **HIGH**
- Linux Filesystems API summary — `filemap_set_wb_err` / `filemap_check_wb_err` — The Linux Kernel documentation — <https://www.kernel.org/doc/html/v5.1/filesystems/api-summary.html> **HIGH**
- Troubleshoot the event ID 50 error message — Microsoft Learn — <https://learn.microsoft.com/en-us/troubleshoot/windows-server/networking/troubleshoot-event-id-50-error> **HIGH**
- File Caching — Win32 apps — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/fileio/file-caching> **HIGH**
- `FlushFileBuffers` function (`fileapi.h`) — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-flushfilebuffers> **HIGH**
- `NtFlushBuffersFileEx` function (`ntifs.h`) — Microsoft Learn (Windows Driver Kit) — <https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/ntifs/nf-ntifs-nt-flushbuffersfileex> **HIGH**
- MS-ERREF: NTSTATUS Values — Microsoft Open Specifications — https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-erref/596a1078-e883-4972-9bbc-49e60bebca55 **HIGH**
- Delayed Write Failed error — data has been lost (KB 2021074) — Microsoft Learn — <https://learn.microsoft.com/en-us/troubleshoot/windows-server/networking/delayed-write-failed-error-data-lost> **HIGH**
- Is it safe to call the `Win32 FlushFileBuffers()` function again if it previously returned an error? — David Sitsky, Stack Overflow — <https://stackoverflow.com/questions/54660780/is-it-safe-to-call-the-win32-flushfilebuffers-function-again-if-it-previously> **MEDIUM**
- Hacker News — "Does anyone know if `FlushFileBuffers()` on Windows also forgets to flush data..." — Hacker News thread, participants incl. Thomas Munro (`macdice`) — <https://news.ycombinator.com/item?id=19240440> **LOW**
- Can Applications Recover from `fsync` Failures? — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20 — <https://www.usenix.org/system/files/atc20-rebello.pdf> **HIGH**

- When Amnesia Strikes: Understanding and Reproducing Data Loss Bugs with Fault Injection — Ramos, Macedo et al., PVLDB vol. 17 — <https://www.vldb.org/pvldb/vol17/p3017-ramos.pdf> **HIGH**
- `fsync` — POSIX Programmer's Manual — IEEE Std 1003.1 / The Open Group Base Specifications — https://www.unix.com/man_page/linux/3/fsync/ **HIGH**
- How Tailscale helped find the SQLite WAL-Reset bug — Tailscale engineering — <https://tailscale.com/blog/sqlite-wal-reset-bug> **MEDIUM**
- All Your GUCs in a Row: `data_sync_retry` — The Build (PostgreSQL Experts) — <https://thebuild.com/blog/all-your-gucs-in-a-row-datasyncretry/> **MEDIUM**

12.36 V54 — The unretryable `fsync()` failure

The note establishes that a failed `fsync()` on Linux cannot be safely retried, because a write-back failure destroys two separate things and only one of them is the data. When the kernel cannot flush a dirty page, it marks the page clean and discards it, since keeping an unwritable page dirty forever would be an unbounded memory leak. It also raises an error flag, `AS_EIO`, on the file's `address_space` — but the act of reading that flag on the next `fsync()` call clears it, so a second call finds no dirty pages and no flag and truthfully reports success, even though the original write never reached disk. The note grounds this in a live reproduction under `strace` and in PostgreSQL's own 2018 incident, where a retried checkpoint reported success after an earlier `fsync()` had failed, letting the write-ahead log segments describing the lost write be recycled. It frames the fix — PostgreSQL, InnoDB, and WiredTiger independently replacing retry with process termination followed by log-based recovery — as the only sound response once the in-memory model of what is on disk can no longer be trusted, and treats the underlying disagreement between kernel and database developers as a genuine, undocumented contract gap rather than a bug on either side.

The note is explicit about where this does not reach. FreeBSD is a stated counter-example, since it re-dirties the page rather than discarding it, so retry is genuinely safe there; unbuffered or direct I/O is likewise excluded, because there is no page-cache copy to lose. Windows is developed as a parallel case on the same terms, and the note is careful to say it does not know the answer for Windows: the OS documents that buffered writes can be silently discarded on a failed lazy write, but whether a retried `FlushFileBuffers` can falsely report success afterward is described as publicly unestablished rather than assumed to mirror Linux. The note also flags as unsettled, not resolved, whether crashing on the first `fsync()` failure remains the right operational trade-off on cloud block storage, where the note observes the error is often transient — noting this is a dispute about cost, not about the underlying mechanism.

Referenced sources

- [PostgreSQL's fsync\(\) surprise](#) — Jonathan Corbet / LWN.net (2018) **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curated archive of the pgsql-hackers thread, 2018) **HIGH**
- [PostgreSQL commit 9ccdd7f6 — "PANIC on fsync\(\) failure"](#) — Craig Ringer / Thomas Munro, PostgreSQL Global Development Group (2018) **HIGH**
- [PostgreSQL documentation — data_sync_retry](#) — PostgreSQL Global Development Group **HIGH**
- [fs: enhanced writeback error reporting with errseq_t \(pile #1\)](#) — Jeff Layton, linux-fsdevel (2017) **HIGH**
- [The errseq_t datatype](#) — Linux Kernel documentation — The Linux Kernel documentation **HIGH**
- [Linux Filesystems API summary — filemap_set_wb_err / filemap_check_wb_err](#) — The Linux Kernel documentation **HIGH**
- [Troubleshoot the event ID 50 error message](#) — Microsoft Learn (originally KB 816004; page updated 2026-02-12) **HIGH**
- [File Caching — Win32 apps](#) — Microsoft Learn **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [NtFlushBuffersFileEx function \(ntifs.h\)](#) — Microsoft Learn (Windows Driver Kit) **HIGH**
- [MS-ERREF: NTSTATUS Values](#) — Microsoft Open Specifications **HIGH**
- [Delayed Write Failed error — data has been lost \(KB 2021074\)](#) — Microsoft Learn (page updated 2026-02-12) **HIGH**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — David Sitsky, Stack Overflow (asked 2019-02-13) **MEDIUM**
- [Hacker News — "Does anyone know if FlushFileBuffers\(\) on Windows also forgets to flush data..."](#) — Hacker News thread, participants incl. Thomas Munro (`macdice`) (2019) **LOW**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20, pp. 753–767 (2020) **HIGH**
- [When Amnesia Strikes: Understanding and Reproducing Data Loss Bugs with Fault Injection](#) — Ramos, Macedo et al., PVLDB vol. 17, pp. 3017–3029 (2024) **HIGH**
- [fsync](#) — POSIX Programmer's Manual — IEEE Std 1003.1 / The Open Group Base Specifications **HIGH**
- [How Tailscale helped find the SQLite WAL-Reset bug](#) — Tailscale engineering (2026) **MEDIUM**

- [All Your GUCs in a Row: data_sync_retry — The Build \(PostgreSQL Experts\)](#)

MEDIUM

12.37 V55 — The non-retryability of a failed fsync()

This note argues that a failed `fsync()` cannot safely be retried, because a Linux write-back failure destroys two separate things and only one is the data. When the kernel fails to flush a dirty page, it marks the page clean and discards it, then records the failure as a one-shot flag, `AS_EIO`, on the file's `address_space`. The first `fsync()` that checks the flag reports EIO and, in reporting, clears it; a second `fsync()` then finds no flag and no dirty pages, and truthfully reports success even though the original write never landed. PostgreSQL's checkpointer hit exactly this: it aborted a checkpoint on the first EIO, retried later, saw success, and recycled WAL segments that were the only remaining record of the lost writes — causing silent data loss for roughly twenty years before a 2018 fix replaced `retry` with `PANIC-and-recover-from-WAL`, reasoning that once both the kernel's error state and the page cache are gone, the process cannot trustworthily re-derive what is on disk. The note frames this as a contract failure, not a bug on either side: POSIX specifies success but stays silent on failure, and kernel and database developers held individually defensible, mutually incompatible, undocumented readings of what a failed `fsync()` means. It also covers the Windows case: Cache Manager discards lazily-written data on failure just as Linux does, documents this, and names `FILE_FLAG_NO_BUFFERING` as the same escape hatch.

The note is explicit about where the claim stops. It is not universal across operating systems: FreeBSD is a live counterexample, clearing the error but re-marking the buffer dirty, so a retried `fsync()` there genuinely re-attempts the write. It does not reach unbuffered or direct I/O — with `O_DIRECT` or `FILE_FLAG_NO_BUFFERING` there is no page-cache copy to discard and no one-shot flag, so reissuing the call is a real retry — and PostgreSQL ships a `data_sync_retry` switch for operating systems that don't share Linux's semantics. On Windows the note grades its confidence lower: it verifies that Cache Manager discards lazily-written data and reports failure out-of-band, but leaves open whether a retried `FlushFileBuffers` can falsely report success after such a loss, noting the question was asked publicly in 2019 and remains unanswered. It also flags as contested, not settled, whether `PANIC` remains the right operational trade-off in 2026 given that EIO on cloud block storage is often transient — a disagreement about cost, not about the underlying correctness argument.

Referenced sources

- [PostgreSQL's fsync\(\) surprise — Jonathan Corbet / LWN.net \(2018\)](#) HIGH

- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curated archive of the postgresql-hackers thread, 2018) **HIGH**
- [PostgreSQL commit 9ccdd7f6 — "PANIC on fsync\(\) failure"](#) — Craig Ringer / Thomas Munro, PostgreSQL Global Development Group (2018) **HIGH**
- [PostgreSQL documentation — data_sync_retry](#) — PostgreSQL Global Development Group **HIGH**
- [fs: enhanced writeback error reporting with errseq_t \(pile #1\)](#) — Jeff Layton, linux-fsdevel (2017) **HIGH**
- [The errseq_t datatype](#) — Linux Kernel documentation — The Linux Kernel documentation **HIGH**
- [Linux Filesystems API summary — filemap_set_wb_err / filemap_check_wb_err](#) — The Linux Kernel documentation **HIGH**
- [Troubleshoot the event ID 50 error message](#) — Microsoft Learn (originally KB 816004; page updated 2026-02-12) **HIGH**
- [File Caching — Win32 apps](#) — Microsoft Learn **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [NtFlushBuffersFileEx function \(ntifs.h\)](#) — Microsoft Learn (Windows Driver Kit) **HIGH**
- [MS-ERREF: NTSTATUS Values](#) — Microsoft Open Specifications **HIGH**
- [Delayed Write Failed error — data has been lost \(KB 2021074\)](#) — Microsoft Learn (page updated 2026-02-12) **HIGH**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — David Sitsky, Stack Overflow (asked 2019-02-13) **MEDIUM**
- [Hacker News — "Does anyone know if FlushFileBuffers\(\) on Windows also forgets to flush data..."](#) — Hacker News thread, participants incl. Thomas Munro (macdice) (2019) **LOW**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20, pp. 753–767 (2020) **HIGH**
- [When Amnesia Strikes: Understanding and Reproducing Data Loss Bugs with Fault Injection](#) — Ramos, Macedo et al., PVLDB vol. 17, pp. 3017–3029 (2024) **HIGH**
- [fsync](#) — POSIX Programmer's Manual — IEEE Std 1003.1 / The Open Group Base Specifications **HIGH**
- [How Tailscale helped find the SQLite WAL-Reset bug](#) — Tailscale engineering (2026) **MEDIUM**
- [All Your GUCs in a Row: data_sync_retry](#) — The Build (PostgreSQL Experts) **MEDIUM**

12.38 V56 — PostgreSQL's `fsync()` error-reporting failure

This note examines "PostgreSQL's `fsync()` surprise," LWN's 2018 report on a defect in how Linux reports write-back errors to `fsync()`. The core finding: when a write-back to disk fails, Linux marks the dirty page clean and discards it, recording the failure only as a one-shot flag that is consumed and cleared the next time any process calls `fsync()` and observes it. PostgreSQL's checkpointer retried after a failure, saw success on the retry, recycled its write-ahead log, and silently lost committed data — because the retry does not re-attempt the failed I/O; it merely reads a status flag the earlier call already reset, against a page cache from which the data has already been evicted. The note documents that this was not a PostgreSQL-only defect: the mechanism was reproduced under `strace`, independently confirmed by fault-injection research across `ext4`, `XFS`, and `Btrfs`, and mirrored by parallel incidents in MySQL's InnoDB and MongoDB's WiredTiger, both of which reached the same conclusion on their own. All three storage engines — PostgreSQL, InnoDB, and WiredTiger — replaced retry-on-failure with crash/abort during 2018, on the reasoning that once an unreported write failure may have occurred, the process's in-memory model of what is on disk cannot be trusted, and the only sound recovery is to discard that model and rebuild it from the write-ahead log, the one artifact whose durability is independently guaranteed.

The note is explicit about what remains unsettled. Whether POSIX's `fsync()` contract obligated the kernel to behave differently is treated as unresolved: POSIX specifies the success case precisely but explicitly declines to guarantee anything about outstanding I/O after a failure, so neither the database nor the kernel side can be shown to have violated the standard. Whether the kernel's page-dropping behavior is a defect or a defensible choice under memory pressure is presented as a live, unresolved disagreement between PostgreSQL and kernel developers rather than a settled question. The note also flags a residual gap in the shipped fix: an error can still be missed if a file is closed, its inode evicted under memory pressure, and then reopened before the error would have been delivered. It further notes that broader academic study found that even after the 2018 fixes, no application — PostgreSQL included — handles `fsync()` failure completely correctly; the crash-based fix converted silent data loss into a loud, visible failure, which is a substantial improvement but not a complete guarantee of buffered-I/O durability.

Referenced sources

- [PostgreSQL's `fsync\(\)` surprise](#) — Jonathan Corbet / LWN.net HIGH
- [Fsyncgate: errors on `fsync` are unrecoverable](#) — Dan Luu HIGH
- [PostgreSQL commit 9ccdd7f66e3324d2b6d3dec282cfa9ff084083f1 — "PANIC on `fsync\(\)` failure"](#) — Craig Ringer / Thomas Munro HIGH

- [PostgreSQL documentation — data_sync_retry](#) — PostgreSQL Global Development Group **HIGH**
- [MySQL Bug #90296 — "Hard error should be report when fsync\(\) return EIO"](#) — Oracle / MySQL bug tracker **HIGH**
- [mysql-server commit 8590c8e12a3374eecb547359750a9d2a128fa6a — Bug#27805553](#) — Oracle / MySQL Server **HIGH**
- [WiredTiger commit ae8bccce3d8a8248afa0e4e0cf67674a43dede96 — WT-4045 "Don't retry fsync calls after EIO failure"](#) — Keith Bostic / Alex Gorrod, MongoDB WiredTiger **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20 **HIGH**
- [fs: introduce new writeback error reporting and convert existing API as a wrapper around it](#) — Jeff Layton, linux-fsdevel **HIGH**
- [PATCH v2 3/5 vfs: track per-sb writeback errors and report them to syncfs](#) — Jeff Layton, LKML **HIGH**
- [Linux Filesystems API summary — filemap_set_wb_err / filemap_check_wb_err](#) — The Linux Kernel documentation **HIGH**
- [PostgreSQL 18 and beyond: From AIO to Direct IO?](#) — Cybertec PostgreSQL International **MEDIUM**
- [debug_io_direct](#) — pgPedia — pgPedia, a PostgreSQL Encyclopedia **MEDIUM**
- [All Your GUCs in a Row: data_sync_retry](#) — The Build (PostgreSQL Experts) **MEDIUM**
- [fsync](#) — POSIX Programmer's Manual — IEEE Std 1003.1 / The Open Group Base Specifications **HIGH**

12.39 V57 — Readiness and Completion I/O Models

This note argues that Linux and Windows answer a different question when a server needs to know whether ten thousand connections are ready for I/O, and that the difference is architectural rather than stylistic. Linux's `select`, `poll`, and `epoll` answer "which descriptor would not block right now?" and leave the program to perform the `read` or `write` itself; Windows' overlapped I/O and I/O completion ports answer "which operation I already submitted has finished?" and hand back a filled buffer. The note traces that one distinction — where the data copy happens relative to the notification — into three consequences it treats as load-bearing: who owns the buffer and for how long (the kernel holds nothing in a readiness model but owns the buffer from submission to completion in a completion model), what shape the control flow takes (a dis-

patch loop versus a per-connection state machine), and when an error becomes visible (synchronously at the call site versus attached to a completion packet identified only by an `OVERLAPPED` pointer). Each claim is backed by primary vendor or kernel documentation — the `epoll(7)` and `select(2)` man pages, Microsoft's IOCP, `GetQueuedCompletionStatus`, and `WSARecv` reference pages — supplemented by a Rust design study showing the two buffer-ownership models are mutually exclusive at the type-signature level, and by evidence from Python's `asyncio` / `selectors` modules and from `libuv`, which emulates one model atop the other and pays for it with two internal architectures.

The note is explicit about what it does not claim. It does not treat the split as a Linux/Windows split: `io_uring` is a completion interface on Linux, and `WSAPoll` is a readiness-shaped (if limited) interface on Windows, so the note frames this as a design-space distinction each platform entered from a different side. Two claims are flagged as resting on secondary rather than primary sources and are marked accordingly in the note: the specific claim that `nginx` uses edge-triggered `epoll` while `Redis` uses level-triggered, and the claim that Windows' Registered I/O mechanism never gained wide adoption. A worked NtTrace transcript is explicitly labelled a constructed illustration rather than a captured trace, since no published capture could be located. The note also surfaces a live, unresolved disagreement among Linux practitioners over whether draining a socket to `EAGAIN` is strictly required to re-arm an edge-triggered `epoll` notification, or merely the reliable way to detect an empty buffer — it reports both readings without adjudicating between them.

Referenced sources

- [epoll\(7\) — Linux manual page](#), Linux man-pages project **HIGH**
- [select\(2\) — Linux manual page](#), Linux man-pages project **HIGH**
- [I/O Completion Ports](#), Microsoft Learn **HIGH**
- [GetQueuedCompletionStatus function \(ioapiset.h\)](#), Microsoft Learn **HIGH**
- [WSARecv function \(winsock2.h\)](#), Microsoft Learn **HIGH**
- [Overlapped Input/Output \(Winsock\)](#), Microsoft Learn **HIGH**
- [WSAPoll function \(winsock2.h\)](#), Microsoft Learn **HIGH**
- [WSAPoll is broken](#), Daniel Stenberg **MEDIUM**
- [The edge-triggered misunderstanding](#), LWN.net comment thread **HIGH**
- [portable-interoperable / io-traits](#), Nick Cameron **MEDIUM**
- [Async IO with completion-model IO systems](#), Nick Cameron **MEDIUM**
- [Design overview — libuv documentation](#), libuv project **HIGH**
- [libuv — iocp-links.html \(design notes\)](#), libuv project **MEDIUM**

- [asyncio — Event Loop \(Python documentation\)](#), Python Software Foundation **HIGH**
- [selectors — High-level I/O multiplexing \(Python documentation\)](#), Python Software Foundation **HIGH**
- [SDL_net: windows: Reimplement poll\(\) with select\(\)](#), Simple DirectMedia Layer project **MEDIUM**
- [PollSet not working as intended on Windows — pocoproject/poco#3248](#), POCO C++ Libraries issue tracker **MEDIUM**
- [Issue 16507: Patch selectmodule.c to support WSApoll on Windows](#), Python issue tracker **HIGH**
- [Under the Hood of AFD.sys Part 1: Investigating Undocumented Interfaces](#), Mateusz Lewczak **MEDIUM**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#), Win-sider Seminars & Solutions **HIGH**
- [libuv Event Loop Internals](#), Sujeet Jaiswal **MEDIUM**
- [Linux io_uring vs Windows I/O: A Technical Comparison](#), Mark Veltzer **LOW**
- [epoll: The API That Powers the Modern Internet](#), rzero.in **LOW**

12.40 V58 — Readiness and Completion Models for Asynchronous I/O

This note argues that Linux and Windows answer different questions when a server must handle many connections without a thread per connection, and that the split is architectural rather than stylistic. Linux's `select`, `poll`, and `epoll` answer "which descriptor would not block right now?" and leave the program to perform the read itself, allocating a buffer only once data is known to be waiting. Windows answers "which operation I already submitted has finished?" and delivers a filled buffer through an I/O completion port, requiring the buffer to exist and be owned by the kernel before any data arrives. The note treats this as changing three structural things — who owns the buffer and for how long, the shape of the program's control flow, and when an error becomes visible — and backs each claim with primary vendor and man-page documentation, a Rust standard-library design study that found the two buffer-ownership models cannot be reconciled into one set of I/O traits, and cross-platform libraries (libuv, CPython asyncio) that must implement both and document the model leaking into their own APIs. Concrete evidence includes the canonical edge-triggered `epoll` bug, the `WSA_IO_PENDING`-is-success contract, and completion order not matching submission order on Windows. Its conclusion: porting event-loop code across models does not make it slow — it makes it wrong.

The note flags what remains unsettled. Its NtTrace transcript of a traced Windows completion is reconstructed from documented output formats and the documented AFD call path, not a captured real trace, and is labelled inference. The claim that nginx favors edge-triggered `epoll` while Redis favors level-triggered, and the claim that Windows' Registered I/O never achieved broad adoption, each rest on a single low-credibility secondary source. The note also declines to resolve a live practitioner disagreement over whether `EAGAIN` is strictly required to re-arm an edge-triggered notification or merely the reliable way to detect an empty buffer, presenting both sides without adjudicating. Finally, it frames the readiness/completion split as "a good first cut, not a complete taxonomy" — `io_uring`, Windows IoRing, and `kqueue` show it is a design-space split each platform entered from a different side, not simply a Linux-versus-Windows one.

Referenced sources

- [epoll\(7\) — Linux manual page](#), Linux man-pages project **HIGH**
- [select\(2\) — Linux manual page](#), Linux man-pages project **HIGH**
- [I/O Completion Ports](#), Microsoft Learn **HIGH**
- [GetQueuedCompletionStatus function \(ioapiset.h\)](#), Microsoft Learn **HIGH**
- [WSARecv function \(winsock2.h\)](#), Microsoft Learn **HIGH**
- [Overlapped Input/Output \(Winsock\)](#), Microsoft Learn **HIGH**
- [WSAPoll function \(winsock2.h\)](#), Microsoft Learn **HIGH**
- [WSAPoll is broken](#), Daniel Stenberg **MEDIUM**
- [The edge-triggered misunderstanding](#), LWN.net comment thread **HIGH**
- [portable-interoperable / io-traits](#), Nick Cameron **MEDIUM**
- [Async IO with completion-model IO systems](#), Nick Cameron **MEDIUM**
- [Design overview — libuv documentation](#), libuv project **HIGH**
- [libuv — iocp-links.html \(design notes\)](#), libuv project **MEDIUM**
- [asyncio — Event Loop \(Python documentation\)](#), Python Software Foundation **HIGH**
- [selectors — High-level I/O multiplexing \(Python documentation\)](#), Python Software Foundation **HIGH**
- [SDL_net: windows: Reimplement poll\(\) with select\(\)](#), Simple DirectMedia Layer project **MEDIUM**
- [PollSet not working as intended on Windows — pocoproject/poco#3248](#), POCO C++ Libraries issue tracker **MEDIUM**
- [Issue 16507: Patch selectmodule.c to support WSAPoll on Windows](#), Python issue tracker **HIGH**

- [Under the Hood of AFD.sys Part 1: Investigating Undocumented Interfaces](#), Mateusz Lewczak **MEDIUM**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#), Win-sider Seminars & Solutions **HIGH**
- [libuv Event Loop Internals](#), Sujeet Jaiswal **MEDIUM**
- [Linux io_uring vs Windows I/O: A Technical Comparison](#), Mark Veltzer **LOW**
- [epoll: The API That Powers the Modern Internet](#), rzero.in **LOW**

12.41 V59 — Readiness and Completion I/O Models

This note argues that Linux and Windows answer a different question when a server must handle many connections without blocking. Linux's `select`, `poll`, and `epoll` answer "which descriptor would not block right now?" and hand the read or write back to the program — the readiness model. Windows' overlapped I/O and completion ports answer "which operation I already submitted has finished?", with the kernel filling the caller's buffer and delivering a completion packet — the completion model. The note treats this as the deepest architectural split between the two networking stacks, changing three things: who owns the buffer and for how long, the shape of the program's control flow, and when an error becomes observable. The evidence is drawn mostly from primary vendor and kernel documentation — the `epoll(7)` and `select(2)` man pages, and Microsoft Learn's I/O Completion Ports, `GetQueuedCompletionStatus`, `WSARecv`, `WSAPoll`, and Overlapped I/O references — supplemented by a Rust standard-library design study showing the two models are opposed at the level of API signatures, by libuv's own design documentation showing it must maintain two internal architectures to unify them, and by convergent independent reports from curl, POCO, SDL_net, and CPython that Windows' readiness-shaped `WSAPoll` cannot be trusted to emulate POSIX `poll` faithfully.

The note is explicit about what it does not claim. It does not present the Linux/Windows split as the whole taxonomy: Linux's `io_uring` is a completion interface, and Windows' newer IoRing is readiness's design arriving from the other direction, so the split is architectural rather than a vendor boundary. Several specific points are flagged as unsettled or unverified rather than established: whether `EAGAIN` is strictly required to re-arm an edge-triggered `epoll` notification is left as a live disagreement quoted from a Linux kernel mailing-list thread rather than resolved; the NtTrace transcript illustrating a Windows completion trace is marked as inference, since no published capture was located; and claims about Registered I/O's adoption and about nginx versus Redis choosing different `epoll` trigger modes are marked as reported from lower-credibility secondary sources rather than verified firsthand.

Referenced sources

- [epoll\(7\) – Linux manual page](#), Linux man-pages project **HIGH**
- [select\(2\) – Linux manual page](#), Linux man-pages project **HIGH**
- [I/O Completion Ports](#), Microsoft Learn **HIGH**
- [GetQueuedCompletionStatus function \(ioapiset.h\)](#), Microsoft Learn **HIGH**
- [WSARecv function \(winsock2.h\)](#), Microsoft Learn **HIGH**
- [Overlapped Input/Output \(Winsock\)](#), Microsoft Learn **HIGH**
- [WSAPoll function \(winsock2.h\)](#), Microsoft Learn **HIGH**
- [WSAPoll is broken](#), Daniel Stenberg **MEDIUM**
- [The edge-triggered misunderstanding](#), LWN.net comment thread **HIGH**
- [portable-interoperable / io-traits](#), Nick Cameron **MEDIUM**
- [Async IO with completion-model IO systems](#), Nick Cameron **MEDIUM**
- [Design overview – libuv documentation](#), libuv project **HIGH**
- [libuv – iocp-links.html \(design notes\)](#), libuv project **MEDIUM**
- [asyncio – Event Loop \(Python documentation\)](#), Python Software Foundation **HIGH**
- [selectors – High-level I/O multiplexing \(Python documentation\)](#), Python Software Foundation **HIGH**
- [SDL_net: windows: Reimplement poll\(\) with select\(\)](#), Simple DirectMedia Layer project **MEDIUM**
- [PollSet not working as intended on Windows – pocoproject/poco#3248](#), POCO C++ Libraries issue tracker **MEDIUM**
- [Issue 16507: Patch selectmodule.c to support WSAPoll on Windows](#), Python issue tracker **HIGH**
- [Under the Hood of AFD.sys Part 1: Investigating Undocumented Interfaces](#), Mateusz Lewczak **MEDIUM**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#), Win-sider Seminars & Solutions **HIGH**
- [libuv Event Loop Internals](#), Sujeet Jaiswal **MEDIUM**
- [Linux io_uring vs Windows I/O: A Technical Comparison](#), Mark Veltzer **LOW**
- [epoll: The API That Powers the Modern Internet](#), rzero.in **LOW**

12.42 V60 — Readiness and completion as the two models of asynchronous network I/O

This note establishes that Linux and Windows answer a different question when a program wants to know whether network I/O can proceed, and that the difference is architectural rather than stylistic. Linux's `select`, `poll`, and `epoll` answer "which descriptor would not block right now?" — the kernel reports state, and the program then performs the read or write itself. Windows' I/O completion ports answer "which operation I already submitted has finished?" — the program posts a buffer with the request, and the kernel fills it before notifying the program. The note traces this single difference — where the data copy happens relative to the notification — into three concrete consequences: who owns the buffer and for how long, what shape the program's control flow takes, and when an error becomes observable. It backs each consequence with vendor documentation: the Linux `epoll` and `select` man pages for the interest-list/ready-list mechanics and the "readiness is a hint, not a contract" caveat; Microsoft's IOCP, `GetQueuedCompletionStatus`, and `WSARecv` references for the completion model's buffer-ownership contract, out-of-order completions, and three-way error return; and a Rust standard-library design study for the claim that the two models cannot share a single buffer-handling abstraction. It also traces the split through Python's `selectors` / `asyncio` and through libuv, showing the model choice propagating into application-level API surfaces on both platforms.

The note is explicit about its limits. It does not claim the readiness/completion split is a complete taxonomy of async I/O designs — it names `io_uring`, Windows IoRing, and `kqueue` as related but distinct points in the same design space, and treats Linux/Windows as one instance of the split rather than its definition. Several claims are flagged as unsettled or reconstructed rather than directly evidenced: the Windows NtTrace transcript in the note is explicitly marked `[inference]`, built from documented output formats rather than a captured trace; the claim that nginx uses edge-triggered `epoll` while Redis uses level-triggered is marked `[reported]` from a single low-credibility source; and the assessment that Registered I/O never gained wide adoption is similarly `[reported]` from a low-credibility, uncited source. The note also records a live, unresolved disagreement among Linux practitioners over whether `EAGAIN` is strictly required to re-arm an edge-triggered notification, and states plainly that it takes no side in resolving it.

Referenced sources

- [epoll\(7\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [select\(2\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [I/O Completion Ports](#) — Microsoft Learn **HIGH**

- [GetQueuedCompletionStatus function \(ioapiset.h\)](#) — Microsoft Learn **HIGH**
- [WSARecv function \(winsock2.h\)](#) — Microsoft Learn **HIGH**
- [Overlapped Input/Output \(Winsock\)](#) — Microsoft Learn **HIGH**
- [WSAPoll function \(winsock2.h\)](#) — Microsoft Learn **HIGH**
- [WSAPoll is broken](#) — Daniel Stenberg **MEDIUM**
- [The edge-triggered misunderstanding](#) — LWN.net comment thread **HIGH**
- [portable-interoperable / io-traits](#) — Nick Cameron **MEDIUM**
- [Async IO with completion-model IO systems](#) — Nick Cameron **MEDIUM**
- [Design overview — libuv documentation](#) — libuv project **HIGH**
- [libuv — iocp-links.html \(design notes\)](#) — libuv project **MEDIUM**
- [asyncio — Event Loop \(Python documentation\)](#) — Python Software Foundation **HIGH**
- [selectors — High-level I/O multiplexing \(Python documentation\)](#) — Python Software Foundation **HIGH**
- [SDL_net: windows: Reimplement poll\(\) with select\(\)](#) — Simple DirectMedia Layer project **MEDIUM**
- [PollSet not working as intended on Windows](#) — pocoproject/poco#3248 — POCO C++ Libraries issue tracker **MEDIUM**
- [Issue 16507: Patch selectmodule.c to support WSAPoll on Windows](#) — Python issue tracker **HIGH**
- [Under the Hood of AFD.sys Part 1: Investigating Undocumented Interfaces](#) — Mateusz Lewczak **MEDIUM**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#) — Win-sider Seminars & Solutions **HIGH**
- [libuv Event Loop Internals](#) — Sujeet Jaiswal **MEDIUM**
- [Linux io_uring vs Windows I/O: A Technical Comparison](#) — Mark Veltzer **LOW**
- [epoll: The API That Powers the Modern Internet](#) — rzero.in **LOW**

12.43 V61 — The Cost of Waiting Behind the C10K Problem

The note argues the C10K problem — Dan Kegel's 1999 goal of serving ten thousand simultaneous connections from one machine — is usually retold as a story about slow I/O, and that this is wrong. Kegel's own page dismisses hardware as the bottleneck outright. What actually bounded servers was the cost of *waiting* on idle connections, not moving bytes for busy ones: under `select()` or `poll()`, an idle descriptor is still handed to the kernel and rescanned on every call, because neither interface remembers the watch set

between calls, so per-call cost tracks the size of the interest set rather than actual I/O performed. This is not only Kegel's framing — it was independently established by measurement. Banga, Mogul and Druschel (1999) state the cost model formally, as work proportional to the number of "interesting" descriptors rather than events, and explain why so many connections sit idle at once: open-connection count outruns request rate once realistic connection durations are accounted for. The decisive evidence is a controlled 2004 experiment (Gammo, Brecht, Shukla and Pariag) holding the request workload fixed while varying only idle connections: `select` and `poll` match `epoll`'s throughput with none idle, then lose up to 79% once 10,000 idle connections are added. A separate CPU-side measurement (Chandra and Mosberger, 2000) reaches the same conclusion four years earlier. The fixes — `epoll`, `kqueue`, `/dev/poll`, Windows IOCP — relocate the interest set so the kernel need not be re-told on every call; none change what `read()` or `write()` does once a descriptor is ready.

The note is explicit about its limits. It is not a claim that I/O was free: Kegel names disk I/O, which cannot be made non-blocking, as a real bottleneck for single-threaded readiness servers. It is not a blanket claim that `select`-based servers cannot perform — Chandra and Mosberger found that amortizing more work per call raises throughput even under heavy load. Several points stay unsettled: Kegel and Banga disagree, within a month of each other in 1999, on whether `poll()` had less overhead than `select()`, recorded as contradiction rather than adjudicated; which revision of Kegel's page first added the idle-descriptor explanation is left open, bounded only to a multi-year window. The lecture's planned demo numbers (28,000 `read` calls versus one `epoll_wait`) are flagged as targets, not measurements — the demo has not been built.

Referenced sources

- [The C10K problem \(current revision\)](#) — Dan Kegel **HIGH**
- [The C10K problem \(8 May 1999 revision\)](#) — Dan Kegel, via the Internet Archive **HIGH**
- [select\(2\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [poll\(2\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [epoll\(7\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [A Scalable and Explicit Event Delivery Mechanism for UNIX](#) — Gaurav Banga, Jeffrey C. Mogul, Peter Druschel, USENIX ATC 1999 **HIGH**
- [Comparing and Evaluating epoll, select, and poll Event Mechanisms](#) — Louay Gammo, Tim Brecht, Amol Shukla, David Pariag, Linux Symposium 2004 **HIGH**
- [Scalability of Linux Event-Dispatch Mechanisms](#) — Abhishek Chandra and David Mosberger, HP Labs TR-HPL-2000-174 **HIGH**

- Kqueue: A generic and scalable event notification facility — Jonathan Lemon, FreeBSD Project **HIGH**
- C10M: The C10M problem — Robert Graham, Errata Security **HIGH**
- C10k problem — Wikipedia **MEDIUM**
- Async IO on Linux: select, poll, and epoll — Julia Evans **MEDIUM**
- poll vs select vs event-based — Daniel Stenberg **MEDIUM**

12.44 V62 — The Cost Model Behind the C10K Problem

This note argues that the C10K problem — Dan Kegel's 1999 goal of serving ten thousand simultaneous clients from one machine — was never a story about slow hardware or slow I/O, despite being retold that way. Kegel's own page opens by ruling out hardware, and the note traces the real bottleneck to the cost of *waiting* on connections that were open but idle. Under a thread-per-connection design, an idle connection still holds a stack and a scheduler slot; under `select()` or `poll()`, an idle descriptor is still re-submitted to the kernel and rescanned on every call, because neither interface remembers the watch set between calls. The evidence is not just Kegel's own framing but independent measurement: Banga, Mogul and Druschel (1999) formalized the cost as proportional to the size of the "interesting" descriptor set rather than to actual events, and a 2004 University of Waterloo experiment isolated the effect directly — holding the request workload fixed while adding 10,000 idle connections degraded `select` and `poll` throughput by up to 79%, with `epoll` unaffected. A 2000 HP Labs study corroborated this from the CPU side. The note concludes that `epoll`, `kqueue`, and IOCP did not speed up `read()` or `write()`; they changed where the interest set lives, letting the kernel remember it instead of having the application re-state it every call.

The note is careful about what it does not claim. It explicitly does not say I/O cost was irrelevant in 1999 — Kegel's own page identifies disk I/O as a genuine bottleneck for single-threaded readiness servers, so "Waiting, not I/O" is a claim about what bounded the number ten thousand specifically, not a claim that I/O was free. It also flags an unresolved contradiction: Kegel's 1999 page and the Banga et al. paper, published a month apart, disagree on whether `poll()` had less overhead than `select()`, and no source consulted adjudicates it, though later measurement favors Banga. Several questions are left explicitly open, including exactly which revision of Kegel's page first introduced the idle-descriptor framing, and whether a contemporaneous production-server comparison (rather than a research prototype) of idle-connection cost exists — none credible was found. A planned classroom demonstration contrasting polled versus event-waited idle pipes is described as not yet built, and its target numbers are marked as unmeasured projections rather than results.

Referenced sources

- [The C10K problem \(current revision\)](#) — Dan Kegel **HIGH**
- [The C10K problem \(8 May 1999 revision\)](#) — Dan Kegel, via the Internet Archive **HIGH**
- [select\(2\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [poll\(2\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [epoll\(7\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [A Scalable and Explicit Event Delivery Mechanism for UNIX](#) — Gaurav Banga, Jeffrey C. Mogul, Peter Druschel **HIGH**
- [Comparing and Evaluating epoll, select, and poll Event Mechanisms](#) — Louay Gammou, Tim Brecht, Amol Shukla, David Pariag **HIGH**
- [Scalability of Linux Event-Dispatch Mechanisms](#) — Abhishek Chandra and David Mosberger **HIGH**
- [Kqueue: A generic and scalable event notification facility](#) — Jonathan Lemon **HIGH**
- [C10M: The C10M problem](#) — Robert Graham **HIGH**
- [C10k problem](#) — Wikipedia **MEDIUM**
- [Async IO on Linux: select, poll, and epoll](#) — Julia Evans **MEDIUM**
- [poll vs select vs event-based](#) — Daniel Stenberg **MEDIUM**

12.45 V63 — The Interest-Set Cost Behind the C10K Problem

This note argues that Dan Kegel's 1999 C10K problem — serving ten thousand simultaneous connections from one machine — was never a story about slow I/O, even though it is routinely retold that way. Kegel's own page opens by ruling out hardware as the bottleneck. The note's claim is that the real cost was waiting: under a thread-per-connection design, an idle connection still holds a stack and a scheduler slot; under `select()` or `poll()`, an idle descriptor is still re-passed to the kernel and rescanned on every call, because neither interface remembers anything between calls. This is not asserted from the man pages alone — `poll(2)` and `epoll(7)` say nothing about scanning cost — but from research: Banga, Mogul and Druschel (USENIX ATC 1999) formalize the cost as proportional to the size of the "interesting" descriptor set rather than the event rate, and a 2004 Linux Symposium experiment (Gammou, Brecht, Shukla, Pariag) holds the request workload fixed and varies only idle-connection count, finding `select / poll` match `epoll` with no idle connections but lose up to 79% of throughput once 10,000 idle connections are added. Chandra and Mosberger (HP Labs, 2000) corroborate from the CPU side, showing saturation from idle connections alone. The answers — `epoll`,

`kqueue`, and Windows IOCP — did not make `read()` or `write()` faster; each changed where the interest set lives and who re-states it, moving it into the kernel so it persists across calls instead of being rebuilt every time.

The note is explicit about what this claim does not cover. It is a claim about the bottleneck, not a claim that I/O was free — Kegel's own page names disk I/O blocking as a real problem for single-threaded readiness servers, a boundary this note defers rather than resolves. It flags a genuine, dated contradiction: Kegel's May 1999 page claims `poll()` had less overhead than `select()`, while Banga et al., published the same month, say the opposite; the note treats the later 2004 measurement (which degrades both mechanisms equally) as favoring Banga, without claiming the dispute is fully settled. Other points are left explicitly open: which exact revision of Kegel's page first introduced the idle-descriptor language, whether a contemporaneous Apache-prefork-versus-event-driven idle-connection comparison exists (none credible was found, so none is cited), and whether the readiness rescan or the per-connection memory footprint dominated at exactly ten thousand connections. The lecture's planned demo numbers (100 idle pipes, 28,000 `read` calls polled versus one `epoll_pwait` waited) are reproduced from the talk outline as targets, explicitly marked not-yet-observed since the demo has not been built.

Referenced sources

- [The C10K problem \(current revision\)](#), Dan Kegel — last updated 5 February 2014 (minor correction: 28 May 2019) **HIGH**
- [The C10K problem \(8 May 1999 revision\)](#), Dan Kegel, via the Internet Archive **HIGH**
- [select\(2\) — Linux manual page](#), Linux man-pages project **HIGH**
- [poll\(2\) — Linux manual page](#), Linux man-pages project **HIGH**
- [epoll\(7\) — Linux manual page](#), Linux man-pages project **HIGH**
- [A Scalable and Explicit Event Delivery Mechanism for UNIX](#), Gaurav Banga, Jeffrey C. Mogul, Peter Druschel — USENIX Annual Technical Conference (1999) **HIGH**
- [Comparing and Evaluating epoll, select, and poll Event Mechanisms](#), Louay Gammo, Tim Brecht, Amol Shukla, David Pariag — Linux Symposium (2004) **HIGH**
- [Scalability of Linux Event-Dispatch Mechanisms](#), Abhishek Chandra and David Mosberger — HP Labs Technical Report HPL-2000-174 (2000) **HIGH**
- [Kqueue: A generic and scalable event notification facility](#), Jonathan Lemon — FreeBSD Project, FREENIX Track, USENIX ATC (2001) **HIGH**
- [C10M: The C10M problem](#), Robert Graham — Errata Security (2013) **HIGH**
- [C10k problem](#), Wikipedia (retrieved 2026-09-20) **MEDIUM**
- [Async IO on Linux: select, poll, and epoll](#), Julia Evans (2017) **MEDIUM**

- [poll vs select vs event-based](#), Daniel Stenberg **MEDIUM**

12.46 V64 — The socket as a kernel object, published differently on Linux and Windows

The note's core claim is that a socket is a kernel object reached through a per-process handle, and that Linux and Windows do not disagree about whether the kernel treats that object as a file — both do — but about which layer makes that treatment a binding contract. On Linux, `socket()` returns a literal file descriptor: the kernel allocates a `struct socket`, binds it to a `struct file` on the `sockfs` pseudo-filesystem, and routes `read`, `write`, `close`, `fstat` and `select` through the same dispatch table used for ordinary files, so those calls work on a socket for a structural reason. The note verifies this directly: a C program and a Python script run on Linux 6.8.0 show `fstat` reporting `S_ISSOCK` and `/proc/self/fd` resolving to a `socket:[inode]` string traced back to the kernel's own formatting code. On Windows, the NT kernel underneath Winsock does the analogous thing — a socket is a file object on `\Device\Afd`, which is what lets `ReadFile`, `WriteFile` and completion ports operate on it — but Microsoft's own documentation makes the Winsock API, not the file-descriptor contract, the thing a programmer may rely on. That divergence in which layer carries the promise is what the note holds responsible for separate initialization and teardown (`WSAStartup` / `WSACleanup`), a separate error channel (`WSAGetLastError` rather than `errno`), a separate close call (`closesocket`, never `CloseHandle`), and a `select()` that works only on sockets on Windows.

The note is explicit about where this account is incomplete. It flags its reconciliation of two apparently contradictory Microsoft Learn pages — one saying a socket descriptor is not a file descriptor, the other saying a socket handle can optionally work with `ReadFile` / `WriteFile` — as its own inference, not a claim either page makes outright. Several supporting details, including specific AFD IOCTL codes and forensic techniques, come from individual researchers' blog posts and are marked as reported rather than independently confirmed. It treats the long-standing "Linux is readiness-based, Windows is completion-based" shorthand as already going stale, since `io_uring` has grown a first-class networking surface. And it describes Windows `AF_UNIX` support as only a partial convergence: the address family matches Linux semantics, but datagram and sequenced-packet socket types, and passing file descriptors over a socket, remain unsupported — so the object-model gap the note describes persists underneath an API surface that looks unified.

Referenced sources

- [socket\(2\) — Linux manual page](#), Michael Kerrisk / Linux man-pages project **HIGH**

- [socket\(7\) — Linux manual page, Michael Kerrisk / Linux man-pages project](#) **HIGH**
- [linux/net/socket.c, Linux kernel source tree](#) **HIGH**
- [Socket Data Type — Win32 apps, Microsoft](#) **HIGH**
- [Socket Handles — Win32 apps, Microsoft](#) **HIGH**
- [Error Codes - errno, h_errno and WSAGetLastError — Win32 apps, Microsoft](#) **HIGH**
- [WSAStartup function \(winsock.h\), Microsoft](#) **HIGH**
- [closesocket function \(winsock.h\), Microsoft](#) **HIGH**
- [CloseHandle function \(handleapi.h\), Microsoft](#) **HIGH**
- [Select, fd_set, and FD_XXX Macros — Win32 apps, Microsoft](#) **HIGH**
- [socket — Low-level networking interface \(Python documentation\), Python Software Foundation](#) **HIGH**
- [select — Waiting for I/O completion \(Python documentation\), Python Software Foundation](#) **HIGH**
- [AF_UNIX comes to Windows, Microsoft, Windows Command Line blog \(2017\)](#) **HIGH**
- [Improving AFD Socket Visibility for Windows Forensics & Troubleshooting, diversenok \(2025\)](#) **MEDIUM**
- [12 Days of HaXmas: Does it Blend Like a Duck? \(Failing Portably\), Rapid7 \(2014\)](#) **MEDIUM**
- [An Introductory 4.4BSD Interprocess Communication Tutorial, Computer Systems Research Group, UC Berkeley](#) **HIGH**
- [An Advanced 4.3BSD Interprocess Communication Tutorial, Samuel J. Leffler et al., UC Berkeley](#) **HIGH**
- [4.4BSD Networking Implementation Notes, Computer Systems Research Group, UC Berkeley](#) **HIGH**
- [rogerorr/NtTrace, Roger Orr](#) **HIGH**
- [Porting Socket Applications to Winsock — Win32 apps, Microsoft](#) **HIGH**
- [Under the Hood of AFD.sys Part 2: TCP handshake, Mateusz Lewczak \(2025\)](#) **MEDIUM**
- [cpython/Modules/socketmodule.h, Python Software Foundation](#) **HIGH**
- [Closing non-blocking socket causes leak, Microsoft](#) **HIGH**
- [vfs: mark pipes and sockets as stream-like file descriptors, Linus Torvalds, Linux kernel \(2019\)](#) **HIGH**
- [io_uring zero copy Rx — The Linux Kernel documentation, The Linux Kernel Organization](#) **HIGH**

- [Winsock Programmer's FAQ: General Winsock Information](#), Warren Young

MEDIUM

12.47 V65 — Sockets as kernel-owned file objects, on Linux and on Windows

This note treats "Everything is a file" as a literal claim about the Linux kernel's implementation of sockets, not a slogan, and builds the case from the source down. `socket()` allocates a `struct socket` and binds it to a `struct file` on the `sockfs` pseudo-filesystem via `sock_alloc_file()`, installing a `socket_file_ops` table so that `read`, `write`, `close`, `select` and `fstat` reach a socket through the exact dispatch path an ordinary file uses. The evidence is layered: the kernel source itself, the `socket(2)` man page's statement that `socket()` "returns a file descriptor that refers to that endpoint," and a reproduced experiment on Linux 6.8.0 in which `fstat` on a live socket reports `S_ISSOCK`, `/proc/self/fd/N` resolves to `socket:[<inode>]` matching that same inode, and a full TCP exchange runs to completion using nothing but `write`, `read` and `close`. The note then extends the argument to Windows: underneath Winsock, the NT kernel also represents a socket as a file object, a handle to a device named `\Device\Afd`, which is exactly what allows `ReadFile`, `WriteFile` and I/O completion ports to operate on it. The divergence it draws is therefore not about whether each kernel models a socket as a file — both do — but about which layer carries the binding promise: Linux makes the file-descriptor type itself the contract, while Windows holds the same kind of kernel object underneath but directs programmers to trust the Winsock API instead of the handle's type.

The note is careful about what this shared object model does not establish. A socket being a file descriptor does not make it behave like one: `read` can return fewer bytes than requested, `write` can block indefinitely on a full send buffer, a `read` of zero signals connection shutdown rather than end-of-file, and `lseek` is meaningless on a socket. It also declines to treat Windows' newer `AF_UNIX` support as closing the gap with Unix-domain sockets — the address family converged, but the Windows implementation still requires Winsock initialization, still closes via `closesocket`, and still omits passing a file descriptor over a socket, which the note calls the single most file-descriptor-dependent thing Unix does with the primitive. Two points are marked explicitly as the author's own reading rather than settled fact: the reconciliation of two Microsoft Learn pages that appear, read naively, to contradict each other over whether a socket handle can be a file handle, and the interpretation of why Windows `AF_UNIX` omits descriptor-passing. The note also names a limit in its own verification: `strace`

was unavailable in the environment used to check the Linux claim, so that check rests on `/proc` inspection rather than a captured syscall trace — a substitution the note discloses rather than hides.

Referenced sources

- [socket\(2\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project **HIGH**
- [socket\(7\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project **HIGH**
- [linux/net/socket.c](#) — Linux kernel source tree **HIGH**
- [Socket Data Type — Win32 apps](#) — Microsoft **HIGH**
- [Socket Handles — Win32 apps](#) — Microsoft **HIGH**
- [Error Codes - errno, h_errno and WSAGetLastError](#) — Win32 apps — Microsoft **HIGH**
- [WSAStartup function \(winsock.h\)](#) — Microsoft **HIGH**
- [closesocket function \(winsock.h\)](#) — Microsoft **HIGH**
- [CloseHandle function \(handleapi.h\)](#) — Microsoft **HIGH**
- [Select, fd_set, and FD_XXX Macros](#) — Win32 apps — Microsoft **HIGH**
- [socket — Low-level networking interface \(Python documentation\)](#) — Python Software Foundation **HIGH**
- [select — Waiting for I/O completion \(Python documentation\)](#) — Python Software Foundation **HIGH**
- [AF_UNIX comes to Windows](#) — Microsoft, Windows Command Line blog (2017) **HIGH**
- [Improving AFD Socket Visibility for Windows Forensics & Troubleshooting](#) — diversenok (2025) **MEDIUM**
- [12 Days of HaXmas: Does it Blend Like a Duck? \(Failing Portably\)](#) — Rapid7 (2014) **MEDIUM**
- [An Introductory 4.4BSD Interprocess Communication Tutorial](#) — Computer Systems Research Group, UC Berkeley **HIGH**
- [An Advanced 4.3BSD Interprocess Communication Tutorial](#) — Samuel J. Leffler et al., UC Berkeley **HIGH**
- [4.4BSD Networking Implementation Notes](#) — Computer Systems Research Group, UC Berkeley **HIGH**
- [rogerorr/NtTrace](#) — Roger Orr **HIGH**
- [Porting Socket Applications to Winsock](#) — Win32 apps — Microsoft **HIGH**
- [Under the Hood of AFD.sys Part 2: TCP handshake](#) — Mateusz Lewczak (2025) **MEDIUM**
- [cpython/Modules/socketmodule.h](#) — Python Software Foundation **HIGH**

- [Closing non-blocking socket causes leak](#) — Microsoft **HIGH**
- [vfs: mark pipes and sockets as stream-like file descriptors](#) — Linus Torvalds, Linux kernel (2019) **HIGH**
- [io_uring zero copy Rx](#) — The Linux Kernel documentation — The Linux Kernel Organization **HIGH**
- [Winsock Programmer's FAQ: General Winsock Information](#) — Warren Young **MEDIUM**

12.48 V66 — Sockets as kernel objects: the file-descriptor/Winsock divergence

This note establishes that a socket is a kernel object reached through a per-process handle, and that Linux and Windows differ only in which layer makes that handle's contract binding — not in whether the kernel treats it as a file. On Linux, `socket()` allocates a `struct socket` and binds it to a `struct file` on the `sockfs` pseudo-filesystem, so `read`, `write`, `close`, `fstat` and `select` dispatch through the same `socket_file_ops` table ordinary files use; the note backs this with the kernel source (`net/socket.c`) and a reproduced `/proc/self/fd` trace whose reported inode matches `fstat`'s across a live TCP exchange. On Windows, `socket()` returns an opaque `SOCKET`, and Microsoft's own documentation states that "a socket descriptor is not a file descriptor and must be used with the Winsock functions" — yet underneath Winsock, the NT kernel's Ancillary Function Driver (`afd.sys`) does hold the socket as a file object on `\Device\Afd`, which is exactly what lets `ReadFile`, `WriteFile` and I/O completion ports operate on it. Three consequences follow from where each platform draws the line: what else can touch the handle, how it is released (`close` versus `closesocket`, with `CloseHandle` explicitly forbidden), and how failure is reported (`errno` versus `WSAGetLastError`) — sharpest in `select()`, which on Windows works only on sockets because `fd_set` is an array rather than a bitmask.

The note is careful about what it does not claim. It does not assert that Windows fails to model sockets as files at the kernel level — it corrects an earlier framing along those lines, since the AFD evidence would not support "Unix treats sockets as files, Windows does not." It marks its own reconciliation of two seemingly conflicting Microsoft Learn pages as an inference rather than a stated position, and leaves the tension visible rather than resolved by either source. It also flags the "Linux is readiness-based, Windows is completion-based" shorthand as aging given `io_uring`'s newer completion-style networking surface, and treats the AFD internals — exact IOCTL call shape and control-code values — as resting on independent researcher write-ups, cross-checked but not first-party or independently reproduced.

Referenced sources

- `socket(2)` — Linux manual page, Michael Kerrisk / Linux man-pages project — <https://man7.org/linux/man-pages/man2/socket.2.html> **HIGH**
- `socket(7)` — Linux manual page, Michael Kerrisk / Linux man-pages project — <https://man7.org/linux/man-pages/man7/socket.7.html> **HIGH**
- `linux/net/socket.c`, Linux kernel source tree — <https://github.com/torvalds/linux/blob/master/net/socket.c> **HIGH**
- Socket Data Type — Win32 apps, Microsoft — <https://learn.microsoft.com/en-us/windows/win32/winsock/socket-data-type-2> **HIGH**
- Socket Handles — Win32 apps, Microsoft — <https://learn.microsoft.com/en-us/windows/win32/winsock/socket-handles-2> **HIGH**
- Error Codes - `errno`, `h_errno` and `WSAGetLastError` — Win32 apps, Microsoft — <https://learn.microsoft.com/en-us/windows/win32/winsock/error-codes-errno-h-errno-and-wsagetlasterror-2> **HIGH**
- `WSAStartup` function (`winsock.h`), Microsoft — <https://learn.microsoft.com/en-us/windows/win32/api/winsock/nf-winsock-wsastartup> **HIGH**
- `closesocket` function (`winsock.h`), Microsoft — <https://learn.microsoft.com/en-us/windows/win32/api/winsock/nf-winsock-closesocket> **HIGH**
- `CloseHandle` function (`handleapi.h`), Microsoft — <https://learn.microsoft.com/en-us/windows/win32/api/handleapi/nf-handleapi-closehandle> **HIGH**
- `Select`, `fd_set`, and `FD_XXX` Macros — Win32 apps, Microsoft — <https://learn.microsoft.com/en-us/windows/win32/winsock/select-and-fd---2> **HIGH**
- `socket` — Low-level networking interface (Python documentation), Python Software Foundation — <https://docs.python.org/3/library/socket.html> **HIGH**
- `select` — Waiting for I/O completion (Python documentation), Python Software Foundation — <https://docs.python.org/3/library/select.html> **HIGH**
- `AF_UNIX` comes to Windows, Microsoft, Windows Command Line blog (2017) — https://devblogs.microsoft.com/commandline/af_unix-comes-to-windows/ **HIGH**
- Improving AFD Socket Visibility for Windows Forensics & Troubleshooting, diversenok (2025) — <https://diversenok.github.io/2025/05/15/AFD-sockets.html> **MEDIUM**
- 12 Days of HaXmas: Does it Blend Like a Duck? (Failing Portably), Rapid7 (2014) — <https://www.rapid7.com/blog/post/2014/12/29/12-days-of-haxmas-failing-portably/> **MEDIUM**
- An Introductory 4.4BSD Interprocess Communication Tutorial, Computer Systems Research Group, UC Berkeley — <https://docs-archive.freebsd.org/44doc/psd/20.ipctut/paper.pdf> **HIGH**

- An Advanced 4.3BSD Interprocess Communication Tutorial, Samuel J. Leffler et al., UC Berkeley — https://sites.units.it/mumolo/reti_nettuno/bsdsocket.pdf **HIGH**
- 4.4BSD Networking Implementation Notes, Computer Systems Research Group, UC Berkeley — <https://docs-archive.freebsd.org/44doc/smm/18.net/paper.pdf> **HIGH**
- rogerorr/NtTrace, Roger Orr — <https://github.com/rogerorr/NtTrace> **HIGH**
- Porting Socket Applications to Winsock — Win32 apps, Microsoft — <https://learn.microsoft.com/en-us/windows/win32/winsock/porting-socket-applications-to-winsock> **HIGH**
- Under the Hood of AFD.sys Part 2: TCP handshake, Mateusz Lewczak (2025) — <https://leftarcode.com/posts/afd-reverse-engineering-part2/> **MEDIUM**
- cpython/Modules/socketmodule.h, Python Software Foundation — <https://github.com/python/cpython/blob/main/Modules/socketmodule.h> **HIGH**
- Closing non-blocking socket causes leak, Microsoft — <https://learn.microsoft.com/en-us/troubleshoot/windows/win32/close-non-blocked-socket-memory-leak> **HIGH**
- vfs: mark pipes and sockets as stream-like file descriptors, Linus Torvalds, Linux kernel (2019) — <https://github.com/torvalds/linux/commit/d8e464ec-c17b4444e9a3e148a9748c4828c6328c> **HIGH**
- io_uring zero copy Rx — The Linux Kernel documentation, The Linux Kernel Organization — <https://docs.kernel.org/networking/iou-zcrx.html> **HIGH**
- Winsock Programmer's FAQ: General Winsock Information, Warren Young — <https://tangentsoft.com/wskfaq/general.html> **MEDIUM**

12.49 V67 — Interprocess Communication

This note establishes a three-branch taxonomy for interprocess communication (IPC), organized around what the kernel guarantees about the shape of data crossing the process boundary: byte-stream transfer (no framing, kernel in the data path every transfer), message passing (kernel-preserved framing, same involvement), and shared state plus synchronisation (kernel present only at setup, framing left to the programmer). Its core principle is that a mechanism should be chosen for its semantics — framing, ordering, lifetime, who owns cleanup — rather than for speed, because a wrong semantic choice becomes a correctness bug. That rests on two practitioner sources, one stating "the choice affects correctness, not just speed" and another ranking correctness first, security second, performance last; both are flagged as practitioner sources, converging with the cited academic literature. Signals and file locking sit outside the split, being non-transports. A further divergence: POSIX named objects persist until explicitly unlinked and leak on an unclean crash, while Windows named kernel objects are reference-counted and disappear when the last handle closes, pulling an object out from un-

der a still-running peer. A cost model follows — branches 1 and 2 pay a per-transfer kernel cost for framing supplied free, while branch 3 hands the synchronisation problem to the programmer — illustrated with worked examples and production cases including Chrome's Mojo, Unix pipelines, and QNX.

The note is explicit about the limits of its own framing. It calls the three-branch split a pedagogical synthesis rather than the field's canonical taxonomy, since the dominant academic treatment splits IPC two ways — message passing versus shared memory — with framing a sub-dimension rather than a top-level branch. It acknowledges that sockets and Windows named pipes can sit in either of the first two branches depending on configuration, so any taxonomy built on mechanism names carries this ambiguity. It omits IPC latency numbers, since published figures vary by an order of magnitude with payload size and hardware, deferring that comparison to a companion note on shared memory. Three claims are marked unsettled within the note itself: a PyWin32 pipe-coalescing result is inference, since no Windows host was available; a Linux `strace` transcript is reconstructed from documented semantics rather than captured, since the authoring environment lacked `strace`; and a Windows NtTrace capture of `CreatePipe` is reproduced from a third party rather than re-captured. The note names all three as gaps to close before the material is taught.

Referenced sources

- [pipe\(7\) — Linux manual page](#), Michael Kerrisk / Linux man-pages project — HIGH
- [mq_overview\(7\) — Linux manual page](#), Michael Kerrisk / Linux man-pages project — HIGH
- [Anonymous Pipe Operations](#), Microsoft Learn (updated 2025) — HIGH
- [Named Pipe Type, Read, and Wait Modes](#), Microsoft Learn (updated 2025) — HIGH
- [Kernel Objects](#), Microsoft Learn (updated 2025) — HIGH
- [CreatePipe function \(namedpipeapi.h\)](#), Microsoft Learn (2021) — HIGH
- [3.2. IPC Models — Computer Systems Fundamentals](#), Charles Kirkpatrick / James Madison University (open textbook) — MEDIUM
- [Operating-System Construction: L10 Inter-Process Communication](#), Horst Schirmeier / TU Dresden (SS2025 lecture notes) — MEDIUM
- [The Cost of IPC: an Architectural Analysis](#), I. Gelado, J. Cabezas, L. Vilanova, N. Navarro (2007) — MEDIUM
- [Taxonomy of Unix IPC Methods — The Art of Unix Programming](#), ch. 7, Eric S. Raymond (2003) — MEDIUM
- [Mojo Style Guide — Chromium security documentation](#), The Chromium Project — HIGH
- [mojo/core — Chromium source documentation](#), The Chromium Project — HIGH

- [Windows: Prefer `ntdll` named pipe APIs over `kernel32` — ziglang/zig issue #19037, The-King-of-Toasters / Zig project \(2024\)](#) — **MEDIUM**
- [Named Pipes Under Attack: Securing Windows Interprocess Communication, BleepingComputer \(22 August 2026\)](#) — **MEDIUM**
- [CVE-2026-28727 — Local privilege escalation due to insecure Unix socket permissions, GitHub Advisory Database \(published 6 March 2026\)](#) — **HIGH**
- [An Empirical Study of Message-Passing Overhead, Michael L. Scott, Alan L. Cox \(1986\)](#) — **MEDIUM**
- [Interprocess Communication \(IPC\) — QNX Neutrino System Architecture, QNX Software Systems](#) — **HIGH**
- [Inter-Process Communication — CS 351 lecture slides, Michael Lee / Illinois Institute of Technology](#) — **MEDIUM**
- [6 Linux IPC Mechanisms: When to Use Each, William / phb-crystal-ball.org \(14 August 2026\)](#) — **LOW**
- [IPC Overview — decision matrices and anti-patterns, Anshik Bansal / OneNoughtOne](#) — **LOW**
- [Linux IPC Like You Actually Need to Use It, Satesh / unixy.io \(20 April 2026\)](#) — **LOW**
- [Concepts/IPC/Pipes and Named Pipes — issue #3419, GitHub \(forward-reference issue, note not yet written\)](#)
- [Concepts/IPC/Signals and Asynchronous Notification — issue #3420, GitHub \(forward-reference issue, note not yet written\)](#)
- [Concepts/IPC/Cross-Process Synchronisation — issue #3421, GitHub \(forward-reference issue, note not yet written\)](#)
- [Concepts/IPC/Local Sockets and Descriptor Passing — issue #3422, GitHub \(forward-reference issue, note not yet written\)](#)
- [Concepts/IPC/File Locking and Advisory Coordination — issue #3423, GitHub \(forward-reference issue, note not yet written\)](#)

12.50 V68 — `io_uring` and Windows `IoRing` as Ring-Based Completion Interfaces

This note describes how Linux's `io_uring` and Windows' `IoRing` replace a per-operation system call with a pair of shared-memory ring buffers: the application writes work descriptions into a submission queue and reads results out of a completion queue, so only the "kick" that wakes the kernel needs to cross the syscall boundary, and one kick can carry an arbitrary number of requests. It documents three properties that follow

from this design — the crossing count is amortised across a batch rather than eliminated (and can reach zero under Linux's `SQPoll` polling mode); completions can arrive out of submission order and must be correlated by a caller-supplied token; and error reporting moves from `errno` into a result field carried by the completion entry itself, on both platforms. The evidence is drawn mainly from the Linux manual pages and Jens Axboe's original design paper, from Microsoft's own `io_uring.h` reference documentation, and from a Windows-internals researcher's reverse-engineering of the Windows implementation, which the note quotes as calling it "almost identical" to the Linux original. It also walks the interface at three levels of directness — raw syscalls, the `liburing` helper library, and the absence of first-class Python support — and covers the 2023 finding that a majority of Google's kCTF kernel exploit submissions targeted `io_uring`, alongside its subsequent, only partial, uptake in PostgreSQL, ScyllaDB, and RocksDB, against Windows `IoRing`'s near-total lack of production adoption.

The note is explicit about what it does not claim. It does not treat `io_uring` as a straightforward performance win: a cited peer-reviewed study found a drop-in substitution for older interfaces gained only 6–10%, against roughly double that from redesigning around batching, and the gain vanishes for workloads that are not I/O-bound. It does not treat the security question as resolved either — it presents the seccomp incompatibility as a structural consequence of moving work off the syscall path, not a bug, and flags a 2026 CVE in which the newer per-task restriction mechanism itself was bypassed, calling this "not yet a clean redemption arc." Several claims are explicitly hedged rather than asserted: a reported-but-unconfirmed Windows API version number beyond what Microsoft documents, a benchmark comparison against SPDK sourced from a blog aggregation rather than a primary measurement, and hardening details drawn from a practitioner site rather than kernel documentation. The note also records, in its own closing section, that a live trace of a working `io_uring` call could not be captured in the sandboxed environment it was written in — the syscall was refused by a seccomp filter — so the predicted "one crossing per batch" trace rests on cited documentation rather than firsthand observation, and one adoption claim was dropped outright because its original source could not be re-fetched.

Referenced sources

- [io_uring\(7\) — Linux manual page — Linux man-pages project / kernel.org](#) **HIGH**
- [io_uring_setup\(2\) — Linux manual page — Linux man-pages project / kernel.org](#) **HIGH**
- [io_uring_enter\(2\) — Linux manual page — Linux man-pages project / kernel.org](#) **HIGH**
- [io_uring_register\(2\) — Linux manual page — Linux man-pages project / kernel.org](#) **HIGH**
- [Efficient IO with io_uring — Jens Axboe \(2019\)](#) **HIGH**
- [axboe/liburing — Jens Axboe \(2019–\)](#) **HIGH**

- [io_uring.h header — Win32 apps — Microsoft Learn \(2022\)](#) **HIGH**
- [CreateIoRing function \(io_uring.h\) — Microsoft Learn \(2021, updated 2025\)](#) **HIGH**
- [BuildIoRingReadFile function \(io_uring.h\) — Microsoft Learn \(2021, updated 2025\)](#) **HIGH**
- [SubmitIoRing function \(io_uring.h\) — Microsoft Learn \(2021, updated 2025\)](#) **HIGH**
- [BuildIoRingRegisterBuffers function \(io_uring.h\) — Microsoft Learn](#) **HIGH**
- [BuildIoRingRegisterFileHandles function \(io_uring.h\) — Microsoft Learn](#) **HIGH**
- [Learnings from kCTF VRP's 42 Linux kernel exploits submissions — Google Security Team / Google Online Security Blog \(2023\)](#) **HIGH**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations — Yarden Shafir / Winsider Seminars & Solutions \(2021\)](#) **HIGH**
- [One Year to I/O Ring: What Changed? — Yarden Shafir / Winsider Seminars & Solutions \(2022\)](#) **HIGH**
- [One I/O Ring to Rule Them All: A Full Read/Write Exploit Primitive on Windows 11 — Yarden Shafir / Winsider Seminars & Solutions \(2022\)](#) **HIGH**
- [Ringing in a new asynchronous I/O API — Jonathan Corbet / LWN.net \(2019\)](#) **HIGH**
- [Task-level io_uring restrictions — LWN.net \(2026\)](#) **HIGH**
- [uringscope: Portable, Low-Overhead Observability for io_uring — Rajarshi Chowdhury / arXiv \(2026\)](#) **MEDIUM**
- [io_uring Security Model — Linux Kernel Internals](#) **MEDIUM**
- [io_uring War Stories — Linux Kernel Internals](#) **MEDIUM**
- [\[PATCHSET RFC 0/2\] Per-task io_uring opcode restrictions — Jens Axboe / io-uring mailing list \(2026\)](#) **HIGH**
- [High-Performance DBMSs with io_uring: When and How to Use It — Jasny et al. / PVLDB vol. 19](#) **HIGH**
- [io_uring Security and Hardening: Disabling, Restricting and Auditing — Systems Hardening \(2026\)](#) **MEDIUM**
- [io_uring, SPDK, and the Kernel Bypass Wars — Murat Karslioglu \(2026\)](#) **MEDIUM**
- [Python io_uring binding landscape — liburing, uringcore, aioring, uring_file, asyncio-uring, and CPython issues 41271 / 44738 — PyPI / CPython issue tracker / GitHub project pages](#) **MEDIUM**
- [Submission Queue Polling — Lord of the io_uring — Shuveb Hussain](#) **MEDIUM**
- [Google Limiting IO_uring Use Due To Security Vulnerabilities — Michael Larabel / Phoronix \(2023\)](#) **MEDIUM**
- [oss-security: Our learnings from 42 Linux kernel exploits, we are limiting io_uring — Tamás Koczka / oss-security mailing list \(2023\)](#) **HIGH**

- [windows-ioring-sys — crate source and design notes — docs.rs \(2026\)](#) **MEDIUM**
- [io_uring Reads Your Files and Opens Sockets Without Calling the Syscalls Your auditd Rules Watch — Trackr.Live \(2026\)](#) **MEDIUM**
- [Asynchronous I/O \(AIO\) — PostgreSQL feature matrix — The PostgreSQL Global Development Group](#) **HIGH**
- [CVE-2026-80713 — io_uring: preserve task restrictions across exec — vuln.today \(2026\), quoting the upstream kernel commit message](#) **MEDIUM**
- [Offloading I/O to Dedicated Cores: An Asymmetric io_uring Backend for Seastar and ScyllaDB — ScyllaDB \(2026\)](#) **HIGH**

12.51 V69 — Blocking as a Thread-Scheduling Decision, Not a Device Property

This note argues that "blocking" is not a property of a socket, a disk, or a network card, but a name for what the operating system's scheduler does to a thread when the operation it requested cannot be satisfied immediately: the kernel takes the thread off the run queue, parks it in a wait state, records a wakeup, and hands the CPU to another runnable thread. It backs this with each kernel's own vocabulary for the same idea — Linux's `ps(1)` process-state codes, where `D` is defined as uninterruptible sleep "usually IO", and Windows' documented `wait` thread state, both framed as scheduling facts rather than hardware facts. From there the note draws its contestable claim: the same operation, on the same device, can block or not depending on how it is requested. Linux's `O_NONBLOCK` flag and Windows' `ioctlsocket(FIONBIO)` toggle the identical property, so the same read either parks the calling thread or returns immediately with `EAGAIN` or `WSAEWOULDBLOCK` — the device does not change, only the scheduler's instruction does. That reframing is what lets the note treat the three standard avoidance mechanisms — non-blocking mode, readiness-based notification (`epoll / kqueue`), and completion-based notification (IOCP/ `io_uring`) — as three answers to one question, who gets parked and where, rather than an unordered feature list. It also uses the reframing to correct a common misconception: marking a function `async` does not itself make its I/O non-blocking, since the keyword only names a suspension/resumption discipline implemented on top of whichever of the three OS mechanisms the platform's runtime happens to use underneath.

The note is explicit about where the claim stops holding: for buffered regular-file I/O, the requesting flag has no vote at all — both Linux and Windows document that path as blocking regardless of how it was asked, which the note treats as a major reason completion-based interfaces exist. It also leaves several questions marked open rather than resolved: how completely `io_uring` closes the buffered-file gap without `O_DIRECT`;

whether Windows' file-I/O caveat is structurally the same phenomenon as Linux's absence of a readiness abstraction, or only similar in effect; a live-observation recipe that works end-to-end on both platforms; and whether the readiness/completion split is exhaustive or just the two families that happened to win. The note treats its own synthesis of the two kernels' vocabulary as inference rather than a claim any single source makes directly.

Referenced sources

- [epoll\(7\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project **HIGH**
- [open\(2\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project **HIGH**
- [I/O Completion Ports — Win32 apps](#) — Microsoft Learn **HIGH**
- [GetQueuedCompletionStatus function \(ioapiset.h\)](#) — Microsoft Learn **HIGH**
- [Efficient IO with io_uring](#) — Jens Axboe **HIGH**
- [Winsock IOCTLS \(Winsock2.h\)](#) — Microsoft Learn **HIGH**
- [Nonblocking Input/Output — Win32 apps](#) — Microsoft Learn **HIGH**
- [Synchronous and Asynchronous I/O — Win32 apps](#) — Microsoft Learn **HIGH**
- [Asynchronous disk I/O appears synchronous](#) — Microsoft Learn **HIGH**
- [asyncio — Event loop \(Python 3.14 documentation\)](#) — Python Software Foundation **HIGH**
- [asyncio — Platform Support \(Python 3.14 documentation\)](#) — Python Software Foundation **HIGH**
- [ps\(1\) — PROCESS STATE CODES](#) — procps-ng / Linux man-pages project **HIGH**
- [Introduction to Kernel Dispatcher Objects](#) — Microsoft Learn **HIGH**
- [ThreadState Enum \(System.Diagnostics\)](#) — Microsoft Learn **HIGH**
- [Exercise 3 — Understand Critical Path and Wait Analysis](#) — Microsoft Learn **HIGH**
- [Kernel Korner — Sleeping in the Kernel](#) — Linux Journal **MEDIUM**
- [Non-blocking I/O with regular files](#) — Rémi Denis-Courmont **MEDIUM**
- [Sync vs async IO under the hood: blocking syscalls, epoll, and the context-switch tax — 0xEAX](#) **LOW**
- [Practical difference between epoll and Windows I/O Completion Ports \(IOCP\)](#) — George / ulduzsoft **MEDIUM**
- [io_uring vs epoll](#) — kernel-internals.org **LOW**
- [I/O Multiplexing — select, poll, epoll, kqueue, io_uring](#) — Unseel **LOW**
- [On Linux asynchronous file I/O](#) — Bert Hubert **MEDIUM**
- [Epoll Is Not Async: Why Non-Blocking I/O Needs a Runtime](#) — Ali Afkhami **LOW**

- The C# Async Cliff: When Synchronous Code Is the Right Engineering Choice — adrianrcs **LOW**

12.52 V70 — Network Buffering and Delivery Semantics

This note establishes that a socket `send()` call is not a transmission but a copy: it moves bytes from the calling program's address space into a kernel-owned send buffer and returns a count of bytes accepted, nothing more. It documents this in parallel on Linux and Windows — the Linux `send(2)` man page states that "no indication of failure to deliver is implicit in a `send()`," and the Winsock reference states directly that successful completion "does not indicate that the data was successfully delivered and received to the recipient." From that bundled non-guarantee it derives two structural consequences. First, per RFC 9293's statement that TCP "guarantees no correlation" between segment boundaries and application read/write buffer boundaries, TCP is a byte stream rather than a message stream: one `send()` call does not correspond to one `recv()` call, and the application must supply its own framing — fixed length, a delimiter, a length prefix, or connection close. Second, because the kernel rather than the application decides when to drain that buffer, small writes can be held by Nagle's algorithm (RFC 896) and further delayed by the receiver's delayed-ACK policy (RFC 1122); the note documents, citing peer-reviewed measurements of production NNTP traffic, that the two mechanisms can interact to stall a request/response exchange for up to 200 milliseconds even on an idle, high-bandwidth link. It supports these claims with syscall-level tracing evidence on both platforms (`strace` on Linux, NtTrace and AFD.sys reverse engineering on Windows) and with each platform's own buffer-tuning documentation.

The note is explicit about what it does not establish. It does not claim that a TCP acknowledgment means the receiving application processed the data — only that the receiving kernel took responsibility for it, per RFC 793's original wording that an acknowledgment "does not guarantee that the data has been delivered to the end user." It treats the absence of any socket-level equivalent to `fsync()` as a structural feature of the network case, not a gap to be filled. Several points are flagged as unsettled rather than resolved: the `strace` trace shape it shows is marked as reconstructed from documented syscall signatures rather than captured live, and is left for lab confirmation; the default Windows `SO_SNDBUF` value when autotuning is disabled is left as an open gap, since no first-party Microsoft figure was located; whether the Nagle/delayed-ACK deadlock reproduces reliably over WSL2 loopback is flagged as needing validation before classroom use; and current Linux delayed-ACK timer behavior is left unstated, since modern kernels use an adaptive timer rather than the fixed 200 ms value documented

for BSD-derived and Windows stacks. The claim that Node.js and .NET disable Nagle by default is marked as reported rather than verified, since it comes from search-result summaries rather than those projects' own documentation.

Referenced sources

- [send\(2\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project (2025) **HIGH**
- [tcp\(7\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project (2025) **HIGH**
- [send function \(winsock2.h\) — Win32 apps](#) — Microsoft Learn (2024) **HIGH**
- [IPPROTO_TCP socket options — Win32 apps](#) — Microsoft Learn (2025) **HIGH**
- [SIO_IDEAL_SEND_BACKLOG_QUERY Control Code — Win32 apps](#) — Microsoft Learn (2021) **HIGH**
- [RFC 896 — Congestion Control in IP/TCP Internetworks](#) — John Nagle / Ford Aerospace and Communications Corporation (1984) **HIGH**
- [RFC 1122 §4.2.3.2 — When to Send an ACK Segment](#) — R. Braden (ed.) / IETF (1989) **HIGH**
- [RFC 9293 — Transmission Control Protocol \(TCP\)](#) — Wesley M. Eddy (ed.) / IETF (2022) **HIGH**
- [socket — Low-level networking interface \(Python 3 documentation\)](#) — Python Software Foundation (2026) **HIGH**
- [Socket Programming HOWTO \(Python 3 documentation\)](#) — Gordon McMillan / Python Software Foundation **HIGH**
- [TCP Performance problems caused by interaction between Nagle's Algorithm and Delayed ACK](#) — Stuart Cheshire (2005) **MEDIUM**
- [Rethinking the TCP Nagle Algorithm](#) — Jeffrey C. Mogul & Greg Minshall / ACM SIGCOMM Computer Communication Review (2001) **HIGH**
- [Application performance pitfalls and TCP's Nagle algorithm](#) — G. Wayne Minshall, Yasushi Saitō, Jeffrey C. Mogul & Ben Verghese / ACM (2000) **HIGH**
- [NtTrace — Native API tracing for Windows](#) — Roger Orr **HIGH**
- [New registry entry for controlling TCP acknowledgment \(ACK\) behavior](#) — Microsoft Learn (2026), KB 328890 **HIGH**
- [Winsock Programmer's FAQ: How to Use TCP Effectively](#) — Warren Young / tangent-soft.com **MEDIUM**
- [Under the Hood of AFD.sys Part 3: Sending TCP packets](#) — Mateusz Lewczak (2025) **MEDIUM**
- [RFC 793 — Transmission Control Protocol](#) — Jon Postel (ed.) / DARPA (1981) **HIGH**

- [Enable TCP_NODELAY by default for TCP sockets \(Disable Nagle Algorithm\)](#) — CPython issue #144254 — python/cpython issue tracker (2026) **HIGH**
- [Interpret Windows Native API call arguments](#) — Stack Overflow **LOW**

12.53 V71 — Destination-dependent buffering of standard output

This note establishes that a C program's standard output does not decide, on its own, when bytes leave the process — the C standard library decides, by testing what file descriptor 1 is attached to at the moment its buffer is first allocated. Attached to a terminal, stdout is line-buffered, so every newline forces a `write()`. Attached to a pipe or a regular file, it is fully (block) buffered, and a newline does nothing until the buffer fills, the stream closes, or the process exits normally. The note traces this to C99 §7.19.3's rule that the streams are "fully buffered if and only if the stream can be determined not to refer to an interactive device," and to glibc's actual implementation in `_IO_file_doallocate`, which `fstat`s the descriptor and only calls `isatty()` if it is a character device. The consequence the note foregrounds is the crash case: because `abort()` and other abnormal terminations "need not close all files properly," a block-buffered stdout sitting in process memory is simply lost, while the always-unbuffered (in C) or line-buffered (in Python, since 3.9) stderr survives — which is why a crashing program tells you why but not what it was doing. The note extends the same test to Python's own buffering layer and to the Windows CRT, where line buffering is not implemented at all and collapses to full buffering.

The note is explicit about what it does not establish. It leaves the Windows UCRT's default stdout mode for a console versus a redirected handle as an open question, since no primary Microsoft source documents an `isatty`-equivalent selection test — only the documented collapse of line buffering into full buffering is asserted with confidence. It similarly declines to resolve why glibc's current tree still branches on `st_blksize` when a 2016 patch proposed an unconditional 8192-byte buffer, and it does not claim a Windows analogue to `LD_PRELOAD`-based `stdbuf` exists, or state how stdout attached to a socket behaves, since no measurement was taken for that case. The note also marks its own `strace` transcript as unexecuted, offering it only as a plausible illustration alongside a byte-count method that was actually run. Most broadly, it confines its claims to POSIX/glibc and to Python's own reimplementations of the same policy, and says explicitly that nothing in it transfers by analogy to Java, .NET, Node, or Rust, each of which would need to be checked separately.

Referenced sources

- [setbuf\(3\) — Linux manual page, Michael Kerrisk / man7.org — Linux man-pages project](#) **HIGH**

- [The C99 Draft \(N869, 18 January 1999\), §7.19.3 Files, ISO/IEC JTC1/SC22/WG14 \(1999\)](#) **HIGH**
- [glibc source: `libio/filedoalloc.c` \(`\_IO\_file\_doallocate`\), GNU C Library, revision `glibc-2.39-288` \(2024\)](#) **HIGH**
- [Output is buffered and lost if the program aborts · Issue #1235 · actions/runner, GitHub / actions-runner issue tracker](#) (2021) **HIGH**
- [sys — System-specific parameters and functions: `sys.stdout` / `sys.stderr`, Python Software Foundation — CPython 3.14 documentation](#) **HIGH**
- [1. Command line and environment — the `-u` option, Python Software Foundation — CPython 3.14 documentation](#) **HIGH**
- [Why pipes sometimes get "stuck": buffering, Julia Evans](#) (2024) **HIGH**
- [How stdbuf works, Harry Marr](#) (2022) **MEDIUM**
- [stdio buffering, Pádraig Brady](#) **HIGH**
- [setvbuf — C runtime library reference, Microsoft Learn](#) (2020, msvc-170) **HIGH**
- [Issue 13601: `sys.stderr` should be line-buffered when `stderr` is not a TTY, Python issue tracker](#) (2011–2019) **HIGH**
- [libio: Always use 8192 bytes as the default buffer size \[BZ #4099\], Florian Weimer / glibc patchwork](#) (2016) **HIGH**
- [Implementation-Defined ISO/IEC C99 Behavior — Oracle Developer Studio C User's Guide, Oracle Corporation](#) **HIGH**
- [Stdout Buffering, Evan Klitzke](#) **MEDIUM**
- [Buffering in pipes, Thomas Habets](#) (2008) **MEDIUM**
- [Why won't journalctl show logs when the program output is piped?, Super User \(community Q&A\)](#) **MEDIUM**
- [Kubernetes "logs" does not show any output for a simple python program running in a container in a pod, Stack Overflow \(community Q&A, 2021\)](#) **MEDIUM**
- [Rationale for International Standard — Programming Languages — C, revision 5.10, ISO/IEC JTC1/SC22/WG14](#) (2003) **HIGH**
- [coreutils source: `src/stdbuf.c`, GNU coreutils](#) **HIGH**
- [SSH output isn't line buffered?, Unix & Linux Stack Exchange \(community Q&A\)](#) **MEDIUM**

12.54 V72 — Virtual memory and the page cache across Linux and Windows

This note's organizing claim is that virtual memory is not a performance feature layered on top of memory but the mechanism by which memory works at all, and that most of the confusing numbers a student meets — a benchmark that runs 40x faster on a second run, a process whose reported memory use does not match what it is actually using, a program with a million page faults that is perfectly healthy — are direct consequences of that mechanism being invisible. It derives this from three properties: address translation is indirect, so "how much memory does this process use?" is ill-posed; population is lazy, so `malloc()` and `mmap()` create address space rather than memory; and physical memory behaves as a reclaimable cache rather than a fixed container. The unifying consequence is that a page fault is the interface by which the kernel enforces those properties, not an error path, and the note tables fault outcomes — minor/soft, demand-zero, copy-on-write, and major/hard — against their approximate cost on Linux and Windows, drawing on Linux kernel documentation and Microsoft's Win32 working-set documentation. It then works through the page cache's role in making repeated reads fast and the resulting benchmarking trap when cold and warm runs are conflated; why RSS misleads because shared pages are double-counted across processes; and `mmap`'s real trade-off — relocating cost from an explicit syscall into an implicit, unpredictable fault — illustrated with a CIDR 2022 measurement of `mmap` throughput collapsing under page-cache eviction.

The note is explicit about what it does not settle. It marks its own page-granular model as a teaching approximation rather than a literal description of current kernel internals, since Linux has largely replaced `struct page` with folios in the page-cache API, and it treats Windows's multi-generational LRU reclaim policy as genuinely contested as of March 2026 rather than textbook-settled. It also flags two of its own claims — the syscall count behind a buffered Python file read, and the fault count from populating a 100 MB byte array — as inference rather than citation, recommending they be backfilled by lab measurement rather than treated as established figures. Two of the five topics it introduces, page faults as mechanism and `mmap`'s cost relocation, are deliberately left undeveloped, with the note stating that the full argument belongs to companion zettels it does not itself contain.

Referenced sources

- [Concepts overview — The Linux Kernel documentation](#), The Linux Kernel Organization (2026) — **HIGH**
- [Windows Kernel Internals: Cache Manager](#), Microsoft Corporation, University of Tokyo lecture series (n.d.) — **HIGH**

- [Page fault — Grokipedia, Grokipedia \(2026\)](#) — MEDIUM
- [Working Set - Win32 apps, Microsoft Learn \(2026\)](#) — HIGH
- [Copy-on-Write and the Page-Fault Lifecycle, Step by Step, PandaStack \(n.d.\)](#) — MEDIUM
- [How to Check Memory Usage per Process in Linux, GoLinuxCloud \(n.d.\)](#) — MEDIUM
- [perf-trace\(1\) — Linux manual page, man7.org / Linux man-pages project \(2026\)](#) — HIGH
- [The Basics of Page Faults, Microsoft Community Hub, AskPerf team blog \(n.d.\)](#) — MEDIUM
- [Windows Performance Monitor basics, SQLShack \(n.d.\)](#) — MEDIUM
- [Memory Management Tracing Events - Win32 apps, Microsoft Learn \(2021\)](#) — HIGH
- [Why does Linux, by default, overcommit memory?, Unix & Linux Stack Exchange \(n.d.\)](#) — MEDIUM
- [Virtual vs Physical vs Resident Memory, Linux Kernel Internals \(n.d.\)](#) — MEDIUM
- [The /proc Filesystem — The Linux Kernel documentation, The Linux Kernel Organization \(2026\)](#) — HIGH
- [RAMMap - Sysinternals, Microsoft Learn / Sysinternals \(2026\)](#) — HIGH
- [Accurately measuring memory usage, Phusion \(n.d.\)](#) — MEDIUM
- [Readahead - Linux Kernel Internals, Linux Kernel Internals \(n.d.\)](#) — MEDIUM
- [Page Cache Writeback - Linux Kernel Internals, Linux Kernel Internals \(n.d.\)](#) — MEDIUM
- [Windows Memory Management in Windows 10 and Windows 11, CyberRaiden \(2025\)](#) — LOW
- [Fair Benchmarking Considered Difficult: Common Pitfalls In Database Performance Testing, Raasveldt, Holanda, Gubner & Mühleisen, DBTest \(2018\)](#) — HIGH
- [Understanding and tuning the page cache on Debian, Big Iron \(2026\)](#) — MEDIUM
- [Excessive Page Faults Generated By Windows Applications, Broadcom Knowledge Base \(n.d.\)](#) — MEDIUM
- [Common disk benchmarking mistakes, Stefan Hajnoczi \(2017\)](#) — HIGH
- [Memory-Mapped I/O - Linux Kernel Internals, Linux Kernel Internals \(n.d.\)](#) — MEDIUM
- [File Mapping - Win32 apps, Microsoft Learn \(2026\)](#) — HIGH
- [Creating Named Shared Memory, MicrosoftDocs/win32 repository \(2026\)](#) — HIGH

- [Memory-Mapped Files \(mmap\): A Practical Guide to Faster I/O and Shared Memory](#), martinuke0 (2025) — MEDIUM
- [mmap SIGBUS error and initializing the file](#), Stack Overflow (2017) — MEDIUM
- [MapViewOfFile function \(memoryapi.h\)](#), Microsoft Learn (2026) — HIGH
- [mmap — Memory-mapped file support](#), Python Software Foundation (2026) — HIGH
- [Are You Sure You Want to Use MMAP in Your Database Management System?](#), Crotty, Leis & Pavlo, CIDR (2022) — HIGH
- [Teaching Operating Systems with FreeBSD through Tracing, Analysis, and Experimentation](#), Neville-Neil et al., USENIX ;login: (2017) — HIGH
- [Teaching Operating Systems Concepts with Execution Visualization](#), Giraldeau, Dagenais & Boucheneb, ASEE (n.d.) — HIGH
- [Mastering strace, perf, and eBPF — 2026 Cheat Sheet](#), Tech Bytes (2026) — MEDIUM
- [CS 110: Principles of Computer Systems — assign3](#), Stanford University (2021) — HIGH
- [Page Cache - Linux Kernel Internals](#), Linux Kernel Internals (n.d.) — MEDIUM
- [On pages and folios](#), Jonathan Corbet, LWN.net (2026) — HIGH
- [Reconsidering the multi-generational LRU](#), Jonathan Corbet, LWN.net (2026-03-05) — HIGH

12.55 V73 — Ring-Based Asynchronous I/O: io_uring and Windows IoRing

This note treats `io_uring` (Linux, kernel 5.1) and `IoRing` (Windows, build 22000) as two implementations of one idea: instead of trapping into the kernel once per operation, the application and the kernel share a submission queue and a completion queue in memory, and only the "kick" that tells the kernel to look at the queue costs a system call. It argues the two platforms converged from opposite starting points — Linux, native to readiness-based polling, adopted completion semantics because `epoll` cannot describe a regular file's readiness and native `aio` was asynchronous only for `O_DIRECT`; Windows, already completion-native through IOCP, adopted the shared-memory ring as a faster transport for semantics it already had. Three properties follow: batching amortises but does not eliminate the per-crossing cost, and a polling mode removes it entirely; completions can arrive out of submission order, correlated by a caller-supplied token; and errors are reported as data inside the completion entry rather than through `errno` or a failed submit call. The note also traces a security history: Google's 2023 disclosure that `io_uring` accounted for 60% of kCTF kernel-exploit submissions, the mechanism behind it (deferred work executed outside the sec-

comp-checked syscall path), and real-world adoption in PostgreSQL 18, ScyllaDB/Seastar, and RocksDB, with measured gains ranging from modest to substantial depending on how deeply the interface is integrated into the surrounding architecture.

The note is explicit about limits. Swapping the interface in as a drop-in replacement for `libaio` or `epoll` yields only modest gains; the larger gains require redesigning around batching and registered buffers, and even then the benefit shrinks for workloads that are not I/O-bound. Availability cannot be assumed: `sysctls`, container `seccomp` profiles, and runtime defaults can and do disable the interface outright, and the note treats this as a first-order limitation rather than an edge case. Observability is degraded — a standard syscall tracer sees only ring setup, not the requests flowing through it — so tracing requires kernel tracepoints, eBPF, or, on Windows, ETW instead. On security, the note does not claim the 2023-era problems are resolved: newer per-task restriction mechanisms have themselves had bypasses, so it frames the mitigation history as ongoing rather than settled. It also does not claim Windows `IoRing` has seen real adoption — the interface's own documenter reported no known applications using it a year after release — and it explicitly does not extend its findings to networking, since `IoRing`'s operation set covers only file reads, writes, flushes, and cancellation.

Referenced sources

- [io_uring\(7\) — Linux manual page](#) — Linux man-pages project / kernel.org **HIGH**
- [io_uring_setup\(2\) — Linux manual page](#) — Linux man-pages project / kernel.org **HIGH**
- [io_uring_enter\(2\) — Linux manual page](#) — Linux man-pages project / kernel.org **HIGH**
- [io_uring_register\(2\) — Linux manual page](#) — Linux man-pages project / kernel.org **HIGH**
- [Efficient IO with io_uring](#) — Jens Axboe (2019) **HIGH**
- [axboe/liburing](#) — Jens Axboe (2019–) **HIGH**
- [io_uring.h header](#) — Win32 apps — Microsoft Learn (2022) **HIGH**
- [CreateIoRing function \(io_uring.h\)](#) — Microsoft Learn (2021, updated 2025) **HIGH**
- [BuildIoRingReadFile function \(io_uring.h\)](#) — Microsoft Learn (2021, updated 2025) **HIGH**
- [SubmitIoRing function \(io_uring.h\)](#) — Microsoft Learn (2021, updated 2025) **HIGH**
- [BuildIoRingRegisterBuffers function \(io_uring.h\)](#) — Microsoft Learn **HIGH**
- [BuildIoRingRegisterFileHandles function \(io_uring.h\)](#) — Microsoft Learn **HIGH**
- [Learnings from kCTF VRP's 42 Linux kernel exploits submissions](#) — Google Security Team / Google Online Security Blog (2023) **HIGH**

- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#) — Yarden Shafir / Winsider Seminars & Solutions (2021) **HIGH**
- [One Year to I/O Ring: What Changed?](#) — Yarden Shafir / Winsider Seminars & Solutions (2022) **HIGH**
- [One I/O Ring to Rule Them All: A Full Read/Write Exploit Primitive on Windows 11](#) — Yarden Shafir / Winsider Seminars & Solutions (2022) **HIGH**
- [Ringing in a new asynchronous I/O API](#) — Jonathan Corbet / LWN.net (2019) **HIGH**
- [Task-level io_uring restrictions](#) — LWN.net (2026) **HIGH**
- [uringscope: Portable, Low-Overhead Observability for io_uring](#) — Rajarshi Chowdhury / arXiv (2026) **MEDIUM**
- [io_uring Security Model](#) — Linux Kernel Internals **MEDIUM**
- [io_uring War Stories](#) — Linux Kernel Internals **MEDIUM**
- [\[PATCHSET RFC 0/2\] Per-task io_uring opcode restrictions](#) — Jens Axboe / io-uring mailing list (2026) **HIGH**
- [High-Performance DBMSs with io_uring: When and How to Use It](#) — Jasny et al. / PVLDB vol. 19 **HIGH**
- [io_uring Security and Hardening: Disabling, Restricting and Auditing](#) — Systems Hardening (2026) **MEDIUM**
- [io_uring, SPDK, and the Kernel Bypass Wars](#) — Murat Karslioglu (2026) **MEDIUM**
- [Python io_uring binding landscape](#) — `liburing`, `uringcore`, `aioring`, `uring_file`, `asyncio-uring`, and CPython issues 41271 / 44738 — PyPI / CPython issue tracker / GitHub project pages **MEDIUM**
- [Submission Queue Polling](#) — Lord of the io_uring — Shuveb Hussain **MEDIUM**
- [Google Limiting IO_uring Use Due To Security Vulnerabilities](#) — Michael Larabel / Phoronix (2023) **MEDIUM**
- [oss-security: Our learnings from 42 Linux kernel exploits, we are limiting io_uring](#) — Tamás Koczka / oss-security mailing list (2023) **HIGH**
- [windows-ioring-sys](#) — crate source and design notes — docs.rs (2026) **MEDIUM**
- [io_uring Reads Your Files and Opens Sockets Without Calling the Syscalls Your auditd Rules Watch](#) — Trackr.Live (2026) **MEDIUM**
- [Asynchronous I/O \(AIO\)](#) — PostgreSQL feature matrix — The PostgreSQL Global Development Group **HIGH**
- [CVE-2026-80713](#) — [io_uring: preserve task restrictions across exec](#) — vuln.today (2026), quoting the upstream kernel commit message **MEDIUM**
- [Offloading I/O to Dedicated Cores: An Asymmetric io_uring Backend for Seastar and ScyllaDB](#) — ScyllaDB (2026) **HIGH**

12.56 V74 — Readiness and completion I/O notification models

This note argues that Linux and Windows answer a different question when a program asks about socket state, and that the difference is architectural rather than a matter of API style. Linux's readiness model — `select`, `poll`, `epoll` — tells a program which descriptor would not block right now, and the program then performs the read or write itself; Windows' completion model — overlapped I/O drained through an I/O completion port — tells a program that an operation it already submitted has finished, with the kernel having copied the bytes into a buffer supplied in advance. The note traces that one distinction through three consequences it treats as structural: who owns the buffer and for how long, what shape the program's control flow must take, and when an I/O error becomes visible. Evidence is drawn primarily from the platforms' own authoritative documentation — the Linux `epoll(7)` and `select(2)` man pages, and Microsoft's reference pages for I/O completion ports, `GetQueuedCompletionStatus`, `WSARecv`, and overlapped I/O — supplemented by a Rust standard-library design study showing the two buffer-ownership models are irreconcilable at the level of trait signatures, and by cross-platform libraries (libuv, CPython's `asyncio`) whose need to support both models makes the split empirically visible rather than theoretical.

The note is explicit about its own limits. It states plainly that the readiness/completion dichotomy is "a good first cut, not a complete taxonomy," and names further positions in the design space — Linux's `io_uring`, Windows' newer IoRing, and BSD's `kqueue` — that it does not treat as fully equivalent to the two anchor cases. Several specific claims are flagged rather than asserted as settled: a Windows NtTrace transcript used to illustrate completion-model tracing is marked as inference reconstructed from documented formats rather than a captured trace; a claim about which notification mode nginx and Redis each use is marked as drawn from a secondary source rather than project documentation; and a live disagreement among Linux practitioners over whether `EAGAIN` is strictly required to re-arm an edge-triggered descriptor, or merely the reliable way to detect a drained buffer, is presented as unresolved, with the note noting only that the practical guidance is identical either way.

Referenced sources

- [epoll\(7\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [select\(2\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [I/O Completion Ports](#) — Microsoft Learn **HIGH**
- [GetQueuedCompletionStatus function \(ioapiset.h\)](#) — Microsoft Learn **HIGH**
- [WSARecv function \(winsock2.h\)](#) — Microsoft Learn **HIGH**
- [Overlapped Input/Output \(Winsock\)](#) — Microsoft Learn **HIGH**
- [WSAPoll function \(winsock2.h\)](#) — Microsoft Learn **HIGH**

- [WSAPoll is broken](#) — Daniel Stenberg **MEDIUM**
- [The edge-triggered misunderstanding](#) — LWN.net comment thread **HIGH**
- [portable-interoperable / io-traits](#) — Nick Cameron **MEDIUM**
- [Async IO with completion-model IO systems](#) — Nick Cameron **MEDIUM**
- [Design overview — libuv documentation](#) — libuv project **HIGH**
- [libuv — iocp-links.html \(design notes\)](#) — libuv project **MEDIUM**
- [asyncio — Event Loop \(Python documentation\)](#) — Python Software Foundation **HIGH**
- [selectors — High-level I/O multiplexing \(Python documentation\)](#) — Python Software Foundation **HIGH**
- [SDL_net: windows: Reimplement poll\(\) with select\(\)](#) — Simple DirectMedia Layer project **MEDIUM**
- [PollSet not working as intended on Windows](#) — pocoproject/poco#3248 — POCO C++ Libraries issue tracker **MEDIUM**
- [Issue 16507: Patch selectmodule.c to support WSApoll on Windows](#) — Python issue tracker **HIGH**
- [Under the Hood of AFD.sys Part 1: Investigating Undocumented Interfaces](#) — Mateusz Lewczak **MEDIUM**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#) — Win-sider Seminars & Solutions **HIGH**
- [libuv Event Loop Internals](#) — Sujeet Jaiswal **MEDIUM**
- [Linux io_uring vs Windows I/O: A Technical Comparison](#) — Mark Veltzer **LOW**
- [epoll: The API That Powers the Modern Internet](#) — rzero.in **LOW**

12.57 V75 — Readiness and Completion Models

This note establishes that Linux and Windows answer a different question when a program waits for network I/O, and argues the difference is structural rather than a matter of API style. Linux's `select`, `poll`, and `epoll` tell a program which descriptors would not block, and the program then performs the read or write itself; Windows accepts a posted operation such as `WSARecv` up front and later delivers a completion packet through an I/O completion port once the kernel has already filled the buffer. That one difference — where the data copy happens relative to the notification — is shown to change who owns the buffer and for how long, what shape the program's control flow takes, and when an error becomes visible. The note works through both platforms at matching depth across Python, C, and the system-call layer, using `strace` and NtTrace output to make claims about the kernel boundary observable rather than asserted, and

it draws on official Linux man pages, Microsoft's own Winsock and IOCP documentation, and independent projects — libuv, CPython's asyncio, curl, and Rust's async-I/O trait design work — that each had to reconcile both models, and did so in ways the note treats as convergent evidence that the split is real. Its worked examples include edge-triggered `epoll` concentrating a distinct class of silent, load-dependent stall, and a completion program requiring a per-connection state machine because an operation and its result are separated by an unbounded, possibly cross-thread gap.

The note is explicit about its own limits. It does not treat the split as a Linux-versus-Windows split: it names `io_uring` as a completion interface on Linux and `WSAPoll` / `select` as readiness-shaped options on Windows, and frames the dichotomy as a design-space split each platform entered from a different side. Several claims are marked [inference] rather than verified — most notably the NtTrace transcript in the note's Windows tracing section, which was not captured from a real system but reconstructed from documented output format and the documented AFD call path, and is labelled as such rather than presented as observed. The note also surfaces a live, unresolved disagreement among Linux practitioners, quoted from an LWN thread, over whether `EAGAIN` is strictly required to re-arm an edge-triggered notification or merely the reliable way to detect it, and states plainly that it does not adjudicate that dispute — only that the practical guidance is identical under either reading.

Referenced sources

- [epoll\(7\) — Linux manual page](#) — Linux man-pages project (ongoing) **HIGH**
- [select\(2\) — Linux manual page](#) — Linux man-pages project (ongoing) **HIGH**
- [I/O Completion Ports](#) — Microsoft Learn (updated 2025-07-18) **HIGH**
- [GetQueuedCompletionStatus function \(ioapiset.h\)](#) — Microsoft Learn (updated 2024-08-22) **HIGH**
- [WSARecv function \(winsock2.h\)](#) — Microsoft Learn (updated 2024-02-22) **HIGH**
- [Overlapped Input/Output \(Winsock\)](#) — Microsoft Learn (updated 2025-03-11) **HIGH**
- [WSAPoll function \(winsock2.h\)](#) — Microsoft Learn (updated 2024-02-22) **HIGH**
- [WSAPoll is broken](#) — Daniel Stenberg (2012, with a 2023 update) **MEDIUM**
- [The edge-triggered misunderstanding](#) — LWN.net comment thread (2021) **HIGH**
- [portable-interoperable / io-traits](#) — Nick Cameron (2021–) **MEDIUM**
- [Async IO with completion-model IO systems](#) — Nick Cameron (2022) **MEDIUM**
- [Design overview — libuv documentation](#) — libuv project (ongoing) **HIGH**
- [libuv — iocp-links.html \(design notes\)](#) — libuv project (early design document) **MEDIUM**
- [asyncio — Event Loop \(Python documentation\)](#) — Python Software Foundation (ongoing) **HIGH**

- [selectors — High-level I/O multiplexing \(Python documentation\)](#) — Python Software Foundation (ongoing) **HIGH**
- [SDL_net: windows: Reimplement poll\(\) with select\(\)](#) — Simple DirectMedia Layer project commit announcement (2025-08-14) **MEDIUM**
- [PollSet not working as intended on Windows](#) — pocoproject/poco#3248 — POCO C++ Libraries issue tracker **MEDIUM**
- [Issue 16507: Patch selectmodule.c to support WSApoll on Windows](#) — Python issue tracker (2012–2013) **HIGH**
- [Under the Hood of AFD.sys Part 1: Investigating Undocumented Interfaces](#) — Mateusz Lewczak **MEDIUM**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#) — Win-sider Seminars & Solutions (published 2021-10-09, updated 2022-04-18) **HIGH**
- [libuv Event Loop Internals](#) — Sujeet Jaiswal **MEDIUM**
- [Linux io_uring vs Windows I/O: A Technical Comparison](#) — Mark Veltzer (2026) **LOW**
- [epoll: The API That Powers the Modern Internet](#) — rzero.in (2026-04-29) **LOW**

12.58 V76 — Readiness and Completion I/O Models

This note argues that Linux and Windows answer a different question when a server needs to handle many connections without blocking a thread per connection: Linux's `select` / `poll` / `epoll` tell a program "which descriptor would not block right now," and the program then performs the read or write itself (readiness), while Windows' I/O completion ports tell a program "the operation you already submitted has finished," with the kernel filling the buffer itself (completion). The note claims this is a structural difference, not a stylistic one, and traces it through three consequences: who owns the buffer and for how long, what shape the program's control flow takes, and when an error becomes observable. It backs each claim primarily with vendor and maintainer-authored primary sources — the Linux `epoll(7)` and `select(2)` man pages, Microsoft's IOCP, `GetQueuedCompletionStatus`, `WSARecv`, and Overlapped I/O reference pages — and corroborates the buffer-ownership tension with a Rust standard-library design document that concluded the two models' requirements cannot be reconciled in one set of traits, and with libuv's own design notes on emulating a completion-style API over a readiness-based kernel interface.

The note is explicit about what it does not claim. It states the readiness/completion split is a design-space division that Linux and Windows entered from different sides, not a Linux/Windows split per se, pointing to Linux's `io_uring` as completion-model and to Windows' `WSAPoll` as a readiness-shaped compatibility surface with documented reli-

ability caveats. Several claims are flagged inline as [inference] or [reported] rather than settled fact: a Windows NtTrace transcript is explicitly marked as constructed rather than captured, since no published trace of a traced overlapped receive was located; the claim that nginx uses edge-triggered `epoll` while Redis uses level-triggered is marked [reported], sourced from a single low-credibility blog rather than either project's own documentation; and a claim about Registered I/O's adoption is drawn from a single low-credibility source and marked accordingly. The note treats a documented disagreement among Linux kernel practitioners over whether `EAGAIN` is strictly required to re-arm edge-triggered notifications as unresolved rather than adjudicating it itself.

Referenced sources

- [epoll\(7\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [select\(2\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [I/O Completion Ports](#) — Microsoft Learn **HIGH**
- [GetQueuedCompletionStatus function \(ioapiset.h\)](#) — Microsoft Learn **HIGH**
- [WSARecv function \(winsock2.h\)](#) — Microsoft Learn **HIGH**
- [Overlapped Input/Output \(Winsock\)](#) — Microsoft Learn **HIGH**
- [WSAPoll function \(winsock2.h\)](#) — Microsoft Learn **HIGH**
- [WSAPoll is broken](#) — Daniel Stenberg **MEDIUM**
- [The edge-triggered misunderstanding](#) — LWN.net comment thread **HIGH**
- [portable-interoperable / io-traits](#) — Nick Cameron **MEDIUM**
- [Async IO with completion-model IO systems](#) — Nick Cameron **MEDIUM**
- [Design overview — libuv documentation](#) — libuv project **HIGH**
- [libuv — iocp-links.html \(design notes\)](#) — libuv project **MEDIUM**
- [asyncio — Event Loop \(Python documentation\)](#) — Python Software Foundation **HIGH**
- [selectors — High-level I/O multiplexing \(Python documentation\)](#) — Python Software Foundation **HIGH**
- [SDL_net: windows: Reimplement poll\(\) with select\(\)](#) — Simple DirectMedia Layer project **MEDIUM**
- [PollSet not working as intended on Windows](#) — pocoproject/poco#3248 — POCO C++ Libraries issue tracker **MEDIUM**
- [Issue 16507: Patch selectmodule.c to support WSAPoll on Windows](#) — Python issue tracker **HIGH**
- [Under the Hood of AFD.sys Part 1: Investigating Undocumented Interfaces](#) — Mateusz Lewczak **MEDIUM**

- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#) — Windows Seminars & Solutions **HIGH**
- [libuv Event Loop Internals](#) — Sujeet Jaiswal **MEDIUM**
- [Linux io_uring vs Windows I/O: A Technical Comparison](#) — Mark Veltzer **LOW**
- [epoll: The API That Powers the Modern Internet](#) — rzero.in **LOW**

12.59 V77 — Readiness and Completion I/O Models

This note argues that Linux and Windows answer a different question about asynchronous network I/O, and that the difference is architectural rather than cosmetic. Linux's readiness model (`select`, `poll`, `epoll`) tells a program which descriptors would not currently block, and the program then performs the read or write itself; Windows' completion model (overlapped I/O plus I/O completion ports) tells a program that an operation it already submitted has finished, with the kernel having done the copy. The note traces three consequences from this split: who owns the data buffer and for how long, what shape the program's control flow takes, and when an I/O error becomes visible. Under readiness, the program owns the buffer, can allocate it late, and may share one buffer across many idle connections, but a ready descriptor is only a hint that a subsequent read will succeed, not a guarantee. Under completion, the kernel holds the buffer from submission to completion, enabling true zero-copy delivery at the cost of one buffer per outstanding operation and a per-connection state machine, with errors surfacing only at dequeue. The claim is grounded in Linux's and Microsoft's own documentation and cross-checked against libuv, CPython, curl, and Rust's async I/O design work.

The note is explicit about its limits. It does not treat the Linux/Windows divide as exhaustive: `io_uring` brings a completion model to Linux, Windows' newer IoRing mirrors `io_uring`, and BSD's `kqueue` is a readiness model outside Linux, so the split is a design-space choice each platform entered from a different side, not a fixed platform rule. Several supporting details are flagged rather than asserted as settled: its reconstructed trace of a Windows completion under NtTrace is marked as inference, since no published capture of that exact scenario was located; the claims that nginx and Redis pick opposite `epoll` trigger modes, and that Windows' Registered I/O never gained adoption, are marked as reported from lower-credibility, single-source material; and a live disagreement among Linux practitioners over whether `EAGAIN` is strictly required to re-arm edge-triggered notifications is left open rather than resolved.

Referenced sources

- [epoll\(7\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [select\(2\) — Linux manual page](#) — Linux man-pages project **HIGH**

- [I/O Completion Ports](#) — Microsoft Learn **HIGH**
- [GetQueuedCompletionStatus function \(ioapiset.h\)](#) — Microsoft Learn **HIGH**
- [WSARecv function \(winsock2.h\)](#) — Microsoft Learn **HIGH**
- [Overlapped Input/Output \(Winsock\)](#) — Microsoft Learn **HIGH**
- [WSAPoll function \(winsock2.h\)](#) — Microsoft Learn **HIGH**
- [WSAPoll is broken](#) — Daniel Stenberg **MEDIUM**
- [The edge-triggered misunderstanding](#) — LWN.net comment thread **HIGH**
- [portable-interoperable / io-traits](#) — Nick Cameron **MEDIUM**
- [Async IO with completion-model IO systems](#) — Nick Cameron **MEDIUM**
- [Design overview](#) — libuv documentation — libuv project **HIGH**
- [libuv — iocp-links.html \(design notes\)](#) — libuv project **MEDIUM**
- [asyncio — Event Loop \(Python documentation\)](#) — Python Software Foundation **HIGH**
- [selectors — High-level I/O multiplexing \(Python documentation\)](#) — Python Software Foundation **HIGH**
- [SDL_net: windows: Reimplement poll\(\) with select\(\)](#) — Simple DirectMedia Layer project **MEDIUM**
- [PollSet not working as intended on Windows](#) — pocoproject/poco#3248 — POCO C++ Libraries issue tracker **MEDIUM**
- [Issue 16507: Patch selectmodule.c to support WSAPoll on Windows](#) — Python issue tracker **HIGH**
- [Under the Hood of AFD.sys Part 1: Investigating Undocumented Interfaces](#) — Mateusz Lewczak **MEDIUM**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#) — Win-sider Seminars & Solutions **HIGH**
- [libuv Event Loop Internals](#) — Sujeet Jaiswal **MEDIUM**
- [Linux io_uring vs Windows I/O: A Technical Comparison](#) — Mark Veltzer **LOW**
- [epoll: The API That Powers the Modern Internet](#) — rzero.in **LOW**

12.60 V78 — The gap between a successful write and durable storage

This note argues that data loss after a program reports "saved" is the normal outcome of a well-understood mechanism, not a malfunction. Between a write call and the physical medium sit four independent buffers — the language runtime, the kernel's user-space entry point, the page cache, and the device's own cache — each with its own flush trigger and cost to cross. A successful `write()` only tells the caller the first kernel-side

buffer accepted the bytes; that is custody, not persistence. Crossing from kernel cache to device requires `fsync()`, which the Linux man page documents as flushing to the device and blocking until the transfer completes — but it carries a documented, non-retryable failure mode: when writeback fails, the kernel reports the error to only the next caller, discards the failed pages, and clears the record, so a retry reports false success. This broke PostgreSQL's checkpointing for roughly two decades; the note frames it as a genuine contract disagreement between two competent groups, since POSIX never specified behavior after a failed `fsync`. PostgreSQL, InnoDB and WiredTiger converged on treating any `fsync` failure as fatal — PANIC and recover from a write-ahead log — since once the page cache discards the data, the application's own log is the only remaining copy. Windows gets equal depth: the same four-layer structure under different names, but on the question that matters most — what happens to cached data after a flush failure — Microsoft has never documented an answer, which the note treats as worse than Linux's documented-but-hostile contract.

The note is explicit about what it does not claim. It does not say Windows shares Linux's non-retryable bug, nor that Windows is safe from it — only that the Windows contract is unspecified, so no correct response can be derived from documentation alone. It does not treat PANIC-on-error as universally correct: that response was calibrated for 2018-era local disks, and on cloud block storage the same error is often transient, changing the availability trade-off without changing the correctness argument. It flags its own four-layer model as a teaching simplification that does not cleanly extend to network filesystems, memory-mapped I/O, or copy-on-write filesystems. Two subclaims are marked open or inferred rather than settled: whether any Windows tool exposes device-level flush commands as cleanly as `blktrace`, and the exact mapping from Python's `os.fsync` to the Windows CRT's `_commit()`.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026) **MEDIUM**
- [fsync\(2\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017) **HIGH**
- [io — Core tools for working with streams](#) — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn **HIGH**
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022) **MEDIUM**

- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla **HIGH**
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode **MEDIUM**
- [Page Cache Writeback](#) — Linux Kernel Internals **MEDIUM**
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn **HIGH**
- [File Caching](#) — Microsoft Learn **HIGH**
- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al. **HIGH**
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026) **LOW**
- [Life of a Write](#) — Linux Kernel Internals **MEDIUM**
- [The errseq_t datatype](#) — The Linux Kernel documentation — Linux kernel documentation **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018) **HIGH**
- [Fsync Errors](#) — PostgreSQL wiki **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020) **HIGH**
- [CreateFileW function — FILE_FLAG_WRITE_THROUGH and FILE_FLAG_NO_BUFFERING semantics](#) — Microsoft Learn **HIGH**
- [File Buffering](#) — Microsoft Learn **HIGH**
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — Dilip Naik (2012) **MEDIUM**
- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News (2019) **LOW**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow (2019) **MEDIUM**
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation **HIGH**
- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow **MEDIUM**
- [DTrace on Windows](#) — Microsoft Learn **HIGH**
- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn **HIGH**
- [Write-Ahead Logging \(SQLite\)](#) — SQLite **HIGH**

12.61 V79 — Ring-Based Completion I/O on Linux and Windows

This note is the detailed layer beneath a two-paragraph comparison in a parent note: it examines Linux's `io_uring` and Windows's `IoRing`, two interfaces that replace the traditional ask-and-be-told system call with a post-and-harvest arrangement, where the application and kernel share submission and completion ring buffers in memory. It establishes that the two platforms converged from opposite starting points — Linux, native to readiness-based I/O, grew a completion interface because `epoll` cannot describe a regular file's readiness and native `aio` was asynchronous only for `O_DIRECT`; Windows, already completion-native through I/O completion ports, adopted the ring as a shared-memory transport rather than a new semantic model. The researcher who reverse-engineered the Windows implementation found it "almost identical" to Linux's, with headers "almost copied" from `io_uring`. The note documents the mechanics across three levels — raw system calls, the `liburing` / Win32 API layer, and the (absent) Python layer — and supports three structural claims with primary sources: crossings are amortised, not eliminated; completions can arrive out of submission order; and errors are returned as data rather than via `errno` or per-call failure. It also traces a well-sourced security history: Google's 2023 disclosure that `io_uring` accounted for 60% of its kCTF exploit submissions, the mechanism by which `io-wq` worker threads bypass `seccomp` filters, and the staged 2023–2026 mitigation response, itself later subject to a restriction-bypass CVE.

The note is explicit about what it does not claim. It does not assert that `io_uring`'s performance benefit is automatic: a peer-reviewed VLDB study found drop-in substitution gains of only 1.06–1.10×, against 2.05–2.31× when an application is redesigned around batching, and the benefit shrinks for compute-bound workloads. It does not claim Windows `IoRing` has reached production use — its own documenter reported a year after launch that "no applications seem to use this mechanism yet." Several points are left open rather than resolved: no Microsoft documentation was found for an `IORING_VERSION` beyond 3, despite a field report of a higher value; the note's own attempt to capture a live `io_uring` trace failed because a `seccomp` filter denied the syscall, so that trace is inferred from documentation rather than observed; a related source could not be re-fetched and its claim is therefore dropped rather than repeated; and several claims are marked `[reported]` or `[inference]` rather than verified, distinguishing survey-level and interpretive statements from primary-sourced fact.

Referenced sources

- [io_uring\(7\) — Linux manual page](#) — Linux man-pages project / kernel.org HIGH
- [io_uring_setup\(2\) — Linux manual page](#) — Linux man-pages project / kernel.org

HIGH

- [io_uring_enter\(2\) — Linux manual page — Linux man-pages project / kernel.org](#) **HIGH**
- [io_uring_register\(2\) — Linux manual page — Linux man-pages project / kernel.org](#) **HIGH**
- [Efficient IO with io_uring — Jens Axboe \(2019\)](#) **HIGH**
- [axboe/liburing — Jens Axboe \(2019-\)](#) **HIGH**
- [io_uring.h header — Win32 apps — Microsoft Learn \(2022\)](#) **HIGH**
- [CreateIoRing function \(io_uring.h\) — Microsoft Learn \(2021, updated 2025\)](#) **HIGH**
- [BuildIoRingReadFile function \(io_uring.h\) — Microsoft Learn \(2021, updated 2025\)](#) **HIGH**
- [SubmitIoRing function \(io_uring.h\) — Microsoft Learn \(2021, updated 2025\)](#) **HIGH**
- [BuildIoRingRegisterBuffers function \(io_uring.h\) — Microsoft Learn](#) **HIGH**
- [BuildIoRingRegisterFileHandles function \(io_uring.h\) — Microsoft Learn](#) **HIGH**
- [Learnings from kCTF VRP's 42 Linux kernel exploits submissions — Google Security Team / Google Online Security Blog \(2023\)](#) **HIGH**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations — Yarden Shafir / Winsider Seminars & Solutions \(2021\)](#) **HIGH**
- [One Year to I/O Ring: What Changed? — Yarden Shafir / Winsider Seminars & Solutions \(2022\)](#) **HIGH**
- [One I/O Ring to Rule Them All: A Full Read/Write Exploit Primitive on Windows 11 — Yarden Shafir / Winsider Seminars & Solutions \(2022\)](#) **HIGH**
- [Ringing in a new asynchronous I/O API — Jonathan Corbet / LWN.net \(2019\)](#) **HIGH**
- [Task-level io_uring restrictions — LWN.net \(2026\)](#) **HIGH**
- [uringscope: Portable, Low-Overhead Observability for io_uring — Rajarshi Chowdhury / arXiv \(2026\)](#) **MEDIUM**
- [io_uring Security Model — Linux Kernel Internals](#) **MEDIUM**
- [io_uring War Stories — Linux Kernel Internals](#) **MEDIUM**
- [\[PATCHSET RFC 0/2\] Per-task io_uring opcode restrictions — Jens Axboe / io-uring mailing list \(2026\)](#) **HIGH**
- [High-Performance DBMSs with io_uring: When and How to Use It — Jasny et al. / PVLDB vol. 19](#) **HIGH**
- [io_uring Security and Hardening: Disabling, Restricting and Auditing — Systems Hardening \(2026\)](#) **MEDIUM**
- [io_uring, SPDK, and the Kernel Bypass Wars — Murat Karslioglu \(2026\)](#) **MEDIUM**

- [Python io_uring binding landscape — liburing, uringcore, aioring, uring_file, asyncio-uring, and CPython issues 41271 / 44738](#) — PyPI / CPython issue tracker / GitHub project pages **MEDIUM**
- [Submission Queue Polling — Lord of the io_uring — Shuveb Hussain](#) **MEDIUM**
- [Google Limiting IO_uring Use Due To Security Vulnerabilities](#) — Michael Larabel / Phoronix (2023) **MEDIUM**
- [oss-security: Our learnings from 42 Linux kernel exploits, we are limiting io_uring](#) — Tamás Koczka / oss-security mailing list (2023) **HIGH**
- [windows-ioring-sys — crate source and design notes](#) — docs.rs (2026) **MEDIUM**
- [io_uring Reads Your Files and Opens Sockets Without Calling the Syscalls Your auditd Rules Watch](#) — Trackr.Live (2026) **MEDIUM**
- [Asynchronous I/O \(AIO\) — PostgreSQL feature matrix](#) — The PostgreSQL Global Development Group **HIGH**
- [CVE-2026-80713 — io_uring: preserve task restrictions across exec](#) — vuln.today (2026), quoting the upstream kernel commit message **MEDIUM**
- [Offloading I/O to Dedicated Cores: An Asymmetric io_uring Backend for Seastar and ScyllaDB](#) — ScyllaDB (2026) **HIGH**

12.62 V80 — Ring-Based I/O on Linux (`io_uring`) and Windows (`IoRing`)

This note treats `io_uring` and Windows `IoRing` as the point where the two platforms' async I/O designs converge from opposite directions, and documents what each interface actually consists of. The shared mechanism is a pair of ring buffers shared between application and kernel — a submission queue the program writes into and a completion queue the kernel writes results into — so that only the "kick" telling the kernel to look at the queue needs a system call, and one kick can carry many requests. Linux, historically a readiness-native platform (`epoll`), adopted completion semantics because readiness cannot describe a regular file or true buffered-I/O asynchrony, both gaps documented by `io_uring`'s own author. Windows, already completion-native via IOCP, adopted only the ring transport. The note works through the mechanics at three levels — raw syscalls, the `liburing` helper library, and the (absent) Python binding — and the parallel Win32 surface (`CreateIoRing`, `BuildIoRingReadFile`, `SubmitIoRing`), backed by man pages, Microsoft's own API reference, and a reverse-engineering comparison of the two implementations. It also covers a documented security history: Google's security team reported that `io_uring` accounted for 60% of kCTF kernel exploit submissions and restricted its use across several products, and the note traces the

mechanism — kernel worker threads that bypass seccomp's syscall-entry filtering — through the kernel maintainer's own statements and subsequent kernel-side mitigations.

The note explicitly does not claim rings make I/O uniformly faster: it cites a peer-reviewed study finding drop-in substitution buys little (roughly 1.06–1.10×) while architecture built around batching and registered buffers buys more (roughly 2.05–2.31×), and reports that an already-optimized system (ScyllaDB/Seastar) found the gains thin outside RPC-heavy paths. It treats Windows `IoRing` adoption as unsettled, quoting its own documenter's finding a year after release that "no applications seem to use this mechanism yet," and it does not claim the seccomp-bypass problem is resolved — it flags a 2026 CVE in the newer per-task restriction mechanism itself as evidence the trajectory is not a finished fix. Several claims are explicitly marked as lower-confidence within the note ("reported," "inference," "unverified") rather than presented as settled fact, including a benchmark comparison against SPDK, an observed Windows API version beyond documented ones, and Python-ecosystem maturity. The note's own appended "Grounding gap" log further states that one prior source could not be re-fetched and was dropped rather than kept, and that its own captured trace was blocked by a seccomp filter in its test environment rather than showing a successful `io_uring` run firsthand.

Referenced sources

- [io_uring\(7\) — Linux manual page](#) — Linux man-pages project / kernel.org HIGH
- [io_uring_setup\(2\) — Linux manual page](#) — Linux man-pages project / kernel.org HIGH
- [io_uring_enter\(2\) — Linux manual page](#) — Linux man-pages project / kernel.org HIGH
- [io_uring_register\(2\) — Linux manual page](#) — Linux man-pages project / kernel.org HIGH
- [Efficient IO with io_uring](#) — Jens Axboe (2019) HIGH
- [axboe/liburing](#) — Jens Axboe (2019–) HIGH
- [ioingapi.h header](#) — Win32 apps — Microsoft Learn (2022) HIGH
- [CreateIoRing function \(ioingapi.h\)](#) — Microsoft Learn (2021, updated 2025) HIGH
- [BuildIoRingReadFile function \(ioingapi.h\)](#) — Microsoft Learn (2021, updated 2025) HIGH
- [SubmitIoRing function \(ioingapi.h\)](#) — Microsoft Learn (2021, updated 2025) HIGH
- [BuildIoRingRegisterBuffers function \(ioingapi.h\)](#) — Microsoft Learn HIGH
- [BuildIoRingRegisterFileHandles function \(ioingapi.h\)](#) — Microsoft Learn HIGH

- [Learnings from kCTF VRP's 42 Linux kernel exploits submissions](#) — Google Security Team / Google Online Security Blog (2023) **HIGH**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#) — Yarden Shafir / Winsider Seminars & Solutions (2021) **HIGH**
- [One Year to I/O Ring: What Changed?](#) — Yarden Shafir / Winsider Seminars & Solutions (2022) **HIGH**
- [One I/O Ring to Rule Them All: A Full Read/Write Exploit Primitive on Windows 11](#) — Yarden Shafir / Winsider Seminars & Solutions (2022) **HIGH**
- [Ringing in a new asynchronous I/O API](#) — Jonathan Corbet / LWN.net (2019) **HIGH**
- [Task-level io_uring restrictions](#) — LWN.net (2026) **HIGH**
- [uringscope: Portable, Low-Overhead Observability for io_uring](#) — Rajarshi Chowdhury / arXiv (2026) **MEDIUM**
- [io_uring Security Model](#) — Linux Kernel Internals **MEDIUM**
- [io_uring War Stories](#) — Linux Kernel Internals **MEDIUM**
- [\[PATCHSET RFC 0/2\] Per-task io_uring opcode restrictions](#) — Jens Axboe / io-uring mailing list (2026) **HIGH**
- [High-Performance DBMSs with io_uring: When and How to Use It](#) — Jasny et al. / PVLDB vol. 19 **HIGH**
- [io_uring Security and Hardening: Disabling, Restricting and Auditing](#) — Systems Hardening (2026) **MEDIUM**
- [io_uring, SPDK, and the Kernel Bypass Wars](#) — Murat Karslioglu (2026) **MEDIUM**
- [Python io_uring binding landscape](#) — `liburing`, `uringcore`, `aioring`, `uring_file`, `asyncio-uring`, and CPython issues 41271 / 44738 — PyPI / CPython issue tracker / GitHub project pages **MEDIUM**
- [Submission Queue Polling](#) — Lord of the io_uring — Shuveb Hussain **MEDIUM**
- [Google Limiting IO_uring Use Due To Security Vulnerabilities](#) — Michael Larabel / Phoronix (2023) **MEDIUM**
- [oss-security: Our learnings from 42 Linux kernel exploits, we are limiting io_uring](#) — Tamás Koczka / oss-security mailing list (2023) **HIGH**
- [windows-ioring-sys](#) — crate source and design notes — docs.rs (2026) **MEDIUM**
- [io_uring Reads Your Files and Opens Sockets Without Calling the Syscalls Your auditd Rules Watch](#) — Trackr.Live (2026) **MEDIUM**
- [Asynchronous I/O \(AIO\)](#) — PostgreSQL feature matrix — The PostgreSQL Global Development Group **HIGH**
- [CVE-2026-80713 — io_uring: preserve task restrictions across exec](#) — vuln.today (2026), quoting the upstream kernel commit message **MEDIUM**

- [Offloading I/O to Dedicated Cores: An Asymmetric io_uring Backend for Seastar and ScyllaDB](#) — ScyllaDB (2026) **HIGH**

12.63 V81 — The Readiness/Completion Split in Network I/O

This note argues that Linux and Windows answer a different question about non-blocking network I/O, and that the difference is architectural rather than stylistic. Linux's `select`, `poll`, and `epoll` answer "which descriptor would not block right now?" — the kernel reports a fact about state, and the program then performs the read or write itself. Windows's I/O completion ports answer "which operation I already submitted has finished?" — the program posts a `WSARecv` with a buffer up front, and the kernel fills that buffer and reports the result later. The note traces this one difference — where the data copy happens relative to the notification — into three consequences it treats as structural: who owns the buffer and for how long, what shape the program's control flow takes (a simple dispatch loop versus a per-connection state machine), and when an I/O error becomes visible. Its evidence is drawn mainly from primary interface documentation — the Linux `epoll(7)` and `select(2)` man pages and Microsoft's own Winsock and IOCP reference pages — supported by a Rust standard-library design study that concluded the two models cannot share one set of I/O traits because one wants the buffer supplied late and the other wants it supplied early and owned by the kernel. Code written for one model and mechanically ported to the other is not slower, the note argues, but wrong: it will stall connections, corrupt buffers, or attribute an error to the wrong operation.

The note is explicit that this is a good first cut, not a complete taxonomy: it names `io_uring` as a completion model that arrived on Linux, Windows IoRing as Linux's ring design arriving on Windows, and `kqueue` as readiness outside Linux, so the split is a design-space choice each platform entered from a different side rather than a strict Linux/Windows boundary. Several supporting claims are explicitly hedged rather than asserted as settled: whether `EAGAIN` is strictly required to re-arm an edge-triggered `epoll` notification is recorded as a live disagreement among practitioners rather than resolved; a worked NtTrace transcript in the note is labelled inference, since no published capture of a traced Winsock receive was located; and claims about nginx/Redis polling-mode choice and about Registered I/O's lack of adoption are marked as reported from lower-credibility secondary sources rather than verified firsthand.

Referenced sources

- [epoll\(7\) — Linux manual page](#) — Linux man-pages project — **HIGH**
- [select\(2\) — Linux manual page](#) — Linux man-pages project — **HIGH**
- [I/O Completion Ports](#) — Microsoft Learn — **HIGH**

- [GetQueuedCompletionStatus function \(ioapiset.h\)](#) — Microsoft Learn — HIGH
- [WSARecv function \(winsock2.h\)](#) — Microsoft Learn — HIGH
- [Overlapped Input/Output \(Winsock\)](#) — Microsoft Learn — HIGH
- [WSAPoll function \(winsock2.h\)](#) — Microsoft Learn — HIGH
- [WSAPoll is broken](#) — Daniel Stenberg — MEDIUM
- [The edge-triggered misunderstanding](#) — LWN.net comment thread — HIGH
- [portable-interoperable / io-traits](#) — Nick Cameron — MEDIUM
- [Async IO with completion-model IO systems](#) — Nick Cameron — MEDIUM
- [Design overview — libuv documentation](#) — libuv project — HIGH
- [libuv — iocp-links.html \(design notes\)](#) — libuv project — MEDIUM
- [asyncio — Event Loop \(Python documentation\)](#) — Python Software Foundation — HIGH
- [selectors — High-level I/O multiplexing \(Python documentation\)](#) — Python Software Foundation — HIGH
- [SDL_net: windows: Reimplement poll\(\) with select\(\)](#) — Simple DirectMedia Layer project — MEDIUM
- [PollSet not working as intended on Windows](#) — pocoproject/poco#3248 — POCO C++ Libraries issue tracker — MEDIUM
- [Issue 16507: Patch selectmodule.c to support WSAPoll on Windows](#) — Python issue tracker — HIGH
- [Under the Hood of AFD.sys Part 1: Investigating Undocumented Interfaces](#) — Mateusz Lewczak — MEDIUM
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#) — Win-sider Seminars & Solutions — HIGH
- [libuv Event Loop Internals](#) — Sujeet Jaiswal — MEDIUM
- [Linux io_uring vs Windows I/O: A Technical Comparison](#) — Mark Veltzer — LOW
- [epoll: The API That Powers the Modern Internet](#) — rzero.in — LOW

12.64 V82 — Interprocess Communication

This note frames interprocess communication as the deliberate, kernel-supervised breach of the wall that memory protection otherwise maintains between processes, and argues that every IPC mechanism is a different bargain struck with that same wall. Its core claim is that a mechanism should be chosen by its semantics — what the kernel promises about framing, ordering, and lifetime — rather than by speed, because IPC is rarely the performance bottleneck while a wrong semantic choice becomes a correct-

ness bug. To make that operational, the note proposes a three-branch taxonomy (byte-stream transfer, message passing, and shared state plus synchronisation), sorted by what the kernel guarantees about the shape of what arrives, with signals and file locking treated as two mechanisms that sit outside the taxonomy rather than being force-fit into it. The evidence is layered: platform manuals and API references (`pipe(7)`, Microsoft's named-pipe documentation) establish the framing distinction as a documented mechanical property rather than a stylistic one; an architectural cost analysis decomposes the price of a kernel-mediated transfer into privilege-switch, address-space-switch, and copy costs; and a real-world case study of Chrome's Mojo IPC layer shows a production system choosing message passing for its trust-boundary properties while composing it with shared memory for bulk data, illustrating that the taxonomy's branches combine rather than compete.

The note is explicit about what it does not establish. It deliberately withholds a ranking among the three branches and does not publish latency numbers, on the grounds that published IPC benchmarks vary too widely by payload size and topology to be durable teaching material. It presents its three-branch split as a pedagogical synthesis rather than the field's canonical taxonomy, contrasting itself with the dominant two-way academic split (message passing versus shared memory) and with a more expressive four-dimension classification, and it flags that sockets and Windows named pipes straddle its own branches depending on configuration. Several worked details carry explicit caveats rather than being asserted as settled: the Windows Python coalescing result and the Linux `strace` transcript are both marked as inferred rather than captured, for lack of a Windows host or `strace` in the authoring environment, and the Windows Nt-Trace capture it relies on is reproduced from a third-party source rather than made for this note. All three are flagged for re-capture before the lecture is delivered, and the note treats them as unverified rather than as evidence.

Referenced sources

- [pipe\(7\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project **HIGH**
- [mq_overview\(7\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project **HIGH**
- [Anonymous Pipe Operations](#) — Microsoft Learn (updated 2025) **HIGH**
- [Named Pipe Type, Read, and Wait Modes](#) — Microsoft Learn (updated 2025) **HIGH**
- [Kernel Objects](#) — Microsoft Learn (updated 2025) **HIGH**
- [CreatePipe function \(namedpipeapi.h\)](#) — Microsoft Learn (2021) **HIGH**
- [3.2. IPC Models — Computer Systems Fundamentals](#) — Charles Kirkpatrick / James Madison University **MEDIUM**
- [Operating-System Construction: L10 Inter-Process Communication](#) — Horst Schirmeier / TU Dresden (SS2025 lecture notes) **MEDIUM**

- [The Cost of IPC: an Architectural Analysis](#) — I. Gelado, J. Cabezas, L. Vilanova, N. Navarro (2007) **MEDIUM**
- [Taxonomy of Unix IPC Methods](#) — [The Art of Unix Programming](#), ch. 7 — Eric S. Raymond (2003) **MEDIUM**
- [Mojo Style Guide](#) — [Chromium security documentation](#) — The Chromium Project **HIGH**
- [mojo/core](#) — [Chromium source documentation](#) — The Chromium Project **HIGH**
- [Windows: Prefer `ntdll` named pipe APIs over `kernel32`](#) — [ziglang/zig issue #19037](#) — The-King-of-Toasters / Zig project (2024) **MEDIUM**
- [Named Pipes Under Attack: Securing Windows Interprocess Communication](#) — BleepingComputer (22 August 2026) **MEDIUM**
- [CVE-2026-28727](#) — [Local privilege escalation due to insecure Unix socket permissions](#) — GitHub Advisory Database (published 6 March 2026) **HIGH**
- [An Empirical Study of Message-Passing Overhead](#) — Michael L. Scott, Alan L. Cox (1986) **MEDIUM**
- [Interprocess Communication \(IPC\)](#) — [QNX Neutrino System Architecture](#) — QNX Software Systems **HIGH**
- [Inter-Process Communication](#) — [CS 351 lecture slides](#) — Michael Lee / Illinois Institute of Technology **MEDIUM**
- [6 Linux IPC Mechanisms: When to Use Each](#) — William / [phb-crystal-ball.org](#) (14 August 2026) **LOW**
- [IPC Overview](#) — [decision matrices and anti-patterns](#) — Anshik Bansal / OneNoughtOne **LOW**
- [Linux IPC Like You Actually Need to Use It](#) — Satesh / [unixy.io](#) (20 April 2026) **LOW**
- [Concepts/IPC/Pipes and Named Pipes](#) — [issue #3419](#) — forward reference, note not yet created
- [Concepts/IPC/Signals and Asynchronous Notification](#) — [issue #3420](#) — forward reference, note not yet created
- [Concepts/IPC/Cross-Process Synchronisation](#) — [issue #3421](#) — forward reference, note not yet created
- [Concepts/IPC/Local Sockets and Descriptor Passing](#) — [issue #3422](#) — forward reference, note not yet created
- [Concepts/IPC/File Locking and Advisory Coordination](#) — [issue #3423](#) — forward reference, note not yet created

12.65 V83 — The unretriability of a failed `fsync()`

This note establishes that a failed Linux `fsync()` cannot be safely retried, because the kernel destroys both of the things a retry would need to find: the dirty page, and the record that it ever failed. When a write-back fails, the kernel marks the page clean and discards it rather than holding it forever, since an unwritable device may simply have been unplugged. It also records the failure as a one-shot flag on the file's `address_space` — and the act of a subsequent `fsync()` reading that flag clears it. A second call therefore finds a clean flag and no dirty pages, and truthfully reports success on data that was never written. This is not a bug either side introduced: the note frames it as a contract failure between two individually defensible, mutually incompatible, and never-written-down readings of what a failed `fsync()` means — the database side assumed a failure stays reported until acknowledged, the kernel side assumed each call demarcates its own epoch of responsibility. PostgreSQL's checkpoint hit exactly this failure mode, retried a failed checkpoint, saw success, and recycled the WAL segments that were the only surviving record of the lost writes. The fix, shipped in 2018 and back-patched across supported releases, was not smarter retry logic but `PANIC`: crash immediately so the WAL is never recycled, and rebuild the in-memory model from the log rather than trusting a syscall return code that can no longer be trusted. InnoDB and WiredTiger converged on the same answer within months, which the note treats as the strongest evidence the mechanism was read correctly.

The note is explicit about what it does not claim. It does not hold across all operating systems or filesystems: FreeBSD re-dirties the page after clearing the error, so retry there is genuinely safe, and the claim's premise fails on that system. It does not reach unbuffered or direct I/O, where there is no page-cache copy to discard and a reissued write is a real retry. On Windows, the note develops the same failure class — a lazy write-back cache that discards data on failure — as a parallel worth equal treatment, but it stops short of a behavioral claim: whether a retried `FlushFileBuffers` on NTFS can falsely report success after a lost delayed write is, as of the note's writing, unestablished in any public source, including direct questions to a PostgreSQL committer and Microsoft's own developer channels. The note treats this explicitly as an open question rather than inferring an answer by analogy to Linux. It also flags as contested, not settled, whether `PANIC` remains the right operational trade-off on cloud block storage in 2026, where a transient and self-clearing `EIO` may make a crash-and-recover cost more than the failure it guards against — the physics of the argument is unchanged, but the price is disputed.

Referenced sources

- [PostgreSQL's `fsync\(\)` surprise](#) — Jonathan Corbet / LWN.net (2018) HIGH

- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curated archive of the postgresql-hackers thread, 2018) **HIGH**
- [PostgreSQL commit 9ccdd7f6 — "PANIC on fsync\(\) failure"](#) — Craig Ringer / Thomas Munro, PostgreSQL Global Development Group (2018) **HIGH**
- [PostgreSQL documentation — data_sync_retry](#) — PostgreSQL Global Development Group **HIGH**
- [fs: enhanced writeback error reporting with errseq_t \(pile #1\)](#) — Jeff Layton, linux-fsdevel (2017) **HIGH**
- [The errseq_t datatype](#) — Linux Kernel documentation — The Linux Kernel documentation **HIGH**
- [Linux Filesystems API summary — filemap_set_wb_err / filemap_check_wb_err](#) — The Linux Kernel documentation **HIGH**
- [Troubleshoot the event ID 50 error message](#) — Microsoft Learn (originally KB 816004; page updated 2026-02-12) **HIGH**
- [File Caching — Win32 apps](#) — Microsoft Learn **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [NtFlushBuffersFileEx function \(ntifs.h\)](#) — Microsoft Learn (Windows Driver Kit) **HIGH**
- [MS-ERREF: NTSTATUS Values](#) — Microsoft Open Specifications **HIGH**
- [Delayed Write Failed error — data has been lost \(KB 2021074\)](#) — Microsoft Learn (page updated 2026-02-12) **HIGH**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — David Sitsky, Stack Overflow (asked 2019-02-13) **MEDIUM**
- [Hacker News — "Does anyone know if FlushFileBuffers\(\) on Windows also forgets to flush data..."](#) — Hacker News thread, participants incl. Thomas Munro (macdice) (2019) **LOW**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20, pp. 753–767 (2020) **HIGH**
- [When Amnesia Strikes: Understanding and Reproducing Data Loss Bugs with Fault Injection](#) — Ramos, Macedo et al., PVLDB vol. 17, pp. 3017–3029 (2024) **HIGH**
- [fsync](#) — POSIX Programmer's Manual — IEEE Std 1003.1 / The Open Group Base Specifications **HIGH**
- [How Tailscale helped find the SQLite WAL-Reset bug](#) — Tailscale engineering (2026) **MEDIUM**
- [All Your GUCs in a Row: data_sync_retry](#) — The Build (PostgreSQL Experts) **MEDIUM**

12.66 V84 — The `fsync()` silent-data-loss defect ("fsyncgate")

This note establishes the mechanism behind "fsyncgate": on a buffered-write failure, Linux marks the dirty page **clean** and discards it, then reports the writeback error to `fsync()` exactly once via an `AS_EIO` flag on the file's `address_space`. The act of reporting the error clears the flag, so a caller that missed the first report and retries `fsync()` later gets a return of 0 — not because the write succeeded, but because there was nothing left in the page cache to check. PostgreSQL's checkpointer hit exactly this: it retried a checkpoint, saw success, recycled its write-ahead log, and silently lost committed data. The claim is backed by the original `pgsql-hackers` thread (including a reader's `strace` reproduction under a `dmsetup` error target), independently corroborated by a peer-reviewed fault-injection study across `ext4`, `XFS`, and `Btrfs`, and by the convergent response of three independent storage engines — PostgreSQL, InnoDB, and WiredTiger — which each replaced retry logic with a hard crash/PANIC during 2018, citing the same mechanism in their own commit messages.

The note is explicit about what remains unsettled. It frames this as a specification vacuum rather than a clean-cut bug: POSIX constrains only the *successful* case of `fsync()` and is silent on the state of the world after a failure, so whether the kernel's one-shot reporting was a defect or a coherent design choice is presented as a genuinely contested, unresolved dispute between the PostgreSQL and kernel communities. It also flags that the eventual kernel-side fix (per-descriptor error cursors, merged in Linux 4.13) makes the *report* reliable without making the *data* recoverable, and that PostgreSQL's own commit acknowledges a residual window — a checkpointer that opens a file after an error already occurred can still miss it. The claim that OpenBSD and NetBSD share this behavior rests on a single mailing-list report and is flagged as unverified. Finally, the note notes that direct I/O is named by all parties as the eventual correct fix but, years later, still has not landed as a production write path in PostgreSQL.

Referenced sources

- [PostgreSQL's `fsync\(\)` surprise](#) — Jonathan Corbet / LWN.net (2018) **HIGH**
- [Fsyncgate: errors on `fsync` are unrecoverable](#) — Dan Luu (archive of the `pgsql-hackers` thread, 2018) **HIGH**
- [PostgreSQL commit 9ccdd7f66e3324d2b6d3dec282cfa9ff084083f1 — "PANIC on `fsync\(\)` failure"](#) — Craig Ringer / Thomas Munro, PostgreSQL Global Development Group (2018) **HIGH**
- [PostgreSQL documentation — `data\_sync\_retry`](#) — PostgreSQL Global Development Group **HIGH**
- [MySQL Bug #90296 — "Hard error should be report when `fsync\(\)` return EIO"](#) — Oracle / MySQL bug tracker (2018) **HIGH**

- [mysql-server commit 8590c8e12a3374eecb547359750a9d2a128fa6a — Bug#27805553](#) — Oracle / MySQL Server (2018) **HIGH**
- [WiredTiger commit ae8bccce3d8a8248afa0e4e0cf67674a43dede96 — WT-4045 "Don't retry fsync calls after EIO failure"](#) — Keith Bostic / Alex Gorrod, MongoDB WiredTiger (2018) **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20 (2020) **HIGH**
- [fs: introduce new writeback error reporting and convert existing API as a wrapper around it](#) — Jeff Layton, linux-fsdevel (2017) **HIGH**
- [PATCH v2 3/5 vfs: track per-sb writeback errors and report them to syncfs](#) — Jeff Layton, LKML (2018) **HIGH**
- [Linux Filesystems API summary — filemap_set_wb_err / filemap_check_wb_err](#) — The Linux Kernel documentation **HIGH**
- [PostgreSQL 18 and beyond: From AIO to Direct IO?](#) — Cybertec PostgreSQL International (2025) **MEDIUM**
- [debug_io_direct](#) — pgPedia — pgPedia, a PostgreSQL Encyclopedia **MEDIUM**
- [All Your GUCs in a Row: data_sync_retry](#) — The Build (PostgreSQL Experts) **MEDIUM**
- [fsync](#) — POSIX Programmer's Manual — IEEE Std 1003.1 / The Open Group Base Specifications **HIGH**

12.67 V85 — The unretryability of a failed fsync() call

This note argues that a failed `fsync()` call cannot safely be retried, because a Linux write-back failure destroys two distinct things and only one of them is the data. When a dirty page cannot be flushed to disk, the kernel discards it and marks it clean, to avoid an unbounded memory leak on a device that may simply be gone; it also records the failure as a one-shot flag on the file, which is cleared the instant it is read. A second `fsync()` therefore finds no dirty pages and no flag, and truthfully reports success — even though the data was never written. The note traces this mechanism through the kernel's own patch history and the 2018 PostgreSQL incident, in which the database's checkpointer retried a failed checkpoint, observed success, and recycled write-ahead log segments that were the only surviving record of the lost writes. PostgreSQL's fix, and independently InnoDB's and WiredTiger's, replaced retry with process termination followed by recovery from the log, on the reasoning that once the error state is destroyed, an in-memory model of what is on disk cannot be trusted and must be rebuilt from a source whose durability was established independently. The note extends the

same question to Windows, showing that Cache Manager discards lazily written data on a failed flush in the same way, while documenting an escape hatch, unbuffered I/O, that Linux never formalized in the same explicit terms.

The note is explicit about what it does not establish. It frames the Linux/PostgreSQL failure as a dispute between two individually defensible, mutually incompatible, and never-written-down contracts — the kernel's best-effort, one-shot notification model versus the database's expectation of durable, total-order failure reporting — and assigns fault to neither side. It names FreeBSD as a live counterexample, where a failed sync leaves the buffer dirty and re-raises the error, so the "retry is unsafe" premise does not hold universally, and it excludes unbuffered or direct I/O, where a retry genuinely does re-attempt the failed write. For Windows specifically, the note distinguishes what is documented — that a failed lazy write loses data — from what is not: whether a retried flush can falsely report success after such a loss is, as the note states, a question asked directly online and left unanswered for years, and the note declines to guess at an answer, treating it instead as an open, unsettled question best resolved by fault injection rather than further documentation review. It also records a 2025 Windows SQLite incident it considered and deliberately rejected as a parallel, and treats whether process termination remains the right operational trade-off on modern cloud storage, as opposed to the right correctness argument, as a live and unresolved disagreement about cost rather than physics.

Referenced sources

- [PostgreSQL's fsync\(\) surprise](#) — Jonathan Corbet / LWN.net (2018) **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curated archive of the pgsq-l-hackers thread, 2018) **HIGH**
- [PostgreSQL commit 9ccdd7f6 — "PANIC on fsync\(\) failure"](#) — Craig Ringer / Thomas Munro, PostgreSQL Global Development Group (2018) **HIGH**
- [PostgreSQL documentation — data_sync_retry](#) — PostgreSQL Global Development Group **HIGH**
- [fs: enhanced writeback error reporting with errseq_t \(pile #1\)](#) — Jeff Layton, linux-fsdevel (2017) **HIGH**
- [The errseq_t datatype — Linux Kernel documentation](#) — The Linux Kernel documentation **HIGH**
- [Linux Filesystems API summary — filemap_set_wb_err / filemap_check_wb_err](#) — The Linux Kernel documentation **HIGH**
- [Troubleshoot the event ID 50 error message](#) — Microsoft Learn (originally KB 816004; page updated 2026-02-12) **HIGH**
- [File Caching — Win32 apps](#) — Microsoft Learn **HIGH**

- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [NtFlushBuffersFileEx function \(ntifs.h\)](#) — Microsoft Learn (Windows Driver Kit) **HIGH**
- [MS-ERREF: NTSTATUS Values](#) — Microsoft Open Specifications **HIGH**
- [Delayed Write Failed error — data has been lost \(KB 2021074\)](#) — Microsoft Learn (page updated 2026-02-12) **HIGH**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — David Sitsky, Stack Overflow (asked 2019-02-13) **MEDIUM**
- [Hacker News — "Does anyone know if FlushFileBuffers\(\) on Windows also forgets to flush data..."](#) — Hacker News thread, participants incl. Thomas Munro (`macdice`) (2019) **LOW**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20, pp. 753–767 (2020) **HIGH**
- [When Amnesia Strikes: Understanding and Reproducing Data Loss Bugs with Fault Injection](#) — Ramos, Macedo et al., PVLDB vol. 17, pp. 3017–3029 (2024) **HIGH**
- [fsync](#) — POSIX Programmer's Manual — IEEE Std 1003.1 / The Open Group Base Specifications **HIGH**
- [How Tailscale helped find the SQLite WAL-Reset bug](#) — Tailscale engineering (2026) **MEDIUM**
- [All Your GUCs in a Row: `data\_sync\_retry`](#) — The Build (PostgreSQL Experts) **MEDIUM**

12.68 V86 — File I/O durability across the buffering stack

This note establishes that durability is not a property of a single write call but of a sequence of independently-failing flushes across four boundaries: a language-runtime buffer, a libc/CRT buffer, the kernel's page cache, and the storage device's own cache. A successful `write()` reports only that the kernel accepted the bytes into the first of these caches — custody, not persistence — and closing a file, exiting a process, or calling `fflush()` / `f.flush()` does not cross the remaining boundaries. On Linux, the note grounds this in the `fsync(2)` man page and the 2018 "fsyncgate" incident: writeback failures are reported once via `errseq_t` and then discarded, so PostgreSQL's retry-on-EIO checkpointer silently lost data for roughly two decades. The resolution across PostgreSQL, InnoDB and WiredTiger was to PANIC and recover from a write-ahead log rather than retry — a genuine contract disagreement, not a bug on either side, since POSIX is silent on post-failure state. For Windows, the note walks the same four layers under different names and reaches its most load-bearing finding: no public source — not Microsoft's API documentation, not the PostgreSQL wiki's per-OS matrix, not the

Hacker News and Stack Overflow threads that asked directly — describes whether Windows discards data after a failed flush the way Linux does. It concludes this makes Windows's position undocumented rather than safe, and argues that is worse than Linux's documented hostility, since no correct response can be engineered against a silent contract.

The note is explicit about what it does not claim. It does not assert that Windows has the same non-retryable failure class as Linux, only that the question is unanswered in any authoritative source; it labels this conclusion [inference] rather than verified. It does not claim PANIC-on-`EIO` is universally correct, noting that on cloud block storage an `EIO` is often transient, which changes the availability cost of crashing without changing the underlying correctness argument. It also flags the four-layer model itself as a teaching simplification that does not cleanly extend to network filesystems, `mmap`-based I/O, or copy-on-write filesystems, and it leaves three points as open grounding gaps: whether any Windows tracing tool exposes device-level `FLUSH_CACHE` commands as cleanly as Linux's `blktrace`, whether the CRT's `_commit()` maps to `FlushFileBuffers` internally, and whether Windows exposes any dirty-page-age control comparable to Linux's `dirty_expire_centisecs`.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026) **MEDIUM**
- [fsync\(2\)](#) — Linux manual page — Linux man-pages project **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017) **HIGH**
- [io](#) — Core tools for working with streams — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn **HIGH**
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022) **MEDIUM**
- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla **HIGH**
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode **MEDIUM**
- [Page Cache Writeback](#) — Linux Kernel Internals **MEDIUM**
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn **HIGH**
- [File Caching](#) — Microsoft Learn **HIGH**

- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al. **HIGH**
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026) **LOW**
- [Life of a Write](#) — Linux Kernel Internals **MEDIUM**
- [The errseq_t datatype](#) — The Linux Kernel documentation — Linux kernel documentation **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018) **HIGH**
- [Fsync Errors](#) — PostgreSQL wiki **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020) **HIGH**
- [CreateFileW function](#) — FILE_FLAG_WRITE_THROUGH and FILE_FLAG_NO_BUFFERING semantics — Microsoft Learn **HIGH**
- [File Buffering](#) — Microsoft Learn **HIGH**
- [Windows Write Caching](#) — Part 2: An Overview for Application Developers — Dilip Naik (2012) **MEDIUM**
- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News (2019) **LOW**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow (2019) **MEDIUM**
- [os](#) — Miscellaneous operating system interfaces — Python Software Foundation **HIGH**
- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow **MEDIUM**
- [DTrace on Windows](#) — Microsoft Learn **HIGH**
- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn **HIGH**
- [Write-Ahead Logging \(SQLite\)](#) — SQLite **HIGH**

12.69 V87 — The socket as a kernel object, and which layer each platform makes its contract

This note's central claim is that a socket is a kernel object reached through a per-process handle, and that Linux and Windows disagree not about the underlying mechanism but about which layer carries the binding contract. On Linux, `socket()` allocates a `struct socket` and binds it to a `struct file` on the pseudo-filesystem `sockfs`, so `read`, `write`, `close`, `select` and `fstat` all work on it for a structural reason: the ker-

nel's dispatch table routes them there, not as a convenience. The note backs this with the kernel source (`net/socket.c`), the `socket(2)` man page, and a reproduced experiment run on Linux 6.8.0 showing `fstat` reporting `S_ISSOCK` and `/proc/self/fd/N` resolving to the same inode a plain `write / read / close` cycle uses. On Windows, `socket()` returns an opaque `SOCKET`, and Microsoft's documentation states plainly that it "is not a file descriptor and must be used with the Winsock functions." Three consequences follow from this divergence: what else can operate on the handle, how it is released (`close` versus `closesocket`, with `CloseHandle` explicitly forbidden), and how failure is reported (`errno` versus `WSAGetLastError`). The note complicates the easy reading, though — it shows that the NT kernel underneath Winsock does model a socket as a file object, a handle to a device called `\Device\Afd`, which is exactly what lets `ReadFile`, `WriteFile` and I/O completion ports work on it. The divergence, it argues, is introduced above the kernel, by the published Win32 contract, not by a difference in what the kernel actually builds.

The note is explicit about what remains unsettled. The reconciliation between two Microsoft Learn pages that look contradictory at a glance — one saying a socket descriptor is not a file descriptor, the other saying a socket handle can optionally be a file handle — is flagged [inference] as the note's own reading, not a claim either page makes outright. Several supporting details about the AFD device and its IOCTL-based call shape are sourced from independent researcher blog posts rather than first-party documentation and are marked [reported] rather than verified firsthand. The note also notes that `strace` was unavailable in its test environment, so the Linux verification relies on `/proc` evidence and a reproduced C/Python experiment rather than an actual syscall trace, a substitution it states rather than glosses over. It does not claim the Linux and Windows object models are truly equivalent — it separately catalogues where the uniform Linux interface hides non-uniform behavior (partial reads, no `lseek`, `EMFILE` from socket leaks) — and it treats the recent convergence of Windows `AF_UNIX` support as partial, since it still omits descriptor-passing and other file-descriptor-dependent features.

Referenced sources

- [socket\(2\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project **HIGH**
- [socket\(7\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project **HIGH**
- [linux/net/socket.c](#) — Linux kernel source tree **HIGH**
- [Socket Data Type — Win32 apps](#) — Microsoft **HIGH**
- [Socket Handles — Win32 apps](#) — Microsoft **HIGH**
- [Error Codes - errno, h_errno and WSAGetLastError — Win32 apps](#) — Microsoft **HIGH**
- [WSAStartup function \(winsock.h\)](#) — Microsoft **HIGH**

- [closesocket function \(winsock.h\)](#) — Microsoft **HIGH**
- [CloseHandle function \(handleapi.h\)](#) — Microsoft **HIGH**
- [Select, fd_set, and FD_XXX Macros](#) — Win32 apps — Microsoft **HIGH**
- [socket](#) — Low-level networking interface (Python documentation) — Python Software Foundation **HIGH**
- [select](#) — Waiting for I/O completion (Python documentation) — Python Software Foundation **HIGH**
- [AF_UNIX comes to Windows](#) — Microsoft, Windows Command Line blog (2017) **HIGH**
- [Improving AFD Socket Visibility for Windows Forensics & Troubleshooting](#) — diversenok (2025) **MEDIUM**
- [12 Days of HaXmas: Does it Blend Like a Duck? \(Failing Portably\)](#) — Rapid7 (2014) **MEDIUM**
- [An Introductory 4.4BSD Interprocess Communication Tutorial](#) — Computer Systems Research Group, UC Berkeley **HIGH**
- [An Advanced 4.3BSD Interprocess Communication Tutorial](#) — Samuel J. Leffler et al., UC Berkeley **HIGH**
- [4.4BSD Networking Implementation Notes](#) — Computer Systems Research Group, UC Berkeley **HIGH**
- [rogerorr/NtTrace](#) — Roger Orr **HIGH**
- [Porting Socket Applications to Winsock](#) — Win32 apps — Microsoft **HIGH**
- [Under the Hood of AFD.sys Part 2: TCP handshake](#) — Mateusz Lewczak (2025) **MEDIUM**
- [cpython/Modules/socketmodule.h](#) — Python Software Foundation **HIGH**
- [Closing non-blocking socket causes leak](#) — Microsoft **HIGH**
- [vfs: mark pipes and sockets as stream-like file descriptors](#) — Linus Torvalds, Linux kernel (2019) **HIGH**
- [io_uring zero copy Rx](#) — The Linux Kernel documentation — The Linux Kernel Organization **HIGH**
- [Winsock Programmer's FAQ: General Winsock Information](#) — Warren Young **MEDIUM**

12.70 V88 — Ring-based I/O interfaces on Linux and Windows

This note describes `io_uring` (Linux, kernel 5.1) and `IoRing` (Windows, build 22000) as two implementations of the same idea: instead of one system call per I/O operation, the application and the kernel share two ring buffers in memory — a submission queue the program writes into and a completion queue it reads out of — so only the "kick" that wakes the kernel needs to cross the syscall boundary, and one kick can carry many requests. It walks the interface at three levels of abstraction (raw syscalls, the `liburing` helper library, and the absence of a Python binding), and the equivalent Windows call sequence (`CreateIoRing`, `BuildIoRing*`, `SubmitIoRing`). Its central argument is about direction of convergence: Linux, historically a readiness-based platform, adopted completion semantics because readiness cannot describe a regular file, which is always "ready"; Windows, already completion-native via IOCP, adopted the ring as a faster transport for a model it already had. Evidence for the interface mechanics comes from Linux man pages, Jens Axboe's original design paper, and Microsoft's API reference; the Windows-specific behavior rests substantially on reverse-engineering work by researcher Yarden Shafir, since Microsoft's own documentation does not cover internals to the same depth.

The note is explicit about what remains unsettled. A large section covers `io_uring`'s security history — Google reported that 60% of kernel exploits submitted to its bug-bounty program in one measurement period targeted `io_uring`, and the note traces this to a structural cause (work queued to kernel worker threads bypasses `seccomp` filtering) rather than a single bug, alongside an admission from Axboe himself that the interface does not compose with `seccomp`. It also documents that Windows `IoRing` has seen negligible real-world adoption a year after shipping, per its own documenter. Several claims are flagged in the note's own terms as `[reported]` (secondhand aggregation, e.g. a Python-binding ecosystem survey and a comparison against `SPDK`) or `[inference]` rather than `[verified]`, and the note records specific gaps it could not close during research — one external source it could not re-fetch, one trace it could not capture in its own sandboxed environment, and search coverage limited by unavailable engines.

Referenced sources

- [io_uring\(7\) — Linux manual page, Linux man-pages project / kernel.org](#) HIGH
- [io_uring_setup\(2\) — Linux manual page, Linux man-pages project / kernel.org](#) HIGH
- [io_uring_enter\(2\) — Linux manual page, Linux man-pages project / kernel.org](#) HIGH

- [io_uring_register\(2\)](#) — Linux manual page, Linux man-pages project / kernel.org **HIGH**
- [Efficient IO with io_uring](#), Jens Axboe (2019) **HIGH**
- [axboe/liburing](#), Jens Axboe (2019–) **HIGH**
- [io_uring.h header](#) — Win32 apps, Microsoft Learn (2022) **HIGH**
- [CreateIoRing function \(io_uring.h\)](#), Microsoft Learn (2021, updated 2025) **HIGH**
- [BuildIoRingReadFile function \(io_uring.h\)](#), Microsoft Learn (2021, updated 2025) **HIGH**
- [SubmitIoRing function \(io_uring.h\)](#), Microsoft Learn (2021, updated 2025) **HIGH**
- [BuildIoRingRegisterBuffers function \(io_uring.h\)](#), Microsoft Learn **HIGH**
- [BuildIoRingRegisterFileHandles function \(io_uring.h\)](#), Microsoft Learn **HIGH**
- [Learnings from kCTF VRP's 42 Linux kernel exploits submissions](#), Google Security Team / Google Online Security Blog (2023) **HIGH**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#), Yarden Shafir / Winsider Seminars & Solutions (2021) **HIGH**
- [One Year to I/O Ring: What Changed?](#), Yarden Shafir / Winsider Seminars & Solutions (2022) **HIGH**
- [One I/O Ring to Rule Them All: A Full Read/Write Exploit Primitive on Windows 11](#), Yarden Shafir / Winsider Seminars & Solutions (2022) **HIGH**
- [Ringin in a new asynchronous I/O API](#), Jonathan Corbet / LWN.net (2019) **HIGH**
- [Task-level io_uring restrictions](#), LWN.net (2026) **HIGH**
- [uringscope: Portable, Low-Overhead Observability for io_uring](#), Rajarshi Chowdhury / arXiv (2026) **MEDIUM**
- [io_uring Security Model](#), Linux Kernel Internals **MEDIUM**
- [io_uring War Stories](#), Linux Kernel Internals **MEDIUM**
- [\[PATCHSET RFC 0/2\] Per-task io_uring opcode restrictions](#), Jens Axboe / io-uring mailing list (2026) **HIGH**
- [High-Performance DBMSs with io_uring: When and How to Use It](#), Jasny et al. / PVLDB vol. 19 **HIGH**
- [io_uring Security and Hardening: Disabling, Restricting and Auditing](#), Systems Hardening (2026) **MEDIUM**
- [io_uring, SPDK, and the Kernel Bypass Wars](#), Murat Karslioglu (2026) **MEDIUM**
- [Python io_uring binding landscape](#) — `liburing`, `uringcore`, `aioring`, `uring_file`, `asyncio-uring`, and CPython issues 41271 / 44738, PyPI / CPython issue tracker / GitHub project pages **MEDIUM**
- [Submission Queue Polling](#) — Lord of the io_uring, Shuveb Hussain **MEDIUM**

- [Google Limiting IO_uring Use Due To Security Vulnerabilities](#), Michael Larabel / Phoronix (2023) **MEDIUM**
- [oss-security: Our learnings from 42 Linux kernel exploits, we are limiting io_uring](#), Tamás Koczka / oss-security mailing list (2023) **HIGH**
- [windows-ioring-sys — crate source and design notes](#), docs.rs (2026) **MEDIUM**
- [io_uring Reads Your Files and Opens Sockets Without Calling the Syscalls Your auditd Rules Watch](#), Trackr.Live (2026) **MEDIUM**
- [Asynchronous I/O \(AIO\) — PostgreSQL feature matrix](#), The PostgreSQL Global Development Group **HIGH**
- [CVE-2026-80713 — io_uring: preserve task restrictions across exec](#), vuln.today (2026), quoting the upstream kernel commit message **MEDIUM**
- [Offloading I/O to Dedicated Cores: An Asymmetric io_uring Backend for Seastar and ScyllaDB](#), ScyllaDB (2026) **HIGH**

12.71 V89 — The unretryable fsync() failure

This note establishes that a failed `fsync()` on Linux cannot be safely retried, because the kernel destroys two separate things on a write-back failure and only one of them is the data. When a dirty page cannot be flushed, the kernel marks it clean and discards it rather than holding it forever, since an unwritable page can otherwise leak memory indefinitely. It then records the failure as a one-shot flag on the file's `address_space`; the next `fsync()` reads that flag, reports `EIO`, and the act of reading clears it. A second `fsync()` therefore finds a clean flag and no dirty pages and truthfully reports success, even though the data was never written. This is documented in the kernel's own patch series and reproduced directly under `strace`. PostgreSQL's checkpointer hit exactly this path — erroring, retrying, succeeding, and recycling WAL segments that were the last surviving record of the lost writes — which is why the 2018 fix was to `PANIC` on the first failure rather than retry, a response independently converged on by InnoDB and WiredTiger. The note frames this as a contract failure rather than a bug in either side, since POSIX never specifies the post-failure state, and extends the same question to Windows, where Cache Manager discards lazily-written data the same way and documents that durability-critical writers should bypass it with unbuffered I/O.

The note is explicit about what it does not claim. FreeBSD is a stated counterexample: it re-marks the failed page dirty instead of clearing it, so a retry there is genuinely safe, and the Linux premise fails on that filesystem's semantics. The claim does not extend to unbuffered or direct I/O, where there is no page-cache copy to discard and a reissued call is a real retry. Whether a retried `FlushFileBuffers` on Windows can likewise falsely report success after a lost delayed write is described as publicly unsettled rather

than answered — the note finds the question asked and unanswered on Stack Overflow and Hacker News, and states directly that it does not claim Windows behaves like Linux, only that the question remains open. It also treats whether `PANIC` is still the right operational trade-off, as opposed to the right correctness argument, as contested: on cloud block storage `EIO` is often transient, so the same crash-and-recover response may now trade a short, self-resolving hiccup for a much longer WAL recovery. Finally, the note is careful to say that the 2018 fixes made the failure loud rather than making buffered-I/O durability correct — later research is cited finding that no application yet handles the failure perfectly.

Referenced sources

- [PostgreSQL's fsync\(\) surprise](#) — Jonathan Corbet / LWN.net (2018) **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu **HIGH**
- [PostgreSQL commit 9ccdd7f6 — "PANIC on fsync\(\) failure"](#) — Craig Ringer / Thomas Munro, PostgreSQL Global Development Group (2018) **HIGH**
- [PostgreSQL documentation — data_sync_retry](#) — PostgreSQL Global Development Group **HIGH**
- [fs: enhanced writeback error reporting with errseq_t \(pile #1\)](#) — Jeff Layton, linux-fsdevel (2017) **HIGH**
- [The errseq_t datatype](#) — Linux Kernel documentation — The Linux Kernel documentation **HIGH**
- [Linux Filesystems API summary — filemap_set_wb_err / filemap_check_wb_err](#) — The Linux Kernel documentation **HIGH**
- [Troubleshoot the event ID 50 error message](#) — Microsoft Learn (originally KB 816004; page updated 2026-02-12) **HIGH**
- [File Caching — Win32 apps](#) — Microsoft Learn **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [NtFlushBuffersFileEx function \(ntifs.h\)](#) — Microsoft Learn (Windows Driver Kit) **HIGH**
- [MS-ERREF: NTSTATUS Values](#) — Microsoft Open Specifications **HIGH**
- [Delayed Write Failed error — data has been lost \(KB 2021074\)](#) — Microsoft Learn (page updated 2026-02-12) **HIGH**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — David Sitsky, Stack Overflow (asked 2019-02-13) **MEDIUM**
- [Hacker News — "Does anyone know if FlushFileBuffers\(\) on Windows also forgets to flush data..."](#) — Hacker News thread, participants incl. Thomas Munro (`macdice`) (2019) **LOW**

- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20, pp. 753–767 (2020) **HIGH**
- [When Amnesia Strikes: Understanding and Reproducing Data Loss Bugs with Fault Injection](#) — Ramos, Macedo et al., PVLDB vol. 17, pp. 3017–3029 (2024) **HIGH**
- [fsync](#) — POSIX Programmer's Manual — IEEE Std 1003.1 / The Open Group Base Specifications **HIGH**
- [How Tailscale helped find the SQLite WAL-Reset bug](#) — Tailscale engineering (2026) **MEDIUM**
- [All Your GUCs in a Row: data_sync_retry](#) — The Build (PostgreSQL Experts) **MEDIUM**

12.72 V90 — The non-retryability of a failed fsync()

The note argues that a failed `fsync()` on Linux cannot be safely retried, because a write-back failure destroys two separate things and only one of them is the data. When the kernel cannot flush a dirty page, it marks the page clean and discards it — keeping an unwritable page dirty forever would be an unbounded memory leak on a device that may simply be gone — and it records the failure as a one-shot flag, `AS_EIO`, on the file's `address_space`, a flag that the act of reporting it clears. A second `fsync()` therefore finds no dirty pages and a cleared flag, and truthfully returns success even though the data was never written. This is documented in the kernel's own patch series and API documentation and was demonstrated directly: PostgreSQL's checkpointer treated the false "success" as confirmation that a checkpoint had completed and recycled WAL segments that were the last surviving record of the lost writes, and a developer reproduced the exact `strace` sequence against a `dmsetup` error target. The fix PostgreSQL shipped in 2018, back-patched to every supported release, was not smarter retry logic but `PANIC` — crashing the process so it must recover from the write-ahead log, because once the notification is destroyed, the process's belief about what is on disk cannot be re-derived from anything it still holds. The note frames this as a contract failure rather than a bug in either side: the kernel's and the database's readings of `fsync()`'s failure semantics were individually defensible and never reconciled, since POSIX specifies what success means and is silent on failure. InnoDB and WiredTiger converged on the same panic-on-failure answer within months, independently.

The note is explicit about where this stops. It is not a universal claim: FreeBSD is a documented counterexample, since it re-marks the buffer dirty after clearing the error instead of discarding it, so a retry there is genuinely safe. The claim also does not reach unbuffered or direct I/O — with `O_DIRECT` or Windows's `FILE_FLAG_NO_BUFFERING` there is no page-cache copy to discard, and a retry re-attempts the actual write. For Win-

dows, the note establishes only that the same class of failure exists — Cache Manager discards data on a failed lazy write and documents that it does so — but treats whether a *retried* flush can falsely report success as genuinely unsettled: an unanswered Stack Overflow question and a Hacker News thread in which a PostgreSQL committer says he does not know are offered as evidence the question is open in public, not as evidence of any particular Windows behavior, and the note explicitly declines to extend the Linux conclusion to Windows by analogy. It also notes that even after the 2018 fixes no application is known to handle `fsync()` failure perfectly, and that whether `PANIC` remains the right *operational* trade-off on cloud block storage, where `EIO` is often transient, is contested — a dispute about cost the note keeps separate from the underlying correctness argument, which it treats as settled.

Referenced sources

- [PostgreSQL's fsync\(\) surprise](#) — Jonathan Corbet / LWN.net (2018) **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curated archive of the `pgsql-hackers` thread, 2018) **HIGH**
- [PostgreSQL commit 9ccdd7f6 — "PANIC on fsync\(\) failure"](#) — Craig Ringer / Thomas Munro, PostgreSQL Global Development Group (2018) **HIGH**
- [PostgreSQL documentation — `data\_sync\_retry`](#) — PostgreSQL Global Development Group **HIGH**
- [fs: enhanced writeback error reporting with `errseq\_t` \(pile #1\)](#) — Jeff Layton, `linux-fsdevel` (2017) **HIGH**
- [The `errseq\_t` datatype](#) — Linux Kernel documentation — The Linux Kernel documentation **HIGH**
- [Linux Filesystems API summary — `filemap\_set\_wb\_err` / `filemap\_check\_wb\_err`](#) — The Linux Kernel documentation **HIGH**
- [Troubleshoot the event ID 50 error message](#) — Microsoft Learn (originally KB 816004; page updated 2026-02-12) **HIGH**
- [File Caching — Win32 apps](#) — Microsoft Learn **HIGH**
- [FlushFileBuffers function \(`fileapi.h`\)](#) — Microsoft Learn **HIGH**
- [NtFlushBuffersFileEx function \(`ntifs.h`\)](#) — Microsoft Learn (Windows Driver Kit) **HIGH**
- [MS-ERREF: NTSTATUS Values](#) — Microsoft Open Specifications **HIGH**
- [Delayed Write Failed error — data has been lost \(KB 2021074\)](#) — Microsoft Learn (page updated 2026-02-12) **HIGH**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — David Sitsky, Stack Overflow (asked 2019-02-13) **MEDIUM**

- [Hacker News — "Does anyone know if FlushFileBuffers\(\) on Windows also forgets to flush data..."](#) — Hacker News thread, participants incl. Thomas Munro (maddice) (2019) **LOW**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20, pp. 753–767 (2020) **HIGH**
- [When Amnesia Strikes: Understanding and Reproducing Data Loss Bugs with Fault Injection](#) — Ramos, Macedo et al., PVLDB vol. 17, pp. 3017–3029 (2024) **HIGH**
- [fsync](#) — POSIX Programmer's Manual — IEEE Std 1003.1 / The Open Group Base Specifications **HIGH**
- [How Tailscale helped find the SQLite WAL-Reset bug](#) — Tailscale engineering (2026) **MEDIUM**
- [All Your GUCs in a Row: data_sync_retry](#) — The Build (PostgreSQL Experts) **MEDIUM**

12.73 V91 — File I/O durability across the write-to-medium path

This note establishes that durability is not a property of a single write but of a sequence of four independent flushes — runtime buffer to kernel, kernel to page cache, page cache to storage device, and device cache to medium — each with its own trigger and its own cost, and each capable of failing on its own. A successful `write()` reports only that the kernel accepted the bytes into the first buffer; it is a statement about custody, not persistence, and POSIX's real guarantee after a successful write is read-after-write visibility, which is a coherence property distinct from durability. The note grounds this in the Linux `fsync()` man page and kernel `errseq_t` documentation, and works through the 2018 "fsyncgate" incident in detail: when Linux writeback fails, the kernel reports the error to the next `fsync()` call and then discards the failed pages, so a retry has nothing left to write and reports false success. PostgreSQL's checkpointer retried on failure and silently lost data for roughly two decades before the field converged on PANIC-and-recover-from-write-ahead-log, a response PostgreSQL, InnoDB and WiredTiger reached independently. The note frames this as a genuine contract disagreement rather than a bug on either side — POSIX is silent on post-failure state, and the kernel and PostgreSQL each read that silence differently. It then does the same analysis for Windows, walking through `FlushFileBuffers`, `FILE_FLAG_WRITE_THROUGH`, and `FILE_FLAG_NO_BUFFERING`, and concludes that Windows arranges the same four layers but documents no equivalent post-failure contract at all.

The note is explicit about what it does not establish. It refuses to claim either "Windows is safe" or "Windows has the same bug," stating instead that nobody has publicly characterized Windows's post-failure behavior and Microsoft has not documented it — the

PostgreSQL wiki's cross-platform matrix carries no Windows row, and direct questions on Hacker News and Stack Overflow went unanswered. It flags several points as speculative or inferred rather than verified: that no Windows tracing tool exposes device flush commands as cleanly as Linux's `blktrace`, that Python's `os.fsync` maps to `_commit()` in a way consistent with `FlushFileBuffers` (an inference, not documented), and that the whole four-layer model is a teaching simplification that breaks down for network filesystems, memory-mapped I/O, and copy-on-write filesystems. It also notes the calibration limit of its own advice: PANIC-on-error is reasoned for local disks with hardware failures that do not self-clear, and the note says explicitly that cloud block storage changes the availability cost without changing the correctness argument.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026) **MEDIUM**
- [fsync\(2\)](#) — Linux manual page — Linux man-pages project **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017) **HIGH**
- [io](#) — Core tools for working with streams — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn **HIGH**
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022) **MEDIUM**
- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla **HIGH**
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode **MEDIUM**
- [Page Cache Writeback](#) — Linux Kernel Internals **MEDIUM**
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn **HIGH**
- [File Caching](#) — Microsoft Learn **HIGH**
- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al. **HIGH**
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026) **LOW**
- [Life of a Write](#) — Linux Kernel Internals **MEDIUM**
- [The errseq_t datatype](#) — The Linux Kernel documentation — Linux kernel documentation **HIGH**

- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018) **HIGH**
- [Fsync Errors](#) — PostgreSQL wiki **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020) **HIGH**
- [CreateFileW function — FILE_FLAG_WRITE_THROUGH and FILE_FLAG_NO_BUFFERING semantics](#) — Microsoft Learn **HIGH**
- [File Buffering](#) — Microsoft Learn **HIGH**
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — Dilip Naik (2012) **MEDIUM**
- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News (2019) **LOW**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow (2019) **MEDIUM**
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation **HIGH**
- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow **MEDIUM**
- [DTrace on Windows](#) — Microsoft Learn **HIGH**
- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn **HIGH**
- [Write-Ahead Logging \(SQLite\)](#) — SQLite **HIGH**

12.74 V92 — Readiness versus Completion I/O Models on Linux and Windows

This note argues that Linux's readiness model (`select` , `poll` , `epoll`) and Windows' completion model (overlapped I/O and I/O completion ports) are not two API styles for the same underlying mechanism but a structural split with three concrete consequences: who owns the I/O buffer and for how long, what shape the program's control flow must take, and when an error becomes observable. Evidence for buffer ownership comes directly from Microsoft's Winsock documentation, which states that a completion-model caller must supply a buffer before data exists and must not touch it until the network has consumed it, and from the `epoll(7)` man page's interest-list/ready-list design, where the kernel holds no buffer at all between notifications. Evidence for error timing comes from `GetQueuedCompletionStatus`'s three-way return value, which can report success, no packet dequeued, or a failed operation identified only by a pointer — a case a readiness model cannot produce, since its errors return synchronously

from the call the program itself makes. The note extends this into a three-level teaching structure (Python, C, and the underlying system call) and shows the split surviving into cross-platform libraries such as libuv and CPython's asyncio, which must maintain two internal architectures to hide it from application code.

The note is explicit that the readiness/completion dichotomy is a useful first cut, not a complete taxonomy: `io_uring` brings a completion interface to Linux, and Windows' newer IoRing mirrors `io_uring`, so the split is a design-space choice each platform entered from a different side rather than simply a Linux-versus-Windows distinction. Several claims are flagged rather than asserted as settled: the note marks its NtTrace transcript of a Windows overlapped receive as `[inference]`, since no published capture was located to verify the exact output shape; the claim that nginx and Redis differ in edge- versus level-triggered `epoll` usage, and the claim that Registered I/O never gained wide adoption, are marked `[reported]`, resting on single secondary sources of medium or low credibility rather than vendor or maintainer statements. The note does not claim to have resolved the LWN dispute over whether `EAGAIN` is strictly required to re-arm edge-triggered notifications — it records the disagreement among practitioners without adjudicating it.

Referenced sources

- [epoll\(7\) — Linux manual page](#), Linux man-pages project **HIGH**
- [select\(2\) — Linux manual page](#), Linux man-pages project **HIGH**
- [I/O Completion Ports](#), Microsoft Learn **HIGH**
- [GetQueuedCompletionStatus function \(ioapiset.h\)](#), Microsoft Learn **HIGH**
- [WSARecv function \(winsock2.h\)](#), Microsoft Learn **HIGH**
- [Overlapped Input/Output \(Winsock\)](#), Microsoft Learn **HIGH**
- [WSAPoll function \(winsock2.h\)](#), Microsoft Learn **HIGH**
- [WSAPoll is broken](#), Daniel Stenberg **MEDIUM**
- [The edge-triggered misunderstanding](#), LWN.net comment thread **HIGH**
- [portable-interoperable / io-traits](#), Nick Cameron **MEDIUM**
- [Async IO with completion-model IO systems](#), Nick Cameron **MEDIUM**
- [Design overview — libuv documentation](#), libuv project **HIGH**
- [libuv — iocp-links.html \(design notes\)](#), libuv project **MEDIUM**
- [asyncio — Event Loop \(Python documentation\)](#), Python Software Foundation **HIGH**
- [selectors — High-level I/O multiplexing \(Python documentation\)](#), Python Software Foundation **HIGH**

- [SDL_net: windows: Reimplement poll\(\) with select\(\), Simple DirectMedia Layer project](#) **MEDIUM**
- [PollSet not working as intended on Windows — pocoproject/poco#3248, POCO C++ Libraries issue tracker](#) **MEDIUM**
- [Issue 16507: Patch selectmodule.c to support WSApoll on Windows, Python issue tracker](#) **HIGH**
- [Under the Hood of AFD.sys Part 1: Investigating Undocumented Interfaces, Mateusz Lewczak](#) **MEDIUM**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations, Win-ster Seminars & Solutions](#) **HIGH**
- [libuv Event Loop Internals, Sujeet Jaiswal](#) **MEDIUM**
- [Linux io_uring vs Windows I/O: A Technical Comparison, Mark Veltzer](#) **LOW**
- [epoll: The API That Powers the Modern Internet, rzero.in](#) **LOW**

12.75 V93 — PostgreSQL's fsync() error-reporting failure ("fsyncgate")

This note summarizes reporting and follow-on evidence on a 2018 Linux/PostgreSQL bug in which a failed `fsync()` call, when retried, returns success even though the underlying write never completed. The kernel marks the page clean and drops it after the first — possibly missed — error report, so a second `fsync()` finds nothing outstanding and truthfully reports nothing wrong. PostgreSQL's checkpointer relied on a successful `fsync()` to license recycling its write-ahead log, so this false-positive success let committed data be silently discarded. The note treats the core mechanism as well established: it is corroborated by the original LWN report, a verbatim archive of the `pgsql-hackers` mailing-list thread that includes a syscall trace reproducing the failure with a `dmsetup` error target, a peer-reviewed 2020 USENIX study that reproduced the behavior under fault injection across three filesystems, and the shipped source-code commits from PostgreSQL, InnoDB, and WiredTiger — three independent engineering teams that each abandoned retry logic in 2018 in favor of crashing and replaying from the log. A kernel-side fix (`errseq_t`, merged in Linux 4.13) made the failure report reliable per open file descriptor but did not make the underlying lost data recoverable.

The note is explicit about what remains unresolved. It flags a residual risk window in PostgreSQL's own fix — an inode-eviction race that can still cause a missed error — as still open, by the commit message's own admission, and it cites the USENIX study's finding that no application, patched PostgreSQL included, handles `fsync` failure perfectly. It treats as contested, not settled, whether POSIX's text favors PostgreSQL's original assumption or the kernel's actual behavior, since POSIX explicitly declines to guarantee anything about outstanding I/O after a failed `fsync()`. It likewise leaves

open whether the kernel's behavior is a defect that should be fixed or a defensible tradeoff bounding unreclaimable memory, presenting substantial arguments on both sides without resolving them. Direct I/O is described as the long-agreed long-term fix that, years later, still has not shipped as a production-ready path. The note's own critical assessment further cautions that the source article underweights the argument that `fsync()`'s post-failure semantics were simply never documented, and that it frames the story as PostgreSQL-specific when other storage engines faced and independently fixed the same defect.

Referenced sources

- [PostgreSQL's fsync\(\) surprise](#) — Jonathan Corbet / LWN.net (2018) **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (archive of the postgresql-hackers thread, 2018) **HIGH**
- [PostgreSQL commit 9ccdd7f66e3324d2b6d3dec282cfa9ff084083f1](#) — "PANIC on fsync() failure" — Craig Ringer / Thomas Munro, PostgreSQL Global Development Group (2018) **HIGH**
- [PostgreSQL documentation — data_sync_retry](#) — PostgreSQL Global Development Group **HIGH**
- [MySQL Bug #90296](#) — "Hard error should be report when fsync() return EIO" — Oracle / MySQL bug tracker (2018) **HIGH**
- [mysql-server commit 8590c8e12a3374eecb547359750a9d2a128fa6a](#) — [Bug#27805553](#) — Oracle / MySQL Server (2018) **HIGH**
- [WiredTiger commit ae8bccce3d8a8248afa0e4e0cf67674a43dede96](#) — WT-4045 "Don't retry fsync calls after EIO failure" — Keith Bostic / Alex Gorrod, MongoDB WiredTiger (2018) **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20 (2020) **HIGH**
- [fs: introduce new writeback error reporting and convert existing API as a wrapper around it](#) — Jeff Layton, linux-fsdevel (2017) **HIGH**
- [PATCH v2 3/5 vfs: track per-sb writeback errors and report them to syncfs](#) — Jeff Layton, LKML (2018) **HIGH**
- [Linux Filesystems API summary — filemap_set_wb_err / filemap_check_wb_err](#) — The Linux Kernel documentation **HIGH**
- [PostgreSQL 18 and beyond: From AIO to Direct IO?](#) — Cybertec PostgreSQL International (2025) **MEDIUM**
- [debug_io_direct](#) — pgPedia — pgPedia, a PostgreSQL Encyclopedia **MEDIUM**
- [All Your GUCs in a Row: data_sync_retry](#) — The Build (PostgreSQL Experts) **MEDIUM**

- [fsync — POSIX Programmer's Manual — IEEE Std 1003.1 / The Open Group Base Specifications](#) **HIGH**

12.76 V94 — The Waiting Cost Behind the C10K Problem

This note argues that the C10K problem — Dan Kegel's 1999 goal of serving ten thousand simultaneous connections from one machine — is misdiagnosed by the usual "I/O was slow" retelling. The actual bottleneck was the cost of waiting on idle connections: under `select()` or `poll()`, the kernel is handed the entire watch list on every call and rescans it regardless of how many descriptors are actually active, so the cost tracks the size of the interest set rather than the amount of I/O performed. This is established both by Kegel's own page and by two independent measurements: Banga, Mogul and Druschel's 1999 formal statement that `select()`'s cost is proportional to the "interesting" descriptor set, and Gammo, Brecht, Shukla and Pariag's 2004 experiment, which held the request workload constant and varied only the number of idle connections, finding `select` and `poll` throughput dropped by up to 79% when 10,000 idle connections were added while `epoll` was unaffected. Chandra and Mosberger's 2000 CPU-side measurement corroborates this from a different angle. The fixes — `epoll`, `kqueue`, Solaris `/dev/poll`, and Windows IOCP — did not make `read()` or `write()` faster; each relocated the interest set into the kernel so it would not need to be re-passed on every call, meaning nothing about the data-transfer path itself changed.

The note is explicit that this is a claim about the bottleneck, not a claim that I/O was free: Kegel's own page identifies blocking disk I/O as a genuine problem in single-threaded readiness servers, just not the one the number ten thousand indexed. It also flags a contradiction within its own sources rather than resolving it: Kegel's 1999 page and the Banga et al. paper, published a month later, directly disagree on whether `poll()` had less overhead than `select()`; the note marks this contested, noting only that later measurement favors Banga. Historical dating is left unsettled too — the note cannot pin down which revision of Kegel's page first introduced the "idle descriptors" framing its whole argument rests on, only that the clause is absent from the archived 8 May 1999 snapshot and present in the current one. It also notes that no credible contemporaneous comparison of Apache prefork against an event-driven server on idle-connection cost was found, and omits that comparison rather than citing a weak source. Demo figures cited in the note's implementation section — 28,000 `read` calls versus one `epoll_pwait` for a hundred idle pipes — are explicitly marked as target numbers for a demonstration that has not yet been built, not as measured results.

Referenced sources

- [The C10K problem \(current revision\)](#) — Dan Kegel **HIGH**

- [The C10K problem \(8 May 1999 revision\)](#) — Dan Kegel, via the Internet Archive **HIGH**
- [select\(2\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [poll\(2\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [epoll\(7\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [A Scalable and Explicit Event Delivery Mechanism for UNIX](#) — Gaurav Banga, Jeffrey C. Mogul, Peter Druschel **HIGH**
- [Comparing and Evaluating epoll, select, and poll Event Mechanisms](#) — Louay Gammo, Tim Brecht, Amol Shukla, David Pariag **HIGH**
- [Scalability of Linux Event-Dispatch Mechanisms](#) — Abhishek Chandra and David Mosberger, HP Labs **HIGH**
- [Kqueue: A generic and scalable event notification facility](#) — Jonathan Lemon **HIGH**
- [C10M: The C10M problem](#) — Robert Graham **HIGH**
- [C10k problem](#) — Wikipedia **MEDIUM**
- [Async IO on Linux: select, poll, and epoll](#) — Julia Evans **MEDIUM**
- [poll vs select vs event-based](#) — Daniel Stenberg **MEDIUM**

12.77 V95 — The C10K Bottleneck: Waiting, Not I/O

The note argues that the C10K problem — Dan Kegel's 1999 goal of serving ten thousand simultaneous connections from one machine — is usually retold as a story about slow I/O, and that this retelling is wrong. The real cost was waiting on connections that were idle. Under a thread- or process-per-connection design, an idle connection still holds a stack and a scheduler slot; under `select()` or `poll()`, an idle descriptor is still handed to the kernel and rescanned on every call, because neither interface remembers anything between calls. The decisive evidence is a 2004 experiment (Gammo, Brecht, Shukla and Pariag) that held the request workload fixed and varied only the number of idle connections: `select` and `poll` matched `epoll`'s throughput with no idle connections present, then lost up to 79% once 10,000 idle connections were added alongside the same workload. A 2000 HP Labs measurement (Chandra and Mosberger) corroborates this from the CPU side, showing `select`-based servers pinned at 100% CPU with idle connections present "right from the beginning." The fixes — `epoll`, `kqueue`, and IOCP — did not make `read()` or `write()` faster; each relocated the interest set into the kernel so the application stopped re-stating it on every call, leaving the actual data-transfer calls unchanged.

The note is explicit about its limits. The "waiting, not I/O" framing is presented as a later sharpening of Kegel's own 1999 page, not a quotation of it: the earliest archived revision denies that hardware was the bottleneck but does not yet name idle descriptors as the cause. It also records, rather than silently resolves, a genuine disagreement between Kegel (1999) and Banga, Mogul and Druschel (1999) over whether `poll()` had lower overhead than `select()`, noting that later measurement favors the latter without treating the dispute as settled by the man pages. It insists the claim concerns the bottleneck, not that I/O was costless — Kegel's own page names blocking disk I/O as a first-class problem that readiness notification does not solve. Several questions are left open: which revision of Kegel's page first introduced the idle-descriptor language, whether Kegel's `poll / select` claim was later corrected, and whether any contemporaneous production comparison isolates idle-connection cost outside a research server. The lecture's planned demo figures are flagged as not-yet-measured targets.

Referenced sources

- [The C10K problem \(current revision\)](#), Dan Kegel **HIGH**
- [The C10K problem \(8 May 1999 revision\)](#), Dan Kegel, via the Internet Archive **HIGH**
- [select\(2\) — Linux manual page](#), Linux man-pages project **HIGH**
- [poll\(2\) — Linux manual page](#), Linux man-pages project **HIGH**
- [epoll\(7\) — Linux manual page](#), Linux man-pages project **HIGH**
- [A Scalable and Explicit Event Delivery Mechanism for UNIX](#), Gaurav Banga, Jeffrey C. Mogul, Peter Druschel **HIGH**
- [Comparing and Evaluating epoll, select, and poll Event Mechanisms](#), Louay Gammo, Tim Brecht, Amol Shukla, David Pariag **HIGH**
- [Scalability of Linux Event-Dispatch Mechanisms](#), Abhishek Chandra and David Mosberger **HIGH**
- [Kqueue: A generic and scalable event notification facility](#), Jonathan Lemon **HIGH**
- [C10M: The C10M problem](#), Robert Graham **HIGH**
- [C10k problem](#), Wikipedia **MEDIUM**
- [Async IO on Linux: select, poll, and epoll](#), Julia Evans **MEDIUM**
- [poll vs select vs event-based](#), Daniel Stenberg **MEDIUM**

12.78 V96 — The C10K problem as a cost of waiting, not a cost of I/O

This note argues that the C10K problem — Dan Kegel's 1999 name for getting one server to handle ten thousand simultaneous connections — was never a story about slow I/O, but about the recurring cost of managing idle connections. Under thread- or process-

per-connection designs, an idle connection still holds a stack and a scheduler slot; under `select()` or `poll()`, an idle descriptor still gets rescanned on every call, because neither interface remembers the watch set between calls. The note traces this cost model to Banga, Mogul and Druschel's 1999 USENIX paper, which states formally that `select()`'s cost is proportional to the size of the interesting descriptor set rather than to the events returned, and to a 2004 Linux Symposium experiment (Gammo et al.) that isolates the effect directly: `select` and `poll` match `epoll`'s throughput when no connections are idle, but lose up to 79% once 10,000 idle connections are added while the request workload stays fixed. An independent 2000 HP Labs measurement corroborates the same effect from the CPU side. The note frames `epoll`, `kqueue`, and Windows IOCP as sharing one design move — relocating the interest set into the kernel so it need not be re-stated on every wait — while the `read()` / `write()` call that follows notification is left completely unchanged.

The note is explicit about its limits. It flags a genuine, unresolved contradiction between Kegel's 1999 claim that `poll()` had less overhead than `select()` and Banga et al.'s contemporaneous claim that `poll()` was no better, noting only that later measurement favors Banga without declaring the earlier disagreement settled either way. It states that the man pages for `poll(2)` and `epoll(7)` supply the mechanism but not the complexity claim, so any $O(1)/O(n)$ formulation attributed to a man page would be an invention. It stresses that "waiting, not I/O" is a claim about the bottleneck at ten thousand connections specifically, not a claim that disk I/O was never a cost — Kegel's own page names blocking disk reads as a separate, real problem it does not dispute. The lecture's planned demonstration numbers (28,000 poll-driven read calls versus one `epoll_pwait` for a hundred idle pipes) are marked explicitly as target figures from a demo that has not yet been built, not as measurements, and the note declines to cite a production Apache-versus-event-driven comparison because no credible source for one was found.

Referenced sources

- [The C10K problem \(current revision\)](#) — Dan Kegel, last updated 5 February 2014 (minor correction: 28 May 2019) **HIGH**
- [The C10K problem \(8 May 1999 revision\)](#) — Dan Kegel, via the Internet Archive **HIGH**
- [select\(2\)](#) — Linux manual page — Linux man-pages project **HIGH**
- [poll\(2\)](#) — Linux manual page — Linux man-pages project **HIGH**
- [epoll\(7\)](#) — Linux manual page — Linux man-pages project **HIGH**
- [A Scalable and Explicit Event Delivery Mechanism for UNIX](#) — Gaurav Banga, Jeffrey C. Mogul, Peter Druschel, USENIX Annual Technical Conference 1999 **HIGH**

- Comparing and Evaluating epoll, select, and poll Event Mechanisms — Louay Gammou, Tim Brecht, Amol Shukla, David Pariag, Linux Symposium 2004 **HIGH**
- Scalability of Linux Event-Dispatch Mechanisms — Abhishek Chandra and David Mosberger, HP Labs, HPL-2000-174, December 2000 **HIGH**
- Kqueue: A generic and scalable event notification facility — Jonathan Lemon, FreeBSD Project, FREENIX Track, 2001 USENIX Annual Technical Conference **HIGH**
- C10M: The C10M problem — Robert Graham (Errata Security), 2013 **HIGH**
- C10k problem — Wikipedia (retrieved 2026-09-20) **MEDIUM**
- Async IO on Linux: select, poll, and epoll — Julia Evans (2017) **MEDIUM**
- poll vs select vs event-based — Daniel Stenberg (author and maintainer of curl) **MEDIUM**